

SIEM Engineer — Complete Study Notes

Boni Yeamin

September 2026

- SIEM Engineer — Study Notes
 - Series Index
 - Topic Summaries
 - Core Flow to Remember
 - How to Use These Notes
 - Suggested Lab Progression
 - Key Terms Glossary (Quick Reference)
 - Next Steps
- SIEM Engineer #01 — Log Management
 - Introduction
- 1. What is a Log?
- 2. Why is Log Management Important?
- 3. Common SIEM Log Sources
 - 3.1 Firewall Logs
- 4. Windows Logs
- 5. Linux Logs
- 6. Active Directory Logs
- 7. VPN Logs
- 8. Proxy Logs
- 9. IDS/IPS Logs
- 10. EDR / Endpoint Logs
- 11. Cloud Logs
- 12. Application Logs

- 13. Database Logs
- 14. Network Device Logs
- 15. How Does a Log Reach the SIEM?
- 16. Log Collection
- 17. Syslog
- 18. Agent-Based Collection
- 19. API-Based Log Collection
- 20. Log Transport
- 21. Log Parsing
- 22. Log Normalization
- 23. Timestamp Normalization
- 24. Log Enrichment
- 25. Data Quality
- 26. Log Loss
- 27. Duplicate Logs
- 28. Log Retention
- 29. Log Indexing & Search
- 30. Log Volume & EPS
- 31. Log Monitoring
- 32. Troubleshooting Missing Logs
- 33. Log Management Security
- 34. Log Management and Detection Engineering
- 35. Log Management and Threat Hunting
- 36. Practical SIEM Log Management Lab
- 37. SIEM Engineer Log Management Checklist
- 38. Real-World SIEM Engineer Mindset
- 39. The Complete Log Management Flow
 - Why Does Log Management Matter?
- Interview Questions for SIEM Engineer
- Final Takeaway
 - SIEM Engineer Learning Series
- SIEM Engineer #02 — SIEM Platforms
 - Introduction
- 1. What is a SIEM Platform?
- 2. What Does a SIEM Engineer Do With a SIEM Platform?
- 3. Major SIEM Platforms
- 4. Splunk

- 5. Splunk Architecture
- 6. SPL — Splunk Search Processing Language
- 7. Microsoft Sentinel
- 8. KQL — Kusto Query Language
- 9. Microsoft Sentinel Data Flow
- 10. IBM QRadar
- 11. QRadar Events
- 12. QRadar DSM
- 13. Elastic Security
- 14. Elasticsearch
- 15. OpenSearch
- 16. Wazuh
- 17. Wazuh Architecture
- 18. SIEM Platforms Have Similar Concepts
- 19. Query Language Is a Core Skill
- 20. Detection Rules
- 21. Alert vs Incident vs Offense
- 22. Dashboard
- 23. SIEM Platform Health
- 24. Choosing a SIEM Platform
- 25. What Should a SIEM Engineer Learn?
- 26. Practical Lab
- SIEM Platform Checklist
- 27. SIEM Engineer Interview Questions
- 28. The Most Important Mindset
- Final Takeaway
 - Next Article
 - SIEM Engineer Learning Series
- SIEM Engineer #03 — Log Integration
 - Introduction
- Log Integration
 - 1. What is Log Integration?
 - 2. Why is Log Integration Important?
 - 3. Common Log Integration Methods
 - 4. Syslog Integration
 - 5. Syslog Facility and Severity
 - 6. Syslog over UDP vs TCP

- 7. CEF Integration
- 8. API Integration
- 9. API Authentication
- 10. API Pagination
- 11. API Rate Limits
- 12. Agent-Based Integration
- 13. Forwarders
- 14. Windows Event Forwarding
- 15. File-Based Integration
- 16. Multiline Logs
- 17. Custom Parsers
- 18. Parsing vs Normalization
- 19. Data Onboarding
- 20. Log Integration Testing
- 21. End-to-End Validation
- 22. Troubleshooting: Logs Not Arriving
- 23. Troubleshooting: Logs Arrive but Fields Are Empty
- 24. Troubleshooting: Duplicate Logs
- 25. Troubleshooting: Log Delay
- 26. Secure Log Integration
- 27. Network Requirements
- 28. Certificates and TLS
- 29. Log Filtering
- 30. Cost and Data Volume
- 31. Critical vs Non-Critical Logs
- 32. Log Integration Documentation
- 33. Real-World Example: Firewall Integration
- 34. Real-World Example: Windows Integration
- 35. Integration Monitoring
- 36. Common Log Integration Mistakes
- 37. SIEM Engineer Log Integration Checklist
- 38. The Complete Integration Flow
 - Why Does Log Integration Matter?
- Interview Questions
- Final Takeaway
- Next Article
 - SIEM Engineer #04 — Detection Engineering

- SIEM Engineer Learning Series
- SIEM Engineer #04 — Detection Engineering
 - Introduction
 - 1. What is Detection Engineering?
 - 2. Why Detection Engineering Matters
 - 3. Detection Engineering Lifecycle
 - 4. Step 1 — Detection Idea
 - 5. Step 2 — Create a Security Hypothesis
 - 6. Step 3 — Identify Required Telemetry
 - 7. Detection Data Quality
 - 8. Detection Logic
 - 9. Types of Detection
 - 9.1 Threshold Detection
 - 9.2 Sequence Detection
 - 9.3 Behavioral Detection
 - 9.4 Anomaly Detection
 - 9.5 Correlation Detection
 - 10. Detection Rules vs Correlation Rules
 - 11. Sigma
 - 12. Sigma Rule Structure
 - 13. MITRE ATT&CK Mapping
 - 14. Detection Coverage
 - 15. Example — SSH Brute Force Detection
 - 16. Windows Failed Login Detection
 - 17. PowerShell Detection
 - 18. Privileged Account Change Detection
 - 19. Threat Intelligence Correlation
 - 20. Severity and Confidence
 - 21. False Positives
 - 22. Detection Tuning
 - 23. Alert Deduplication
 - 24. Exceptions and Allowlisting
 - 25. Detection Testing
 - 26. Example Detection Test
 - 27. Splunk Detection Example
 - 28. Microsoft Sentinel / KQL Concept
 - 29. Wazuh Detection Concept

- 30. Detection-as-Code
- 31. Detection Rule Documentation
- 32. Detection Health Monitoring
- 33. Detection Performance and Cost
- 34. Detection Coverage vs Detection Count
- 35. Detection Improvement Cycle
- 36. Practical SIEM Lab
- 37. Detection Engineering Checklist
- 38. Complete Detection Engineering Flow
- 39. SIEM Engineer Interview Questions
- 40. Important Lessons
- Final Takeaway
 - SIEM Engineer Learning Series
- SIEM Engineer #05 — Query & Investigation
 - Introduction
- 1. What is Query & Investigation?
- 2. Why Query Skills Matter
- 3. The Basic Investigation Questions
- 4. Query Building Fundamentals
- 5. Time Range
- 6. Investigation Timeline
- 7. Splunk SPL
- 8. Microsoft Sentinel and KQL
- 9. Elasticsearch / OpenSearch Query
- 10. Wazuh Investigation
- 11. Filtering
- 12. Field Selection
- 13. Aggregation
- 14. Group By
- 15. Correlation
- 16. Pivoting
- 17. Investigation Pivot Example
- 18. IP Investigation
- 19. User Investigation
- 20. Host Investigation
- 21. Process Investigation
- 22. Authentication Investigation

- 23. Alert Investigation
- 24. True Positive vs False Positive
- 25. Threat Hunting
- 26. Threat Hunting Questions
- 27. Query → Hypothesis → Evidence
- 28. Time-Based Investigation
- 29. Baseline Analysis
- 30. Statistical Investigation
- 31. Example — Investigating SSH Brute Force
- 32. Example — Windows Account Investigation
- 33. Example — Suspicious Process Investigation
- 34. IOC Investigation
- 35. Search Broad → Narrow
- 36. Common Query Mistakes
- 37. Query Performance
- 38. Query Language Skills
- 39. Query Skill Progression
- 40. Practical SIEM Lab
- 41. Investigation Checklist
- 42. Complete Investigation Flow
- 43. SIEM Engineer Interview Questions
- 44. Final Takeaway
- SIEM Engineer Learning Series
 - SIEM Engineer #06 — SOC Operations
 - Introduction
 - 1. What is SOC Operations?
 - 2. Main Responsibilities of a SOC
 - 3. SOC Team Structure
 - 4. SOC L1 Analyst
 - 5. SOC L2 Analyst
 - 6. SOC L3 / Senior Security Roles
 - 7. SIEM Engineer in SOC Operations
 - 8. Event vs Alert vs Incident
 - Event
 - Alert
 - Incident
 - 9. SOC Alert Lifecycle

- 10. Step 1 — Detect
- 11. Step 2 — Validate
- 12. Step 3 — Investigate
- 13. Step 4 — Determine Scope
- 14. Step 5 — Contain
- 15. Step 6 — Eradicate
- 16. Step 7 — Recover
- 17. Step 8 — Document
- 18. Complete Incident Response Flow
- 19. Alert Triage
- 20. Alert Priority
- 21. Severity vs Priority vs Confidence
- 22. Ticket Management
- 23. Alert Assignment
- 24. Escalation
- 25. Good Escalation Information
- 26. Root Cause Analysis
- 27. Attack Chain Analysis
- 28. SOC Runbooks
- 29. Playbook vs Runbook
- 30. Automation in SOC
- 31. Analyst Feedback
- 32. SOC Feedback Loop
- 33. SOC Metrics
- 34. MTTD
- 35. MTTR
- 36. SOC Shift Handover
- 37. Evidence Handling
- 38. Least Privilege in SOC
- 39. SOC Access Control
- 40. SOC Dashboard
- 41. Practical SOC Lab
- 42. Realistic SOC Investigation Example
- 43. SOC and SIEM Relationship
- 44. SIEM Engineer + SOC Analyst
- 45. SOC Operations Checklist
- 46. SIEM Engineer / SOC Interview Questions

- 47. Important SOC Principles
- 48. Final Takeaway
 - SIEM Engineer Learning Series
- SIEM Engineer #07 — Threat Intelligence
 - Introduction
- 1. What is Threat Intelligence?
- 2. Why Threat Intelligence Matters in SIEM
- 3. What is an IOC?
- 4. IOC ≠ Incident
- 5. Types of Threat Intelligence
 - Strategic Intelligence
 - Tactical Intelligence
 - Operational Intelligence
 - Technical Intelligence
- 6. Threat Intelligence Lifecycle
- 7. Step 1 — Intelligence Requirements
- 8. Step 2 — Collection
- 9. Step 3 — Processing
- 10. Step 4 — Analysis
- 11. Step 5 — Dissemination
- 12. Step 6 — Feedback
- 13. Threat Intelligence in SIEM
- 14. IOC Enrichment
- 15. IOC Correlation
- 16. IP Reputation
- 17. Domain Intelligence
- 18. URL Intelligence
- 19. Hash Intelligence
- 20. Email Threat Intelligence
- 21. Threat Intelligence Confidence
- 22. Freshness Matters
- 23. Threat Feed Quality
- 24. Threat Intelligence False Positives
- 25. Internal Intelligence
- 26. Internal IOC Lifecycle
- 27. Threat Intelligence Platform
- 28. SIEM Threat Intelligence Integration

- 29. STIX and TAXII
- 30. Threat Intelligence + Detection Engineering
- 31. Threat Intelligence + Threat Hunting
- 32. Historical Search
- 33. Threat Intelligence Correlation Across Data Sources
- 34. Threat Intelligence and MITRE ATT&CK
- 35. Automated Enrichment
- 36. Security of Threat Intelligence Integrations
- 37. Threat Intelligence Cost Management
- 38. Threat Intelligence Dashboard
- 39. Practical SIEM Lab
- 40. Threat Intelligence Checklist
- 41. Complete Threat Intelligence Flow
- 42. SIEM Engineer Interview Questions
- 43. Important Lessons
- 44. Final Takeaway
- SIEM Engineer Learning Series
 - SIEM Engineer #08 — Automation & SOAR
 - Introduction
 - 1. What is SOAR?
 - 2. Why Automation Matters in SOC
 - 3. Automation vs SOAR
 - 4. What is Orchestration?
 - 5. Basic SOAR Architecture
 - 6. What is a Playbook?
 - 7. Runbook vs Playbook
 - 8. Alert Automation
 - 9. Enrichment Automation
 - 10. Automated Investigation
 - 11. Automatic Ticket Creation
 - 12. Notification Automation
 - 13. Automated Response
 - 14. Why Automated Response Is Risky
 - 15. Human-in-the-Loop
 - 16. Fully Automated vs Semi-Automated
 - 17. Automation Decision Matrix
 - 18. APIs in SOAR

- 19. API Authentication
- 20. API Rate Limits
- 21. Retry Logic
- 22. Idempotency
- 23. Duplicate Alert Handling
- 24. Automation Metrics
- 25. Testing Automation
- 26. Automation Development Lifecycle
- 27. Detection + SOAR
- 28. Example — Suspicious IP Workflow
- 29. Example — Phishing Workflow
- 30. Example — Compromised Account Workflow
- 31. Example — Malware Alert Workflow
- 32. SOAR and Threat Intelligence
- 33. SOAR and Threat Hunting
- 34. SOAR Error Handling
- 35. SOAR Security
- 36. Audit Logging
- 37. Rollback
- 38. Human Approval Workflow
- 39. Playbook Documentation
- 40. Practical SOAR Lab
- 41. SOC Automation Architecture
- 42. Automation Maturity
- 43. Common SOAR Mistakes
- 44. SOAR Checklist
- 45. Complete SOAR Flow
- 46. SIEM Engineer Interview Questions
- 47. Important Lessons
- 48. Final Takeaway
- SIEM Engineer Learning Series
 - Introduction
 - 1. What is a SIEM Dashboard?
 - 2. Why Dashboard Matters
 - Why does this matter?
 - 3. Dashboard vs Report

- 4. SIEM Engineer's Role
- 5. Different Dashboard Users
- 6. SOC Analyst Dashboard
- 7. Alert Dashboard
- 8. Alert Trend
- 9. Severity Dashboard
- 10. Priority vs Severity
- 11. User-Based Dashboard
- 12. Endpoint Dashboard
- 13. Network Security Dashboard
- 14. Firewall Dashboard
- 15. Authentication Dashboard
- 16. Detection Dashboard
- 17. MITRE ATT&CK Coverage Dashboard
- 18. Detection Coverage ≠ Attack Prevention
- 19. Incident Dashboard
- 20. MTTD
- 21. MTTR
- 22. SOC Metrics Dashboard
- 23. Automation Dashboard
- 24. Log Source Health Dashboard
- 25. EPS Monitoring
- 26. Data Volume Dashboard
- 27. Query Design for Dashboards
- 28. Time-Series Visualization
- 29. Geographic Visualization
- 30. Table Visualization
- 31. Dashboard Filters
- 32. Dashboard Drill-Down
- 33. Dashboard → Investigation
- 34. Dashboard Design Principles
- 35. Common Dashboard Mistakes
- 36. Dashboard Validation
- 37. Dashboard Security
- 38. Security Reports
- 39. Daily SOC Report
- 40. Monthly Security Report

- 41. Executive Dashboard
- 42. Analyst Dashboard vs Executive Dashboard
- 43. Data Quality and Reporting
- 44. SIEM Reporting Pipeline
- 45. Practical Lab — SOC Dashboard
- 46. Practical Wazuh Dashboard Exercise
- 47. Dashboard Development Workflow
- 48. Dashboard Maintenance
- 49. Dashboard as a Detection Engineering Tool
- 50. Dashboard as an Investigation Tool
- 51. SIEM Engineer Interview Questions
- 52. Dashboard Checklist
- 53. Complete Dashboard & Reporting Flow
- 54. Key Skills for a SIEM Engineer
- 55. Final Takeaway
- SIEM Engineer Learning Series
 - Introduction
 - 1. What is SIEM Infrastructure?
 - 2. Why SIEM Infrastructure Matters
 - 3. Basic SIEM Architecture
 - 4. Main SIEM Infrastructure Components
 - 5. Log Collectors
 - 6. Log Forwarders
 - 7. Ingestion Layer
 - 8. EPS
 - 9. Data Volume
 - 10. Storage Planning
 - 11. Hot, Warm and Cold Data
 - 12. Retention Planning
 - 13. Storage Full Problem
 - 14. CPU Monitoring
 - 15. Memory Monitoring
 - 16. Disk I/O
 - 17. Network Monitoring
 - 18. Network Segmentation
 - 19. SIEM Server Hardening

- 20. Access Control
- 21. High Availability
- 22. Single Node vs Cluster
- 23. Load Balancing
- 24. Failover
- 25. Queue and Buffer
- 26. Disaster Recovery
- 27. Backup
- 28. Restore Testing
- 29. RPO and RTO
- 30. SIEM Upgrade
- 31. Patch Management
- 32. SIEM Health Monitoring
- 33. Log Pipeline Health
- 34. Detecting Missing Logs
- 35. Log Latency
- 36. Search Performance
- 37. Capacity Planning
- 38. Scaling
- 39. Data Tiering
- 40. Encryption
- 41. Certificate Management
- 42. Infrastructure Automation
- 43. Configuration Management
- 44. Change Management
- 45. SIEM Infrastructure Testing
- 46. Common Infrastructure Failures
- 47. SIEM Troubleshooting Method
- 48. Root Cause Analysis
- 49. SIEM Infrastructure Monitoring Dashboard
- 50. Practical SIEM Infrastructure Lab
- 51. Docker-Based SIEM Lab
- 52. SIEM Infrastructure Security Checklist
- 53. SIEM Engineer Interview Questions
- 54. Complete SIEM Infrastructure Checklist
- 55. Complete SIEM Engineer Infrastructure Flow
- 56. Complete SIEM Engineer Skill Map

- 57. Final Takeaway
- SIEM Engineer — Complete Learning Series
 - What Next?
- SIEM Engineer #11 — SIEM Architecture Design
 - 1. What is SIEM Architecture?
 - 2. Why does SIEM Architecture matter?
 - 3. SIEM Architecture Design Principles
 - 4. Small SIEM Architecture
 - 5. Medium SIEM Architecture
 - 6. Enterprise SIEM Architecture
 - 7. Centralized vs Distributed Architecture
 - Centralized Architecture
 - 8. Distributed Architecture
 - 9. Single-node vs Multi-node
 - Single-node
 - Multi-node
 - 10. Collector
 - 11. Forwarder
 - 12. Ingestion Layer
 - 13. Processing Layer
 - 14. Indexer / Storage Layer
 - 15. Search Layer
 - 16. Dashboard Layer
 - 17. Detection Layer
 - 18. SIEM Components — Complete View
 - 19. High Availability — HA
 - 20. Load Balancing
 - 21. Scalability
 - Vertical Scaling
 - Horizontal Scaling
 - 22. Architecture Design-এর আগে কী জানতে হবে?
 - 23. EPS — Events Per Second
 - 24. Storage Architecture
 - 25. SIEM Network Architecture
 - 26. SIEM Security Architecture
 - 27. Role-Based Access Control — RBAC
 - 28. On-Prem SIEM Architecture

- 29. Cloud SIEM Architecture
- 30. Hybrid SIEM Architecture
- 31. Disaster Recovery — DR
- 32. RPO and RTO
- 33. Observability & Health Monitoring
- 34. Common SIEM Architecture Mistakes
- 35. Practical SIEM Architecture Lab
- 36. Real-World Example
- 37. SIEM Architecture Design Process
- 38. Architecture Documentation
- 39. SIEM Platform Architecture — Conceptual View
- 40. SIEM Architecture Interview Questions & Answers
 - Q1. What is SIEM Architecture?
 - Q2. Single-node and distributed SIEM-এর মধ্যে পার্থক্য কী?
 - Q3. What is a SIEM Collector?
 - Q4. What is an Indexer?
 - Q5. Why is High Availability important in SIEM?
 - Q6. What is the difference between vertical and horizontal scaling?
 - Q7. What factors do you consider when designing SIEM architecture?
 - Q8. Why should peak EPS be considered?
 - Q9. What is RPO?
 - Q10. What is RTO?
 - Q11. How would you troubleshoot a SIEM architecture where logs are delayed?
- 41. SIEM Architecture Checklist
- 42. Complete SIEM Architecture Flow
- 43. What should a SIEM Engineer remember?
- Practical Security Takeaway
- Next in the SIEM Engineer Series
 - SIEM Engineer #12 — SIEM Capacity Planning
 - 1. What is SIEM Capacity Planning?
 - 2. Why does Capacity Planning matter?
 - 3. Key Capacity Metrics
 - 4. What is EPS?
 - 5. EPS Calculation
 - 6. Events Per Day
 - 7. Average Event Size
 - 8. GB Per Day Calculation

- 9. General Storage Formula
- 10. Storage Planning
- 11. Hot Storage
- 12. Warm Storage
- 13. Cold Storage
- 14. Archive
- 15. Retention Planning
- 16. Growth Estimation
- 17. Simple Growth Example
- 18. Average EPS vs Peak EPS
- 19. Capacity Buffer
- 20. Network Capacity Planning
- 21. CPU Planning
- 22. RAM Planning
- 23. Disk I/O Planning
- 24. Replication Factor
- 25. Complete Storage Estimation Example
- 26. Real-World Example
- 27. Capacity Planning Worksheet
- Capacity Planning Checklist
- 28. Capacity Monitoring
- 29. Capacity Alerting
- 30. Capacity Planning Lifecycle
- 31. Practical Lab
- 32. Interview Questions & Answers
 - Q1. What is SIEM Capacity Planning?
 - Q2. What is EPS?
 - Q3. How do you calculate events per day?
 - Q4. What factors affect SIEM storage?
 - Q5. Why is peak EPS important?
 - Q6. What is retention?
 - Q7. Is raw GB/day enough for SIEM storage planning?
 - Q8. How would you handle future SIEM growth?
 - Q9. What happens if SIEM storage becomes full?
 - Q10. Which metrics do you monitor for capacity?
- 33. Common Capacity Planning Mistakes
- 34. SIEM Engineer Mindset

- 35. Complete Capacity Planning Flow
- Final Takeaway
- SIEM Engineer Series Navigation
- SIEM Engineer #13 — SIEM Performance Tuning
- 1. Why does SIEM Performance matter?
- 2. SIEM Performance-এর Main Areas
- 3. Performance Troubleshooting Mindset
- 4. Important Performance Metrics
- 5. Slow Query — First Problem
- 6. Time Range Optimization
- 7. Filter Early
- 8. Search the Right Data
- 9. Query Optimization
- 10. Wildcard Problem
- 11. Too Many Fields
- 12. Aggregation Performance
- 13. CPU Performance
- 14. High CPU Troubleshooting Flow
- 15. RAM / Memory Performance
- 16. Swap Problem
- 17. Disk I/O
- 18. Important Disk Metrics
- 19. High Disk I/O Symptoms
- 20. Storage Optimization
- 21. Index Optimization
- 22. Index Size
- 23. Sharding / Partitioning
- 24. Replication vs Performance
- 25. Ingestion Performance
- 26. Queue Monitoring
- 27. Event Latency
- 28. Performance Baseline
- 29. Before & After Testing
- 30. Performance Tuning Process
- 31. Real-World Problem — Slow Search
- 32. Real-World Problem — Logs Delayed
- 33. Real-World Problem — High CPU

- 34. Performance Optimization Areas
- 35. Detection Rule Performance
- 36. Dashboard Performance
- 37. Search Concurrency
- 38. Performance vs Capacity
- 39. Practical Lab
- 40. Performance Troubleshooting Checklist
- 41. Interview Questions & Answers
 - Q1. What is SIEM Performance Tuning?
 - Q2. What would you check if SIEM searches are slow?
 - Q3. How do you troubleshoot high CPU?
 - Q4. Why is disk I/O important for SIEM?
 - Q5. What is event latency?
 - Q6. What causes SIEM ingestion delay?
 - Q7. How can you optimize a SIEM query?
 - Q8. What is the difference between capacity and performance?
- 42. Common Performance Tuning Mistakes
- 43. SIEM Engineer Performance Mindset
- 44. Complete SIEM Performance Flow
- Final Security Takeaway
- SIEM Engineer Series Navigation
- SIEM Engineer #14 — Advanced Detection Engineering
 - Introduction
 - What is Advanced Detection Engineering?
 - Why does this matter?
 - Detection Engineering Lifecycle
 - Security Hypothesis
 - Data Source Identification
 - Understand Your Fields
 - Single-Event Detection
 - Correlation Detection
 - Threshold Detection
 - Sequence Detection
 - Behavioral Detection
 - Baseline Analysis
 - Entity-Based Detection

- Risk-Based Detection
- Severity vs Risk vs Confidence
- False Positive Reduction
- Exceptions and Allowlists
- Suppression
- Detection for Brute Force
- PowerShell Detection
- Privilege Escalation Detection
- Lateral Movement Detection
- Firewall Scanning Detection
- MITRE ATT&CK Mapping
- Detection Coverage
- Sigma Rules
- Detection-as-Code
- Detection Testing
- Detection Performance
- Detection Monitoring
- Detection Tuning Process
- Real-World Example — Suspicious Login
- Practical Detection Engineering Lab
- Detection Engineering Documentation
- Detection Engineering + SOC
- Common Mistakes
- SIEM Engineer Interview Questions
- Detection Engineering Checklist
- Complete Advanced Detection Flow
- Final Takeaway
- SIEM Engineer Learning Series
- SIEM Engineer #15 — Threat Hunting with SIEM
 - Introduction
 - What is Threat Hunting?
 - Why does this matter?
 - Threat Hunting vs Alert Investigation
 - Threat Hunting Lifecycle
 - Start with a Hypothesis
 - Types of Threat Hunting

- [IOC Hunting](#)
- [Historical IOC Hunting](#)
- [TTP Hunting](#)
- [MITRE ATT&CK and Threat Hunting](#)
- [Data Sources for Threat Hunting](#)
- [Search Strategy](#)
- [Time-Based Hunting](#)
- [Baseline Hunting](#)
- [User Behavior Hunting](#)
- [Host-Based Hunting](#)
- [Process Hunting](#)
- [Network Threat Hunting](#)
- [DNS Hunting](#)
- [Authentication Hunting](#)
- [Impossible Travel / Abnormal Location](#)
- [Threat Hunting with Multiple Data Sources](#)
- [Pivoting](#)
- [Entity Relationship Hunting](#)
- [Threat Intelligence Enrichment](#)
- [Hunt Results](#)
- [Threat Hunting → Detection Engineering](#)
- [Practical Hunt — SSH Activity](#)
- [Practical Hunt — Suspicious PowerShell](#)
- [Practical Hunt — Lateral Movement](#)
- [Threat Hunting Query Design](#)
- [Hunting Notebook](#)
- [Threat Hunting Metrics](#)
- [Common Threat Hunting Mistakes](#)
- [Threat Hunting Lab](#)
- [SIEM Engineer Interview Questions](#)
- [Threat Hunting Checklist](#)
- [Complete Threat Hunting Flow](#)
- [Final Takeaway](#)
 - [Practical Security Takeaway](#)
 - [SIEM Engineer Learning Series](#)

- SIEM Engineer #16 — SIEM Data Quality & Troubleshooting
 - Introduction
 - What is SIEM Data Quality?
 - Why does this matter?
 - Data Quality Dimensions
 - Common SIEM Data Problems
 - The Data Troubleshooting Flow
 - Missing Logs
 - Missing Log Troubleshooting
 - Source Health Monitoring
 - Log Silence Detection
 - Duplicate Logs
 - Why Duplicate Logs Matter
 - Detecting Duplicates
 - Parsing Failure
 - Why Parsing Matters
 - Parsing Troubleshooting
 - Field Mapping Problems
 - Timestamp Problems
 - Why Timestamp Accuracy Matters
 - Time Synchronization
 - Log Latency
 - Sources of Log Latency
 - Queue and Backpressure
 - Event Drops
 - Log Completeness
 - Data Quality Validation
 - Log Source Onboarding Validation
 - Raw vs Parsed Data
 - Detection Failure Because of Data Quality
 - Data Quality and Threat Hunting
 - Data Quality Monitoring Dashboard
 - Data Quality Alerts
 - Troubleshooting Example — Firewall Logs Missing
 - Troubleshooting Example — Duplicate Events

- Troubleshooting Example — Wrong Timestamp
- Troubleshooting Example — Parsing Failure
- Data Quality SLA
- Critical vs Non-Critical Sources
- Data Quality and Compliance
- Practical Lab
- SIEM Data Quality Checklist
- SIEM Engineer Interview Questions
- Complete SIEM Data Quality Flow
- Final Takeaway
 - Practical Security Takeaway
 - SIEM Engineer Learning Series
- SIEM Engineer #17 — Hands-On SIEM: Multi-Platform Practical Guide
 - Introduction
- 1. Platform-Independent Core Skills
- 2. The Unified Lab Architecture
- 3. Platform Setup — কোনটা কীভাবে
 - 3.1 Wazuh (Free, Open-Source) — Primary
 - 3.2 Splunk (Free/Trial Lab Option — 500 MB/day)
 - 3.3 Elastic Security (Basic License Lab Option)
 - 3.4 OpenSearch (Open-Source Lab Option)
 - 3.5 Microsoft Sentinel (Cloud)
- 4. The One-Dataset Approach — একই Data, সব Platform
 - 4.1 Standard Dataset — যা generate করবেন
 - 4.2 Attack Simulation Script (SSH Brute Force)
 - 4.3 Log Delivery — কোন platform-এ কীভাবে পৌঁছাবে
- 5. Same Task, Different Platform — Mapping Table
- 6. Exercise 1 — Search: Same Query, 4 Languages
 - Wazuh
 - Splunk (SPL)
 - Elastic (Kibana KQL)
 - Sentinel (Kusto KQL)
- 7. Exercise 2 — Detection: Brute Force Rule Everywhere
 - 7.1 Wazuh Custom Rule
 - 7.2 Splunk Correlation Search
 - 7.3 Elastic Detection Rule

- 7.4 OpenSearch Security Analytics
- 7.5 Sentinel Analytics Rule (KQL)
- ◦ 8. Exercise 3 — Investigation: Attack Chain খোঁজা
 - Splunk version:
 - Sentinel version:
- 9. Exercise 4 — Dashboard: একই Panel, সব Platform
- ◦ 10. Exercise 5 — Automation: SOAR Mini-Lab
 - 10.1 Webhook Alert → Python Script
 - 10.2 Sentinel Playbook (Logic App)
 - 10.3 Shuffle (Free SOAR)
- ◦ 11. Exercise 6 — Threat Intel Integration
 - MISP Lab (Free TIP)
- ◦ 12. 6-Week Combined Learning Plan — সব একসাথে
- ◦ 13. Sigma — একবার লিখুন, সব জায়গায় চালান
- ◦ 14. Windows Event ID Cheat Sheet (Lab-এ যা collect করবেন)
- ◦ 15. Common Multi-Platform Lab Mistakes
- 16. Multi-Platform SIEM Checklist
- 17. Interview Questions
- Final Takeaway
 - SIEM Engineer Learning Series

SIEM Engineer — Study Notes

A complete study-note series covering the **SIEM Engineer** learning path — from log management fundamentals to enterprise SIEM architecture design. Written for SOC/SIEM engineer interview preparation and hands-on lab practice.

Languages: Articles #01–#03, #11–#13, #17 are written in **Bengali + English (Banglish)**. Articles #04–#10 and #14–#16 are in **English**.

Note: The full series (all 17 articles) lives in the [a11/](#) folder.

Series Index

#	File	Topic	Status
01	01_soc.md	Log Management	Complete
02	02_soc.md	SIEM Platforms	Complete
03	03_soc.md	Log Integration	Complete
04	04_soc.md	Detection Engineering	Complete
05	05_soc.md	Query & Investigation	Complete
06	06_soc.md	SOC Operations	Complete
07	07_soc.md	Threat Intelligence	Complete
08	08_soc.md	Automation & SOAR	Complete
09	09_soc.md	Dashboard & Reporting	Complete
10	10_soc.md	SIEM Infrastructure	Complete
11	11_soc.md	SIEM Architecture Design	Complete
12	12_soc.md	SIEM Capacity Planning	Complete
13	13_soc.md	SIEM Performance Tuning	Complete
14	14_soc.md	Advanced Detection Engineering	Complete
15	15_SIEM.md	Threat Hunting with SIEM	Complete
16	16_SIEM.md	SIEM Data Quality & Troubleshooting	Complete

#	File	Topic	Status
17	17_SIEM.md	Hands-On SIEM: Multi-Platform Practical Guide	Complete

Topic Summaries

Each summary follows the same format: **what it covers** → **key flow** → **hands-on outcome**.
Click the title to open the full note.

Phase 1 — Foundation (#01–#03)

#01 — Log Management

What logs are and how they reach the SIEM. - **Covers:** Log sources (Firewall, Windows, Linux, AD, VPN, Proxy, IDS/IPS, EDR, Cloud, App, DB, Network), Syslog / Agent / API collection, parsing, normalization, enrichment - **Key flow:** Collect → Transport → Parse → Normalize → Enrich → Store → Search → Monitor → Retain - **Hands-on:** EPS basics, data quality, log loss & duplicates, retention, missing-log troubleshooting

#02 — SIEM Platforms

How the major SIEMs work and how concepts map across them. - **Covers:** Splunk (SPL, Forwarder / Indexer / Search Head), Sentinel (KQL, Analytics Rules), QRadar (DSM, Offenses, AQL), Elastic, OpenSearch, Wazuh - **Key idea:** Log Sources → Collection → SIEM Platform → Search / Detection / Alert → Investigation → Dashboard - **Hands-on:** Compare the same workflow on different platforms, platform-health checks, lab with Wazuh

#03 — Log Integration

Connecting a new source and making it usable in the SIEM. - **Covers:** Syslog (facility / severity, UDP vs TCP vs TLS), CEF, API (auth / pagination / rate limits), Agents, Forwarders, WEF, file-based, cloud connectors, custom parsers - **Key flow:** Identify → Configure → Transport → Parse → Normalize → Test Search → Detect → Monitor - **Hands-on:** 10-step data onboarding, parsing vs normalization, end-to-end validation, troubleshooting

Phase 2 — Detection & Investigation (#04–#05, #14–#15)

#04 — Detection Engineering

Turning telemetry into reliable detections. - **Covers:** Detection lifecycle, threshold / sequence / behavioral / anomaly / correlation types, Sigma rules, MITRE ATT&CK mapping (e.g. T1059.001, T1110), severity vs confidence - **Key flow:** Hypothesis → Data → Logic → Rule → Test → Tune → Deploy → Monitor → Improve - **Hands-on:** SSH brute-force, Windows failed-login, PowerShell rules, FP tuning, deduplication, Detection-as-Code

#05 — Query & Investigation

Finding the story in the data with SPL / KQL. - **Covers:** Filtering, aggregation, group by, sorting, pivoting (IP → User → Host → Process → Network), timeline analysis, before / during / after method - **Key flow:** Alert → Validate → Filter → Pivot → Correlate → Scope → Escalate / Close - **Hands-on:** IP / user / host / process / auth investigations, TP vs FP verdicts, IOC hunting, query-performance pitfalls

#14 — Advanced Detection Engineering

Accurate, context-aware detections beyond single events. - **Covers:** Single-event vs correlation vs behavioral vs risk-based detections, baselines, entity-based logic, allowlists, suppression, FP-rate reduction - **Key idea:** Risk = Event + Behavior + TI + Context - **Hands-on:** Brute-force, PowerShell, privesc, lateral-movement, scanning detections + Sigma YAML structure

#15 — Threat Hunting with SIEM

Proactively looking for what alerts missed. - **Covers:** Hypothesis-driven hunting, IOC vs TTP hunting, MITRE-based hunts, time / baseline / user / host / process / DNS / auth hunting, pivoting - **Key flow:** Hypothesis → Search → Evidence → Verdict → New / Improved Detection - **Hands-on:** 6 starter hunts, hunt notebook template, TI enrichment, hunt → detection promotion

Phase 3 — SOC, Intel & Automation (#06–#08)

#06 — SOC Operations

How a SOC turns alerts into resolved incidents. - **Covers:** L1 / L2 / L3 roles, **Event vs Alert vs Incident**, triage, severity vs priority vs confidence, ticketing, escalation, RCA, attack-chain analysis - **Key flow:** Detect → Validate → Investigate → Scope → Contain → Eradicate → Recover → Document - **Hands-on:** Suspicious-login runbook, runbook vs playbook, MTTD / MTTR, shift handover, evidence handling

#07 — Threat Intelligence

Giving alerts context with IOCs and intel lifecycle. - **Covers:** IOC types (IP, domain, URL, hash, email), strategic / tactical / operational / technical intel, CTI lifecycle, STIX / TAXII, TIPs, enrichment & correlation - **Key flow:** Direction → Collection → Processing → Analysis → Dissemination → Feedback - **Hands-on:** IP / domain / URL / hash / email intel, feed quality, freshness & confidence, internal IOCs

#08 — Automation & SOAR

Automating the repetitive SOC work safely. - **Covers:** Orchestration, playbooks, enrichment, auto-ticketing, notification, automated response risks, **human-in-the-loop**, fully vs semi-automated, APIs, idempotency, rollback - **Key flow:** Alert → Enrich → Decide → Act → Ticket → Notify → Audit - **Hands-on:** Suspicious-IP, phishing & compromised-account workflows, decision matrix, automation metrics

Phase 4 — Reporting & Infrastructure (#09–#13, #16)

#09 — Dashboard & Reporting

Making SOC activity visible and measurable. - **Covers:** Dashboard vs report, analyst / manager / executive views, alert trends, severity, auth / endpoint / network / firewall panels, MITRE coverage, drill-down design - **Key metrics:** MTTD / MTTR, EPS & source-health, detection coverage - **Hands-on:** Time-series / geo / table visualizations, filters, dashboard → investigation workflow

#10 — SIEM Infrastructure

What the SIEM runs on and how to keep it healthy. - **Covers:** Collectors, forwarders, ingestion, EPS & data-volume math, Hot → Warm → Cold storage, retention, CPU / memory / disk / network monitoring - **Key topics:** HA & clustering, load balancing, failover, queues / buffers, DR, backup & restore testing, RPO / RTO, upgrades - **Hands-on:** Storage estimation (~172 GB/day example), health monitoring, scaling vertical vs horizontal

#11 — SIEM Architecture Design

Designing small, medium and enterprise SIEMs. - **Covers:** Design principles, centralized vs distributed, single-node vs multi-node, collector / ingestion / processing / indexer / search / dashboard / detection layers - **Key topics:** HA, scalability, network & security architecture, RBAC, on-prem vs cloud vs hybrid, observability, DR - **Hands-on:** Real-world design example, common architecture mistakes, sizing inputs for #12

#12 — SIEM Capacity Planning

Sizing the SIEM with real numbers. - **Covers:** EPS, events/day ($\text{EPS} \times 86400$), GB/day, retention sizing, peak vs average, growth, replication & buffer - **Key formulas:** $\text{Events/day} = \text{EPS} \times 86400 \rightarrow \text{GB/day} = \text{events} \times \text{avg size} \rightarrow \text{Retention} = \text{GB/day} \times \text{days}$ - **Hands-on:** Planning worksheet, $2000 \text{ EPS} \times 1\text{KB} \times 30\text{d} \approx 5.18\text{TB}$ example, capacity alerts (80% warn / 90% crit)

#13 — SIEM Performance Tuning

Finding and fixing slowness. - **Covers:** Bottlenecks (ingestion, storage, query, CPU, memory, indexing), ingestion delay, slow-search analysis, time-range & filter-early optimization, index / shard tuning - **Key flow:** Identify → Measure Baseline → Tune → Before / After Test → Monitor - **Hands-on:** Slow-search, delayed-logs & high-CPU scenarios, queue & latency checks

#16 — SIEM Data Quality & Troubleshooting

Making sure the data is trustworthy for detection. - **Covers:** Availability, completeness, accuracy, timeliness, parsing & field mapping, timestamps, duplicates, latency, drops - **Key flow:** Source → Network → Collector → Ingestion → Parsing → Search → Detection - **Hands-on:** Silence detection, EPS-drop alerts, parsing-failure triage, example SLAs (99.5% availability, ≤5 min latency)

Capstone (#17)

#17 — Hands-On SIEM: Multi-Platform Practical Guide

One lab, one dataset, same exercises on 5 platforms. - **Covers:** Unified lab (Ubuntu + Windows + Attacker VM), Wazuh / Splunk / Elastic / OpenSearch / Sentinel setup, one-dataset approach - **Key work:** Same brute-force detection in 5 query languages, attack-chain investigation, dashboards, SOAR / Python automation, MISP intel, Sigma rules, Windows Event ID cheat sheet - **Hands-on:** 6-week plan (Wazuh → Splunk → Elastic → Sentinel), portfolio-ready screenshots & write-up

Core Flow to Remember

The single pipeline that connects the whole series:

Log Source → Collection → Transport → Parsing → Normalization → Enrichment
 → Indexing → Storage → Detection → Alert → Investigation
 → Response → Automation → Dashboard → Reporting

How to Use These Notes

1. **Read in order (#01 → #17)** — each article builds on the previous one.
2. **#17 is the hands-on capstone** — do its 6-week lab plan alongside everything else.
3. **Every article ends with:**
 - Interview questions & answers
 - Practical checklist
 - Final takeaway
4. **Build a lab alongside reading** — the recommended setup used across the series:
 - Windows VM + Linux VM + Firewall (or pfSense) + **Wazuh** as the SIEM
 - Practice: generate failed SSH/Windows logins → collect → parse → detect → investigate → tune → dashboard
5. **Interview prep:** review the Q&A sections of each article, plus the checklists.

Suggested Lab Progression

Theory (#01-#03) → Build Wazuh lab & onboard logs
 Detection (#04, #14) → Write & test detection rules
 Investigation (#05) → Pivot and build timelines
 SOC (#06) → Run full alert triage workflow
 TI + SOAR (#07-#08) → Enrich alerts & automate
 Reporting (#09) → Build SOC dashboard
 Infra (#10-#13) → Size, tune, scale & document the architecture
 Hunting (#15) → Run hypothesis-driven threat hunts
 Quality (#16) → Validate data quality end-to-end
 Hands-On (#17) → Same exercises on Wazuh + Splunk + Elastic + Sentinel

Key Terms Glossary (Quick Reference)

Term	Meaning
SIEM	Security Information and Event Management
SOC	Security Operations Center
EPS	Events Per Second — ingestion rate metric
IOC	Indicator of Compromise (IP, domain, hash, URL...)
CEF	Common Event Format — standardized log format
DSM	QRadar Device Support Module (log parsing)
SPL / KQL / AQL	Query languages: Splunk / Sentinel / QRadar
Sigma	SIEM-independent detection rule format
MITRE ATT&CK	Adversary tactics & techniques knowledge base
SOAR	Security Orchestration, Automation and Response
STIX / TAXII	Threat intel representation / exchange standards
TIP	Threat Intelligence Platform
WEF	Windows Event Forwarding
MTTD / MTTR	Mean Time to Detect / Respond (or Resolve)
RPO / RTO	Recovery Point / Time Objective
HA	High Availability
RBAC	Role-Based Access Control
FIM	File Integrity Monitoring
IOC Enrichment	Adding context to an indicator (source, confidence, freshness)

Term	Meaning
Data Onboarding	Connecting a new log source & making it usable in the SIEM

Next Steps

- ☐ Write #12 — **SIEM Capacity Planning** → [all/12_soc.md](#)
 - ☐ Practice hands-on: complete the Wazuh lab exercises from each article
 - ☐ Review all interview Q&A sections before applying
 - ☐ Build a SOC dashboard project from #09 concepts
 - ☐ Run a full threat-hunt exercise (#15) in the lab and document findings
 - ☐ Complete the #17 **multi-platform lab plan** (Wazuh → Splunk → Elastic → Sentinel) and build a GitHub portfolio
-

Last updated: September 2026

SIEM Engineer #01 — Log Management

Introduction

A SIEM Engineer works with a huge amount of security data every day.

Firewall, Windows Server, Linux Server, Active Directory, VPN, Proxy, IDS/IPS, EDR, Cloud Platform, Database, and Applications—সব জায়গা থেকেই বিভিন্ন ধরনের logs generate হয়।

কিন্তু শুধু logs collect করলেই হবে না।

একজন SIEM Engineer-এর দায়িত্ব হলো:

Collect → Transport → Parse → Normalize → Enrich → Store → Search → Monitor → Retain

এই পুরো process-কে বুঝে এবং maintain করে security monitoring-এর জন্য logs-কে usable করা।

এই article-এ আমরা **SIEM Engineer-এর Log Management** area-টি শুরু থেকে advanced level পর্যন্ত বুঝাব।

1. What is a Log?

একটি system, application, network device বা security product-এ কোনো activity ঘটলে সেটি সাধারণত একটি record হিসেবে তৈরি হয়। এই record-কে আমরা **log** বলি।

উদাহরণ:

```
Sep 16 14:20:15 server01 sshd[1234]:  
Failed password for admin from 192.168.10.50
```

এই একটি log থেকে আমরা জানতে পারি:

==> Time: 14:20:15 ==> Host: server01 ==> Service: sshd ==> Username: admin ==> Source IP: 192.168.10.50 ==> Event: Failed password

একজন SIEM Engineer-এর কাজ হলো এই ধরনের raw information-কে security investigation-এর জন্য useful data-তে পরিণত করা।

2. Why is Log Management Important?

SIEM-এর মূল শক্তি হলো **data**।

যদি গুরুত্বপূর্ণ security logs SIEM-এ না আসে, তাহলে detection এবং investigation দুটোই দুর্বল হয়ে যায়।

ধরুন কোনো organization-এর firewall থেকে logs আসছে না।

তাহলে SOC analyst হয়তো জানতে পারবে না:

==> কোন IP internal network-এ connection করার চেষ্টা করেছে ==> কোন connection blocked হয়েছে ==>

কোন port scan হয়েছে ==> কোন suspicious outbound connection হয়েছে ==> কোন VPN activity হয়েছে

তাই SIEM Engineer-এর প্রথম লক্ষ্য:

Make sure the right logs are collected, delivered, parsed, stored, and searchable.

3. Common SIEM Log Sources

একটি enterprise environment-এ অনেক ধরনের log source থাকতে পারে।

3.1 Firewall Logs

Firewall হলো SIEM-এর অন্যতম গুরুত্বপূর্ণ log source।

Firewall logs থেকে পাওয়া যেতে পারে:

==> Source IP ==> Destination IP ==> Source Port ==> Destination Port ==> Protocol ==> Action —
Allow/Block ==> Policy ID ==> Username ==> Application ==> URL ==> NAT information ==> VPN
activity ==> Threat/IPS events

Example:

```
src_ip=10.10.10.20  
dst_ip=8.8.8.8  
dst_port=443  
protocol=TCP  
action=allow
```

এগুলো ব্যবহার করে network activity investigate করা যায়।

4. Windows Logs

Windows environment-এ অনেক security-relevant events generate হয়।

বিশেষ করে:

==> Windows Security Event Log ==> System Logs ==> Application Logs ==> PowerShell Logs ==> Windows Defender Logs ==> Sysmon Logs

Important security events-এর মধ্যে থাকতে পারে:

==> Successful Login ==> Failed Login ==> Account Creation ==> Account Modification ==> Privilege Changes ==> Process Creation ==> PowerShell Activity ==> Service Creation ==> Security Policy Changes

একজন SIEM Engineer-এর Windows Event ID এবং event structure সম্পর্কে ভালো ধারণা থাকা প্রয়োজন।

Key Windows Security Event IDs for SIEM:

```
4624 = Successful logon
4625 = Failed logon
4672 = Special privileges assigned (admin logon)
4720 = User account created
4732 = Member added to security-enabled group
4688 = New process created
4689 = Process exited
7045 = Service installed
4104 = PowerShell script block logging
```

Always validate field names per platform (e.g. Splunk EventCode, Sentinel EventID, Elastic winlog.event_id).

5. Linux Logs

Linux system-এ বিভিন্ন security এবং operational logs পাওয়া যায়।

Common locations:

```
/var/log/auth.log  
/var/log/syslog  
/var/log/messages  
/var/log/secure
```

Security investigation-এ গুরুত্বপূর্ণ হতে পারে:

==> SSH Login ==> Failed SSH Login ==> sudo activity ==> User creation ==> Authentication events
==> Service activity ==> System events

Example:

```
Failed password for root from 10.10.10.50
```

এই ধরনের events ব্যবহার করে SSH brute-force detection তৈরি করা যায়।

6. Active Directory Logs

Enterprise Windows environment-এ **Active Directory** অত্যন্ত গুরুত্বপূর্ণ log source।

AD logs থেকে security team দেখতে পারে:

==> User authentication ==> Failed authentication ==> Account lockout ==> Group membership changes ==> Privilege changes ==> Kerberos activity ==> Domain Controller events ==> Account creation/deletion

উদাহরণ:

একজন সাধারণ user হঠাৎ privileged group-এ যোগ হলো।

এই activity SIEM-এর মাধ্যমে detect করা সম্ভব, যদি প্রয়োজনীয় logs properly collected হয়।

7. VPN Logs

Remote users এবং employees VPN ব্যবহার করলে VPN logs অত্যন্ত গুরুত্বপূর্ণ।

VPN logs থেকে পাওয়া যেতে পারে:

==> Username ==> Source IP ==> Login time ==> Logout time ==> Authentication status ==>
Assigned IP ==> VPN gateway ==> Connection duration

Possible detections:

==> Multiple failed VPN login ==> Login from unusual location ==> Suspicious account usage ==>
Login outside normal working hours ==> Impossible travel-type patterns

8. Proxy Logs

Proxy বা Secure Web Gateway থেকে web browsing activity সম্পর্কিত logs পাওয়া যায়।

Common information:

==> Source IP ==> Username ==> Requested URL ==> Domain ==> HTTP Method ==> Status Code
==> User-Agent ==> Category ==> Action

এগুলো ব্যবহার করে malicious domain access, suspicious downloads এবং unusual web activity investigate করা যায়।

9. IDS/IPS Logs

IDS/IPS network traffic monitor করে suspicious activity detect করতে পারে।

Logs may contain:

==> Signature ==> Source IP ==> Destination IP ==> Port ==> Protocol ==> Attack Type ==> Severity
==> Action

Example:

```
Possible SQL Injection detected  
Source: 10.10.10.50  
Destination: 10.10.20.10
```

এই alert-এর সাথে অন্যান্য SIEM logs correlate করে incident-এর scope বোঝা যায়।

10. EDR / Endpoint Logs

Modern SOC environment-এ endpoint telemetry অত্যন্ত গুরুত্বপূর্ণ।

EDR logs may include:

==> Process creation ==> Parent-child process relationship ==> Command line ==> File creation
==> File modification ==> Network connection ==> Registry activity ==> Malware detection ==>
User activity

Example:

```
powershell.exe  
  ↓  
cmd.exe  
  ↓  
suspicious.exe
```

এই ধরনের process relationship detection engineering-এর জন্য গুরুত্বপূর্ণ।

11. Cloud Logs

Modern organizations cloud services ব্যবহার করে।

Common cloud log sources:

==> AWS CloudTrail ==> AWS VPC Flow Logs ==> Azure Activity Logs ==> Microsoft Entra ID logs
==> Google Cloud Audit Logs ==> Cloud firewall logs ==> Cloud application logs

Cloud logs থেকে detect করা যেতে পারে:

==> Unusual login ==> Privilege escalation ==> API activity ==> Resource creation ==>
Configuration changes ==> Suspicious access

12. Application Logs

Security শুধু infrastructure level-এ সীমাবদ্ধ নয়।

Application logs-ও গুরুত্বপূর্ণ।

Example:

```
User login failed
User login successful
Password reset requested
API request received
Access denied
Database query failed
```

Application logs ব্যবহার করে detect করা যেতে পারে:

==> Brute-force attempts ==> Account abuse ==> Privilege abuse ==> Suspicious API activity ==>
Authentication attacks ==> Application errors

13. Database Logs

Database logs থেকেও গুরুত্বপূর্ণ security information পাওয়া যায়।

Examples:

==> Database login ==> Failed authentication ==> Query activity ==> Permission changes ==> User creation ==> Data access ==> Configuration changes

Sensitive database activity monitoring-এর ক্ষেত্রে এগুলো গুরুত্বপূর্ণ।

14. Network Device Logs

Network infrastructure থেকেও logs collect করা হয়।

Examples:

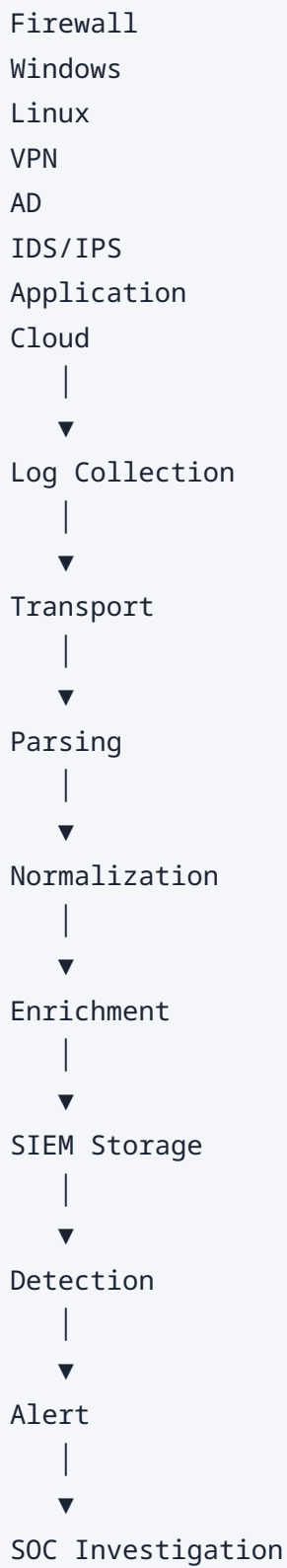
==> Router ==> Switch ==> Wireless Controller ==> Load Balancer ==> DNS Server ==> DHCP Server

These logs can provide information about:

==> Network connections ==> Configuration changes ==> Authentication ==> Device health ==> Network errors ==> Suspicious activity

15. How Does a Log Reach the SIEM?

একটি সাধারণ architecture হতে পারে:



এই পুরো pipeline সম্পর্কে SIEM Engineer-এর ধারণা থাকা দরকার।

16. Log Collection

Log collection হলো source system থেকে logs সংগ্রহ করার process।

Common approaches:

==> Syslog ==> Agent-based collection ==> Forwarder ==> API ==> Windows Event Forwarding ==>
File monitoring ==> Cloud connector ==> Message queue

কোন method ব্যবহার হবে তা depend করে:

==> Log source ==> Vendor ==> Network architecture ==> Security requirement ==> SIEM platform

17. Syslog

Syslog network এবং security devices থেকে logs পাঠানোর জন্য বহুল ব্যবহৃত একটি protocol।

Commonly used ports:

==> UDP 514 ==> TCP 514 ==> TCP 6514 — commonly used for Syslog over TLS

Example architecture:

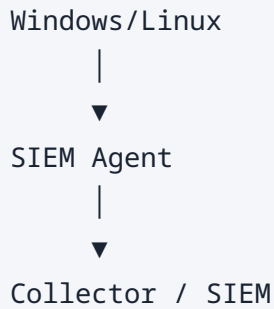


একজন SIEM Engineer-এর Syslog facility, severity, transport এবং TLS-based secure transport সম্পর্কে ধারণা থাকা উচিত।

18. Agent-Based Collection

কিছু ক্ষেত্রে endpoint/server-এ agent install করা হয়।

Architecture:



Agent-এর সুবিধা হতে পারে:

==> Reliable collection ==> Local file monitoring ==> Structured event collection ==> Secure transport ==> Filtering ==> Buffering

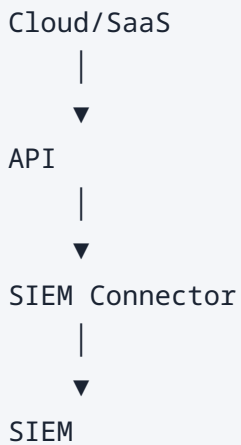
Wazuh environment-এ Wazuh Agent একটি common example।

19. API-Based Log Collection

সব system Syslog support করে না।

কিছু cloud service এবং SaaS platform API-এর মাধ্যমে data provide করে।

Architecture:



SIEM Engineer-এর API authentication, pagination, rate limits, tokens, error handling এবং data parsing সম্পর্কে ধারণা থাকা useful।

20. Log Transport

Log source থেকে SIEM পর্যন্ত data কীভাবে যাচ্ছে সেটিও গুরুত্বপূর্ণ।

Potential problems:

==> Network failure ==> Firewall blocking ==> Wrong port ==> DNS issue ==> TLS certificate problem ==> Authentication failure ==> Collector service down ==> Queue overflow

তাই শুধু SIEM dashboard দেখা যথেষ্ট নয়।

পুরো data path বুঝতে হবে।

21. Log Parsing

Raw log অনেক সময় structured নয়।

Example:

```
Failed password for admin from 192.168.1.20
```

Parsing-এর পরে:

```
event.action = authentication_failure  
user.name = admin  
source.ip = 192.168.1.20
```

Parsing-এর উদ্দেশ্য হলো raw data থেকে meaningful fields তৈরি করা।

22. Log Normalization

Different vendors একই information-এর জন্য different field names ব্যবহার করতে পারে।

Example:

```
src_ip  
source_ip  
sourceAddress  
client_ip
```

Normalization করলে এগুলো common field-এ map করা যায়:

```
source.ip
```

এতে বিভিন্ন log source-এর উপর একই ধরনের detection তৈরি করা সহজ হয়।

23. Timestamp Normalization

Security investigation-এ time অত্যন্ত গুরুত্বপূর্ণ।

একটি device যদি Bangladesh time ব্যবহার করে এবং অন্য device UTC ব্যবহার করে, তাহলে timeline analysis-এ সমস্যা হতে পারে।

তাই SIEM environment-এ:

==> Timezone ==> Timestamp format ==> NTP synchronization ==> Event time vs ingestion time

এসব properly manage করা প্রয়োজন।

24. Log Enrichment

Raw logs-এর সাথে additional context যোগ করাকে **Log Enrichment** বলা যায়।

Example:

```
Source IP: 185.x.x.x
```

এর সাথে যোগ হতে পারে:

==> GeoIP ==> ASN ==> Threat Intelligence reputation ==> Asset information ==> User information ==> Domain information

তখন analyst আরও useful context পায়।

25. Data Quality

SIEM-এ data আসছে মানেই কাজ শেষ নয়।

SIEM Engineer-কে দেখতে হবে:

==> সব expected sources থেকে logs আসছে কি না ==> Events drop হচ্ছে কি না ==> Parsing ঠিক হচ্ছে কি না
==> Fields populated হচ্ছে কি না ==> Timestamp ঠিক আছে কি না ==> Duplicate logs হচ্ছে কি না ==> Data delay হচ্ছে কি না

এটাকে **Log/Data Quality Management** হিসেবে দেখা যায়।

26. Log Loss

High-volume environment-এ log loss একটি গুরুত্বপূর্ণ concern।

Possible causes:

==> Network congestion ==> Collector overload ==> Queue overflow ==> Disk full ==> Rate limiting
==> Incorrect configuration ==> SIEM ingestion limit

একজন SIEM Engineer-এর লক্ষ্য হলো critical security data যেন silently lost না হয় তা নিশ্চিত করা।

27. Duplicate Logs

একই event কখনো multiple paths দিয়ে SIEM-এ আসতে পারে।

Example:

```
Firewall
|
├── Syslog → Collector
|
└── API → SIEM
```

এর ফলে duplicate events তৈরি হতে পারে।

Duplicate data:

==> Storage বাড়ায় ==> Search result distort করতে পারে ==> Alert volume বাড়াতে পারে ==> SOC analyst-এর workload বাড়াতে পারে

তাই deduplication এবং proper ingestion design গুরুত্বপূর্ণ।

28. Log Retention

সব logs একই সময়ের জন্য রাখা হয় না।

Retention policy নির্ভর করতে পারে:

==> Security requirements ==> Compliance ==> Legal requirements ==> Business requirements

==> Storage capacity ==> Investigation needs

একটি সাধারণ storage model:

```
graph TD; Hot --> Warm; Warm --> Cold; Cold --> Archive;
```

Hot
↓
Warm
↓
Cold
↓
Archive

Recent data সাধারণত দ্রুত search-এর জন্য রাখা হয় এবং পুরোনো data কম-cost storage-এ archive করা যেতে পারে।

29. Log Indexing & Search

Large-scale SIEM environment-এ data দ্রুত search করার জন্য indexing গুরুত্বপূর্ণ।

একজন SIEM Engineer-এর বুঝতে হবে:

==> Data organization ==> Index/storage structure ==> Search performance ==> Time-based searching ==> Field-based searching ==> Data lifecycle

Poor indexing বা inefficient queries investigation slow করে দিতে পারে।

30. Log Volume & EPS

SIEM infrastructure planning-এর সময় **EPS (Events Per Second)** গুরুত্বপূর্ণ metric।

ধরুন:

$$\text{EPS} = 1,000$$

$$\text{Events/day} = \text{EPS} \times 86400 = 86.4\text{M}$$

$$\text{GB/day} = \text{Events/day} \times \text{avg event size (e.g. 86.4M} \times 1\text{KB} \sim 86.4 \text{ GB)}$$

$$\text{Retention storage} = \text{GB/day} \times \text{retention days}$$

See #10 SIEM Infrastructure and #12 Capacity Planning for full sizing with index, replication and buffer overhead.

মানে প্রতি second-এ প্রায় 1,000 events আসছে।

একটি বড় environment-এ EPS অনেক বেশি হতে পারে।

SIEM Engineer-কে consider করতে হয়:

==> Average EPS ==> Peak EPS ==> Data size ==> Daily ingestion ==> Storage requirement ==> Search workload

Capacity planning-এর জন্য শুধু average volume দেখলে হবে না; peak traffic-ও consider করতে হবে।

31. Log Monitoring

SIEM Engineer নিজেও SIEM-এর health monitor করেন।

Monitor করা যেতে পারে:

==> Ingestion rate ==> Event count ==> EPS ==> Queue size ==> Collector health ==> Parser errors
==> Dropped events ==> Disk usage ==> CPU ==> Memory ==> Network ==> Data latency

32. Troubleshooting Missing Logs

ধরুন:

Firewall logs হঠাৎ **SIEM**-এ আসছে না।

একজন SIEM Engineer কী করবেন?

Step 1 — Check the Source

Firewall কি logs generate করছে?

Step 2 — Check Configuration

Syslog/API configuration ঠিক আছে?

Step 3 — Check Network

Firewall থেকে collector পর্যন্ত connectivity আছে?

Step 4 — Check Port

Required port open আছে?

Step 5 — Check Collector

Collector service running আছে?

Step 6 — Check Transport

Packets/events আসছে কি?

Step 7 — Check Parsing

Events আসছে কিন্তু parse হচ্ছে না?

Step 8 — Check SIEM

Data indexed/searchable হচ্ছে?

Step 9 — Check Storage

Disk বা ingestion capacity full হয়েছে কি?

Step 10 — Validate

শেষে নতুন events search করে নিশ্চিত করতে হবে যে log flow আবার ঠিক হয়েছে।

33. Log Management Security

Logs নিজেরাও sensitive security data হতে পারে।

তাই consider করতে হবে:

==> Access Control ==> Encryption in Transit ==> Encryption at Rest ==> Integrity ==> Least Privilege ==> Audit Logging ==> Secure Retention ==> Backup

কারণ attacker যদি logs modify বা delete করতে পারে, তাহলে investigation এবং forensic evidence ক্ষতিগ্রস্ত হতে পারে।

34. Log Management and Detection Engineering

Log Management এবং Detection Engineering একে অপরের সাথে directly connected।

Example:

Detection requirement:

Detect SSH Brute Force

প্রথমে প্রয়োজন:

Linux SSH Authentication Logs

তারপর:

```
Collection
  ↓
Parsing
  ↓
Normalization
  ↓
Detection Rule
  ↓
Alert
```

যদি SSH logs SIEM-এ না আসে, তাহলে detection rule যত ভালোই হোক, সেটি কাজ করবে না।

35. Log Management and Threat Hunting

Threat hunting-এর জন্যও quality logs দরকার।

একজন analyst যদি hunt করতে চায়:

“গত 24 ঘণ্টায় কোন **user unusual login** করেছে?”

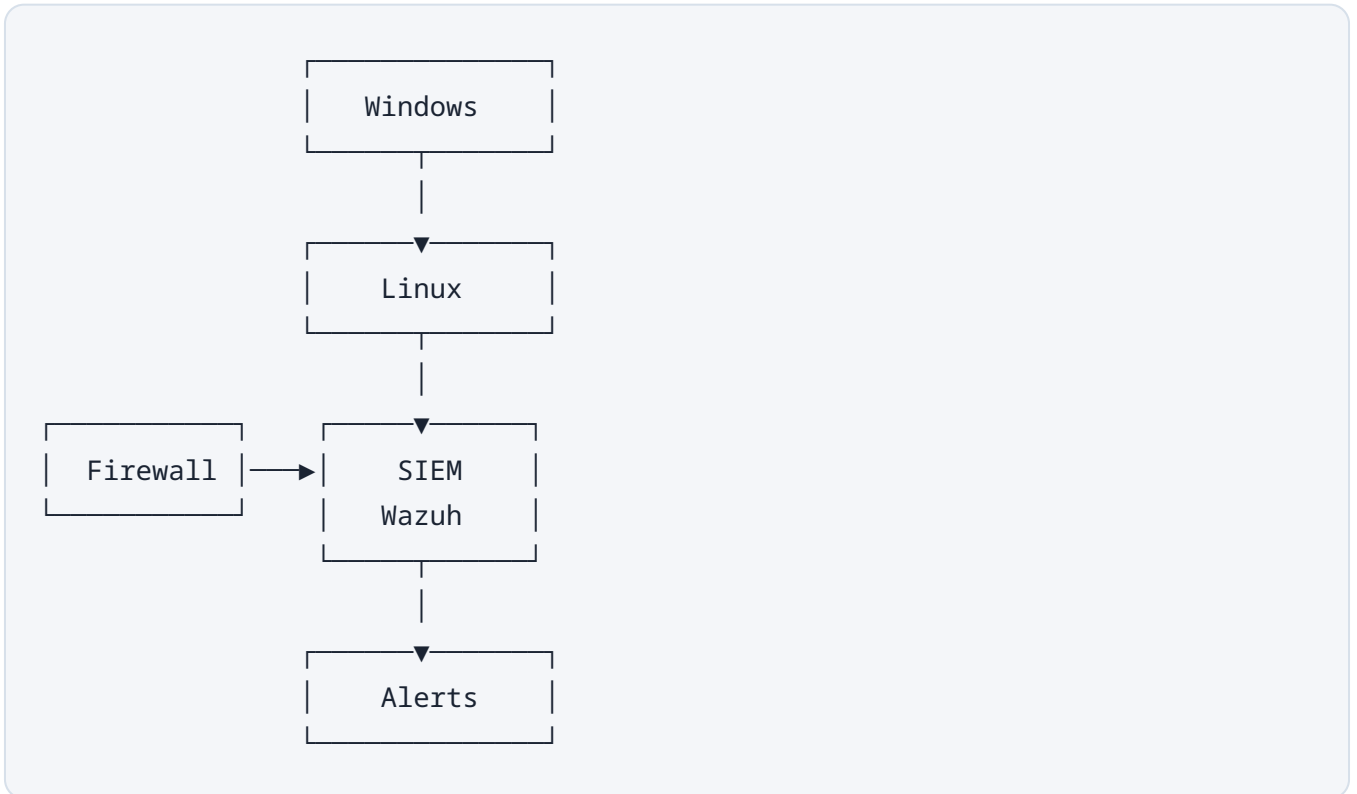
তাহলে প্রয়োজন হতে পারে:

==> Authentication logs ==> VPN logs ==> AD logs ==> Endpoint logs ==> GeoIP information

তাই ভালো threat hunting শুরু হয় ভালো log visibility থেকে।

36. Practical SIEM Log Management Lab

একজন aspiring SIEM Engineer নিজের lab-এ এমন environment তৈরি করতে পারে:



Lab-এ practice করতে পারেন:

==> Linux SSH failed login generate করা ==> Windows login events generate করা ==> Firewall logs forward করা ==> Wazuh Agent configure করা ==> Syslog configure করা ==> Logs search করা ==> Custom parser তৈরি করা ==> Detection rule তৈরি করা ==> Alert generate করা ==> False Positive tune করা

37. SIEM Engineer Log Management Checklist

একজন SIEM Engineer-এর জন্য এই checklist useful:

Log Sources

==> Firewall connected? ==> Windows connected? ==> Linux connected? ==> AD connected? ==> VPN connected? ==> Proxy connected? ==> IDS/IPS connected? ==> EDR connected? ==> Cloud connected? ==> Application connected?

Collection

==> Correct protocol? ==> Correct port? ==> Agent healthy? ==> API authentication working? ==> Network connectivity available?

Processing

==> Parsing working? ==> Fields extracted? ==> Normalization correct? ==> Timestamp correct? ==> Enrichment working?

Storage

==> Data indexed? ==> Retention configured? ==> Storage available? ==> Backup/archive working?

Monitoring

==> EPS normal? ==> Data delay normal? ==> Dropped events? ==> Parser errors? ==> Collector health?

Security

==> Access control configured? ==> Encryption enabled? ==> Audit logging enabled? ==> Sensitive data protected?

38. Real-World SIEM Engineer Mindset

একজন SIEM Engineer শুধু এই প্রশ্ন করবেন না:

“Logs are coming?”

বরং প্রশ্ন করবেন:

“Are the right logs coming?”

“Are they complete?”

“Are they parsed correctly?”

“Are timestamps accurate?”

“Are important fields available?”

“Are we losing events?”

“Can SOC analysts search them quickly?”

“Can our detection rules use them?”

“Can we investigate an incident using these logs?”

এটাই একজন SIEM Engineer-এর practical mindset।

39. The Complete Log Management Flow

শেষে পুরো process-টাকে মনে রাখুন:

```
graph TD; A[Log Source] --> B[Collection]; B --> C[Transport]; C --> D[Ingestion]; D --> E[Parsing]; E --> F[Normalization]; F --> G[Enrichment]; G --> H[Filtering]; H --> I[Indexing]; I --> J[Storage]; J --> K[Retention]; K --> L[Search]; L --> M[Detection]; M --> N[Alert]; N --> O[Investigation];
```

Log Source
↓
Collection
↓
Transport
↓
Ingestion
↓
Parsing
↓
Normalization
↓
Enrichment
↓
Filtering
↓
Indexing
↓
Storage
↓
Retention
↓
Search
↓
Detection
↓
Alert
↓
Investigation

প্রতিটি step-এর reliability গুরুত্বপূর্ণ।

একটি জায়গায় সমস্যা হলে পুরো security monitoring pipeline প্রভাবিত হতে পারে।



Why Does Log Management Matter?

একটি SIEM-এর detection quality অনেকটাই নির্ভর করে **data quality** এবং **visibility**-এর উপর।

যদি প্রয়োজনীয় logs না থাকে:

No Logs → No Visibility → Weak Detection → Difficult Investigation

আর যদি logs properly collected, parsed, normalized এবং enriched হয়:

Good Logs → Better Visibility → Better Detection → Faster Investigation

তাই **Log Management** শুধু SIEM-এর একটি ছোট অংশ নয়; এটি পুরো **SIEM ecosystem**-এর **foundation**।

Interview Questions for SIEM Engineer

Q1. What is Log Management?

Answer: Log Management is the process of collecting, transporting, parsing, normalizing, storing, monitoring, and retaining logs from different systems so they can be used for security monitoring, detection, investigation, and compliance.

Q2. What are common SIEM log sources?

Answer: Common sources include firewalls, Windows/Linux servers, Active Directory, VPN, Proxy, IDS/IPS, EDR, cloud platforms, network devices, databases, and applications.

Q3. What is log parsing?

Answer: Log parsing is the process of extracting meaningful fields from raw log data, such as source IP, username, timestamp, event type, and destination IP.

Q4. What is log normalization?

Answer: Normalization converts different log formats and field names into a consistent structure so that common searches and detection rules can work across multiple log sources.

Q5. What would you check if logs suddenly stop coming into the SIEM?

Answer:

Source → **Configuration** → **Network** → **Port** → **Collector** → **Transport** → **Parser** → **Index** → **Storage** → **Search**

I would check each stage to identify where the log flow has stopped.

Q6. What is EPS?

Answer: EPS means **Events Per Second**. It represents the number of events being generated or ingested per second and is important for SIEM capacity planning.

Final Takeaway

A SIEM Engineer should not think of logs as simple text files.

Think of logs as **security evidence**.

Your responsibility is to make sure that evidence is:

Collected → Reliable → Structured → Searchable → Protected → Available for Detection and Investigation

Once you understand Log Management properly, the next major area becomes much easier:

SIEM Engineer #02 — SIEM Platforms

In the next article, we will explore:

Splunk → Microsoft Sentinel → QRadar → Elastic Security → OpenSearch → Wazuh

and understand how these platforms work, where they fit, and what a SIEM Engineer needs to know about each one.



SIEM Engineer Learning Series

#01 — Log Management ← You are here

#02 — SIEM Platforms → Next

#03 — Log Integration

#04 — Detection Engineering

#05 — Query & Investigation

#06 — SOC Operations

#07 — Threat Intelligence

#08 — Automation & SOAR

#09 — Dashboard & Reporting

#10 — SIEM Infrastructure

#11 — SIEM Architecture Design

#12 — SIEM Capacity Planning

#13 — SIEM Performance Tuning

#14 — Advanced Detection Engineering

#15 — Threat Hunting with SIEM

#16 — SIEM Data Quality & Troubleshooting

#17 — Hands-On SIEM: Multi-Platform Practical Guide

Navigation

SIEM Engineer Index

Next Article: #02 — SIEM Platforms →

SIEM Engineer #02 — SIEM Platforms

Introduction

In the previous article, we discussed **Log Management**—how logs are collected, transported, parsed, normalized, enriched, stored, and monitored.

But where do all these logs actually go?

This is where **SIEM Platforms** come in.

A SIEM platform provides a centralized place where security data can be:

Collected → Stored → Searched → Correlated → Detected → Investigated → Reported

For a SIEM Engineer, knowing only the name of a SIEM product is not enough.

You need to understand **how the platform works, how data enters it, how searches are performed, how detections are created, and how the platform is maintained.**

In this article, we will explore the major SIEM and security monitoring platforms a SIEM Engineer should know.

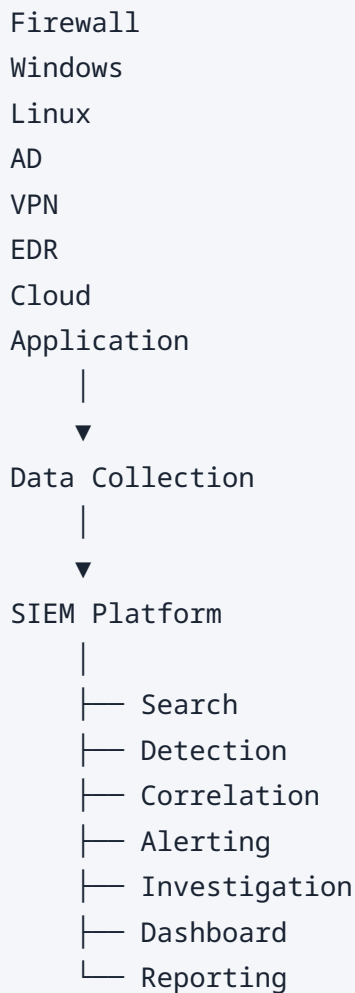
1. What is a SIEM Platform?

SIEM stands for:

Security Information and Event Management

A SIEM platform collects security-related events from different sources and provides capabilities for centralized monitoring, searching, correlation, detection, investigation, alerting, and reporting.

A simplified architecture looks like this:



2. What Does a SIEM Engineer Do With a SIEM Platform?

A SIEM Engineer may be responsible for:

==> Deploying the SIEM ==> Configuring data sources ==> Onboarding logs ==> Creating indexes/
data streams ==> Building parsers ==> Creating detection rules ==> Writing queries ==> Creating
dashboards ==> Managing alerts ==> Tuning false positives ==> Monitoring ingestion ==>
Managing storage ==> Troubleshooting performance ==> Integrating threat intelligence ==>
Automating security workflows ==> Maintaining and upgrading the platform

So, SIEM engineering is much broader than simply watching a dashboard.

3. Major SIEM Platforms

Some commonly encountered platforms include:

==> Splunk ==> Microsoft Sentinel ==> IBM QRadar ==> Elastic Security ==> OpenSearch ==> Wazuh

Each platform has its own architecture, terminology, query language, integrations, and operational model.

The concepts are often similar even though the implementation differs.

4. Splunk

Splunk is a widely used platform for collecting, searching, analyzing, and monitoring machine-generated data.

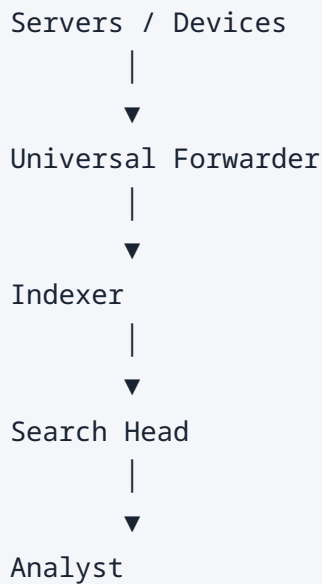
In security environments, **Splunk Enterprise Security** provides security monitoring and analytics capabilities.

For a SIEM Engineer, important Splunk concepts include:

==> Index ==> Sourcetype ==> Source ==> Host ==> Forwarder ==> Indexer ==> Search Head ==> SPL ==> Data Model ==> Correlation Search ==> Risk-based Alerting ==> Dashboard ==> Alert

5. Splunk Architecture

A simplified Splunk environment can look like:



In larger environments, Splunk architecture can become distributed with multiple indexers and search heads.

==> **Search Head**

The Search Head handles user searches and search-related workloads.

==> **Indexer**

The Indexer stores and indexes incoming data.

==> **Forwarder**

A Forwarder can collect and forward data from systems to Splunk.

6. SPL — Splunk Search Processing Language

One of the most important skills for a Splunk-focused SIEM Engineer is **SPL**.

Example:

```
index=windows EventCode=4625
```

This searches for Windows failed authentication events based on the specified fields.

You can then perform operations such as:

==> Filtering ==> Counting ==> Grouping ==> Statistical analysis ==> Time-based analysis ==> Correlation

Example:

```
index=windows EventCode=4625  
| stats count by src_ip  
| sort - count
```

This can help identify source IPs generating large numbers of failed authentication events.

7. Microsoft Sentinel

Microsoft Sentinel is Microsoft's cloud-native SIEM and security analytics platform.

It integrates closely with the Microsoft security ecosystem and Azure services.

Important Sentinel concepts include:

==> Data Connectors ==> Log Analytics Workspace ==> KQL ==> Analytics Rules ==> Incidents ==> Workbooks ==> Hunting Queries ==> Automation Rules ==> Playbooks

8. KQL — Kusto Query Language

For Microsoft Sentinel, **KQL** is an essential skill.

Example:

```
SecurityEvent
| where EventID == 4625
| summarize FailedLogins=count() by Account
| order by FailedLogins desc
```

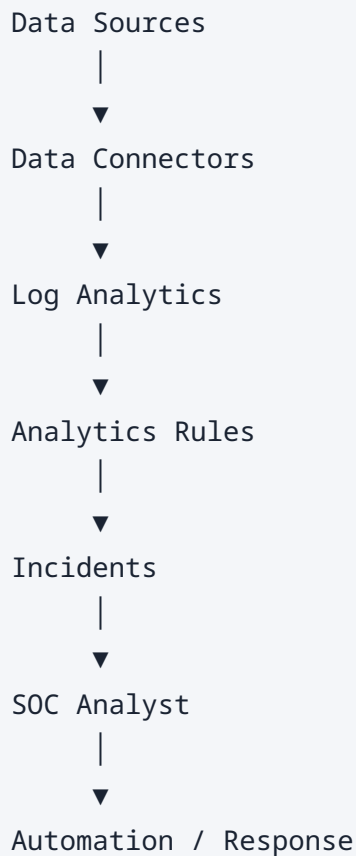
This can help identify accounts with a high number of failed authentication events.

A SIEM Engineer working with Sentinel should become comfortable with:

==> where ==> summarize ==> project ==> extend ==> join ==> count() ==> Time filtering ==>
Aggregation ==> Correlation

9. Microsoft Sentinel Data Flow

A simplified architecture:



Sentinel supports connectors for bringing data from Microsoft and non-Microsoft sources into the platform.

10. IBM QRadar

IBM QRadar is another enterprise SIEM platform.

QRadar is known for centralized security event monitoring, correlation, offense management, and network/security data analysis.

Important concepts include:

==> Events ==> Flows ==> Log Sources ==> Offenses ==> Rules ==> CRE ==> DSM ==> Ariel ==>

Reference Sets

11. QRadar Events

A QRadar environment can receive events from different sources:

==> Firewall ==> Windows ==> Linux ==> IDS/IPS ==> VPN ==> Proxy ==> Applications

QRadar can correlate events and generate an **Offense** when activity meets configured conditions.

For example:

```
graph TD; A[Multiple Failed Logins] -- "+" --> B[Successful Login]; B -- "+" --> C[Unusual Source]; C -- "↓" --> D[Offense];
```

Multiple Failed Logins
+
Successful Login
+
Unusual Source
↓
Offense

The SOC analyst can then investigate the offense.

12. QRadar DSM

DSM stands for **Device Support Module**.

It helps QRadar understand and process events from different device types.

A SIEM Engineer working with QRadar should understand:

==> Log Source configuration ==> DSM ==> Event parsing ==> Rules ==> Offenses ==> Reference Sets ==> Event properties

13. Elastic Security

Elastic Security provides security monitoring, detection, investigation, and threat hunting capabilities on top of the Elastic Stack.

Important components include:

==> Elasticsearch ==> Kibana ==> Elastic Agent ==> Beats ==> Data Streams ==> Detection Rules
==> Cases ==> Timeline ==> Threat Intelligence

14. Elasticsearch

Elasticsearch is a distributed search and analytics engine.

In an Elastic Security environment, security events can be stored and searched through Elasticsearch.

A simplified flow:



A SIEM Engineer should understand:

==> Indexing ==> Data Streams ==> Mappings ==> Fields ==> Querying ==> Shards ==> Replicas
==> Cluster health

15. OpenSearch

OpenSearch is an open-source search and analytics platform that is also used for security monitoring and log analytics.

An OpenSearch-based security environment can provide:

==> Log collection ==> Search ==> Dashboards ==> Alerting ==> Security analytics ==> Detection capabilities

For an engineer, useful concepts include:

==> OpenSearch ==> OpenSearch Dashboards ==> Index ==> Index Pattern ==> Data Stream ==> Query ==> Alert ==> Security Analytics

16. Wazuh

Wazuh is an open-source security platform commonly used for security monitoring, endpoint protection, threat detection, vulnerability detection, compliance monitoring, and log analysis.

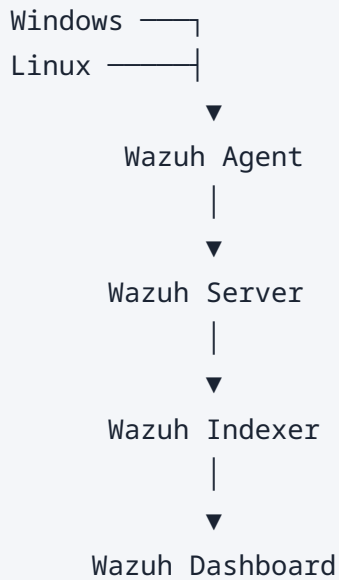
For someone starting SIEM engineering, Wazuh can be a useful hands-on platform because it allows you to build a practical lab environment.

Important Wazuh components include:

==> Wazuh Agent ==> Wazuh Server ==> Wazuh Indexer ==> Wazuh Dashboard

17. Wazuh Architecture

A simplified Wazuh architecture:



Wazuh can also receive logs from network devices and other systems through supported integration methods.

18. SIEM Platforms Have Similar Concepts

Although the products are different, many concepts are similar.

Concept	Splunk	Sentinel	QRadar	Elastic	Wazuh
Log/Data	Events	Logs	Events	Events	Alerts/ Events
Search	SPL	KQL	AQL	Query DSL/ KQL	Search
Detection	Correlation Search	Analytics Rule	Rule	Detection Rule	Rules
Investigation	Search	Investigation	Offense	Timeline	Alert/Events
Dashboard	Dashboard	Workbook	Dashboard	Dashboard	Dashboard
Automation	SOAR/ Integrations	Playbooks	Integrations	Connectors	Active Response/ Integrations

The exact terminology and capabilities vary by product and version, but the underlying security workflow is similar.

19. Query Language Is a Core Skill

A SIEM Engineer should not depend entirely on the graphical interface.

You should know how to query the underlying data.

Splunk

SPL

```
index=auth  
| stats count by user
```

Microsoft Sentinel

KQL

```
SigninLogs  
| summarize count() by UserPrincipalName
```

QRadar

AQL

Used to search and analyze QRadar event/flow data.

Elastic/OpenSearch

Query languages and query APIs are used to search indexed security data.

The important skill is not memorizing syntax.

The important skill is understanding:

What data do I need? → Where is it stored? → How do I filter it? → How do I correlate it?

20. Detection Rules

SIEM platforms provide different mechanisms for detection.

Example:

Possible Brute Force

```
graph TD; A[Failed Login] --> B[Failed Login]; B --> C[Failed Login]; C --> D[Failed Login]; D --> E[Threshold Reached]; E --> F[Alert];
```

Failed Login
↓
Failed Login
↓
Failed Login
↓
Failed Login
↓
Threshold Reached
↓
Alert

The implementation differs between platforms, but the security logic remains similar.

21. Alert vs Incident vs Offense

Different platforms use different terminology.

Generally:

Event

A single piece of activity.

↓

Detection

A rule identifies suspicious activity.

↓

Alert

The platform creates a security notification.

↓

Incident/Offense

Related suspicious activity may be grouped for investigation.

↓

Investigation

SOC analyst reviews evidence.

Understanding this distinction is important for SIEM Engineer interviews.

22. Dashboard

SIEM platforms provide dashboards to visualize security data.

Common dashboard components:

==> Alert count ==> Severity ==> Top source IPs ==> Top targeted assets ==> Failed authentication
==> Malware events ==> Network activity ==> Incident trends ==> Detection trends

But remember:

A dashboard is not a detection strategy.

A beautiful dashboard is useful only when the underlying data and detection logic are reliable.

23. SIEM Platform Health

A SIEM Engineer also needs to monitor the platform itself.

Important metrics include:

==> CPU ==> Memory ==> Disk ==> Network ==> EPS ==> Ingestion latency ==> Queue size ==>
Search performance ==> Storage utilization ==> Service health ==> Cluster health

The SIEM should be treated as a production security system.

24. Choosing a SIEM Platform

There is no single platform that fits every organization.

The choice may depend on:

==> Organization size ==> Existing infrastructure ==> Cloud strategy ==> Budget ==> Compliance requirements ==> Data volume ==> Security requirements ==> Existing Microsoft environment ==> Required integrations ==> Available engineering skills

For example, an organization heavily invested in Microsoft services may evaluate Microsoft Sentinel, while another organization may already have a mature Splunk environment.

The technical requirements and operational context should drive the decision.

25. What Should a SIEM Engineer Learn?

You do not need to master every platform at the same depth.

A practical approach is:

Foundation

==> Networking ==> Linux ==> Windows ==> Authentication ==> Logs ==> Security fundamentals

Choose One Primary SIEM

Go deep into one:

==> Splunk **or** ==> Microsoft Sentinel **or** ==> QRadar **or** ==> Elastic **or** ==> Wazuh

Then Learn Others

After understanding one platform deeply, learn how the same concepts are implemented elsewhere.

For example:

Splunk SPL

↓

KQL

↓

QRadar AQL

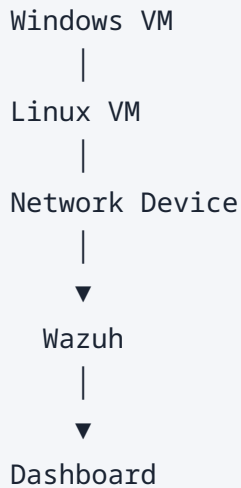
↓

Elastic/OpenSearch Query

This makes it easier to move between SIEM technologies.

26. Practical Lab

You can create a small SIEM lab using:



Then practice:

==> Generate failed SSH logins ==> Generate Windows authentication events ==> Collect logs ==> Search events ==> Create detection rules ==> Generate alerts ==> Investigate alerts ==> Tune false positives ==> Build dashboard ==> Configure Active Response

After that, repeat similar exercises using another SIEM platform.

This is much more valuable than only watching product demonstrations.

SIEM Platform Checklist

- [] Can explain SIEM collection → detection → alert → investigation flow
 - [] Can describe Splunk roles: Forwarder → Indexer → Search Head
 - [] Can write basic SPL search with index, sourcetype, stats
 - [] Can describe Sentinel flow: Data Connector → Log Analytics → Analytics Rule → Incident
 - [] Can write basic KQL with where, summarize, project
 - [] Knows QRadar DSM, Offense, AQL basics and Elastic/OpenSearch index concepts
 - [] Knows Wazuh manager, agent, rule, Active Response basics
 - [] Can compare Event vs Alert vs Incident/Offense across platforms
-

27. SIEM Engineer Interview Questions

Q1. What is SIEM?

Answer:

SIEM stands for Security Information and Event Management. It provides centralized collection, analysis, correlation, detection, alerting, investigation, and reporting of security-related data.

Q2. What SIEM platforms have you worked with?

A good answer should be honest.

For example:

I have hands-on experience with Wazuh and have worked with Splunk-related concepts and security monitoring workflows. I understand log ingestion, parsing, detection rules, alert investigation, and SIEM operations, and I am continuously expanding my knowledge of other enterprise SIEM platforms.

Q3. What is the difference between an Event and an Alert?

Event is an individual activity recorded by a system.

Alert is generated when a detection mechanism identifies activity that requires attention.

Q4. What is a correlation rule?

A correlation rule combines multiple events or conditions to identify a potentially meaningful security event.

Q5. Why is query language important?

Because SIEM Engineers and SOC analysts need to search large amounts of security data, investigate incidents, create detections, and perform threat hunting efficiently.

28. The Most Important Mindset

Do not learn SIEM platforms by memorizing menus.

Learn them by understanding the security workflow:

Where does data come from?

↓

How is it collected?

↓

How is it stored?

↓

How do I search it?

↓

How do I detect threats?

↓

How do I investigate?

↓

How do I respond?

↓

How do I automate?

↓

How do I report?

Once you understand this workflow, learning a new SIEM becomes much easier.

Final Takeaway

A SIEM Engineer is not simply a person who operates a SIEM dashboard.

A strong SIEM Engineer understands the relationship between:

Log Sources → SIEM Platform → Query → Detection → Alert → Investigation → Response → Automation

The platform may change:

Splunk → Sentinel → QRadar → Elastic → OpenSearch → Wazuh

but the fundamental security concepts remain connected.

If you understand those concepts deeply, you can adapt to different SIEM technologies much more effectively.

Next Article

SIEM Engineer #03 — Log Integration

In the next article, we will go deeper into:

==> Syslog ==> CEF ==> API Integration ==> Agents ==> Forwarders ==> Cloud Connectors ==>
Custom Parsers ==> Data Onboarding ==> Log Pipeline Troubleshooting

Collecting logs is one thing. Successfully integrating them into a SIEM is another.



SIEM Engineer Learning Series

#01 — Log Management

#02 — SIEM Platforms ← You are here

#03 — Log Integration → Next

#04 — Detection Engineering

#05 — Query & Investigation

#06 — SOC Operations

#07 — Threat Intelligence

#08 — Automation & SOAR

#09 — Dashboard & Reporting

#10 — SIEM Infrastructure

#11 — SIEM Architecture Design

#12 — SIEM Capacity Planning

#13 — SIEM Performance Tuning

#14 — Advanced Detection Engineering

#15 — Threat Hunting with SIEM

#16 — SIEM Data Quality & Troubleshooting

#17 — Hands-On SIEM: Multi-Platform Practical Guide

Navigation

← **Previous Article: #01 — Log Management**

SIEM Engineer Index

Next Article: #03 — Log Integration →

SIEM Engineer #03 — Log Integration

Introduction

A SIEM platform is only as useful as the data it receives.

In the previous articles, we discussed:

#01 — Log Management #02 — SIEM Platforms

Now we will focus on one of the most practical responsibilities of a SIEM Engineer:

Log Integration

Log Integration means connecting different **servers, network devices, security products, applications, cloud platforms, and other data sources** to the SIEM so their security events can be collected, processed, and analyzed.

A simple way to think about it is:

Log Source → Connector/Agent → Transport → SIEM → Parsing → Normalization → Detection

1. What is Log Integration?

Suppose an organization has:

==> FortiGate Firewall ==> Windows Servers ==> Linux Servers ==> Active Directory ==> VPN ==> EDR ==> Web Application ==> AWS ==> Wazuh

The SIEM Engineer needs to connect these systems to the SIEM.

For example:

```
graph TD
    FortiGate -- Syslog --> LogCollector[Log Collector]
    LogCollector --> SIEM
```

Another system might use an API:

```
graph TD
    CloudPlatform[Cloud Platform] -- API --> SIEMConnector[SIEM Connector]
    SIEMConnector --> SIEM
```

The integration method depends on the source and the SIEM platform.

2. Why is Log Integration Important?

Imagine a SOC has a detection rule for:

Multiple Failed VPN Logins

But the VPN logs are not connected to the SIEM.

The detection rule cannot work properly.

This gives us a simple principle:

No Data → No Visibility → No Detection

Therefore, a SIEM Engineer must ensure that critical data sources are properly onboarded.

3. Common Log Integration Methods

The most common methods include:

==> Syslog ==> CEF ==> API Integration ==> Agents ==> Forwarders ==> Windows Event Forwarding ==> File-based collection ==> Cloud Connectors ==> Message Queues ==> Custom Connectors ==> Custom Parsers

Let's understand them one by one.

4. Syslog Integration

Syslog is one of the most common methods for collecting logs from network and security devices.

Common sources include:

==> Firewalls ==> Routers ==> Switches ==> VPN Gateways ==> IDS/IPS ==> Proxy Servers ==>
Network Appliances

A basic architecture:



Common Syslog ports include:

==> UDP 514 ==> TCP 514 ==> TCP 6514 for Syslog over TLS

The exact configuration depends on the vendor and environment.

5. Syslog Facility and Severity

A SIEM Engineer should understand the basic Syslog structure.

Syslog messages commonly contain information such as:

==> Timestamp ==> Host ==> Facility ==> Severity ==> Message

Severity levels commonly range from:

0 — Emergency

to

7 — Debug

This helps systems classify the importance of messages.

However, severity values should not automatically be treated as the actual business/security risk of an event. Detection context matters.

6. Syslog over UDP vs TCP

UDP

Advantages:

==> Simple ==> Low overhead ==> Commonly supported

Limitations:

==> No delivery guarantee ==> Packets may be lost

TCP

Advantages:

==> Reliable transport ==> Better delivery behavior

Limitations:

==> More connection overhead ==> Requires appropriate configuration

For security-sensitive environments, encrypted transport such as **Syslog over TLS** may be appropriate.

7. CEF Integration

CEF means:

Common Event Format

CEF is a standardized event format commonly used to send security events to SIEM systems.

A simplified example:

```
CEF:0|Vendor|Firewall|1.0|100|Blocked Connection|8|  
src=192.168.1.10 dst=10.10.10.20
```

The event contains structured information such as:

==> Vendor ==> Product ==> Event ID ==> Event Name ==> Severity ==> Source IP ==> Destination IP

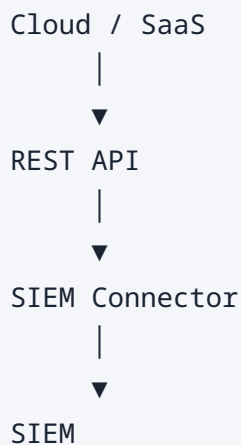
Using standardized formats can make integration and parsing easier.

8. API Integration

Not every platform sends logs through Syslog.

Cloud services and SaaS platforms often expose APIs.

A typical architecture:



API-based integration may require:

==> API Endpoint ==> Authentication ==> API Key/Token ==> OAuth ==> Pagination ==> Rate-limit handling ==> Error handling ==> Data transformation

A SIEM Engineer should understand basic REST API concepts.

9. API Authentication

Security data should not be collected using insecure authentication methods.

Common authentication mechanisms include:

==> API Keys ==> Bearer Tokens ==> OAuth 2.0 ==> Certificates ==> Service Accounts

Credentials should be stored securely and should not be hardcoded into scripts or configuration files unnecessarily.

10. API Pagination

Some APIs do not return all events in one response.

Instead:

Request

↓

Page 1

↓

Page 2

↓

Page 3

↓

Page 4

The connector must handle pagination correctly.

Otherwise, the SIEM may receive only part of the available data.

11. API Rate Limits

Cloud APIs may limit the number of requests that can be made within a specific period.

Example:

100 requests/minute

If the SIEM connector sends too many requests, the API may return:

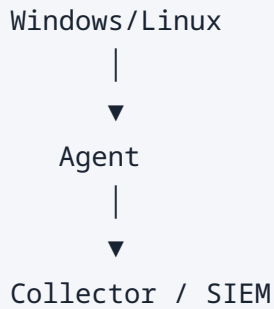
HTTP 429 — Too Many Requests

A properly designed integration should handle rate limits using appropriate retry/backoff mechanisms.

12. Agent-Based Integration

An **agent** is software installed on a system that collects and forwards data.

Architecture:



Agents can provide capabilities such as:

==> Log collection ==> File monitoring ==> Process monitoring ==> Security event collection ==>
Secure transport ==> Buffering ==> Filtering

Wazuh Agent is one example of agent-based security data collection.

13. Forwarders

Some SIEM platforms use **forwarders** to collect and send data.

For example, a forwarder can collect local logs and forward them to a central SIEM component.

Conceptually:

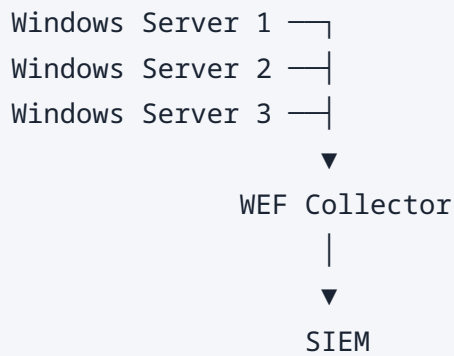


Forwarders can help distribute collection responsibilities and reduce the need for direct connections from every data source to the central platform.

14. Windows Event Forwarding

Windows environments can use **Windows Event Forwarding (WEF)** to centralize Windows event data.

A simplified architecture:



This can be useful when an organization wants centralized Windows event collection.

15. File-Based Integration

Some applications write logs to local files.

Example:

```
/var/log/application.log
```

A collector or agent can monitor the file and forward new entries to the SIEM.

Important considerations include:

==> File rotation ==> Encoding ==> Permissions ==> Multiline events ==> Log format ==> File path changes

16. Multiline Logs

Some applications generate events across multiple lines.

Example:

```
ERROR: Application failed
Exception:
    Connection refused
    Database unavailable
```

If the SIEM treats every line as a separate event, the original event may become fragmented.

Therefore, the collector/parser may need **multiline handling**.

17. Custom Parsers

Sometimes a SIEM does not understand a vendor's log format automatically.

Then a custom parser may be required.

Example raw log:

```
AUTHFAIL|admin|192.168.10.20|VPN01|2026-09-16 14:20:10
```

We may want:

```
event.action = authentication_failure
user.name = admin
source.ip = 192.168.10.20
device.name = VPN01
event.time = 2026-09-16 14:20:10
```

Custom parsing converts vendor-specific data into usable security fields.

18. Parsing vs Normalization

These two concepts are related but different.

Parsing

Extract information from raw data.

```
Raw Log
  ↓
source.ip
username
event.action
timestamp
```

Normalization

Map different field names/formats into a common structure.

```
src_ip
sourceIP
source_address
  ↓
source.ip
```

A SIEM Engineer should understand both.

19. Data Onboarding

Data Onboarding means bringing a new log source into the SIEM and making sure it is usable.

A practical onboarding process:

Step 1 — Identify the Source

Example:

FortiGate Firewall

Step 2 — Identify Available Logs

==> Traffic ==> VPN ==> Authentication ==> IPS ==> Malware

Step 3 — Select Integration Method

Example:

Syslog

Step 4 — Configure Source

Configure the firewall to send logs to the collector/SIEM.

Step 5 — Verify Transport

Check whether the events are reaching the collector.

Step 6 — Parse

Make sure fields are extracted correctly.

Step 7 — Normalize

Map fields into the expected schema.

Step 8 — Test Search

Search for newly received events.

Step 9 — Create Detection

Build relevant detection rules.

Step 10 — Monitor

Monitor ingestion and data quality continuously.

20. Log Integration Testing

Never assume that an integration is working simply because the configuration was saved.

Test:

==> Are logs being generated? ==> Are packets/events reaching the collector? ==> Is authentication working? ==> Are events being parsed? ==> Are fields populated? ==> Are timestamps correct? ==> Are events searchable? ==> Are alerts working?

21. End-to-End Validation

A good validation process follows:

```
graph TD; Source --> Transport; Transport --> Collector; Collector --> Parser; Parser --> Normalizer; Normalizer --> Indexer; Indexer --> SIEM_Search[SIEM Search]; SIEM_Search --> Detection; Detection --> Alert;
```

Source
↓
Transport
↓
Collector
↓
Parser
↓
Normalizer
↓
Indexer
↓
SIEM Search
↓
Detection
↓
Alert

If something fails, troubleshoot from the source toward the SIEM.

22. Troubleshooting: Logs Not Arriving

Suppose:

“The firewall is configured, but no logs are appearing in the SIEM.”

Do not immediately change the SIEM rule.

Start from the source.

Check 1 — Firewall

Is the firewall generating logs?

Check 2 — Configuration

Is the destination IP correct?

Check 3 — Port

Is the correct port configured?

Check 4 — Network

Can the firewall reach the collector?

Check 5 — Collector

Is the Syslog/collection service running?

Check 6 — Packet Capture

Are packets actually arriving?

Tools such as tcpdump can help verify network traffic.

Example:

```
tcpdump -ni any port 514
```

Check 7 — Parser

Are events arriving but failing to parse?

Check 8 — Index/Data Stream

Are events being stored correctly?

Check 9 — Search

Can you find the latest event?

This approach is much better than guessing.

23. Troubleshooting: Logs Arrive but Fields Are Empty

Suppose the SIEM receives:

```
Firewall login failed admin 192.168.10.20
```

But the fields show:

```
username = null  
source.ip = null  
event.action = null
```

Possible causes:

==> Parser mismatch ==> Vendor format changed ==> Wrong parser selected ==> Incorrect regular expression ==> Unexpected delimiter ==> New log format

The SIEM Engineer should inspect the raw event and update the parsing logic.

24. Troubleshooting: Duplicate Logs

Suppose the same firewall is sending logs through:

==> Syslog ==> API

Both may contain the same events.

The result could be duplicate events.

Possible solutions:

==> Disable unnecessary ingestion path ==> Configure deduplication ==> Adjust source configuration ==> Add unique event identifiers ==> Review ingestion architecture

25. Troubleshooting: Log Delay

Sometimes logs arrive several minutes late.

Possible causes:

==> Network latency ==> Collector queue ==> High ingestion volume ==> API polling interval ==> SIEM processing delay ==> Storage/indexing load

Monitor:

Event Time vs Ingestion Time

This helps identify where the delay is occurring.

26. Secure Log Integration

Log integration itself must be secured.

Important practices:

==> Use encrypted transport where appropriate ==> Protect API credentials ==> Use least privilege ==> Restrict network access ==> Validate certificates ==> Monitor collector access ==> Protect log integrity ==> Avoid exposing collectors unnecessarily

Security infrastructure should not create a new attack surface.

27. Network Requirements

Before integrating a source, identify:

==> Source IP ==> Destination IP ==> Protocol ==> Port ==> DNS requirement ==> Firewall rules
==> Routing ==> NAT requirements ==> TLS requirements

Example:

```
Firewall
10.10.10.10
  |
  | TCP/6514
  ▼
Collector
10.10.20.10
```

Both network and security teams may need to coordinate the implementation.

28. Certificates and TLS

When TLS is used, certificate problems can stop ingestion.

Check:

==> Certificate validity ==> Certificate chain ==> Hostname/SAN ==> Trusted CA ==> Expiration ==> TLS version ==> Client/server configuration

A certificate may be valid but still fail if the trust chain or hostname validation is incorrect.

29. Log Filtering

Not every event must always be sent to the SIEM.

Filtering may be used to reduce unnecessary data.

For example:

```
All Events
|
├─ Security Events → SIEM
├─ Authentication → SIEM
├─ Critical Errors → SIEM
└─ Low-value Debug → Filter/Archive
```

However, filtering must be carefully designed.

If you filter too aggressively, you may remove data required for future detection or investigation.

30. Cost and Data Volume

Log integration also affects cost.

More logs mean:

==> More network traffic ==> More storage ==> More indexing ==> More processing ==> More search workload ==> Potentially higher licensing/cloud costs

Therefore, SIEM Engineers need to balance:

Security Visibility + Data Quality + Cost + Performance

31. Critical vs Non-Critical Logs

Not every data source has the same security value.

An organization may prioritize:

High Priority

==> Authentication ==> Firewall ==> Active Directory ==> EDR ==> VPN ==> IDS/IPS ==> Privileged activity

Medium Priority

==> Application ==> Proxy ==> DNS ==> Network devices

Contextual

==> General operational logs ==> Debug logs ==> Low-value informational events

The exact priority depends on the organization's architecture and monitoring requirements.

32. Log Integration Documentation

Every onboarded data source should ideally have documentation.

Record:

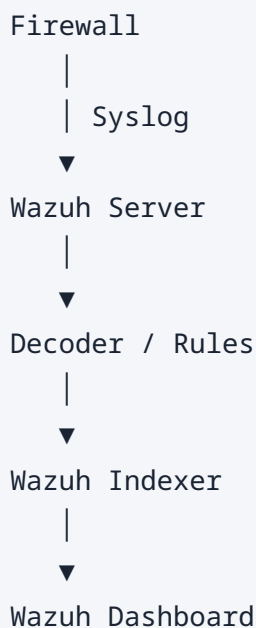
==> Source name ==> Vendor/product ==> IP/hostname ==> Integration method ==> Protocol ==>
Port ==> Authentication ==> Parser ==> Normalized fields ==> Expected EPS ==> Retention ==>
Owner ==> Detection rules ==> Troubleshooting steps

This becomes extremely useful when the environment grows.

33. Real-World Example: Firewall Integration

Imagine an organization wants to connect a firewall to Wazuh.

Basic flow:



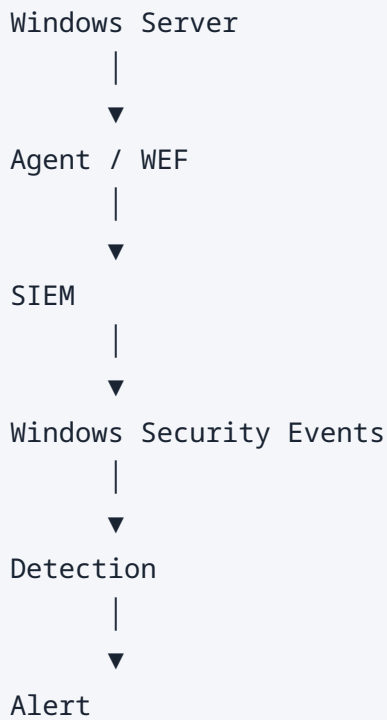
The SIEM Engineer should verify:

==> Firewall logging enabled ==> Correct destination configured ==> Correct protocol/port ==> Network connectivity ==> Wazuh receiving logs ==> Decoder extracting fields ==> Rules generating alerts ==> Dashboard showing events

This is a complete integration—not simply “send Syslog.”

34. Real-World Example: Windows Integration

A Windows environment might look like:



Potential detections:

==> Brute Force ==> Suspicious Account Creation ==> Privilege Changes ==> Suspicious PowerShell ==> Abnormal Authentication

Again, detection quality depends on the underlying log collection.

35. Integration Monitoring

After onboarding, the job is not finished.

You need continuous monitoring.

Ask:

Is the source still sending logs?

Has the volume suddenly dropped?

Did the vendor change the log format?

Are parsers still working?

Are events delayed?

Are fields still populated?

Is the collector healthy?

This is why production SIEM environments need **data-source health monitoring**.

36. Common Log Integration Mistakes

Avoid these common mistakes:

==> Connecting a source without defining the use case ==> Sending logs without validating them
==> Ignoring timestamps ==> Ignoring time synchronization ==> Not monitoring log volume ==>
Creating parsers without testing raw events ==> Filtering too much data ==> Hardcoding API
credentials ==> Not documenting the integration ==> Assuming “configured” means “working”

37. SIEM Engineer Log Integration Checklist

Before Integration

==> Identify log source ==> Identify security use cases ==> Determine required events ==> Select integration method ==> Identify network requirements

During Integration

==> Configure source ==> Configure collector/connector ==> Configure authentication ==> Verify network ==> Verify transport ==> Configure parser

After Integration

==> Verify events ==> Verify fields ==> Verify timestamps ==> Verify normalization ==> Test searches ==> Test detection ==> Monitor ingestion ==> Document configuration

38. The Complete Integration Flow

Remember this:

```
graph TD; Identify --> Design; Design --> Configure; Configure --> Connect; Connect --> Transport; Transport --> Collect; Collect --> Parse; Parse --> Normalize; Normalize --> Validate; Validate --> Detect; Detect --> Monitor; Monitor --> Document;
```

Identify
↓
Design
↓
Configure
↓
Connect
↓
Transport
↓
Collect
↓
Parse
↓
Normalize
↓
Validate
↓
Detect
↓
Monitor
↓
Document

This is the practical lifecycle of SIEM Log Integration.



Why Does Log Integration Matter?

A SIEM platform cannot protect an environment if it cannot see what is happening inside that environment.

Good integration provides:

Better Visibility → Better Detection → Better Investigation → Better Response

But remember:

Successful SIEM integration is not just about making logs arrive. It is about making sure the right security data arrives correctly, consistently, securely, and in a form that the SIEM can actually use.

Interview Questions

Q1. What is Log Integration?

Log Integration is the process of connecting systems, applications, network devices, and security tools to a SIEM so their events can be collected, processed, searched, and used for security monitoring.

Q2. What are common log integration methods?

==> Syslog ==> CEF ==> API ==> Agents ==> Forwarders ==> Windows Event Forwarding ==> Cloud connectors ==> File-based collection

Q3. What would you check if Syslog logs are not arriving?

Source → **Configuration** → **Network** → **Port** → **Collector** → **Transport** → **Parser** → **Index** → **Search**

Q4. What is the difference between parsing and normalization?

Parsing extracts fields from raw logs.

Normalization maps different formats and field names into a common structure.

Q5. Why is API integration important?

Because many cloud and SaaS platforms expose security data through APIs instead of traditional Syslog.

Q6. What is CEF?

CEF stands for **Common Event Format**, a standardized format commonly used to represent security events for SIEM ingestion.

Q7. What is log onboarding?

Log onboarding is the process of connecting a new data source to the SIEM, validating the data, parsing/normalizing it, testing searches and detections, and monitoring the integration.

Q8. What is EPS?

EPS = Events Per Second.

It measures event ingestion/generation rate and is important for capacity planning and monitoring.

Final Takeaway

A SIEM Engineer should be able to take an unknown data source and ask:

What data does it generate?

↓

How can I collect it?

↓

How will I transport it securely?

↓

How will I parse it?

↓

How will I normalize it?

↓

How will I validate it?

↓

What detection can I build from it?

↓

How will I monitor its health?

That is the real skill behind **SIEM Log Integration**.

Next Article

SIEM Engineer #04 — Detection Engineering

In the next article, we will move from **collecting security data** to **detecting threats from that data**.

We will cover:

==> Detection Rules ==> Correlation Rules ==> Sigma ==> MITRE ATT&CK ==> Detection Logic ==> Threshold Detection ==> Behavioral Detection ==> False Positive Reduction ==> Alert Tuning ==> Detection Testing ==> Detection Lifecycle ==> Real-world Brute Force Detection

Logs give us visibility. Detection Engineering turns that visibility into security alerts.



SIEM Engineer Learning Series

#01 — Log Management

#02 — SIEM Platforms

#03 — Log Integration ← You are here

#04 — Detection Engineering → Next

#05 — Query & Investigation

#06 — SOC Operations

#07 — Threat Intelligence

#08 — Automation & SOAR

#09 — Dashboard & Reporting

#10 — SIEM Infrastructure

#11 — SIEM Architecture Design

#12 — SIEM Capacity Planning

#13 — SIEM Performance Tuning

#14 — Advanced Detection Engineering

#15 — Threat Hunting with SIEM

#16 — SIEM Data Quality & Troubleshooting

#17 — Hands-On SIEM: Multi-Platform Practical Guide

Navigation

← **Previous Article: #02 — SIEM Platforms**

SIEM Engineer Index

Next Article: #04 — Detection Engineering →

SIEM Engineer #04 — Detection Engineering

Introduction

A SIEM is only useful when it can help the security team identify suspicious activity.

Collecting thousands or millions of logs is not enough.

The SIEM Engineer needs to answer an important question:

“What activity should generate a security alert, and why?”

This is where **Detection Engineering** comes in.

Detection Engineering is the process of designing, developing, testing, tuning, deploying, and maintaining security detections that identify suspicious or malicious activity from security telemetry.

A simple detection flow is:

Log Source → Telemetry → Detection Logic → Rule → Alert → Investigation → Tuning



1. What is Detection Engineering?

Detection Engineering means creating security logic that can identify suspicious behavior from available data.

For example:

Suppose a Linux server receives:

```
Failed SSH login
Failed SSH login
Failed SSH login
Failed SSH login
Failed SSH login
```

One failed login may be normal.

But:

```
50 failed SSH logins
from the same IP
against the same server
within 5 minutes
```

could indicate a brute-force attempt.

A detection could therefore be:

```
IF
failed SSH login >= 20
FROM same source IP
WITHIN 5 minutes

THEN
generate alert
```

This is a simple example of detection logic.

2. Why Detection Engineering Matters

A SOC may receive millions of events every day.

Analysts cannot manually inspect every event.

Detection Engineering converts raw telemetry into meaningful security signals.

Example

Without detection:

```
10,000 Windows events
      ↓
Analyst manually searches
      ↓
High workload
```

With detection:

```
10,000 Windows events
      ↓
Detection Rules
      ↓
Suspicious activity
      ↓
🔔 Security Alert
      ↓
SOC Analyst
```

Why does this matter?

Good detections help SOC teams:

==> Reduce manual investigation

==> Detect suspicious behavior faster

==> Improve incident response

==> Reduce unnecessary alerts

==> Improve MITRE ATT&CK visibility

==> Support threat hunting

==> Standardize security monitoring

3. Detection Engineering Lifecycle

Detection Engineering is not simply:

Write Rule → Deploy

A mature process looks like:

```
Detection Idea
    ↓
Security Hypothesis
    ↓
Identify Data Sources
    ↓
Check Required Fields
    ↓
Develop Detection Logic
    ↓
Create Rule
    ↓
Test
    ↓
Validate
    ↓
Deploy
    ↓
Monitor
    ↓
Tune
    ↓
Review
    ↓
Retire / Update
```

Let's understand each step.



4. Step 1 — Detection Idea

Everything starts with a security question.

Examples:

==> Can we detect SSH brute-force activity?

==> Can we detect suspicious PowerShell execution?

==> Can we detect unexpected administrator creation?

==> Can we detect privilege group changes?

==> Can we detect suspicious login behavior?

==> Can we detect communication with known malicious IPs?

This is the starting point.



5. Step 2 — Create a Security Hypothesis

A detection should have a clear reason.

For example:

“An attacker attempting to gain access to a Linux server may generate multiple failed SSH authentication events from the same source IP.”

This becomes the detection hypothesis.

Then we determine:

What behavior?
Which system?
Which log?
Which fields?
What threshold?
What time window?
What response?



6. Step 3 — Identify Required Telemetry

A detection cannot work without the required data.

For SSH brute-force detection, we may need:

```
timestamp
source_ip
destination_ip
username
event_type
authentication_result
hostname
```

For Windows authentication detection:

```
timestamp
user
source_ip
hostname
event_id
logon_type
authentication_result
```

For process detection:

```
timestamp
hostname
user
parent_process
process_name
command_line
process_id
```

Important

Before writing a rule, ask:

“Do I actually have the data required for this detection?”

A great detection idea is useless if the SIEM does not receive the necessary telemetry.



7. Detection Data Quality

Detection quality depends heavily on log quality.

For example:

```
source_ip = NULL  
username = NULL  
event_type = NULL
```

makes many detections difficult.

Therefore:

Good Detection = Good Telemetry + Good Logic

You should verify:

==> Logs are arriving

==> Parsing is correct

==> Fields are normalized

==> Timestamps are correct

==> Required fields exist

==> Events are not duplicated

==> Events are not heavily delayed



8. Detection Logic

Detection logic defines what behavior should trigger an alert.

Example:

```
failed_login >= 10  
AND  
same_source_ip  
AND  
within 5 minutes
```

Another example:

```
new_admin_account = TRUE  
AND  
account_created_outside_change_window = TRUE
```

Another:

```
known_malicious_ip = TRUE  
AND  
internal_host_connection = TRUE
```

Detection logic can be simple or complex.



9. Types of Detection

There are several common detection approaches.

9.1 Threshold Detection

A threshold detection looks for an event count above a certain value.

Example:

More than 20 failed logins
from one IP
within 5 minutes

Useful for:

- ==> Brute force
 - ==> Repeated authentication failures
 - ==> Excessive connection attempts
 - ==> Repeated security events
-

9.2 Sequence Detection

Sequence detection looks for multiple events occurring in a specific order.

Example:

```

Login
  ↓
Privilege escalation
  ↓
Sensitive command
  ↓
Outbound connection
  
```

Individually, each event may not be highly suspicious.

Together, the sequence may be more meaningful.



9.3 Behavioral Detection

Behavioral detection looks for activity that differs from expected behavior.

Example:

A user normally logs in from:

Dhaka

Suddenly the same account authenticates from:

Another geographic location

The detection may investigate the unusual behavior.

Behavioral detections often require additional context.



9.4 Anomaly Detection

Anomaly detection attempts to identify activity that differs significantly from an established baseline.

Example:

Normal:
20-50 login events/day

Observed:
2,000 login events/day

This could trigger investigation.

However, anomaly does not automatically mean malicious.

It means:

“This behavior is unusual and should be investigated.”



9.5 Correlation Detection

Correlation combines multiple events or data sources.

Example:

```
Firewall connection
      +
Endpoint process
      +
Threat Intelligence IOC
      ↓
Security Alert
```

Correlation is one of the most important SIEM capabilities.



10. Detection Rules vs Correlation Rules

Detection Rule

Usually focuses on a specific event or behavior.

Example:

```
PowerShell process created
```

Correlation Rule

Combines multiple events or conditions.

Example:

```
Suspicious PowerShell  
+  
Network connection  
+  
Known malicious destination
```

Both are useful.

11. Sigma

Sigma is a generic, open detection-rule format designed to describe suspicious activity in a SIEM-independent way.

A simplified Sigma-style rule can look like:

```
title: Multiple Failed SSH Logins
status: experimental

logsource:
  product: linux
  service: ssh

detection:
  selection:
    event_type: "authentication_failure"

    condition: selection

level: medium
```

The exact fields depend on the telemetry and backend.

The important idea is:

```
Detection Logic
  ↓
Sigma Rule
  ↓
Backend-specific query/rule
```

Sigma helps security teams maintain detection logic in a portable format.



12. Sigma Rule Structure

Common Sigma components include:

```
title
id
status
description
references
author
date
tags
logsource
detection
falsepositives
level
```

Example concept:

title: Suspicious Authentication Activity

description: Detects repeated authentication failures.

logsource:

product: linux

service: ssh

detection:

selection:

event_type: authentication_failure

condition: selection

falsepositives:

- Administrative activity

level: medium

The quality of the rule depends on the quality of the underlying data.

13. MITRE ATT&CK Mapping

Detection Engineers often map detections to **MITRE ATT&CK** techniques.

MITRE ATT&CK provides a common knowledge base for describing adversary tactics and techniques.

A detection can contain tags such as:

```
Tactic
Technique
Sub-technique
```

For example, a PowerShell-related detection can be associated with:

```
Tactic: Execution (TA0002)
Technique: T1059 Command and Scripting Interpreter
Sub-technique: T1059.001 PowerShell
```

Other common mappings: T1110 Brute Force, T1078 Valid Accounts, T1003 Credential Dumping, T1021 Remote Services. See #14 Advanced Detection and #17 hands-on for full examples.

The exact mapping should be based on what the detection actually identifies.

Important

Do not map a detection to a technique simply because the technique sounds related.

The mapping should reflect the behavior the rule is actually detecting.



14. Detection Coverage

A mature SOC should understand:

What can we detect?
What cannot we detect?
What telemetry do we have?
What telemetry is missing?

Example:

Area	Telemetry	Detection
Windows Login	Available	Yes
Linux SSH	Available	Yes
Firewall	Available	Yes
Cloud Identity	Missing	No
Endpoint Process	Partial	Limited

This helps identify detection gaps.



15. Example — SSH Brute Force Detection

Suppose the SIEM receives:

```
timestamp
source_ip
username
hostname
event_type
```

Example events:

```
10:01:01  10.10.10.50  admin  server01  failed
10:01:10  10.10.10.50  admin  server01  failed
10:01:18  10.10.10.50  root   server01  failed
...
```

Detection:

```
IF

authentication_failure >= 10

FROM same source_ip

WITHIN 5 minutes

THEN

generate alert
```

Possible alert:

Alert:
Possible SSH Brute Force

Source IP:
10.10.10.50

Target:
server01

Failed Attempts:
24

Time Window:
5 minutes

The analyst then investigates whether it is:

True Positive
False Positive
Benign Activity



16. Windows Failed Login Detection

A detection may monitor repeated authentication failures.

Concept:

Event:

Authentication Failure

Group By:

user + source_ip + destination_host

Threshold:

multiple failures

Time Window:

5 minutes

Potential alert:

Multiple Windows Authentication Failures

The analyst can then investigate:

==> Source IP

==> Username

==> Target host

==> Login type

==> Historical activity

==> Related events

==> Endpoint activity



17. PowerShell Detection

PowerShell is legitimate administrative software.

Therefore:

PowerShell execution does not automatically mean malicious activity.

A useful detection may look for suspicious combinations of:

```
PowerShell execution
+
unusual parent process
+
encoded/suspicious command characteristics
+
unexpected user
+
unusual host
```

This is an important Detection Engineering principle:

Context matters.



18. Privileged Account Change Detection

Another useful detection:

User added to privileged group

For example:

```
User
  ↓
Added to Administrator Group
  ↓
Outside approved change window
  ↓
Alert
```

The detection becomes stronger when additional context is available:

==> Who performed the change?

==> Which account was modified?

==> Which host?

==> Was there an approved ticket?

==> Was the change expected?



19. Threat Intelligence Correlation

Suppose the organization has a threat intelligence feed.

The SIEM receives:

```
Source IP:  
203.x.x.x
```

Threat Intelligence says:

```
IP = Known Malicious
```

The SIEM can correlate:

```
Internal Host  
    ↓  
Connection  
    ↓  
External IP  
    ↓  
Threat Intelligence Match  
    ↓  
Alert
```

This can increase investigation priority.

But remember:

IOC match ≠ automatic confirmation of compromise.

Analysts still need context and validation.

20. Severity and Confidence

A common mistake is treating severity and confidence as the same thing.

They are different concepts.

Severity

How serious could the activity be?

Confidence

How strongly does the detection indicate the suspected behavior?

Example:

High Severity
Low Confidence

could mean:

The potential impact is serious, but the detection has a significant chance of being benign.

Another example:

Medium Severity
High Confidence

could mean:

The detection is very likely to represent the behavior it was designed to identify, but the impact may be limited.

This distinction helps SOC analysts prioritize investigation.



21. False Positives

False positives are one of the biggest problems in SOC environments.

Example:

A rule detects:

20 failed logins

But the source IP belongs to:

Internal vulnerability scanner

The alert may be legitimate activity.

This does not necessarily mean the detection is useless.

It may need tuning.



22. Detection Tuning

Detection tuning can include:

- ==> Adjusting thresholds
- ==> Changing time windows
- ==> Adding conditions
- ==> Adding context
- ==> Creating exceptions
- ==> Adding trusted sources
- ==> Excluding approved service accounts
- ==> Improving field normalization
- ==> Correlating additional events

Example:

Before:

```
failed_login > 10
```

After:

```
failed_login > 10  
AND  
source_ip NOT IN approved_scanner_list
```

The goal is not simply to reduce alert numbers.

The goal is to improve **signal quality**.

23. Alert Deduplication

Sometimes the same activity generates many alerts.

Example:

```
100 events
  ↓
100 alerts
```

This can overwhelm analysts.

A better design may group related events:

```
100 events
  ↓
Correlation
  ↓
1 security alert
```

This is called **alert aggregation/deduplication**, depending on the implementation.



24. Exceptions and Allowlisting

Some activities are known and approved.

Examples:

```
Security Scanner
Backup Server
Monitoring Server
Patch Management Server
Approved Administrator
```

Instead of disabling the detection completely, create controlled exceptions where appropriate.

Bad approach:

```
Too many alerts
    ↓
Disable rule
```

Better approach:

```
Too many alerts
    ↓
Investigate reason
    ↓
Tune rule
    ↓
Add controlled exception
    ↓
Monitor
```




25. Detection Testing

Never deploy an important detection without testing it.

Testing can include:

Unit Testing

Test individual detection conditions.

Replay Testing

Use historical or representative events.

Synthetic Events

Generate controlled test events.

Attack Simulation

Use authorized security testing to produce expected telemetry.

Regression Testing

Make sure a rule change does not break previously working behavior.



26. Example Detection Test

Detection:

Multiple SSH authentication failures

Test cases:

Test 1:

1 failed login

Expected:

No alert

Test 2:

5 failed logins

Expected:

No alert

Test 3:

20 failed logins from same IP

Expected:

Alert

Test 4:

20 failed logins from approved scanner

Expected:

Exception / controlled handling

This is much better than simply writing the rule and assuming it works.

27. Splunk Detection Example

A simplified Splunk SPL concept:

```
index=linux authentication_failure  
| stats count by source_ip  
| where count >= 10
```

This demonstrates the basic idea:

```
Search  
↓  
Group  
↓  
Count  
↓  
Threshold  
↓  
Detection
```

A production rule should normally include appropriate time windows, fields, exclusions, and context.

28. Microsoft Sentinel / KQL Concept

A simplified KQL example:

```
SigninLogs
| where ResultType != 0
| summarize FailedAttempts=count() by IPAddress
| where FailedAttempts >= 10
```

Again, this is a simplified learning example.

A production detection should account for:

==> Time window

==> Identity context

==> Trusted sources

==> User risk

==> Related events

==> Appropriate exclusions



29. Wazuh Detection Concept

Wazuh can use rules to identify specific security events.

Conceptually:

```
Log
↓
Decoder
↓
Rule
↓
Alert
↓
Active Response / SOC Investigation
```

For example:

```
SSH authentication failure
↓
Wazuh decoder
↓
Rule matching
↓
Alert
↓
SOC investigation
```

The important lesson is that Detection Engineering is not limited to one SIEM platform.

The concepts apply across:

==> Wazuh

==> Splunk

==> Microsoft Sentinel

==> QRadar

==> Elastic Security

==> OpenSearch

and other security monitoring platforms.



30. Detection-as-Code

Modern security teams increasingly manage detections similarly to software.

Instead of keeping rules only inside a SIEM:

Detection Rules



Git Repository



Version Control



Testing



Review



Deployment

Benefits include:

==> Version history

==> Peer review

==> Change tracking

==> Testing

==> Rollback

==> Collaboration

A detection rule should be treated as production security logic.



31. Detection Rule Documentation

Every important detection should have documentation.

A useful detection document can contain:

- Detection Name
- Description
- Purpose
- Data Sources
- Required Fields
- Detection Logic
- MITRE ATT&CK Mapping
- Severity
- Confidence
- False Positives
- Exceptions
- Response Guidance
- Owner
- Version
- Last Reviewed

Example:

Detection:

SSH Brute Force

Purpose:

Detect repeated SSH authentication failures.

Data Source:

Linux / SSH

Threshold:

10 failures

Time Window:

5 minutes

MITRE Mapping:

Relevant authentication technique mapping

False Positives:

Approved security scanners

Response:

Investigate source IP and target host



32. Detection Health Monitoring

Creating a rule is not the end.

You should monitor whether the rule is healthy.

Useful metrics include:

==> Alert volume

==> False-positive rate

==> Detection firing frequency

==> Data freshness

==> Rule execution errors

==> Query performance

==> Missing telemetry

==> Detection coverage

==> Analyst feedback

A rule that worked six months ago may become ineffective after an infrastructure or logging change.



33. Detection Performance and Cost

Detection rules consume resources.

Poorly designed queries can cause:

- High CPU
- High memory
- Slow searches
- High SIEM cost
- Delayed alerts

Therefore:

Detection Engineering = Security + Performance

Optimize:

==> Search scope

==> Time range

==> Indexed fields

==> Query structure

==> Event volume

==> Correlation frequency

==> Data retention



34. Detection Coverage vs Detection Count

A SOC should not measure maturity only by:

We have 1,000 detection rules.

More rules do not automatically mean better security monitoring.

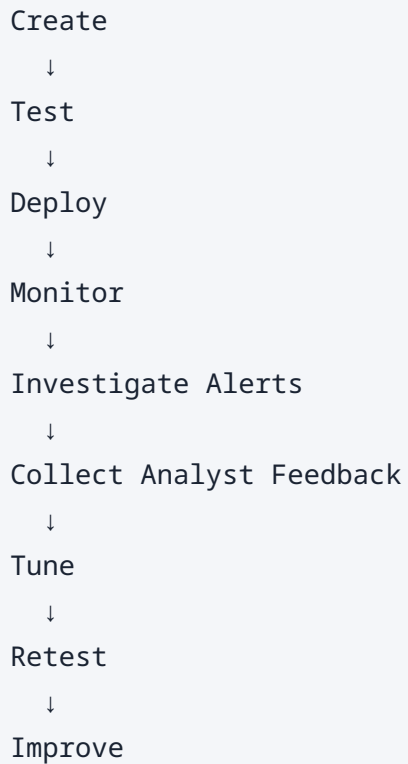
Instead ask:

What threats can we detect?
What telemetry supports them?
How reliable are the detections?
What are the detection gaps?

A small number of well-tested detections can be more useful than a huge collection of poorly maintained rules.

35. Detection Improvement Cycle

A mature detection process looks like:



```
graph TD; Create --> Test; Test --> Deploy; Deploy --> Monitor; Monitor --> InvestigateAlerts[Investigate Alerts]; InvestigateAlerts --> CollectFeedback[Collect Analyst Feedback]; CollectFeedback --> Tune; Tune --> Retest; Retest --> Improve; Improve --> Create;
```

Create
↓
Test
↓
Deploy
↓
Monitor
↓
Investigate Alerts
↓
Collect Analyst Feedback
↓
Tune
↓
Retest
↓
Improve

Detection Engineering is a continuous process.

36. Practical SIEM Lab

You can build a small detection engineering lab using:

```
Linux VM
Windows VM
Wazuh / SIEM
Syslog
Windows Event Logs
```

Lab 1 — SSH Detection

Generate several failed SSH authentication events in your authorized lab.

Then:

```
Collect logs
  ↓
Parse logs
  ↓
Identify source IP
  ↓
Create threshold detection
  ↓
Generate alert
  ↓
Investigate
  ↓
Tune
```

Lab 2 — Windows Authentication

Collect Windows security events.

Create a detection for repeated authentication failures.

Test:

Normal login
Failed login
Repeated failures
Approved administrative activity

Lab 3 — Privilege Change

Monitor administrative group membership changes.

Create a detection:

Privilege Group Change

↓

Alert

↓

Validate

↓

Investigate



37. Detection Engineering Checklist

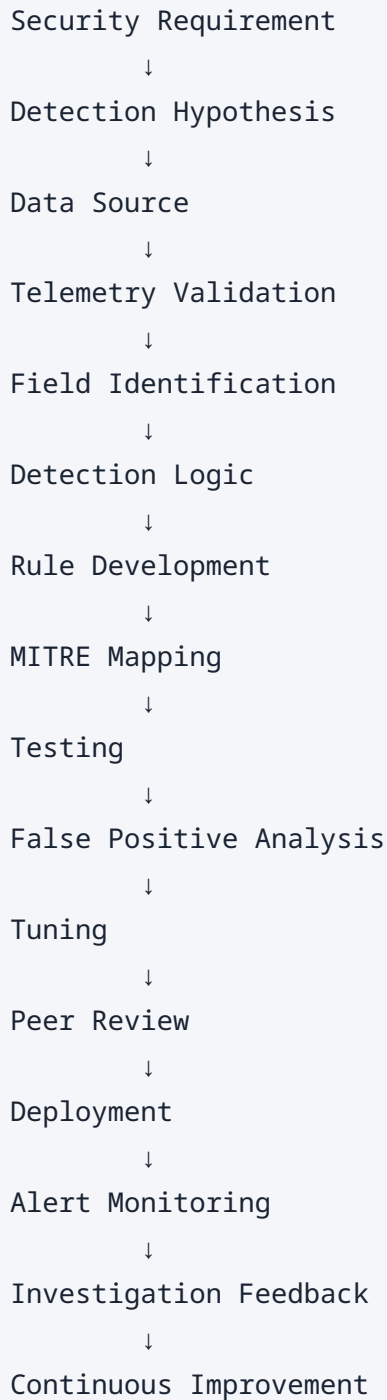
Before deploying a detection, ask:

- [] What behavior am I detecting?
- [] What is the security hypothesis?
- [] Which data source provides the evidence?
- [] Are the required logs available?
- [] Are required fields parsed?
- [] Is the timestamp correct?
- [] Is the detection logic clear?
- [] Is a threshold required?
- [] Is a time window required?
- [] Is correlation required?
- [] What are the expected false positives?
- [] Are exceptions required?
- [] What severity should it have?
- [] What confidence should it have?
- [] Is MITRE ATT&CK mapping appropriate?
- [] Has the rule been tested?
- [] Has it been reviewed?
- [] Is it documented?
- [] Is it version controlled?
- [] How will it be monitored?

[] When will it be reviewed again?

38. Complete Detection Engineering Flow

Remember this flow:



```
graph TD; A[Security Requirement] --> B[Detection Hypothesis]; B --> C[Data Source]; C --> D[Telemetry Validation]; D --> E[Field Identification]; E --> F[Detection Logic]; F --> G[Rule Development]; G --> H[MITRE Mapping]; H --> I[Testing]; I --> J[False Positive Analysis]; J --> K[Tuning]; K --> L[Peer Review]; L --> M[Deployment]; M --> N[Alert Monitoring]; N --> O[Investigation Feedback]; O --> P[Continuous Improvement];
```

Security Requirement

↓

Detection Hypothesis

↓

Data Source

↓

Telemetry Validation

↓

Field Identification

↓

Detection Logic

↓

Rule Development

↓

MITRE Mapping

↓

Testing

↓

False Positive Analysis

↓

Tuning

↓

Peer Review

↓

Deployment

↓

Alert Monitoring

↓

Investigation Feedback

↓

Continuous Improvement

This is the core workflow of a SIEM Detection Engineer.



39. SIEM Engineer Interview Questions

Q1. What is Detection Engineering?

Answer:

Detection Engineering is the process of designing, developing, testing, deploying, tuning, and maintaining security detections that identify suspicious or malicious activity from security telemetry.

Q2. What is a detection rule?

Answer:

A detection rule contains logic that identifies a specific suspicious event or behavior and generates an alert when the defined conditions are met.

Q3. What is a correlation rule?

Answer:

A correlation rule combines multiple events, conditions, or data sources to identify a more meaningful security pattern.

Q4. What is Sigma?

Answer:

Sigma is a generic and SIEM-independent format for describing detection rules. It allows detection logic to be shared and converted for different SIEM backends.

Q5. Why is MITRE ATT&CK important for Detection Engineering?

Answer:

MITRE ATT&CK provides a common framework for describing adversary tactics and techniques. Mapping detections to ATT&CK helps security teams understand detection coverage and identify gaps.

Q6. What is a false positive?

Answer:

A false positive occurs when a detection generates an alert for activity that is not actually malicious according to the detection's intended purpose.

Q7. How do you reduce false positives?

Answer:

I first investigate why the alert is firing. Then I can improve the detection using better conditions, thresholds, time windows, contextual fields, correlation, or carefully controlled exceptions.

Q8. What is detection tuning?

Answer:

Detection tuning is the process of improving a detection's accuracy, relevance, and performance based on real-world alert results and analyst feedback.

Q9. What is Detection-as-Code?

Answer:

Detection-as-Code means managing detection rules using software-engineering practices such as Git, version control, testing, peer review, documentation, and controlled deployment.

Q10. What makes a good detection?

Answer:

A good detection should have a clear purpose, reliable telemetry, understandable logic, acceptable false-positive behavior, appropriate context, testing, documentation, and continuous monitoring.



40. Important Lessons

Remember these principles:

Logs are not detections

Log $\neq$ Alert

Logs provide evidence.

Detections turn evidence into security signals.

More alerts do not mean better security

High Alert Volume

$\neq$

High Security

Signal quality matters.

Context is extremely important

A single event may be normal.

Multiple related events may tell a different story.

Detection Engineering is continuous

A detection should be:

Built $\rightarrow$ Tested $\rightarrow$ Monitored $\rightarrow$ Tuned $\rightarrow$ Improved

Detection depends on telemetry

If you do not collect the required data, you cannot reliably detect the behavior.



Final Takeaway

A SIEM Engineer should not only know how to collect logs.

A strong SIEM Engineer should understand:

What should we detect?
Why should we detect it?
Which logs provide the evidence?
How should the events be correlated?
How do we test the detection?
How do we reduce false positives?
How do we measure detection quality?
How do we maintain the rule?

The complete Detection Engineering mindset is:

Hypothesis → Data → Logic → Rule → Test → Tune → Deploy → Monitor → Improve

If you master this process, you move beyond simply operating a SIEM and start thinking like a **Detection Engineer**.



SIEM Engineer Learning Series

#01 — Log Management

#02 — SIEM Platforms

#03 — Log Integration

#04 — Detection Engineering ← You are here

#05 — Query & Investigation → Next

#06 — SOC Operations

#07 — Threat Intelligence

#08 — Automation & SOAR

#09 — Dashboard & Reporting

#10 — SIEM Infrastructure

#11 — SIEM Architecture Design

#12 — SIEM Capacity Planning

#13 — SIEM Performance Tuning

#14 — Advanced Detection Engineering

#15 — Threat Hunting with SIEM

#16 — SIEM Data Quality & Troubleshooting

#17 — Hands-On SIEM: Multi-Platform Practical Guide

Navigation

← **Previous Article: #03 — Log Integration**

SIEM Engineer Index

Next Article: #05 — Query & Investigation →

SIEM Engineer #05 — Query & Investigation

Introduction

A SIEM collects and stores huge amounts of security data.

But collecting logs is only the beginning.

A SIEM Engineer and SOC Analyst must be able to **search, filter, correlate, analyze, and investigate** that data.

This is where **Query & Investigation** becomes important.

A simple security investigation looks like:

Alert → Search → Filter → Correlate → Timeline → Evidence → Verdict

If you know how to write good SIEM queries, you can investigate incidents much faster.



1. What is Query & Investigation?

Query means asking the SIEM for specific information from collected data.

For example:

```
Show me all failed logins from 10.10.10.50
during the last 30 minutes.
```

The SIEM searches the available data and returns matching events.

Investigation goes further.

You ask:

```
Who?
What?
When?
Where?
How?
Why?
What happened before?
What happened after?
```

The goal is to understand the complete activity.

2. Why Query Skills Matter

Imagine your SIEM contains:

10,000,000 events

An alert says:

Multiple failed login attempts

You cannot manually read 10 million events.

Instead, you query:

```
source_ip = suspicious IP
username = affected user
time = incident window
event_type = authentication failure
```

Then you gradually expand the investigation.

Why does this matter?

Good query skills help you:

- ==> Investigate alerts faster
- ==> Find related events
- ==> Build incident timelines
- ==> Perform threat hunting
- ==> Validate detections
- ==> Identify false positives
- ==> Find suspicious behavior

==> Collect evidence



3. The Basic Investigation Questions

During an investigation, start with:

WHO?

Which user?
Which account?
Which process?
Which administrator?

WHAT?

What happened?
What event occurred?
What command was executed?

WHEN?

Exact timestamp?
First event?
Last event?
Duration?

WHERE?

Source IP?
Destination IP?
Hostname?
Geographic location?

HOW?

How did the activity happen?
Which protocol?
Which process?
Which authentication method?

WHY?

This is usually the most difficult question.

You should not assume the reason.

Use available evidence to determine whether the activity was:

Expected
Benign
Suspicious
Malicious
Unknown



4. Query Building Fundamentals

Most SIEM queries use some combination of:

Search
Filter
Fields
Aggregation
Grouping
Sorting
Time Range
Correlation

Conceptually:

Raw Events
↓
Filter
↓
Select Fields
↓
Group
↓
Count
↓
Sort
↓
Investigate



5. Time Range

Time is one of the most important investigation fields.

For example:

14:00 → Alert

Don't only investigate the exact alert timestamp.

Check:

13:00 → 15:00

You may discover:

13:20 Initial login
13:35 Failed authentication
13:42 Successful login
13:47 Privilege change
13:52 Process execution
14:00 SIEM alert
14:10 Network connection

Now you have a timeline.

6. Investigation Timeline

A timeline helps explain an incident.

Example:

```
graph TD; T1[10:00] --> E1[Failed Login]; T2[10:02] --> E2[Failed Login]; T3[10:04] --> E3[Successful Login]; T4[10:06] --> E4[New Process]; T5[10:08] --> E5[Privilege Change]; T6[10:10] --> E6[Outbound Connection];
```

10:00
↓
Failed Login

10:02
↓
Failed Login

10:04
↓
Successful Login

10:06
↓
New Process

10:08
↓
Privilege Change

10:10
↓
Outbound Connection

The individual events become more meaningful when placed together.



7. Splunk SPL

Splunk uses **Search Processing Language (SPL)**.

A simple search:

```
index=security
```

Filter:

```
index=security source_ip="10.10.10.50"
```

Filter by event:

```
index=security event_type="authentication_failure"
```

Count events:

```
index=security event_type="authentication_failure"  
| stats count
```

Group by source IP:

```
index=security event_type="authentication_failure"  
| stats count by source_ip
```

Sort:

```
index=security event_type="authentication_failure"  
| stats count by source_ip  
| sort - count
```

The basic idea is:

Search



Filter



Aggregate



Sort

8. Microsoft Sentinel and KQL

Microsoft Sentinel commonly uses **Kusto Query Language (KQL)**.

Example:

```
SigninLogs
| where ResultType != 0
```

Group authentication failures:

```
SigninLogs
| where ResultType != 0
| summarize count() by IPAddress
```

Sort:

```
SigninLogs
| where ResultType != 0
| summarize FailedAttempts=count() by IPAddress
| order by FailedAttempts desc
```

The concept is similar:

```
Table
↓
Filter
↓
Aggregate
↓
Sort
```



9. Elasticsearch / OpenSearch Query

Elastic and OpenSearch investigations commonly use field-based searches and query DSL.

Conceptually:

```
event.action = authentication_failure
```

Then filter by:

```
source.ip  
user.name  
host.name  
@timestamp
```

You may also use aggregations to identify:

```
Top source IPs  
Top users  
Top destinations  
Event counts
```

The exact syntax depends on the interface and query language being used.



10. Wazuh Investigation

In Wazuh, an analyst may investigate:

```
Rule ID
Agent
Timestamp
Source IP
Username
Location
Event
MITRE ATT&CK information
```

A useful investigation process is:

```
Wazuh Alert
  ↓
Open Alert Details
  ↓
Identify Source
  ↓
Check Related Events
  ↓
Check Host
  ↓
Check User
  ↓
Check Timeline
  ↓
Determine Verdict
```



11. Filtering

Filtering is one of the most important query skills.

Suppose you have:

```
1,000,000 events
```

You can reduce the dataset by filtering.

Example:

```
event_type = authentication_failure
```

Then:

```
source_ip = specific IP
```

Then:

```
username = specific account
```

Then:

```
hostname = specific server
```

Now the investigation becomes manageable.

12. Field Selection

Do not always return every field.

For authentication investigation, useful fields may be:

```
timestamp
username
source_ip
destination_ip
hostname
event_type
status
logon_type
```

For process investigation:

```
timestamp
username
hostname
process_name
parent_process
command_line
process_id
```

For network investigation:

```
timestamp
source_ip
destination_ip
source_port
destination_port
protocol
action
bytes
```

Selecting useful fields makes investigations easier to understand.



13. Aggregation

Aggregation converts many events into useful statistics.

Example:

Authentication Failures:

```
10.10.10.10 → 5  
10.10.10.20 → 12  
10.10.10.30 → 97
```

This immediately tells you which source deserves further investigation.

Common aggregation concepts include:

==> Count

==> Sum

==> Average

==> Minimum

==> Maximum

==> Distinct count

==> Group by



14. Group By

Suppose you have:

```
User A → 10 failures  
User B → 2 failures  
User C → 75 failures
```

Grouping by username can highlight unusual accounts.

Similarly, group by:

```
source_ip  
destination_ip  
username  
hostname  
process_name  
parent_process
```

The correct grouping depends on the investigation question.

15. Correlation

One event rarely tells the complete story.

Suppose:

Authentication Failure

Then:

Successful Login

Then:

Privilege Change

Then:

Suspicious Process

Correlation connects these events.

Conceptually:

```

Event A
+
Event B
+
Event C
+
Event D
↓
Investigation Story
  
```




16. Pivoting

Pivoting means using information from one event to search for related activity.

Example:

You receive:

```
Source IP:  
10.10.10.50
```

Pivot:

```
Search all activity from 10.10.10.50
```

You discover:

```
User:  
admin
```

Pivot again:

```
Search all activity by admin
```

You discover:

```
Hostname:  
SERVER-01
```

Pivot again:

```
Search SERVER-01 activity
```

This creates an investigation chain.



17. Investigation Pivot Example

Start:

```
Alert
↓
Source IP
↓
Username
↓
Hostname
↓
Process
↓
Destination IP
↓
Threat Intelligence
↓
Related Alerts
```

This is a fundamental SOC investigation technique.



18. IP Investigation

Suppose an alert contains:

```
source_ip = 203.x.x.x
```

You may investigate:

- ==> How many connections?
- ==> Which internal systems contacted it?
- ==> Which ports?
- ==> When did communication begin?
- ==> Which users were involved?
- ==> Was it seen before?
- ==> Is it associated with known threat intelligence?

Do not conclude that an IP is malicious based only on one indicator.

Context matters.



19. User Investigation

For a suspicious account:

```
username = admin
```

Search:

```
All authentication events  
All failed logins  
All successful logins  
Privilege changes  
Process activity  
Endpoint activity  
Network activity
```

Then construct:

```
User Timeline
```

Example:

```
09:00 Login  
09:05 Failed authentication  
09:07 Successful authentication  
09:10 Privilege change  
09:12 Process execution  
09:15 Network connection
```

20. Host Investigation

For a suspicious host:

```
hostname = SERVER-01
```

Investigate:

==> Authentication

==> Processes

==> Network connections

==> File activity

==> Privilege changes

==> Security alerts

==> Endpoint detections

==> Configuration changes

This helps determine whether the host is affected or simply generated an alert.



21. Process Investigation

For endpoint activity, look at:

```
Process Name
Parent Process
User
Command Line
Path
Timestamp
Host
Network Activity
```

The **parent-child relationship** can be particularly useful.

Example:

```
Process A
  ↓
Process B
  ↓
Process C
```

Unexpected process relationships may deserve investigation.

But again:

Unusual does not automatically mean malicious.

22. Authentication Investigation

Authentication investigations often involve:

```
User
Source IP
Destination Host
Authentication Result
Logon Type
Timestamp
Location
Device
```

Questions:

==> Was the login successful?

==> Was the account expected?

==> Was the source known?

==> Was the destination expected?

==> Were there repeated failures?

==> Was there a privilege change afterward?



23. Alert Investigation

When receiving a SIEM alert, do not immediately close it.

Use a structured process:

```
Alert
↓
Understand Detection
↓
Validate Event
↓
Identify Assets
↓
Identify Users
↓
Search Related Events
↓
Build Timeline
↓
Check Threat Intelligence
↓
Determine Scope
↓
Determine Verdict
↓
Document
```

24. True Positive vs False Positive

After investigation, classify the alert according to your SOC process.

True Positive

The detection identified the intended suspicious/malicious behavior.

False Positive

The detection fired, but the activity was not the behavior it was intended to identify.

Benign / Expected

The activity is legitimate and expected.

Unknown

There is insufficient evidence to confidently determine the nature of the activity.

The exact verdict categories depend on the organization's SOC process.



25. Threat Hunting

Threat hunting is proactive.

Traditional workflow:

```
Alert
↓
Investigation
```

Threat hunting:

```
Hypothesis
↓
Query
↓
Search Environment
↓
Find Evidence
↓
Investigate
```

Example hypothesis:

“There may be unusual authentication activity against privileged accounts.”

Then search:

Privileged accounts

+

Authentication failures

+

Authentication successes

+

Source IPs

+

Time patterns

26. Threat Hunting Questions

Good hunters ask questions such as:

==> Which accounts have unusual login behavior?

==> Which systems have unexpected outbound connections?

==> Which hosts are communicating with suspicious destinations?

==> Which users suddenly have privilege changes?

==> Which processes are rarely observed?

==> Which authentication failures are increasing?

==> Which systems have missing security telemetry?



27. Query → Hypothesis → Evidence

Do not search randomly.

Use:

```
Hypothesis
  ↓
Required Evidence
  ↓
Query
  ↓
Results
  ↓
Analysis
```

Example:

```
Hypothesis:
An account may have unusual login activity.

Required Evidence:
Login time
Source IP
Destination host
Login result
Historical behavior

Query:
Search authentication events.

Result:
Multiple unusual login events.

Next:
Pivot into source IP and host.
```




28. Time-Based Investigation

Always think about:

Before
During
After

If an alert occurred at:

15:00

Investigate:

Before:
What happened before the alert?

During:
What happened when the alert triggered?

After:
What happened after the alert?

This can reveal the attack chain or explain why the detection fired.



29. Baseline Analysis

You need to understand normal behavior.

Example:

```
User normally:  
Login 09:00-18:00  
  
Observed:  
Login at 03:30
```

This is unusual.

But you should investigate context:

```
Was the user on-call?  
Was it a scheduled task?  
Was VPN involved?  
Was the device known?  
Was there an approved activity?
```

Baseline is useful, but unusual behavior is not automatically malicious.



30. Statistical Investigation

Queries can help identify unusual volumes.

Example:

Normal:
20 events/hour

Current:
800 events/hour

This may indicate:

==> Misconfiguration

==> System issue

==> Scanning

==> Attack activity

==> Logging problem

The query identifies the anomaly.

Investigation determines the cause.



31. Example — Investigating SSH Brute Force

Alert:

Multiple SSH Authentication Failures

Step 1

Identify:

Source IP
Destination Host
Username
Time

Step 2

Search all events from the source IP.

Source IP
↓
All related authentication events

Step 3

Check whether a successful login occurred.

Failed
Failed
Failed
Successful

This changes the investigation priority.

Step 4

Search activity after successful login.

```
Login
↓
Process
↓
Privilege
↓
Network
```

Step 5

Determine scope.

```
One host?
Multiple hosts?
One account?
Multiple accounts?
```

Step 6

Document findings.



32. Example — Windows Account Investigation

Alert:

Multiple Authentication Failures

Search:

```
username
source_ip
hostname
event_id
logon_type
timestamp
```

Then ask:

```
Was there a successful login?
Was the account privileged?
Was the source device known?
Were other systems targeted?
Did privilege changes occur?
```



33. Example — Suspicious Process Investigation

Alert:

Suspicious Process Activity

Collect:

Process
Parent Process
User
Host
Command Line
Timestamp
Network Activity

Then search:

What happened before process creation?
What happened after?
Were similar processes observed elsewhere?
Did the same user perform related activity?

34. IOC Investigation

Indicators can include:

```
IP
Domain
URL
Hash
Email
Username
Hostname
```

Suppose you receive a suspicious hash.

Search:

```
hash = X
```

Then:

```
Which hosts?
Which users?
When observed?
How many times?
What process?
What network connections?
```

This determines the scope of the IOC.



35. Search Broad → Narrow

A useful investigation strategy is:

```
Broad Search
  ↓
Identify Pattern
  ↓
Narrow Search
  ↓
Pivot
  ↓
Correlate
  ↓
Confirm
```

Do not begin with an extremely narrow query if you don't yet understand the event.

You may accidentally exclude important evidence.



36. Common Query Mistakes

Mistake 1 — Wrong Time Range

You may miss the initial activity.

Mistake 2 — Wrong Field

For example:

```
src_ip
```

when your normalized field is:

```
source.ip
```

Mistake 3 — Case Sensitivity

Some systems or fields may behave differently depending on query syntax.

Mistake 4 — Searching Only One Event

The surrounding events may contain the real story.

Mistake 5 — Ignoring Normal Activity

You may incorrectly classify legitimate administrative activity.

Mistake 6 — Overly Broad Queries

Searching everything can create:

```
Slow Query  
Huge Result  
Analyst Confusion
```




37. Query Performance

A SIEM Engineer must care about query performance.

Poor queries can create:

- High CPU
- High Memory
- Slow Search
- High Storage/Compute Cost
- Delayed Investigation

Good practices:

- ==> Use appropriate time ranges
 - ==> Filter early
 - ==> Search relevant indexes/data sources
 - ==> Select necessary fields
 - ==> Avoid unnecessary expensive operations
 - ==> Use indexed/search-optimized fields where applicable
 - ==> Test queries before production use
-



38. Query Language Skills

A SIEM Engineer should become comfortable with the query language of their platform.

Splunk

SPL

Microsoft Sentinel

KQL

QRadar

AQL

Elastic

KQL / ES | QL / Query DSL, depending on the workflow

OpenSearch

DQL / Query DSL, depending on the interface and configuration

The syntax differs.

The investigation mindset remains similar.



39. Query Skill Progression

Learn in this order:

1. Basic Search
↓
2. Filtering
↓
3. Fields
↓
4. Sorting
↓
5. Aggregation
↓
6. Grouping
↓
7. Time Windows
↓
8. Correlation
↓
9. Sub-search / Advanced Logic
↓
10. Threat Hunting

Do not try to memorize everything at once.

Understand the logic first.



40. Practical SIEM Lab

Build a lab with:

```
Windows VM
Linux VM
Wazuh / SIEM
Firewall Logs
Authentication Logs
Process Logs
```

Create alerts and investigate them.

Lab 1

Search failed SSH authentication.

Lab 2

Find top source IPs.

Lab 3

Find top targeted usernames.

Lab 4

Find successful login after multiple failures.

Lab 5

Build a timeline around the login.

Lab 6

Pivot from IP → User → Host.

Lab 7

Search related alerts.

Lab 8

Document the final verdict.



41. Investigation Checklist

Use this checklist during an alert investigation:

- [] Read the alert carefully
- [] Understand why the detection triggered
- [] Record timestamp
- [] Identify source IP
- [] Identify destination
- [] Identify username
- [] Identify hostname
- [] Check related events
- [] Search before the alert
- [] Search after the alert
- [] Check authentication activity
- [] Check process activity
- [] Check network activity
- [] Check privilege changes
- [] Check threat intelligence
- [] Determine affected assets
- [] Determine affected accounts
- [] Determine scope
- [] Decide verdict according to SOC process
- [] Document evidence
- [] Escalate if required

[] Provide detection feedback

42. Complete Investigation Flow

Remember this:

```
Alert
↓
Validate
↓
Identify
↓
Search
↓
Filter
↓
Pivot
↓
Correlate
↓
Build Timeline
↓
Enrich
↓
Determine Scope
↓
Analyze Evidence
↓
Verdict
↓
Document
↓
Escalate / Close
```

This is one of the most important workflows for a SOC environment.



43. SIEM Engineer Interview Questions

Q1. What is a SIEM query?

Answer:

A SIEM query is a structured search used to retrieve, filter, aggregate, correlate, or analyze security data stored in a SIEM platform.

Q2. What is SPL?

Answer:

SPL stands for Search Processing Language and is used in Splunk to search and analyze machine-generated data.

Q3. What is KQL?

Answer:

KQL stands for Kusto Query Language and is used across Microsoft security and Azure data-analysis workflows, including Microsoft Sentinel.

Q4. What is pivoting?

Answer:

Pivoting means using information discovered from one event to search for related activity.

For example:

IP → User → Host → Process → Network

Q5. Why is time range important during investigation?

Answer:

Time range helps identify what happened before, during, and after an alert. A narrow time range may miss important related events.

Q6. What is threat hunting?

Answer:

Threat hunting is a proactive process where analysts develop hypotheses and search security telemetry for evidence of suspicious activity that may not have generated an alert.

Q7. How do you investigate a suspicious IP?

Answer:

I would identify where the IP appeared, which hosts communicated with it, which users were involved, when the activity occurred, what services or protocols were used, whether related events exist, and whether threat intelligence provides additional context.

Q8. What is correlation?

Answer:

Correlation means connecting multiple related events or data sources to identify a larger security pattern.

Q9. How do you investigate a suspicious login?

Answer:

I would examine the username, source IP, destination host, authentication result, timestamp, login type, historical activity, related authentication events, privilege changes, endpoint activity, and network activity.

Q10. What is the difference between searching and investigating?

Answer:

Searching retrieves relevant data. Investigation analyzes that data, correlates related events, builds context, determines scope, and supports a security verdict.



44. Final Takeaway

A SIEM Engineer should not only know how to write queries.

They should know **how to think during an investigation**.

The key mindset is:

Don't just search for events.

Understand the story behind the events.

A strong investigation follows:

Question → Query → Evidence → Context → Correlation → Timeline → Verdict

The most important skills are:

==> SPL

==> KQL

==> Elasticsearch/OpenSearch Query

==> Filtering

==> Aggregation

==> Correlation

==> Pivoting

==> Timeline Analysis

==> Threat Hunting

==> IOC Investigation

==> Evidence-based Investigation

SIEM Engineer Learning Series

- #01 — Log Management
- #02 — SIEM Platforms
- #03 — Log Integration
- #04 — Detection Engineering
- #05 — Query & Investigation ← You are here
- #06 — SOC Operations → Next
- #07 — Threat Intelligence
- #08 — Automation & SOAR
- #09 — Dashboard & Reporting
- #10 — SIEM Infrastructure
- #11 — SIEM Architecture Design
- #12 — SIEM Capacity Planning
- #13 — SIEM Performance Tuning
- #14 — Advanced Detection Engineering
- #15 — Threat Hunting with SIEM
- #16 — SIEM Data Quality & Troubleshooting
- #17 — Hands-On SIEM: Multi-Platform Practical Guide

Navigation

← Previous Article: #04 — Detection Engineering

SIEM Engineer Index

Next Article: #06 — SOC Operations →

SIEM Engineer #06 — SOC Operations

Introduction

A SIEM can generate thousands of alerts.

But an alert by itself is not an incident.

A SOC team needs a structured process to understand:

==> What happened?

==> Is the alert valid?

==> Which user or system is affected?

==> What is the scope?

==> What action should be taken?

==> Does the incident need escalation?

This is where **SOC Operations** becomes important.

The core SOC workflow is:

Detect → Validate → Investigate → Contain → Eradicate → Recover → Document

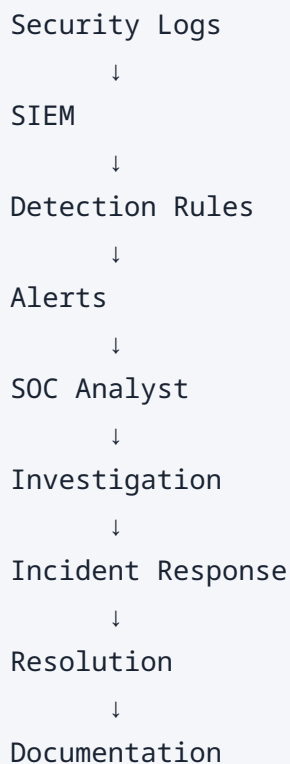
A SIEM Engineer works closely with SOC Analysts to make this workflow effective.



1. What is SOC Operations?

SOC Operations means the day-to-day process of monitoring, detecting, investigating, responding to, and documenting cybersecurity events and incidents.

A typical SOC contains:



The SOC continuously monitors the organization's security environment.

2. Main Responsibilities of a SOC

A SOC generally performs activities such as:

- ==> Security monitoring
 - ==> Alert triage
 - ==> Incident investigation
 - ==> Threat detection
 - ==> Threat hunting
 - ==> Incident response
 - ==> Threat intelligence enrichment
 - ==> Vulnerability awareness
 - ==> Security reporting
 - ==> Detection tuning
 - ==> Incident documentation
 - ==> Continuous improvement
-



3. SOC Team Structure

A traditional SOC may have several levels.

```
L1 Analyst
  ↓
L2 Analyst
  ↓
L3 Analyst
  ↓
SOC Engineer / Detection Engineer
  ↓
Incident Response / Threat Hunting
```

The exact structure varies between organizations.



4. SOC L1 Analyst

L1 usually focuses on initial alert handling.

Responsibilities can include:

==> Monitor alerts

==> Validate alerts

==> Perform initial investigation

==> Identify false positives

==> Collect basic evidence

==> Follow runbooks

==> Escalate complex incidents

Example:

```
SIEM Alert
  ↓
L1 Analyst
  ↓
Validate
  ↓
Initial Investigation
  ↓
Close / Escalate
```



5. SOC L2 Analyst

L2 generally performs deeper investigation.

Responsibilities may include:

==> Advanced log analysis

==> Correlation

==> Threat hunting

==> Incident scoping

==> Root-cause investigation

==> Detection tuning

==> Supporting containment

==> L1 guidance

Example:

L1 Escalation

↓

L2 Investigation

↓

Deep Analysis

↓

Scope

↓

Containment Recommendation



6. SOC L3 / Senior Security Roles

L3-level work may involve:

==> Advanced threat hunting

==> Malware analysis

==> Detection engineering

==> Incident response

==> Advanced forensic analysis

==> Complex attack-chain investigation

==> Security architecture

==> Engineering improvements

Not every organization uses the same L1/L2/L3 structure.



7. SIEM Engineer in SOC Operations

A SIEM Engineer is not simply a person who manages a SIEM server.

They may be responsible for:

==> Log onboarding

==> Parser management

==> Detection rules

==> Correlation rules

==> Alert quality

==> SIEM health

==> Query performance

==> Dashboards

==> Integrations

==> Automation

==> Data retention

==> Troubleshooting

==> Detection coverage

The SIEM Engineer enables analysts to work effectively.



8. Event vs Alert vs Incident

These terms are often confused.

Event

An event is a recorded activity.

Example:

```
User login failed
```

Alert

An alert is generated when a detection identifies activity that meets defined conditions.

```
Multiple failed logins
      ↓
Detection Rule
      ↓
Alert
```

Incident

An incident is a security case requiring investigation and potentially response.

Multiple alerts

+

Evidence

+

Impact / Risk

↓

Security Incident

An alert does not automatically mean an incident.

9. SOC Alert Lifecycle

A practical alert lifecycle is:

```
Alert Created
  ↓
Assigned
  ↓
Triaged
  ↓
Validated
  ↓
Investigated
  ↓
Verdict
  ↓
Escalated / Contained / Closed
  ↓
Documented
```

This workflow should be clearly defined inside a SOC.



10. Step 1 — Detect

The SIEM identifies suspicious activity.

Example:

```
20 failed logins  
from same source IP  
within 5 minutes
```

Detection rule:

```
Authentication Failure  
+  
Threshold  
+  
Time Window
```

Result:

```
Security Alert
```



11. Step 2 — Validate

The analyst checks whether the alert is valid.

Questions:

==> Did the event actually occur?

==> Are the logs correct?

==> Is the source known?

==> Is the user legitimate?

==> Is this expected administrative activity?

==> Is the detection working correctly?

Validation prevents analysts from spending unnecessary time on bad data or broken detections.



12. Step 3 — Investigate

After validation, investigate the activity.

Look at:

- User
- Source IP
- Destination
- Host
- Process
- Timestamp
- Authentication
- Network Activity
- Related Alerts
- Threat Intelligence

Then build the timeline.

- Before
 - ↓
- During
 - ↓
- After



13. Step 4 — Determine Scope

One of the most important SOC questions is:

“How much of the environment is affected?”

Check:

==> One user?

==> Multiple users?

==> One host?

==> Multiple hosts?

==> One IP?

==> Multiple IPs?

==> One application?

==> Multiple applications?

Example:

```
Alert
↓
Host A
↓
User A
↓
Host B
↓
User B
```

The scope may be larger than the original alert.

14. Step 5 — Contain

Containment attempts to limit further impact.

Depending on the incident and organizational procedures, actions may include:

- ==> Isolating an affected endpoint
- ==> Blocking a malicious network indicator
- ==> Disabling a compromised account
- ==> Revoking sessions
- ==> Restricting network access
- ==> Applying emergency firewall controls

Containment should be authorized and carefully documented.



15. Step 6 — Eradicate

Eradication means removing the cause or malicious artifacts from the environment.

Examples can include:

- ==> Removing malicious persistence
- ==> Removing unauthorized accounts
- ==> Removing malicious files
- ==> Patching exploited vulnerabilities
- ==> Rotating compromised credentials
- ==> Removing unauthorized configurations

The exact action depends on the incident.

16. Step 7 — Recover

Recovery returns affected systems to normal operation.

Activities may include:

- ==> Restoring systems
- ==> Rebuilding compromised hosts
- ==> Restoring clean backups
- ==> Validating security controls
- ==> Monitoring affected systems
- ==> Confirming normal operation

Recovery should not simply mean:

System is working

You should also confirm:

System is secure
+
Monitoring is active
+
Root cause is addressed



17. Step 8 — Document

Documentation is a critical SOC responsibility.

Record:

What happened?
When?
Which system?
Which user?
Which IP?
What evidence?
What actions?
Who performed them?
What was the verdict?
What was the impact?
What was the root cause?
What recommendations?

A well-documented incident helps future investigations.



18. Complete Incident Response Flow

Remember:

```
Detect
↓
Validate
↓
Investigate
↓
Scope
↓
Contain
↓
Eradicate
↓
Recover
↓
Document
↓
Lessons Learned
↓
Improve Detection
```

This is one of the most important workflows for a SOC Engineer.

19. Alert Triage

Triage means quickly determining what an alert represents and what should happen next.

During triage, ask:

What triggered the alert?
Who is involved?
Which asset?
When?
Where?
Is it expected?
What is the potential impact?
Is there related activity?
Does it require escalation?

The goal is to prioritize investigation appropriately.



20. Alert Priority

Organizations may classify alerts using:

Low
Medium
High
Critical

But priority should be based on organizational criteria.

Consider:

==> Asset importance

==> User privilege

==> Detection confidence

==> Potential impact

==> Scope

==> Business context

==> Evidence

For example, an alert involving a critical production server may require different handling from an identical alert on a test machine.



21. Severity vs Priority vs Confidence

These concepts should not be confused.

Severity

Potential seriousness of the security activity.

Confidence

How strongly the evidence supports the detection.

Priority

How urgently the SOC should handle the case.

Example:

```
High Severity
+
High Confidence
+
Critical Asset
=
High Investigation Priority
```

The exact prioritization model should be defined by the organization.



22. Ticket Management

A SOC should track incidents using a case or ticketing system.

A case may contain:

- Case ID
- Title
- Severity
- Priority
- Status
- Assigned Analyst
- Affected Asset
- Affected User
- Detection
- Evidence
- Timeline
- Comments
- Actions
- Verdict
- Resolution

Common statuses include:

- New
- Assigned
- In Progress
- Waiting
- Escalated
- Contained
- Resolved
- Closed

23. Alert Assignment

When an alert is generated:

```
Alert
↓
Queue
↓
Analyst Assignment
↓
Investigation
```

Assignment can be based on:

==> Shift

==> Analyst skill

==> Severity

==> Workload

==> Incident type

Good assignment prevents important alerts from being ignored.

24. Escalation

Escalation is necessary when the current analyst or team needs additional expertise or authority.

Examples:

L1

↓

L2

or:

SOC

↓

Incident Response

or:

Security Team

↓

Network Team

or:

Security Team

↓

System Owner

Escalation should include useful evidence instead of simply saying:

“Please investigate.”



25. Good Escalation Information

A useful escalation should include:

Incident Summary

Detection Name

Timestamp

Affected Host

Affected User

Source IP

Destination IP

Relevant Events

Timeline

Investigation Performed

Evidence

Current Impact

Actions Already Taken

Recommended Next Step

This makes the next analyst's job much easier.



26. Root Cause Analysis

After an incident, ask:

“Why did this happen?”

Possible causes:

==> Stolen credentials

==> Misconfiguration

==> Vulnerability

==> Weak access control

==> Malicious email

==> Exposed service

==> Insider activity

==> Compromised endpoint

Do not assume the root cause before collecting evidence.



27. Attack Chain Analysis

A security incident may contain multiple stages.

For example:

```
graph TD; A[Initial Access] --> B[Execution]; B --> C[Persistence]; C --> D[Privilege Escalation]; D --> E[Discovery]; E --> F[Lateral Movement]; F --> G[Collection]; G --> H[Command & Control];
```

Initial Access
↓
Execution
↓
Persistence
↓
Privilege Escalation
↓
Discovery
↓
Lateral Movement
↓
Collection
↓
Command & Control

MITRE ATT&CK can help analysts describe observed behaviors using common terminology.



28. SOC Runbooks

A **runbook** provides predefined instructions for handling common events.

Example:

Suspicious Login Runbook

1. Validate login event
2. Identify user
3. Identify source IP
4. Check authentication history
5. Check device
6. Check geographic context
7. Check related events
8. Determine whether activity is expected
9. Escalate if required
10. Document findings

Runbooks help maintain consistency across analysts and shifts.



29. Playbook vs Runbook

These terms are sometimes used differently by organizations.

Generally:

Runbook

Step-by-step operational instructions.

Playbook

A broader response workflow that can include decisions, automation, people, tools, and actions.

Example:

```
Alert
  ↓
Playbook
  ↓
Enrichment
  ↓
Decision
  ↓
Containment
  ↓
Ticket Update
```



30. Automation in SOC

Automation can reduce repetitive work.

Example:

```
SIEM Alert
↓
Automation
↓
Extract IP
↓
Threat Intelligence Lookup
↓
Enrich Alert
↓
Update Ticket
```

Automation should be carefully controlled.

Not every alert should automatically trigger destructive or disruptive actions.



31. Analyst Feedback

SOC analysts interact with detections every day.

Their feedback is extremely valuable.

Example:

```
Detection
↓
100 Alerts
↓
80 Benign
↓
15 Suspicious
↓
5 Confirmed
```

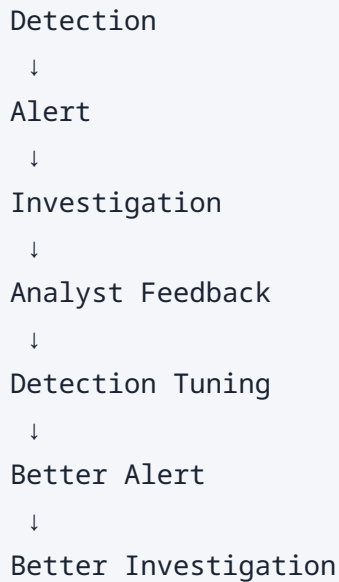
The analyst can report:

“Most alerts are generated by an approved scanner.”

The Detection Engineer can then investigate and tune the rule.

32. SOC Feedback Loop

A mature SOC operates as a continuous loop:



```
graph TD; A[Detection] --> B[Alert]; B --> C[Investigation]; C --> D[Analyst Feedback]; D --> E[Detection Tuning]; E --> F[Better Alert]; F --> G[Better Investigation];
```

Detection
↓
Alert
↓
Investigation
↓
Analyst Feedback
↓
Detection Tuning
↓
Better Alert
↓
Better Investigation

This is how SOC operations improve over time.



33. SOC Metrics

SOC teams may track metrics such as:

==> Alert volume

==> Mean Time to Detect (MTTD)

==> Mean Time to Respond (MTTR)

==> False-positive rate

==> Escalation rate

==> Incident volume

==> Detection coverage

==> Response time

==> Case closure time

Metrics should be interpreted in context rather than used in isolation.

34. MTTD

MTTD = Mean Time to Detect

It measures how long it takes to identify a security event or incident.

Conceptually:

Security Activity

↓

Time passes

↓

Detection

The smaller the detection delay, the sooner the SOC can begin investigation.



35. MTTR

MTTR = Mean Time to Respond/Resolve, depending on the organization's definition.

Always check how your organization defines the metric.

Conceptually:

```
Detection
  ↓
Investigation
  ↓
Response
  ↓
Resolution
```

MTTR helps organizations understand response efficiency.



36. SOC Shift Handover

SOC operations often run across multiple shifts.

A good handover should include:

- Open Incidents
- Pending Investigations
- High-Priority Alerts
- Affected Assets
- Actions Taken
- Expected Follow-up
- Escalations
- Important Changes

Example:

```
Case #1024

Status:
Investigation

Affected Host:
SERVER-01

Current Finding:
Suspicious authentication activity

Action:
Network team notified

Pending:
Review endpoint telemetry
```

This prevents important information from being lost between shifts.



37. Evidence Handling

During investigations, preserve relevant evidence.

Examples:

==> Log events

==> Alert details

==> Screenshots

==> Hashes

==> Network information

==> Timeline

==> Analyst notes

==> Ticket history

Maintain accurate timestamps and document actions.

For incidents requiring formal forensic handling, follow the organization's evidence-preservation procedures.



38. Least Privilege in SOC

SOC analysts should have only the permissions needed for their role.

Example:

L1 Analyst



Read / Investigate

More sensitive actions may require:

L2 / Incident Response



Containment

and administrative changes may require additional authorization.

This reduces operational risk.



39. SOC Access Control

A SOC platform should ideally support:

==> RBAC

==> MFA

==> Audit logs

==> Session management

==> Access reviews

==> Separation of duties

For example:

Analyst



Investigate

Engineer



Modify Detection

Administrator



Manage Platform



40. SOC Dashboard

A SOC dashboard may show:

- Open Alerts
- Open Incidents
- Critical Alerts
- Alert Trends
- Top Source IPs
- Top Affected Hosts
- Top Users
- Detection Health
- Incident Status
- MTTD
- MTTR

The dashboard should help analysts make decisions rather than simply display large numbers.

41. Practical SOC Lab

You can build a basic SOC lab using:

```
Windows VM
Linux VM
Wazuh
Firewall
SIEM Dashboard
Threat Intelligence
Ticket / Case Management
```

Scenario

Generate several authentication failures in your authorized lab.

Then follow:

```
Event
↓
SIEM Detection
↓
Alert
↓
L1 Triage
↓
Investigation
↓
Scope
↓
Verdict
↓
Containment Decision
↓
Documentation
```



42. Realistic SOC Investigation Example

Suppose the SIEM generates:

Alert:
Multiple Failed SSH Logins

Step 1 — Validate

Check:

Timestamp
Source IP
Destination Host
Username

Step 2 — Investigate

Search:

All activity from source IP

Step 3 — Check Successful Login

Failed
Failed
Failed
Successful

Now investigate the successful login.

Step 4 — Check Host Activity

Search:

Processes
Authentication
Privilege changes
Network connections

Step 5 — Determine Scope

One host?
Multiple hosts?
One account?
Multiple accounts?

Step 6 — Decide Response

Based on evidence and organizational procedures:

Close
or
Escalate
or
Contain

Step 7 — Document

Record:

Finding
Evidence
Timeline
Actions
Verdict



43. SOC and SIEM Relationship

Think of it this way:

SIEM
=
Security Data + Detection + Investigation Platform

SOC:

People
+
Process
+
Technology

The SIEM is one important component of the SOC.

A strong SOC needs all three.

44. SIEM Engineer + SOC Analyst

The relationship is very important.

SOC Analyst says:

“This detection generates too many false positives.”

SIEM Engineer:

```

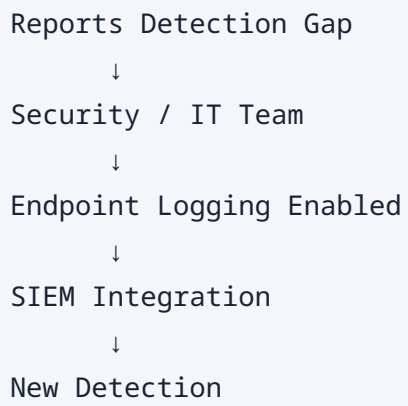
Investigates Rule
    ↓
Checks Data
    ↓
Tunes Logic
    ↓
Tests
    ↓
Deploys
    
```

Another example:

SIEM Engineer says:

“We are missing endpoint telemetry.”

SOC:



This creates a continuous security improvement cycle.



45. SOC Operations Checklist

- [] Monitor SIEM
- [] Review new alerts
- [] Validate alert
- [] Assign alert
- [] Investigate
- [] Identify affected assets
- [] Identify affected users
- [] Search related events
- [] Build timeline
- [] Check threat intelligence
- [] Determine scope
- [] Determine verdict
- [] Escalate if required
- [] Contain when authorized
- [] Eradicate root cause
- [] Recover affected systems
- [] Document evidence
- [] Close incident
- [] Perform lessons learned
- [] Improve detection



46. SIEM Engineer / SOC Interview Questions

Q1. What is SOC Operations?

Answer:

SOC Operations is the continuous process of monitoring security telemetry, detecting suspicious activity, investigating alerts, responding to incidents, and documenting the results.

Q2. What is the difference between an event, alert, and incident?

Answer:

An event is recorded activity. An alert is generated when a detection identifies activity matching defined conditions. An incident is a security case requiring investigation and potentially response.

Q3. Explain the incident response lifecycle.

Answer:

A practical SOC workflow is:

Detect → Validate → Investigate → Scope → Contain → Eradicate → Recover → Document → Lessons Learned

Q4. What do you do when a SIEM alert is triggered?

Answer:

First, I validate the alert and understand why it triggered. Then I identify the affected user, host, IP, and timestamp, search related events, build a timeline, determine the scope and verdict, and escalate or respond according to the SOC procedure.

Q5. How do you handle false positives?

Answer:

I investigate why the detection triggered, determine whether the activity is expected, and provide feedback to the Detection Engineer. The rule can then be tuned using better conditions, thresholds, context, or controlled exceptions.

Q6. When should an alert be escalated?

Answer:

An alert should be escalated when it requires deeper technical expertise, additional authority, incident-response support, broader investigation, or involvement from another team.

Q7. What information should be included in an escalation?

Answer:

The escalation should contain the alert summary, timestamps, affected users and assets, source and destination information, relevant events, timeline, investigation findings, evidence, actions already taken, and recommended next steps.

Q8. What is MTTD?

Answer:

MTTD means Mean Time to Detect. It measures the average time between a security event or incident occurring and its detection, according to the organization's measurement definition.

Q9. What is MTTR?

Answer:

MTTR is a time-based response metric. Organizations may define it as Mean Time to Respond or Mean Time to Resolve/Recover, so the exact definition should be confirmed before comparing metrics.

Q10. What is a SOC runbook?

Answer:

A runbook is a documented set of operational steps that helps analysts consistently handle a particular alert or incident type.

Q11. Why is documentation important in SOC operations?

Answer:

Documentation preserves evidence, explains decisions and actions, supports handovers and escalations, provides an audit trail, and helps improve future detection and response.

Q12. What is the role of a SIEM Engineer in a SOC?

Answer:

A SIEM Engineer manages and improves the SIEM ecosystem, including log collection, integrations, parsing, detection rules, alert quality, dashboards, performance, data retention, troubleshooting, and support for SOC investigations.



47. Important SOC Principles

Principle 1 — Alert ≠ Incident

Never treat every alert as a confirmed incident.

Principle 2 — Evidence Before Conclusion

Do not make conclusions without supporting evidence.

Principle 3 — Context Matters

An event must be understood in its environment.

Principle 4 — Document Everything Important

If an important action or finding is not documented, it becomes difficult to verify later.

Principle 5 — Learn From Every Incident

Every investigation can improve:

- Detection
- Process
- Runbook
- Automation
- Training



48. Final Takeaway

A SOC is not just a room full of analysts watching dashboards.

A mature SOC combines:

People
+
Process
+
Technology
+
Data
+
Detection
+
Investigation
+
Response

For a SIEM Engineer, understanding SOC Operations is essential because the SIEM exists to support the security operation.

Remember the core flow:

Detect → Validate → Investigate → Scope → Contain → Eradicate → Recover → Document → Improve

If you understand this workflow and can explain it with real examples during an interview, you demonstrate practical SOC knowledge—not just SIEM tool knowledge.



SIEM Engineer Learning Series

#01 — Log Management

- #02 — SIEM Platforms
 - #03 — Log Integration
 - #04 — Detection Engineering
 - #05 — Query & Investigation
 - #06 — SOC Operations ← You are here
 - #07 — Threat Intelligence → Next
 - #08 — Automation & SOAR
 - #09 — Dashboard & Reporting
 - #10 — SIEM Infrastructure
 - #11 — SIEM Architecture Design
 - #12 — SIEM Capacity Planning
 - #13 — SIEM Performance Tuning
 - #14 — Advanced Detection Engineering
 - #15 — Threat Hunting with SIEM
 - #16 — SIEM Data Quality & Troubleshooting
 - #17 — Hands-On SIEM: Multi-Platform Practical Guide
-

Navigation

← Previous Article: #05 — Query & Investigation

SIEM Engineer Index

Next Article: #07 — Threat Intelligence →

SIEM Engineer #07 — Threat Intelligence

Introduction

A SIEM can tell us:

```
Which IP connected?  
Which user logged in?  
Which host generated the event?  
What process executed?  
When did it happen?
```

But sometimes we need additional context.

For example:

```
203.x.x.x
```

A SIEM may know that an internal system communicated with this IP.

But Threat Intelligence may provide additional information such as:

```
Is this IP associated with known malicious activity?  
Has it been reported before?  
What type of threat has it been associated with?  
When was it last observed?
```

This additional context can make SIEM investigations more useful.

The core concept is:

Security Telemetry + Threat Intelligence → Better Detection & Investigation



1. What is Threat Intelligence?

Cyber Threat Intelligence (CTI) is information about threats that helps an organization understand, detect, investigate, and respond to security risks.

Threat Intelligence can provide information about:

==> Malicious IP addresses

==> Malicious domains

==> URLs

==> File hashes

==> Malware families

==> Threat actors

==> Attack techniques

==> Vulnerabilities

==> Campaigns

==> Indicators of compromise

But raw information is not automatically useful intelligence.

The information needs:

Context

+

Validation

+

Relevance

+

Timeliness

2. Why Threat Intelligence Matters in SIEM

Suppose the SIEM detects:

```
Internal Host
  ↓
Connection
  ↓
External IP
```

Without intelligence:

```
Unknown external IP
```

With enrichment:

```
External IP
  ↓
Threat Intelligence Lookup
  ↓
Known suspicious indicator
  ↓
Additional context
```

This can help the analyst prioritize and investigate the event.



3. What is an IOC?

IOC = Indicator of Compromise

Common IOCs include:

- IP Address
- Domain
- URL
- File Hash
- Email Address
- Hostname

Examples:

- IP:
203.x.x.x
- Domain:
example-suspicious-domain
- Hash:
SHA256 value

An IOC is an indicator—not necessarily proof of compromise by itself.



4. IOC ≠ Incident

This is a very important SOC concept.

Suppose your threat feed contains:

203.x.x.x

Your firewall logs show:

Internal Host → 203.x.x.x

This is useful evidence.

But you still need to investigate:

==> Which host communicated?

==> Which process initiated the connection?

==> Which user was logged in?

==> When did it happen?

==> Was the indicator still relevant?

==> Was the connection blocked?

==> Was there any additional suspicious activity?

Therefore:

IOC Match ≠ Automatically Confirmed Compromise



5. Types of Threat Intelligence

Threat Intelligence is commonly discussed in several categories.

Strategic Intelligence

Focuses on high-level risks and trends.

Audience:

==> Management

==> Security leadership

==> Risk teams

Example:

Increasing ransomware activity
against a particular industry.

Tactical Intelligence

Focuses on adversary techniques and behaviors.

Useful for:

==> Detection Engineering

==> Threat Hunting

==> SOC teams

Example:

Adversaries commonly use specific techniques for credential access.

Operational Intelligence

Focuses on campaigns and ongoing threat activity.

Example:

Current campaign
Target
Infrastructure
Attack behavior
Timeline

Technical Intelligence

Focuses heavily on technical indicators.

Examples:

IP
Domain
URL
Hash
Email

This is especially useful for SIEM enrichment and automated correlation.



6. Threat Intelligence Lifecycle

Threat Intelligence should follow a lifecycle.

```
graph TD; A[Direction] --> B[Collection]; B --> C[Processing]; C --> D[Analysis]; D --> E[Dissemination]; E --> F[Feedback]; F --> G[Improvement];
```

Direction
↓
Collection
↓
Processing
↓
Analysis
↓
Dissemination
↓
Feedback
↓
Improvement

Let's understand it.

7. Step 1 — Intelligence Requirements

First ask:

What information does the organization actually need?

Examples:

==> Which threats target our industry?

==> Which malicious IPs are contacting our infrastructure?

==> Which vulnerabilities are actively being exploited?

==> Which threat behaviors should our SOC detect?

Without clear requirements, teams may collect large amounts of irrelevant information.



8. Step 2 — Collection

Threat information can come from multiple sources.

Examples:

==> Threat intelligence platforms

==> Commercial feeds

==> Open-source intelligence

==> Security vendors

==> CERT/CSIRT organizations

==> Internal incident data

==> Malware analysis

==> Security research

The source should be evaluated for reliability and relevance.



9. Step 3 — Processing

Raw intelligence often needs processing.

Example:

```
Raw Feed
  ↓
Remove duplicates
  ↓
Normalize fields
  ↓
Validate format
  ↓
Add timestamps
  ↓
Assign confidence
  ↓
Store
```

For an IP feed:

```
IP
First Seen
Last Seen
Source
Confidence
Threat Type
Expiration
```



10. Step 4 — Analysis

Analysis converts collected information into useful intelligence.

Suppose a feed contains:

```
IP A  
IP B  
IP C
```

The analyst asks:

```
Are these relevant to us?  
Have our systems communicated with them?  
What activity is associated with them?  
How recent is the intelligence?  
How reliable is the source?
```

This is where context becomes important.



11. Step 5 — Dissemination

Useful intelligence should reach the right people and systems.

For example:

Threat Intelligence



SIEM



SOC Analyst

or:

Threat Intelligence



Firewall

or:

Threat Intelligence



EDR

The format and level of detail should match the consumer.

12. Step 6 — Feedback

SOC analysts should provide feedback.

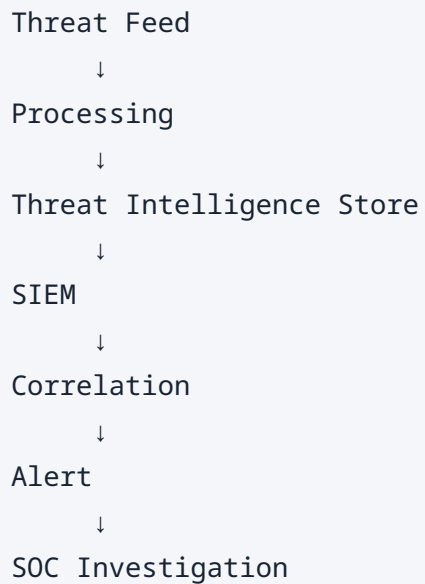
Example:

```
Feed Indicator
  ↓
Matched 500 alerts
  ↓
Most were irrelevant
  ↓
Analyst Feedback
  ↓
Feed Filtering / Tuning
```

This improves intelligence quality.

13. Threat Intelligence in SIEM

The basic architecture is:



The SIEM can compare observed events against known indicators.

14. IOC Enrichment

Suppose an alert contains:

Destination IP:
203.x.x.x

The SIEM can enrich the event:

Destination IP:
203.x.x.x

Threat Intelligence:
Known suspicious indicator

Source:
Threat Feed A

Last Seen:
Recent

Confidence:
High

Now the analyst has more context.



15. IOC Correlation

Imagine your firewall produces:

Internal Host → External IP

Threat Intelligence contains:

External IP = Known Indicator

Correlation:

Firewall Event
+
Threat Intelligence Match
↓
Security Alert

This is one of the most common uses of Threat Intelligence in SIEM.



16. IP Reputation

IP reputation attempts to provide context about an IP address.

Possible classifications include:

- Known malicious
- Suspicious
- Unknown
- Benign

But reputation information must be treated carefully.

An IP can be:

==> Shared

==> Dynamic

==> Reassigned

==> Compromised temporarily

==> Used by legitimate services

Therefore, reputation should be one part of the investigation.



17. Domain Intelligence

Similar analysis can be performed for domains.

Example:

```
Internal Host
  ↓
DNS Request
  ↓
Suspicious Domain
```

Investigate:

==> Which host requested it?

==> Which user?

==> When?

==> Was there an HTTP/HTTPS connection afterward?

==> Was the domain recently registered?

==> Is it present in trusted intelligence sources?

18. URL Intelligence

URLs can be useful in:

==> Phishing investigation

==> Web security monitoring

==> Email security

==> Proxy analysis

Example:

```

Email
↓
URL
↓
User Click
↓
Proxy Event
↓
Endpoint Event
  
```

Correlating these events can provide much more context than looking at the URL alone.

19. Hash Intelligence

File hashes can be used to identify known files.

Common hash types include:

MD5
SHA-1
SHA-256

For security investigations, SHA-256 is commonly preferred for identifying files.

Example:

```
Endpoint
  ↓
File Hash
  ↓
Threat Intelligence Lookup
  ↓
Known / Unknown
```

Again:

Unknown does not mean safe.

And:

Known malicious does not replace investigation of the surrounding activity.



20. Email Threat Intelligence

Email investigations can involve:

```
Sender
Recipient
Domain
URL
Attachment Hash
Source IP
Message ID
```

A SIEM can correlate:

```
Email
↓
URL
↓
DNS
↓
Proxy
↓
Endpoint
```

This can help analysts understand the complete activity chain.



21. Threat Intelligence Confidence

Not all intelligence sources are equally reliable.

A useful intelligence record can contain:

```
Indicator
Source
First Seen
Last Seen
Confidence
Threat Type
Expiration
```

For example:

```
Indicator:
203.x.x.x

Source:
Feed-A

Confidence:
High

Last Seen:
Recent
```

Confidence should come from the source methodology or your organization's intelligence process.

Do not invent confidence values simply to make an alert look stronger.



22. Freshness Matters

Threat intelligence changes over time.

An indicator that was malicious months ago may not have the same relevance today.

Therefore consider:

```
First Seen
Last Seen
Expiration
Current Status
```

Example:

```
Indicator
↓
Old intelligence
↓
Current investigation
↓
Validate freshness
```

This is especially important for IP and domain reputation.



23. Threat Feed Quality

A feed should not be judged only by its size.

For example:

Feed A:
1,000,000 indicators

does not automatically mean it is better than:

Feed B:
20,000 high-quality indicators

Important factors include:

==> Accuracy

==> Relevance

==> Freshness

==> Confidence

==> Coverage

==> Update frequency

==> False-positive rate

==> Data format

==> Integration capability



24. Threat Intelligence False Positives

Suppose an intelligence feed identifies:

IP = Suspicious

Your SIEM generates:

500 alerts

But investigation shows:

Most traffic belongs to a legitimate cloud service.

Possible reasons:

- ==> Shared infrastructure
- ==> Stale intelligence
- ==> Poor feed quality
- ==> Incorrect classification
- ==> Legitimate infrastructure reuse

The solution is investigation and controlled tuning—not blindly trusting or ignoring the feed.



25. Internal Intelligence

Not all Threat Intelligence comes from external feeds.

Your organization generates valuable intelligence from its own incidents.

Examples:

Previously compromised IP
Compromised account
Malicious hash
Suspicious domain
Observed attack behavior
Internal attacker infrastructure

This can become:

Internal Threat Intelligence



26. Internal IOC Lifecycle

Example:

```
Incident
↓
IOC Discovered
↓
Validate
↓
Document
↓
Add to Intelligence Store
↓
Correlate Future Events
↓
Detect Reuse
```

This creates organizational memory.



27. Threat Intelligence Platform

A **TIP** can help manage threat intelligence.

Common functions include:

==> IOC storage

==> Enrichment

==> Relationships

==> Feeds

==> Confidence

==> Expiration

==> Sharing

==> Investigation

==> Automation

A TIP can integrate with the SIEM.

Architecture:

Threat Sources



TIP



Enrichment



SIEM



Detection



SOC

28. SIEM Threat Intelligence Integration

A SIEM integration may use:

```
API
STIX/TAXII
CSV
JSON
Syslog
Custom Connector
```

The integration process is:

```
Threat Source
↓
Connector
↓
Authentication
↓
Data Retrieval
↓
Parsing
↓
Normalization
↓
Validation
↓
Storage
↓
Correlation
```



29. STIX and TAXII

Two important standards in Threat Intelligence are:

STIX

Structured Threat Information Expression

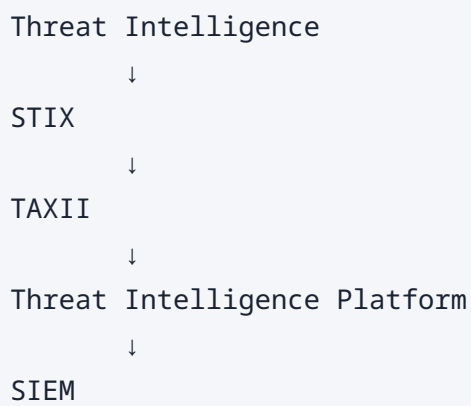
STIX provides a structured way to represent threat intelligence.

TAXII

Trusted Automated Exchange of Intelligence Information

TAXII provides mechanisms for exchanging threat intelligence.

Conceptually:





30. Threat Intelligence + Detection Engineering

Threat Intelligence can directly support Detection Engineering.

Example:

```
graph TD;
    A[Threat Research] --> B[Known Behavior];
    B --> C[Detection Hypothesis];
    C --> D[Detection Rule];
    D --> E[SIEM Alert];
```

Another example:

```
graph TD;
    A[Known IOC] --> B[IOC Detection];
    B --> C[SIEM Correlation];
    C --> D[Alert];
```

Threat Intelligence therefore provides input into the detection lifecycle.

31. Threat Intelligence + Threat Hunting

Threat Intelligence can also create hunting hypotheses.

Example:

Threat Intelligence:
Specific malicious infrastructure observed

Hunt:

Search historical DNS
Search proxy logs
Search firewall logs
Search endpoint logs

Then:

Indicator Found?
↓
Yes → Investigate
No → Continue / Document



32. Historical Search

When you discover a new IOC, do not only search current events.

If retention allows, search historical telemetry.

Example:

New IOC

↓

Search last 30 days

↓

Search last 90 days

↓

Identify previous observations

This may reveal earlier activity.

The appropriate lookback period depends on the environment and investigation requirements.



33. Threat Intelligence Correlation Across Data Sources

One indicator can appear in many places.

Example:

```
IP
↓
Firewall
↓
DNS
↓
Proxy
↓
EDR
↓
SIEM
```

Another:

```
Hash
↓
Email
↓
Endpoint
↓
File Monitoring
↓
EDR
```

Correlation across multiple sources can provide stronger context.



34. Threat Intelligence and MITRE ATT&CK

Threat Intelligence may describe:

Adversary
Campaign
Technique
Sub-technique
Infrastructure
Malware

Detection Engineers can use this information to identify detection opportunities.

For example:

Threat Research
↓
Observed Technique
↓
Telemetry Requirement
↓
Detection
↓
MITRE ATT&CK Mapping

This connects intelligence with detection coverage.



35. Automated Enrichment

Automation can enrich alerts automatically.

Example:

```
SIEM Alert
  ↓
Extract IP
  ↓
Threat Intelligence API
  ↓
Reputation / Context
  ↓
Update Alert
  ↓
SOC Analyst
```

This saves analyst time.

But API integrations must handle:

==> Authentication

==> Rate limits

==> Timeouts

==> API failures

==> Invalid responses

==> Data freshness

==> Duplicate data



36. Security of Threat Intelligence Integrations

Protect your integrations.

Use:

==> Secure API credentials

==> Secret management

==> TLS

==> Least privilege

==> API access controls

==> Monitoring

==> Rate limiting

Do not hard-code sensitive API keys inside detection rules or scripts.



37. Threat Intelligence Cost Management

Large feeds can create significant data volume.

Consider:

```
graph TD; A[1 Million Indicators] --> B[Continuous Correlation]; B --> C[High Storage / Compute];
```

Before integrating everything, determine:

==> Which indicators are relevant?

==> Which data types are needed?

==> How long should indicators be retained?

==> How often should feeds update?

==> What should expire?

Quality and relevance are more important than blindly collecting huge amounts of data.



38. Threat Intelligence Dashboard

A SIEM/TIP dashboard may show:

- Active Indicators
- New Indicators
- Expired Indicators
- IOC Matches
- Top Sources
- Top Threat Types
- High-Confidence Matches
- Recent Matches

SOC analysts may also need:

- Top Matched IPs
- Top Matched Domains
- Affected Hosts
- Affected Users
- First Seen
- Last Seen



39. Practical SIEM Lab

Build a small lab:

```
Wazuh / SIEM
+
Firewall Logs
+
DNS Logs
+
Threat Intelligence Feed
```

Lab 1 — IP Enrichment

Take a test indicator.

```
IP
↓
Threat Intelligence
↓
Enrichment
```

Lab 2 — SIEM Correlation

Generate a controlled network event in your lab.

```
Network Event
+
Threat Intelligence Match
↓
Alert
```

Lab 3 — Historical Search

Search whether the indicator appeared previously.

IOC

↓

Historical SIEM Search

↓

Previous Events

Lab 4 — Detection

Create a detection for a known test indicator.

IOC Match

↓

Detection

↓

Alert

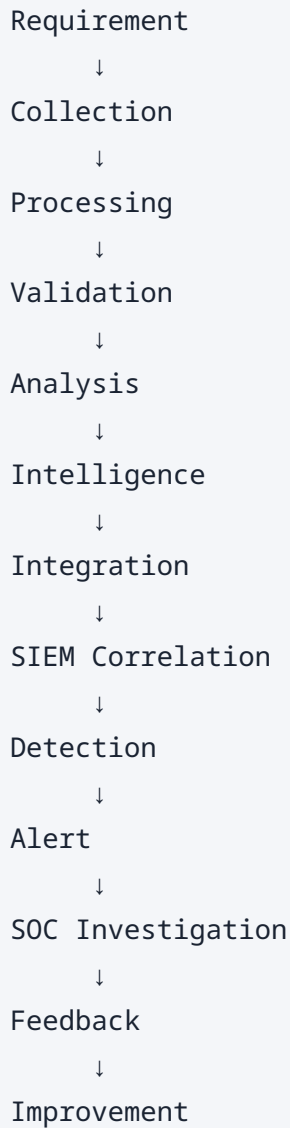


40. Threat Intelligence Checklist

- [] Define intelligence requirements
- [] Identify relevant sources
- [] Validate source reliability
- [] Collect intelligence
- [] Normalize indicators
- [] Remove duplicates
- [] Assign source information
- [] Track first seen / last seen
- [] Track expiration
- [] Track confidence
- [] Integrate with SIEM
- [] Test correlation
- [] Monitor false positives
- [] Monitor feed freshness
- [] Search historical events
- [] Provide analyst enrichment
- [] Feed intelligence into detection engineering
- [] Protect API credentials
- [] Monitor integration health
- [] Review intelligence quality

41. Complete Threat Intelligence Flow

Remember:



```
graph TD; Requirement --> Collection; Collection --> Processing; Processing --> Validation; Validation --> Analysis; Analysis --> Intelligence; Intelligence --> Integration; Integration --> SIEM_Correlation[SIEM Correlation]; SIEM_Correlation --> Detection; Detection --> Alert; Alert --> SOC_Investigation[SOC Investigation]; SOC_Investigation --> Feedback; Feedback --> Improvement;
```

Requirement
↓
Collection
↓
Processing
↓
Validation
↓
Analysis
↓
Intelligence
↓
Integration
↓
SIEM Correlation
↓
Detection
↓
Alert
↓
SOC Investigation
↓
Feedback
↓
Improvement

This is the complete relationship between Threat Intelligence and SOC operations.



42. SIEM Engineer Interview Questions

Q1. What is Threat Intelligence?

Answer:

Threat Intelligence is analyzed information about threats, adversaries, indicators, vulnerabilities, and attack behavior that helps an organization improve detection, investigation, and response.

Q2. What is an IOC?

Answer:

IOC means Indicator of Compromise. Common examples include IP addresses, domains, URLs, file hashes, and email addresses associated with suspicious or malicious activity.

Q3. Does an IOC match mean the system is compromised?

Answer:

No. An IOC match is an important investigation signal, but analysts should validate the context, timing, source reliability, related activity, and affected assets before concluding that a compromise occurred.

Q4. What is IOC enrichment?

Answer:

IOC enrichment means adding additional context to an indicator, such as threat source, confidence, first-seen/last-seen information, threat type, or reputation.

Q5. What is the difference between Threat Intelligence and an IOC?**Answer:**

An IOC is a specific indicator. Threat Intelligence is broader information that can include indicators, adversary behavior, campaigns, techniques, vulnerabilities, context, and analysis.

Q6. What is STIX?**Answer:**

STIX is a structured language and framework for representing cyber threat intelligence.

Q7. What is TAXII?**Answer:**

TAXII is a protocol and set of services designed to exchange cyber threat intelligence.

Q8. How can Threat Intelligence be integrated with SIEM?**Answer:**

It can be integrated through APIs, STIX/TAXII, files, feeds, or custom connectors. The data is then normalized, stored, enriched, and correlated with SIEM events.

Q9. How do you handle a high number of IOC false positives?**Answer:**

I would investigate the source and quality of the intelligence, check indicator freshness, understand why the matches occur, validate the affected infrastructure, and then apply controlled filtering or tuning where appropriate.

Q10. How does Threat Intelligence support Threat Hunting?

Answer:

Threat Intelligence can provide hypotheses and indicators that analysts can search across historical and current telemetry to identify related activity.

Q11. What makes a good Threat Intelligence feed?

Answer:

Important characteristics include accuracy, relevance, freshness, transparency of sourcing, useful context, appropriate confidence, manageable false positives, and reliable integration.

Q12. How can Threat Intelligence improve Detection Engineering?

Answer:

Threat Intelligence can identify new indicators, adversary behaviors, and techniques that can be converted into detection hypotheses, telemetry requirements, and SIEM detection rules.

43. Important Lessons

Intelligence needs context

IOC

≠

Complete Investigation

Freshness matters

Old intelligence may have reduced relevance.

Quality matters more than quantity

A huge feed is not automatically a useful feed.

Internal intelligence is valuable

Your own incidents can generate valuable future detection data.

Intelligence should support action

The objective is not simply collecting indicators.

The objective is:

Understand

↓

Detect

↓

Investigate

↓

Respond

↓

Improve



44. Final Takeaway

A SIEM Engineer should understand how Threat Intelligence becomes operational security data.

The important flow is:

Threat Intelligence → Enrichment → Correlation → Detection → Investigation

The SIEM Engineer should be able to work with:

==> IP Intelligence

==> Domain Intelligence

==> URL Intelligence

==> Hash Intelligence

==> IOC Feeds

==> STIX/TAXII

==> Threat Intelligence Platforms

==> SIEM Correlation

==> Threat Hunting

==> Detection Engineering

==> Automated Enrichment

The most important mindset is:

Don't blindly trust an indicator. Validate the indicator, understand the context, and investigate the evidence.

SIEM Engineer Learning Series

- #01 — Log Management
- #02 — SIEM Platforms
- #03 — Log Integration
- #04 — Detection Engineering
- #05 — Query & Investigation
- #06 — SOC Operations
- #07 — Threat Intelligence ← You are here
- #08 — Automation & SOAR → Next
- #09 — Dashboard & Reporting
- #10 — SIEM Infrastructure
- #11 — SIEM Architecture Design
- #12 — SIEM Capacity Planning
- #13 — SIEM Performance Tuning
- #14 — Advanced Detection Engineering
- #15 — Threat Hunting with SIEM
- #16 — SIEM Data Quality & Troubleshooting
- #17 — Hands-On SIEM: Multi-Platform Practical Guide

Navigation

← Previous Article: #06 — SOC Operations

SIEM Engineer Index

Next Article: #08 — Automation & SOAR →

SIEM Engineer #08 — Automation & SOAR

Introduction

A modern SOC may receive hundreds or thousands of alerts every day.

Analysts cannot manually perform every repetitive task.

For example:

```
SIEM Alert
  ↓
Extract IP
  ↓
Check Threat Intelligence
  ↓
Enrich Alert
  ↓
Create Ticket
  ↓
Notify Analyst
```

Doing this manually for every alert takes time.

This is where **Automation** and **SOAR** become important.

SOAR = Security Orchestration, Automation and Response

The main idea is:

Automate repetitive tasks so analysts can spend more time on complex investigations.



1. What is SOAR?

SOAR is a security technology and operational approach that connects security tools, automates workflows, and supports incident response.

A simple architecture:

```
SIEM
↓
SOAR
↓
Threat Intelligence
↓
EDR
↓
Firewall
↓
Ticketing
↓
Notification
```

Instead of analysts manually switching between many tools, SOAR can coordinate the workflow.

2. Why Automation Matters in SOC

Imagine:

100 Alerts

For every alert, an analyst manually:

```
Copy IP
↓
Threat Intelligence Lookup
↓
Copy Result
↓
Update Ticket
↓
Send Message
```

This creates:

==> Repetitive work

==> Human error

==> Slow response

==> Analyst fatigue

Automation can reduce these repetitive activities.



3. Automation vs SOAR

These concepts are related but not exactly identical.

Automation

A system performs a predefined task automatically.

Example:

```
Extract IP
↓
Threat Intelligence Lookup
```

SOAR

Coordinates multiple tools and actions into a broader security workflow.

Example:

```
SIEM Alert
↓
Enrich IOC
↓
Create Case
↓
Notify Analyst
↓
Request Approval
↓
Contain
↓
Update Case
```

4. What is Orchestration?

Orchestration means coordinating multiple systems and actions.

Example:

```

SIEM
  ↓
SOAR
  ↓
Threat Intelligence
  ↓
Firewall
  ↓
EDR
  ↓
Ticketing
  
```

The SOAR platform acts as the workflow coordinator.

5. Basic SOAR Architecture

A typical workflow:

Security Event



SIEM



Detection



Alert



SOAR



Playbook



Enrichment



Decision



Response



Case Update



6. What is a Playbook?

A **Playbook** is an automated or semi-automated workflow for handling a specific security scenario.

Example:

Suspicious IP Playbook

```
graph TD; A[Alert Received] --> B[Extract IP]; B --> C[Threat Intelligence Lookup]; C --> D[Check Reputation]; D --> E[Check Internal History]; E --> F[Update Alert]; F --> G[Notify Analyst];
```

Another example:

Phishing Playbook

Phishing Alert

↓

Extract URL

↓

Extract Domain

↓

Extract Attachment Hash

↓

Threat Intelligence Lookup

↓

Enrich Case

↓

Notify SOC



7. Runbook vs Playbook

These terms can vary by organization.

Generally:

Runbook

Human-readable operational instructions.

```
Step 1
Step 2
Step 3
```

Playbook

A broader workflow that can include:

```
Automation
Decision
Tools
Analyst Approval
Response
Documentation
```

Example:

```
Runbook:
How to investigate suspicious IP

Playbook:
Automatically enrich suspicious IP
and create an investigation case.
```



8. Alert Automation

Suppose a SIEM generates:

Suspicious IP Alert

SOAR can automatically:

1. Extract IP
2. Lookup intelligence
3. Find previous SIEM activity
4. Identify affected hosts
5. Create case
6. Add enrichment
7. Notify analyst

The analyst receives a more complete alert.

9. Enrichment Automation

Enrichment means adding additional information.

Example:

```
IOC:  
203.x.x.x
```

Automation can query an approved intelligence source and add:

```
Reputation  
Source  
Confidence  
First Seen  
Last Seen  
Related Information
```

Then:

```
Raw Alert  
+  
Enrichment  
=  
Context-Rich Alert
```



10. Automated Investigation

Automation can perform repetitive investigation tasks.

Example:

```
Alert
↓
Extract User
↓
Search Authentication History
↓
Search Endpoint Activity
↓
Search Network Activity
↓
Collect Results
↓
Update Case
```

This does not necessarily replace the analyst.

Instead, it gives the analyst useful context faster.



11. Automatic Ticket Creation

A SOAR workflow can create a ticket automatically.

Example:

```
SIEM Alert
↓
SOAR
↓
Case Creation
```

Ticket fields:

```
Title
Severity
Priority
User
Host
Source IP
Detection
Timestamp
Evidence
Status
```

This reduces manual data entry.



12. Notification Automation

SOAR can notify the appropriate team.

Examples:

```
SIEM
↓
SOAR
↓
Email
```

or:

```
SIEM
↓
SOAR
↓
Chat / Collaboration Platform
```

or:

```
SIEM
↓
SOAR
↓
Incident Management System
```

Notifications should contain enough context to be useful.

13. Automated Response

Some actions can be automated.

Examples may include:

==> Disable a compromised account

==> Isolate an endpoint

==> Block an indicator

==> Revoke a session

==> Create a firewall block

==> Add an IOC to a security control

But automated response requires careful design.



14. Why Automated Response Is Risky

Suppose the SIEM incorrectly identifies an IP as malicious.

Automation:

Alert

↓

Automatic Firewall Block

could accidentally block a legitimate service.

Therefore:

Detection

↓

Validation

↓

Confidence

↓

Approval / Policy

↓

Response

may be more appropriate for higher-impact actions.



15. Human-in-the-Loop

A **Human-in-the-Loop** workflow requires analyst approval before a potentially disruptive action.

Example:

```
Alert
↓
Enrichment
↓
High Confidence
↓
Request Approval
↓
Analyst Approves
↓
Containment
```

This is useful when an automated action could affect business operations.

16. Fully Automated vs Semi-Automated

Fully Automated

```
Alert
↓
Decision
↓
Action
```

Useful for low-risk, well-understood tasks.

Semi-Automated

```
Alert
↓
Enrichment
↓
Recommendation
↓
Analyst Approval
↓
Action
```

Useful for actions with higher operational impact.



17. Automation Decision Matrix

A SOC can classify actions by risk.

Action	Automation Approach
Extract IOC	Automatic
Threat Intelligence Lookup	Automatic
Case Creation	Automatic
Alert Enrichment	Automatic
Analyst Notification	Automatic
Endpoint Isolation	Approval / Policy-based
Account Disable	Approval / Policy-based
Firewall Block	Approval / Policy-based
Production System Shutdown	Strong authorization

The exact policy depends on the organization.

18. APIs in SOAR

APIs are extremely important.

A SOAR platform may communicate with:

```
SIEM API
EDR API
Firewall API
Threat Intelligence API
Ticketing API
Identity API
Email API
Cloud API
```

Example:

```
SOAR
↓
GET /indicator/203.x.x.x
↓
Threat Intelligence
↓
Result
↓
SOAR
```

Then:

```
SOAR
↓
POST /cases
↓
Ticketing System
```



19. API Authentication

Automation requires secure authentication.

Common methods include:

==> API Keys

==> OAuth

==> Service Accounts

==> Bearer Tokens

==> Client Certificates

Secrets should be stored securely.

Never put sensitive credentials directly inside source code or public repositories.



20. API Rate Limits

External APIs may have rate limits.

Example:

100 requests/minute

If the SOAR platform suddenly sends:

1,000 requests/minute

the API may reject requests.

Automation should handle:

==> Rate limits

==> Retries

==> Backoff

==> Timeouts

==> API errors

==> Duplicate requests

21. Retry Logic

Suppose:

```
graph TD; A[Threat Intelligence API] --> B[Temporary Failure];
```

The automation should not immediately fail the entire incident workflow.

A controlled retry strategy can be:

```
graph TD; A[Request] --> B[Failure]; B --> C[Wait]; C --> D[Retry]; D --> E[Success / Failure Handling];
```

Avoid infinite retries.



22. Idempotency

Automation should avoid performing the same action repeatedly.

Example:

Block IP

If the same alert triggers five times, the system should not create five duplicate actions unnecessarily.

A good workflow checks:

Already Blocked?
Already Ticketed?
Already Enriched?
Already Isolated?

This concept is called **idempotency**.



23. Duplicate Alert Handling

Suppose:

100 identical alerts

Without aggregation:

100 tickets

With proper logic:

100 alerts
↓
Correlation / Deduplication
↓
1 Case

This reduces unnecessary workload.

24. Automation Metrics

A SOC should measure automation.

Useful metrics include:

==> Number of automated workflows

==> Automation success rate

==> Automation failure rate

==> Average execution time

==> Manual effort reduced

==> Number of enriched alerts

==> Number of duplicate cases prevented

==> Analyst approval rate

==> Rollback/failure incidents

Metrics should help identify whether automation is actually improving operations.

25. Testing Automation

Never deploy automation directly into production without testing.

Test:

```
Success
Failure
Timeout
Invalid Input
Duplicate Event
API Rate Limit
Missing Field
Permission Error
```

Example:

```
Input:
IP = 203.x.x.x

Expected:
Threat Intelligence lookup

API Failure:
Retry

Repeated Alert:
No duplicate case

Invalid IP:
Graceful error
```



26. Automation Development Lifecycle

Use:

```
Requirement
↓
Design
↓
Build
↓
Unit Test
↓
Integration Test
↓
Security Review
↓
Pilot
↓
Production
↓
Monitor
↓
Improve
```

This is similar to software engineering.



27. Detection + SOAR

Detection Engineering and SOAR work together.

Example:

```
Detection
↓
Alert
↓
SOAR Playbook
↓
Enrichment
↓
Response
```

A detection answers:

“What happened?”

SOAR helps answer:

“What should happen next?”



28. Example — Suspicious IP Workflow

Suppose the SIEM detects:

Internal Host

↓

Suspicious External IP

SOAR workflow:

1. Receive Alert
2. Extract IP
3. Validate IP Format
4. Threat Intelligence Lookup
5. Search Historical Activity
6. Identify Affected Host
7. Enrich Case
8. Assign Priority
9. Notify Analyst
10. Request Approval if Containment Is Required
11. Record Action

This can significantly reduce repetitive analyst work.



29. Example — Phishing Workflow

A phishing alert arrives.

SOAR can:

```
Email Alert
↓
Extract Sender
↓
Extract URLs
↓
Extract Domains
↓
Extract Hashes
↓
Threat Intelligence
↓
Search Similar Emails
↓
Identify Recipients
↓
Create Case
↓
Notify SOC
```

If organizational policy allows, further response actions can be performed after validation and authorization.



30. Example — Compromised Account Workflow

Suppose there is strong evidence that an account may be compromised.

Workflow:

```
SIEM Alert
↓
Identify User
↓
Search Login History
↓
Check Endpoint Activity
↓
Check Risk Context
↓
Create Case
↓
Notify Analyst
↓
Approval
↓
Session Revocation / Account Action
↓
Document
```

High-impact identity actions should have strong authorization controls.



31. Example — Malware Alert Workflow

Endpoint generates:

Suspicious File Detection

SOAR can:

```
Extract Hash
↓
Threat Intelligence Lookup
↓
Search Other Endpoints
↓
Check Related Network Events
↓
Create Case
↓
Notify Analyst
↓
Recommend Containment
```

This helps determine whether the event is isolated or widespread.



32. SOAR and Threat Intelligence

Threat Intelligence is one of the easiest areas to automate.

```
Alert
↓
Extract IOC
↓
Threat Intelligence API
↓
Reputation
↓
Confidence
↓
Context
↓
Update Case
```

This can happen in seconds instead of requiring manual lookups.



33. SOAR and Threat Hunting

SOAR can also automate predefined searches.

Example:

```
IOC Discovered
↓
Search SIEM
↓
Search DNS
↓
Search Proxy
↓
Search EDR
↓
Aggregate Results
↓
Create Investigation Case
```

This is especially useful when the same investigation steps are repeated frequently.



34. SOAR Error Handling

Every automation workflow should expect failures.

Possible failures:

```
SIEM unavailable
API unavailable
Invalid token
Timeout
Rate limit
Missing data
Permission denied
Unexpected response
```

Good automation should:

- ==> Detect the failure
 - ==> Record the error
 - ==> Retry when appropriate
 - ==> Avoid destructive assumptions
 - ==> Notify the responsible team
 - ==> Continue safely when possible
-

35. SOAR Security

SOAR has powerful permissions.

Therefore protect it carefully.

Use:

==> RBAC

==> MFA

==> Least privilege

==> Secure secret storage

==> API access controls

==> Audit logging

==> Approval workflows

==> Change management

==> Network restrictions

A compromised SOAR platform could potentially affect multiple security systems.



36. Audit Logging

Every important automated action should be traceable.

Record:

Who triggered it?
What automation ran?
When?
Which API?
What action?
What result?
What error?
Who approved it?

Example:

Action:
Firewall Block

Trigger:
SIEM Alert #1024

Automation:
Suspicious-IP Playbook

Approval:
SOC Analyst

Time:
14:32

Result:
Success

37. Rollback

Some automated actions should have rollback capability.

Example:

```
Firewall Block
↓
False Positive Identified
↓
Rollback
↓
Remove Block
↓
Document
```

Rollback capability is especially important for actions that can disrupt business operations.



38. Human Approval Workflow

A mature workflow may look like:

```
Detection
↓
SOAR
↓
Enrichment
↓
Risk Evaluation
↓
Analyst Approval
↓
Response
↓
Verification
↓
Documentation
```

This balances:

Automation + Human Judgment



39. Playbook Documentation

Every important playbook should document:

- Playbook Name
- Purpose
- Trigger
- Required Data
- Actions
- APIs
- Permissions
- Conditions
- Approval Requirements
- Failure Handling
- Rollback
- Logging
- Owner
- Version
- Last Review

This makes automation maintainable.

40. Practical SOAR Lab

You can create a small lab with:

```
Wazuh / SIEM
+
Threat Intelligence
+
Python
+
Webhook / REST API
+
Ticketing System
```

Lab 1 — IOC Enrichment

```
SIEM Alert
↓
Extract IP
↓
API Lookup
↓
Add Result
```

Lab 2 — Automatic Case Creation

```
SIEM Alert
↓
Webhook
↓
Create Ticket
```

Lab 3 — Notification

High-Priority Alert

↓

Automation

↓

SOC Notification

Lab 4 — Approval-Based Response

Alert

↓

Enrichment

↓

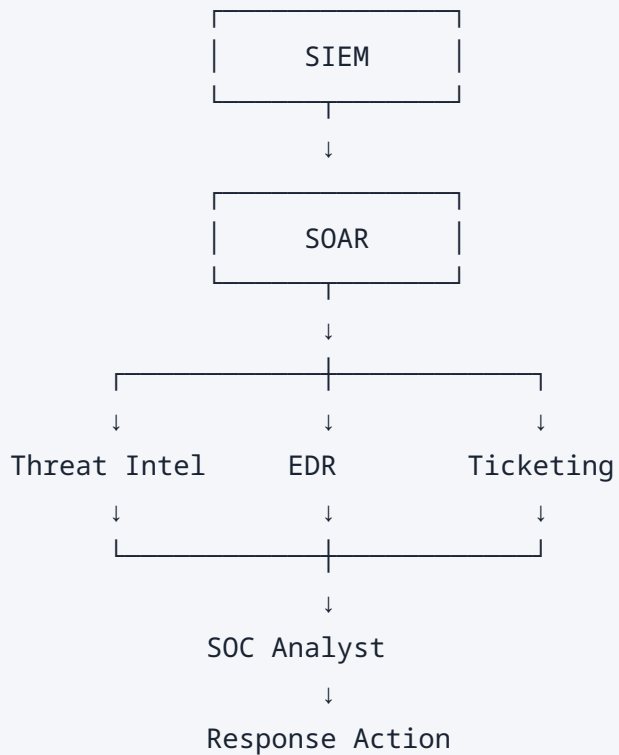
Analyst Approval

↓

Authorized Response

41. SOC Automation Architecture

A practical architecture:



The SOAR layer coordinates the workflow.



42. Automation Maturity

Automation can mature gradually.

Level 1

Manual investigation.

Analyst does everything.

Level 2

Automated enrichment.

Alert → IOC Lookup

Level 3

Automated case management.

Alert → Enrichment → Ticket

Level 4

Semi-automated response.

Alert → Enrichment → Analyst Approval → Action

Level 5

Controlled automated response.

Alert → Validation → Policy → Automated Action

The organization should decide which level is appropriate for each use case.

43. Common SOAR Mistakes

Mistake 1 — Automating Everything

Not every task should be automated.

Mistake 2 — No Error Handling

APIs will fail.

Mistake 3 — Excessive Permissions

SOAR should follow least privilege.

Mistake 4 — No Approval

High-impact actions may require human approval.

Mistake 5 — No Logging

Every important action should be auditable.

Mistake 6 — No Rollback

Some actions need recovery mechanisms.

Mistake 7 — No Testing

Untested automation can create operational problems.

Mistake 8 — Duplicate Actions

Automation must handle repeated alerts safely.



44. SOAR Checklist

- [] Define automation objective
- [] Identify repetitive task
- [] Define trigger
- [] Identify required data
- [] Identify APIs
- [] Configure authentication
- [] Apply least privilege
- [] Build workflow
- [] Add validation
- [] Add error handling
- [] Add retry logic
- [] Handle duplicates
- [] Add approval where required
- [] Add audit logging
- [] Add rollback where appropriate
- [] Test success path
- [] Test failure path
- [] Test API timeout
- [] Test invalid input
- [] Test duplicate alert
- [] Deploy gradually

☐ Monitor performance

☐ Review regularly

45. Complete SOAR Flow

Remember:

```
Security Event
  ↓
SIEM Detection
  ↓
Alert
  ↓
SOAR Trigger
  ↓
Data Extraction
  ↓
Enrichment
  ↓
Correlation
  ↓
Decision
  ↓
Human Approval if Required
  ↓
Response
  ↓
Verification
  ↓
Case Update
  ↓
Audit Log
  ↓
Lessons Learned
```



46. SIEM Engineer Interview Questions

Q1. What is SOAR?

Answer:

SOAR stands for Security Orchestration, Automation and Response. It connects security tools and automates or coordinates repetitive security workflows and response processes.

Q2. What is a SOAR playbook?

Answer:

A playbook is a predefined workflow that describes the steps, decisions, integrations, and actions used to handle a specific security scenario.

Q3. What is orchestration?

Answer:

Orchestration means coordinating multiple security tools and systems so they can work together as part of a single workflow.

Q4. Why is automation important in a SOC?

Answer:

Automation reduces repetitive manual work, improves response speed, standardizes workflows, and allows analysts to focus on more complex investigations.

Q5. Should every response action be automated?

Answer:

No. Low-risk and repetitive actions can often be automated, while high-impact actions may require validation, policy controls, or analyst approval.

Q6. What is human-in-the-loop automation?

Answer:

It is a workflow where automation performs investigation or enrichment but waits for human approval before executing a potentially high-impact action.

Q7. How do you secure a SOAR platform?

Answer:

Use least privilege, RBAC, MFA, secure secret storage, API access controls, audit logging, network restrictions, approval workflows, and strong change management.

Q8. How do you handle API failures in automation?

Answer:

I would implement timeout handling, controlled retries, backoff, error logging, rate-limit handling, validation of responses, and safe failure behavior.

Q9. What is idempotency in automation?

Answer:

Idempotency means repeating the same operation does not create unwanted duplicate effects. For example, repeated alerts should not unnecessarily create multiple identical firewall blocks or tickets.

Q10. How would you automate suspicious IP investigation?

Answer:

I would trigger the workflow from the SIEM, extract and validate the IP, perform approved Threat Intelligence enrichment, search historical SIEM activity, identify affected hosts, update the case, and notify the analyst. Any containment action would follow organizational authorization requirements.

Q11. What should be included in a SOAR playbook?

Answer:

The playbook should define the trigger, required data, actions, integrations, permissions, conditions, approval requirements, error handling, logging, rollback where appropriate, owner, and version.

Q12. How do you test a SOAR workflow?

Answer:

I test normal execution as well as API failures, timeouts, invalid inputs, missing fields, duplicate alerts, permission errors, rate limits, and rollback or recovery scenarios where applicable.



47. Important Lessons



Automation should solve a problem

Don't automate simply because you can.

Automate repetitive work first

Start with:

- Enrichment
- Ticket Creation
- Notification
- Data Collection

High-impact actions need control

Use:

- Validation
- +
- Policy
- +
- Approval

when appropriate.

Automation must be observable

You should always know:

What happened?
When?
Why?
Who approved it?
Did it succeed?

SOAR is not a replacement for analysts

SOAR handles repetitive workflows.

Analysts provide:

Context
Judgment
Investigation
Decision-making



48. Final Takeaway

A modern SIEM Engineer should understand how SIEM connects with automation.

The complete concept is:

SIEM → Detection → SOAR → Enrichment → Decision → Response → Documentation

Important skills include:

==> REST API

==> Webhooks

==> Python automation

==> Playbooks

==> Threat Intelligence integration

==> Ticketing integration

==> Error handling

==> Authentication

==> RBAC

==> Approval workflows

==> Audit logging

==> Rollback

==> Automation testing

The goal is not:

“Automate everything.”

The goal is:

“Automate the right things safely.”

SIEM Engineer Learning Series

- #01 — Log Management
- #02 — SIEM Platforms
- #03 — Log Integration
- #04 — Detection Engineering
- #05 — Query & Investigation
- #06 — SOC Operations
- #07 — Threat Intelligence
- #08 — Automation & SOAR ← You are here
- #09 — Dashboard & Reporting → Next
- #10 — SIEM Infrastructure
- #11 — SIEM Architecture Design
- #12 — SIEM Capacity Planning
- #13 — SIEM Performance Tuning
- #14 — Advanced Detection Engineering
- #15 — Threat Hunting with SIEM
- #16 — SIEM Data Quality & Troubleshooting
- #17 — Hands-On SIEM: Multi-Platform Practical Guide

Navigation

← **Previous Article: #07 — Threat Intelligence**

SIEM Engineer Index

Next Article: #09 — Dashboard & Reporting →

SIEM Engineer #09 — Dashboard & Reporting

Introduction

A SIEM can collect millions of security events.

But collecting data is not enough.

A SOC team needs to **see, understand, measure, and communicate** what is happening.

This is where **Dashboard & Reporting** becomes important.

A SIEM Engineer should know how to transform raw security data into:

==> SOC Dashboards

==> Investigation Dashboards

==> Security Metrics

==> Management Reports

==> Detection Coverage

==> Alert Trends

==> Incident Reports

The basic idea is:

Raw Logs



SIEM



Search / Query



Aggregation



Visualization



Dashboard



Report



Security Decision



1. What is a SIEM Dashboard?

A SIEM Dashboard is a visual representation of security data.

Instead of reading thousands of events individually, analysts can quickly see important information.

Example:

SOC DASHBOARD	
Critical Alerts	12
High Alerts	47
Open Incidents	18
False Positive	31%
Top Source IPs	
Top Attacked Hosts	
Top Detection Rules	
Alert Trend	

2. Why Dashboard Matters

Imagine an organization receives:

50,000 events/day

An analyst cannot manually review every event.

A dashboard can immediately show:

==> Which alerts are increasing?

==> Which systems are generating alerts?

==> Which users are involved?

==> Which detection rules are triggering?

==> Which incidents are still open?

==> Are critical alerts increasing?

This helps analysts understand the security environment faster.



Why does this matter?

A good dashboard converts:

Data → Information → Visibility → Action

A bad dashboard creates:

Data → Noise → Confusion

The objective is not to put every available field on the screen.

The objective is to show the **right information for the right audience**.



3. Dashboard vs Report

These are related but different.

Dashboard

Usually provides interactive and near-real-time visibility.

Example:

```
Current Alerts
Current Incidents
Current Threats
Current Log Status
```

Report

Usually provides a structured summary for a specific period.

Example:

```
Monthly Security Report
Weekly SOC Report
Incident Report
Compliance Report
```

Simple difference:

```
Dashboard = What is happening?
```

```
Report = What happened during a period?
```



4. SIEM Engineer's Role

A SIEM Engineer may be responsible for:

- ==> Creating dashboards
 - ==> Building queries
 - ==> Designing visualizations
 - ==> Creating reports
 - ==> Maintaining dashboard data
 - ==> Validating metrics
 - ==> Optimizing queries
 - ==> Troubleshooting missing data
 - ==> Supporting SOC analysts
 - ==> Supporting management reporting
 - ==> Monitoring SIEM health
-



5. Different Dashboard Users

Not everyone needs the same dashboard.

A mature SOC may have:

SOC Analyst Dashboard



SOC Manager Dashboard



Security Management Dashboard



Executive Dashboard

Each audience needs different information.



6. SOC Analyst Dashboard

The analyst needs operational information.

Example:

Current Alerts
Open Incidents
Critical Alerts
Top Source IPs
Top Destination Hosts
Authentication Failures
Endpoint Alerts
Firewall Events
Detection Rule Activity

A simple structure:

Open Alerts		
Critical	High	Medium
5	21	74
Alert Trend		
		
Top Detection Rules		
1. Brute Force		
2. PowerShell		
3. Malware		



7. Alert Dashboard

One of the most important SOC dashboards.

Useful metrics:

==> Alerts by severity

==> Alerts by status

==> Alerts by detection rule

==> Alerts by source

==> Alerts by host

==> Alerts by user

==> Alerts over time

Example:

Alert Status

Open	→ 42
Investigating	→ 18
Escalated	→ 7
Resolved	→ 91
Closed	→ 125



8. Alert Trend

A trend shows how alerts change over time.

Example:

Monday	120
Tuesday	145
Wednesday	132
Thursday	210
Friday	340

A sudden increase may require investigation.

But remember:

An increase in alerts does not automatically mean an increase in successful attacks.

It could be caused by:

==> New detection rules

==> Configuration changes

==> New log sources

==> Scanner activity

==> Increased legitimate activity

==> Detection tuning problems

Therefore, dashboards provide visibility; analysts still need investigation.

9. Severity Dashboard

Show alerts by severity.

Example:

```
Critical → 8  
High      → 37  
Medium    → 102  
Low       → 450
```

Severity should have a documented meaning.

Do not simply assign every interesting alert as Critical.

10. Priority vs Severity

These are not always the same.

Severity

How serious the security event may be.

Priority

How urgently the SOC should handle it.

Example:

```
Medium Severity
+
Critical Production Server
=
High Priority
```

Context can change priority.



11. User-Based Dashboard

A dashboard can show authentication activity.

Example:

Top Users with Failed Logins

user01 → 87

admin → 54

user02 → 41

Additional metrics:

==> Successful logins

==> Failed logins

==> MFA failures

==> Login source countries/locations where legitimately available

==> Unusual login times

==> Privileged account activity

This can help identify suspicious authentication behavior.



12. Endpoint Dashboard

For endpoint security:

Total Endpoints
Healthy Endpoints
Disconnected Agents
Malware Alerts
Suspicious Processes
High-Risk Hosts

Example:

Endpoints	
Healthy	950
Disconnected	32
Critical	8

The exact fields depend on the endpoint security platform and integration.



13. Network Security Dashboard

Useful network metrics include:

==> Firewall events

==> IDS/IPS alerts

==> VPN activity

==> Proxy activity

==> DNS events

==> Network anomalies

==> Blocked connections

Example:

Network Activity

Firewall Events	125K
Blocked Connections	18K
IDS Alerts	420
VPN Logins	2.3K
DNS Events	95K



14. Firewall Dashboard

A firewall dashboard might include:

- Top Blocked IPs
- Top Allowed Connections
- Top Destination Ports
- Top Source Countries
- Denied Traffic
- Allowed Traffic
- Policy Violations

Example:

Top Blocked Sources

- IP-A → 2,430
- IP-B → 1,890
- IP-C → 1,210

This can help analysts identify patterns.



15. Authentication Dashboard

Authentication dashboards are extremely useful.

Monitor:

==> Failed logins

==> Successful logins

==> Account lockouts

==> Privileged logins

==> New accounts

==> Password changes

==> MFA failures

==> Authentication anomalies

Example:

Authentication

Successful → 45,230

Failed → 3,420

Lockouts → 120

Privileged → 340



16. Detection Dashboard

Detection Engineers need their own visibility.

Monitor:

==> Detection rules

==> Rule execution

==> Alert count

==> False positives

==> True positives

==> Detection coverage

==> Disabled rules

==> Detection errors

Example:

Detection Rule Health

Active Rules	320
Disabled Rules	12
New Rules	8
Rules with Errors	3

17. MITRE ATT&CK Coverage Dashboard

A SOC can map detections to MITRE ATT&CK techniques.

Example:

```
graph TD; A[Initial Access] --> B[Execution]; B --> C[Persistence]; C --> D[Privilege Escalation]; D --> E[Defense Evasion]; E --> F[Credential Access];
```

A dashboard can show:

==> Techniques covered

==> Techniques without detection

==> Number of detections per technique

==> Alert activity by technique

This helps identify detection coverage gaps.



18. Detection Coverage $\neq$ Attack Prevention

An important point:

If a technique appears in a dashboard, it does not mean the organization is fully protected against it.

It may simply mean:

Telemetry Available
+
Detection Rule Exists

Coverage should be interpreted carefully.



19. Incident Dashboard

A SOC manager may need:

- Open Incidents
- Critical Incidents
- Average Resolution Time
- Escalated Incidents
- Incidents by Category
- Incidents by Severity
- Incidents by Business Unit

Example:

Incident Status

Open	18
Investigating	12
Escalated	4
Resolved	31
Closed	76

20. MTTD

MTTD = Mean Time to Detect

It measures how long it takes to detect an incident or security event after it occurs, according to the organization's defined measurement method.

Conceptually:

MTTD =
Detection Time - Event Start Time

Example:

Event:
10:00

Detection:
10:12

MTTD:
12 minutes

The exact calculation should follow the organization's measurement definition.

21. MTTR

MTTR = Mean Time to Respond/Resolve, depending on the organization's definition.

For example, an organization may measure:

Incident Detected



Response Started

or:

Incident Detected



Incident Resolved

Therefore, always document what your organization means by MTTR.



22. SOC Metrics Dashboard

A SOC metrics dashboard can include:

==> MTTD

==> MTTR

==> Alert volume

==> Incident volume

==> False-positive rate

==> Escalation rate

==> SLA compliance

==> Automation rate

==> Open-case backlog

==> Detection coverage

Example:

SOC Performance

Alerts	12,450
Incidents	86
False Positive	24%
MTTD	12 min
MTTR	1.8 hr
SLA Compliance	96%

Metrics must have clearly defined calculation methods.



23. Automation Dashboard

After implementing SOAR, track automation.

Example:

Automated Workflows	32
Successful Runs	4,320
Failed Runs	42
Manual Escalations	87
Automation Success	99%

This helps the team identify automation reliability problems.



24. Log Source Health Dashboard

A SIEM Engineer should also monitor whether logs are arriving.

Example:

Log Source Health

Firewall-01	Healthy
Windows-DC01	Healthy
Linux-Server-02	Delayed
VPN-01	Offline
Wazuh-Agent-22	Healthy

Important metrics:

==> Last received event

==> EPS

==> Event delay

==> Parsing failures

==> Connection status

==> Agent status



25. EPS Monitoring

EPS = Events Per Second

It represents the event ingestion rate.

Example:

Current EPS:

2,500

Peak EPS:

4,800

A sudden change can indicate:

==> Log source configuration change

==> Attack activity

==> Application issue

==> Duplicate logging

==> Integration problem

==> SIEM ingestion problem



26. Data Volume Dashboard

For some SIEM architectures, data volume has operational or cost implications.

Track:

Daily Ingestion
Weekly Ingestion
Monthly Ingestion
Top Data Sources
Storage Usage
Retention Usage

Example:

Daily Ingestion

Firewall	40 GB
Windows	32 GB
Linux	12 GB
Application	25 GB
Cloud	55 GB

This can help identify unexpected data growth.



27. Query Design for Dashboards

A dashboard is only as good as the query behind it.

Typical query operations include:

==> Filtering

==> Grouping

==> Counting

==> Aggregation

==> Time bucketing

==> Sorting

==> Correlation

Example concept:

```
Search Events
  ↓
Filter severity=high
  ↓
Group by rule
  ↓
Count events
  ↓
Sort descending
  ↓
Visualization
```



28. Time-Series Visualization

Security data is often time-based.

Useful visualizations include:

==> Line charts

==> Bar charts

==> Area charts

==> Tables

==> Heatmaps

==> Single-value metrics

Choose the visualization based on the question.

Example:

Question:
Are failed logins increasing?

Use:
Time-series chart

Another:

Question:
Which users have the most failed logins?

Use:
Bar chart/table



29. Geographic Visualization

Some organizations visualize security events geographically.

Example:

Country



Source IP



Authentication Activity

However, IP geolocation is imperfect.

Therefore:

Geographic visualization should be treated as contextual information, not definitive proof of user location or attacker identity.



30. Table Visualization

Tables are useful when analysts need exact values.

Example:

Time	User	Source IP	Host	Detection	Severity
10:21	admin	IP-A	DC01	Failed Login	High
10:23	user01	IP-B	PC02	Malware	Critical
10:25	user02	IP-C	WEB01	Exploit Attempt	High

Tables are often better than charts for investigation.



31. Dashboard Filters

Good dashboards allow filtering.

Useful filters:

- Time Range
- Severity
- User
- Host
- Source IP
- Destination IP
- Detection
- Status
- Department
- Log Source

Example:

- Time:
Last 24 Hours
- Severity:
High + Critical
- Source:
Firewall
- Status:
Open

Now the analyst sees only relevant information.

32. Dashboard Drill-Down

A powerful dashboard should allow analysts to move from summary to evidence.

Example:

```

Dashboard
  ↓
High Alert Count
  ↓
Detection Rule
  ↓
Specific Alert
  ↓
Raw Event
  ↓
Investigation
  
```

This is called **drill-down**.



33. Dashboard → Investigation

Suppose the dashboard shows:

Failed Login Spike

The analyst should be able to investigate:

User
↓
Source IP
↓
Host
↓
Authentication Logs
↓
Network Logs
↓
Endpoint Logs

A dashboard should support investigation rather than simply look attractive.



34. Dashboard Design Principles

Principle 1 — Keep it Simple

Do not display everything.

Principle 2 — Prioritize Important Information

Critical information should be visible quickly.

Principle 3 — Use Consistent Terminology

For example:

Open
Investigating
Resolved
Closed

Principle 4 — Provide Context

A number without context may be misleading.

Principle 5 — Support Drill-Down

Allow analysts to move from summary to evidence.



35. Common Dashboard Mistakes

Mistake 1 — Too Many Charts

More charts do not automatically mean better visibility.

Mistake 2 — No Time Context

Always understand:

Last hour?
Last day?
Last week?

Mistake 3 — Incorrect Queries

A beautiful dashboard with incorrect data is dangerous.

Mistake 4 — No Validation

Every important metric should be validated.

Mistake 5 — Mixing Different Populations

For example:

Production Alerts
+
Lab Alerts

can produce misleading metrics.

Mistake 6 — No Ownership

Every important dashboard should have an owner.

36. Dashboard Validation

Before publishing a dashboard:

```
Query
↓
Test Data
↓
Validate Count
↓
Compare Raw Events
↓
Validate Time Range
↓
Check Filters
↓
Check Permissions
↓
Publish
```

Example:

If the dashboard says:

```
Critical Alerts = 100
```

verify the underlying SIEM query actually returns the expected events.



37. Dashboard Security

Dashboards may contain sensitive information.

Protect them with:

==> RBAC

==> Authentication

==> Least privilege

==> Appropriate data access

==> Audit logging

==> Secure sharing

For example, an executive dashboard may not need the same raw event details available to a SOC analyst.



38. Security Reports

Common SIEM reports include:

==> Daily SOC Report

==> Weekly Security Report

==> Monthly Security Report

==> Incident Report

==> Vulnerability-related Security Report

==> Compliance Report

==> Detection Coverage Report

==> Log Health Report

39. Daily SOC Report

A daily report may contain:

- Reporting Period
- Alert Summary
- Incident Summary
- Critical Events
- Major Investigations
- Top Detection Rules
- Top Affected Assets
- Notable Threat Intelligence
- Open Cases
- SLA Status

40. Monthly Security Report

A monthly report can include:

- Executive Summary
- Alert Trends
- Incident Trends
- Top Threat Categories
- Detection Performance
- False Positives
- MTTD
- MTTR
- Automation
- Log Coverage
- Detection Coverage
- Major Incidents
- Recommendations

Reports should be based on clearly defined and validated metrics.



41. Executive Dashboard

Executives generally need high-level information.

Example:

Security Overview

Critical Incidents	3
Open Incidents	14
SLA Compliance	96%
Security Trend	Stable*

The exact interpretation should be supported by defined metrics.

Avoid exposing unnecessary technical details.



42. Analyst Dashboard vs Executive Dashboard

Area	Analyst	Executive
Raw Events	Important	Usually unnecessary
Alerts	Detailed	Summary
Incidents	Detailed	High-level
Detection Rules	Important	Summary
MTTD/MTTR	Useful	Important
Risk Context	Detailed	Summary
Technical IOC	Important	Usually limited
Business Impact	Useful	Important
Compliance	Some detail	Important



43. Data Quality and Reporting

Reporting depends on data quality.

Consider:

```
Missing Logs
+
Parsing Errors
+
Duplicate Events
+
Wrong Timestamp
+
Incorrect Classification
=
Incorrect Report
```

Therefore:

Log Management → Detection → Investigation → Reporting

are connected.

44. SIEM Reporting Pipeline

The complete process:

```
Log Sources
↓
Collection
↓
Parsing
↓
Normalization
↓
Storage
↓
Detection
↓
Alert
↓
Investigation
↓
Classification
↓
Aggregation
↓
Dashboard
↓
Report
```



45. Practical Lab — SOC Dashboard

You can build a basic dashboard using your SIEM.

Step 1

Collect logs from:

- Windows
- Linux
- Firewall
- Wazuh
- Application

Step 2

Create queries for:

- Alert Count
- Severity
- Top Source IP
- Top User
- Top Host
- Detection Rules

Step 3

Create visualizations.

- Alert Trend
- Severity Distribution
- Top IPs
- Top Users

Step 4

Add filters.

Time
Severity
Host
User
Detection

Step 5

Add drill-down.

Chart
↓
Alert
↓
Raw Event



46. Practical Wazuh Dashboard Exercise

In a Wazuh lab, create a dashboard concept around:

```
Agent Status
↓
Authentication Events
↓
File Integrity Monitoring
↓
Vulnerability Findings
↓
Security Alerts
```

For example:

```
SOC Dashboard

Total Agents
Active Agents
Disconnected Agents

Security Alerts
High
Medium
Low

Top Rule IDs
Top Source IPs
Top Affected Hosts
```

Always validate the fields and queries against the actual Wazuh version and data available in your environment.



47. Dashboard Development Workflow

Use:

```
Requirement
↓
Audience
↓
Data Sources
↓
Metrics
↓
Queries
↓
Visualization
↓
Filters
↓
Drill-down
↓
Validation
↓
Security Review
↓
Publish
↓
Monitor
↓
Improve
```

48. Dashboard Maintenance

Dashboards need maintenance.

Things change:

==> Log sources

==> Field names

==> Detection rules

==> SIEM versions

==> Data schemas

==> Business requirements

==> Security policies

Therefore:

```
Build
  ↓
Monitor
  ↓
Review
  ↓
Update
```



49. Dashboard as a Detection Engineering Tool

Dashboards are not only for management.

Detection Engineers can use dashboards to identify:

==> Noisy rules

==> Detection spikes

==> Detection gaps

==> Data source problems

==> False-positive trends

==> Rule performance

Example:

```
Detection Rule
↓
Alert Volume
↓
Trend Analysis
↓
False Positive Review
↓
Rule Tuning
```




50. Dashboard as an Investigation Tool

An analyst may start with:

Suspicious IP

Then pivot:

IP
↓
Users
↓
Hosts
↓
Processes
↓
Authentication
↓
Network
↓
Timeline

Therefore a dashboard can act as the starting point for a larger investigation.



51. SIEM Engineer Interview Questions

Q1. What is a SIEM dashboard?

Answer:

A SIEM dashboard is a visual interface that presents security data, metrics, alerts, trends, and other information from the SIEM in a form that supports monitoring and investigation.

Q2. What is the difference between a dashboard and a report?

Answer:

A dashboard generally provides interactive and often near-real-time visibility, while a report provides a structured summary for a defined reporting period.

Q3. What metrics would you include in a SOC dashboard?

Answer:

I would consider alert volume, severity, open incidents, detection trends, MTTD, MTTR, SLA compliance, false-positive trends, log-source health, and other metrics defined by the SOC.

Q4. What is MTTD?

Answer:

MTTD means Mean Time to Detect. It measures the average time between the defined start of a security event or incident and its detection.

Q5. What is MTTR?

Answer:

MTTR is an organization-defined metric commonly used for Mean Time to Respond or Mean Time to Resolve. The SOC should clearly document which definition and timestamps it uses.

Q6. Why are dashboards important for SOC operations?

Answer:

They provide visibility into alerts, incidents, trends, system health, and operational metrics, allowing analysts and managers to understand security activity more efficiently.

Q7. How would you troubleshoot an incorrect dashboard number?

Answer:

I would verify the time range, query logic, filters, field mappings, data source, normalization, duplicate events, and underlying raw events. Then I would compare the dashboard result with an independent query or known dataset.

Q8. What is dashboard drill-down?

Answer:

Drill-down allows a user to move from a high-level visualization into the underlying alerts or events for detailed investigation.

Q9. What is EPS?

Answer:

EPS means Events Per Second. It represents the rate at which events are being generated or ingested, depending on the measurement point.

Q10. How do you design an executive dashboard?

Answer:

I would focus on validated high-level metrics such as critical incidents, trends, response metrics, SLA status, and business-relevant security information rather than overwhelming executives with raw technical events.



52. Dashboard Checklist

- ☐ Define audience
- ☐ Define purpose
- ☐ Identify data sources
- ☐ Define metrics
- ☐ Validate data
- ☐ Create queries
- ☐ Create visualizations
- ☐ Add time filters
- ☐ Add useful filters
- ☐ Add drill-down
- ☐ Validate calculations
- ☐ Check permissions
- ☐ Add ownership
- ☐ Document metrics
- ☐ Monitor performance
- ☐ Review regularly

53. Complete Dashboard & Reporting Flow

Remember this:

```
Data
↓
Collection
↓
Normalization
↓
Detection
↓
Investigation
↓
Classification
↓
Aggregation
↓
Query
↓
Visualization
↓
Dashboard
↓
Metrics
↓
Report
↓
Security / Operational Decision
```



54. Key Skills for a SIEM Engineer

To become strong in Dashboard & Reporting, learn:

==> SPL

==> KQL

==> Elasticsearch/OpenSearch queries

==> Data aggregation

==> Time-series analysis

==> Visualization

==> Dashboard design

==> SIEM data models

==> SOC metrics

==> MTTD/MTTR concepts

==> Detection coverage

==> Log health monitoring

==> Reporting

==> Data validation



55. Final Takeaway

A SIEM Engineer should not only know how to collect logs and create detection rules.

They should also know how to **turn security data into useful visibility**.

The complete concept is:

Data → Query → Visualization → Dashboard → Metrics → Report → Action

A good SIEM dashboard should answer:

==> What is happening?

==> Where is it happening?

==> Which users or systems are involved?

==> Which detections are firing?

==> How serious is the activity?

==> What is changing over time?

==> What needs investigation?

And a good report should answer:

==> What happened?

==> How frequently?

==> How did the SOC respond?

==> What trends were observed?

==> What should be monitored or improved?

Remember:

A dashboard is not decoration. It is a security visibility and investigation tool.

SIEM Engineer Learning Series

- #01 — Log Management
 - #02 — SIEM Platforms
 - #03 — Log Integration
 - #04 — Detection Engineering
 - #05 — Query & Investigation
 - #06 — SOC Operations
 - #07 — Threat Intelligence
 - #08 — Automation & SOAR
 - #09 — Dashboard & Reporting ← You are here
 - #10 — SIEM Infrastructure → Next
 - #11 — SIEM Architecture Design
 - #12 — SIEM Capacity Planning
 - #13 — SIEM Performance Tuning
 - #14 — Advanced Detection Engineering
 - #15 — Threat Hunting with SIEM
 - #16 — SIEM Data Quality & Troubleshooting
 - #17 — Hands-On SIEM: Multi-Platform Practical Guide
-

Navigation

← **Previous Article: #08 — Automation & SOAR**

SIEM Engineer Index

Next Article: #10 — SIEM Infrastructure →

SIEM Engineer #10 — SIEM Infrastructure

Introduction

A SIEM is not only a software application.

Behind every SIEM platform, there is an infrastructure that handles:

==> Log ingestion

==> Data processing

==> Parsing

==> Indexing

==> Searching

==> Storage

==> Retention

==> Detection

==> Dashboard

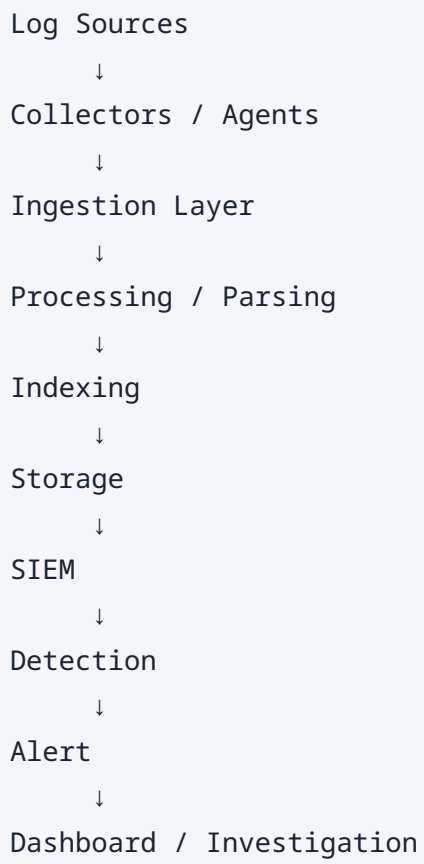
==> Alerting

==> High availability

==> Backup and recovery

A SIEM Engineer therefore needs both **Security Knowledge + Infrastructure Knowledge**.

The basic architecture is:





1. What is SIEM Infrastructure?

SIEM Infrastructure is the collection of:

```
Hardware
+
Operating Systems
+
Network
+
SIEM Components
+
Storage
+
Database / Index
+
Security Controls
+
Monitoring
+
Backup
```

that allows the SIEM to operate reliably.

2. Why SIEM Infrastructure Matters

Suppose your detection rules are perfect.

But:

SIEM Server

↓

CPU = 100%

or:

Storage = Full

or:

Logs = Delayed

Then the SOC may not receive timely security visibility.

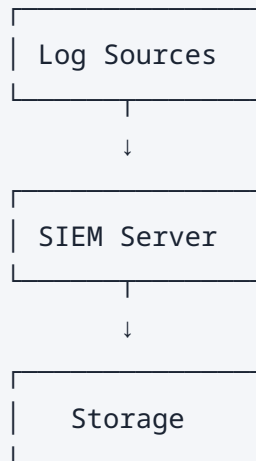
Therefore:

A good detection is only useful when the SIEM infrastructure can reliably process and retain the required data.

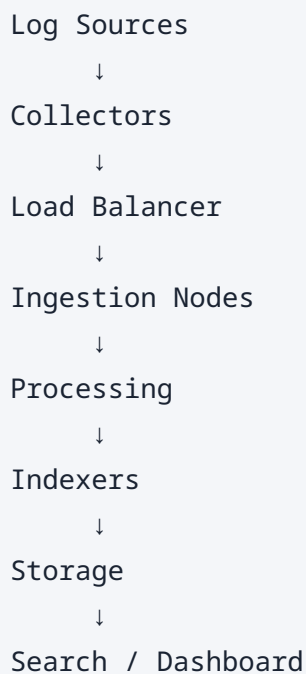


3. Basic SIEM Architecture

A small environment may look like:



A larger environment may use separate components:





4. Main SIEM Infrastructure Components

Common components include:

==> Data Collectors

==> Log Forwarders

==> Ingestion Nodes

==> Processing Nodes

==> Indexers

==> Search Nodes

==> Management Nodes

==> Storage

==> Database / Index Cluster

==> Dashboard

==> API Services

==> Monitoring

The exact architecture depends on the SIEM platform and organization.



5. Log Collectors

Collectors receive logs from different sources.

Examples:

```
Firewall
Windows
Linux
VPN
Proxy
Application
Network Device
Cloud
EDR
```

The collector may then:

```
Receive
↓
Buffer
↓
Forward
```

logs toward the SIEM.



6. Log Forwarders

A forwarder sends logs from one location to another.

Example:

```
graph TD; Server --> Forwarder; Forwarder --> SIEM;
```

Server
↓
Forwarder
↓
SIEM

Forwarders can reduce the need to send every source directly to the central SIEM.



7. Ingestion Layer

The ingestion layer is responsible for receiving security data.

Example:

```
graph TD; A[Syslog] --> F[Ingestion]; B[API] --> F; C[Agent] --> F; D[CEF] --> F; E[JSON] --> F; G[Cloud Connector] --> F;
```

Syslog
API
Agent
CEF
JSON
Cloud Connector
↓
Ingestion

The ingestion layer must handle the expected event rate.



8. EPS

EPS = Events Per Second

It is an important SIEM infrastructure metric.

Example:

Normal EPS = 2,000

Peak EPS = 5,000

Infrastructure planning should consider both normal and peak ingestion.



9. Data Volume

EPS is not the only consideration.

You also need to estimate data volume.

Example:

Average Event Size = 1 KB
EPS = 2,000

Approximate raw data per second:

$2,000 \times 1 \text{ KB}$
 $= 2,000 \text{ KB/sec}$
 $\approx 2 \text{ MB/sec}$

Daily volume:

$\approx 172.8 \text{ GB/day}$

This is only a simplified raw-data estimate.

Actual SIEM storage can be significantly different because of:

==> Indexing

==> Metadata

==> Replication

==> Compression

==> Retention

==> Internal overhead



10. Storage Planning

Storage is one of the most important SIEM infrastructure considerations.

You need to consider:

==> Daily ingestion

==> Retention period

==> Replication

==> Index overhead

==> Compression

==> Search requirements

==> Backup requirements

Example:

Daily Data = 100 GB

Retention = 30 days

Raw retention:

100×30

= 3 TB

But actual required storage may be higher.

11. Hot, Warm and Cold Data

Many SIEM architectures divide data by how frequently it is accessed.

Hot Data

Frequently searched.

Recent Events

Warm Data

Older but still accessible.

Historical Events

Cold Data

Long-term retention.

Archive

Conceptually:

```

Hot
↓
Warm
↓
Cold
↓
Archive / Delete
  
```

12. Retention Planning

Retention means how long security data is stored.

Example:

Hot → 7 days
Warm → 30 days
Archive → 1 year

These are only examples.

The actual retention period should follow:

==> Business requirements

==> Security requirements

==> Legal requirements

==> Compliance requirements

==> Storage capacity

==> Incident investigation needs



13. Storage Full Problem

One of the most dangerous infrastructure problems is:

Storage

↓

90%

↓

95%

↓

100%

Possible consequences:

==> Log ingestion failure

==> Indexing problems

==> Search problems

==> Service instability

==> Data loss

Therefore, storage monitoring is essential.



14. CPU Monitoring

SIEM systems perform many CPU-intensive tasks:

==> Parsing

==> Indexing

==> Searching

==> Correlation

==> Detection

==> Compression

Monitor:

CPU Usage
Load Average
Process CPU
System Load

A sudden increase may indicate:

==> Increased event volume

==> Expensive query

==> Detection problem

==> Misconfiguration

==> Infrastructure issue



15. Memory Monitoring

SIEM components can require significant memory.

Monitor:

==> RAM usage

==> Swap

==> JVM memory where applicable

==> Cache

==> Memory pressure

Example:

Memory

↓

80%

↓

90%

↓

Swap Activity

↓

Performance Degradation

Memory issues can affect both ingestion and search performance.



16. Disk I/O

SIEM workloads can generate heavy disk activity.

Monitor:

==> Read IOPS

==> Write IOPS

==> Disk latency

==> Throughput

==> Queue depth

High disk latency can affect:

Indexing
Search
Ingestion



17. Network Monitoring

SIEM infrastructure depends heavily on networking.

Monitor:

==> Bandwidth

==> Packet loss

==> Latency

==> Connections

==> Interface errors

==> Network saturation

Example:

```
Firewall
  ↓
Network
  ↓
Collector
  ↓
SIEM
```

If the network is unstable, logs may be delayed or lost.



18. Network Segmentation

SIEM infrastructure should be appropriately segmented.

For example:

```
Production Network
  ↓
  Firewall
  ↓
Security / SIEM Network
  ↓
  SIEM Cluster
```

Access should be restricted to required communication paths.



19. SIEM Server Hardening

The SIEM infrastructure itself must be secured.

Use:

- ==> Minimal services
- ==> Secure configuration
- ==> Patch management
- ==> Strong authentication
- ==> MFA where supported
- ==> RBAC
- ==> Firewall restrictions
- ==> Secure management access
- ==> File permissions
- ==> Audit logging
- ==> Endpoint protection where appropriate

Remember:

The SIEM is itself a high-value security asset.



20. Access Control

Use least privilege.

Different users may need different permissions:

SOC Analyst



Investigation Access

Detection Engineer



Detection Management

SIEM Engineer



Infrastructure Management

Administrator



Restricted Administrative Access

Avoid giving every user administrator permissions.

21. High Availability

HA = High Availability

The objective is to reduce service disruption.

Instead of:

Single SIEM Server

a larger environment may use:

SIEM Node 1
SIEM Node 2
SIEM Node 3

with appropriate clustering or redundancy.



22. Single Node vs Cluster

Single Node

Sources



SIEM

Advantages:

==> Simple

==> Easy to maintain

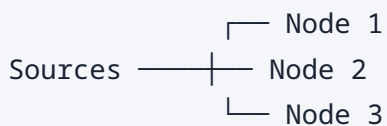
==> Lower infrastructure complexity

Limitations:

==> Limited scalability

==> Single point of failure

Cluster



Advantages:

==> Scalability

==> Redundancy

==> Better workload distribution

But:

==> More complexity

==> More infrastructure

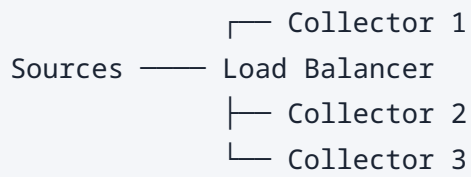
==> More monitoring

==> More operational requirements

23. Load Balancing

Large SIEM deployments may distribute incoming traffic across multiple nodes.

Conceptually:



This can help distribute workload.

24. Failover

Failover means moving workload to another available component when one fails.

Example:

```
Primary Collector
  ↓
  Failure
  ↓
Secondary Collector
```

Failover design should be tested rather than assumed to work.



25. Queue and Buffer

A buffer can temporarily hold logs when the downstream SIEM component is unavailable or slow.

Example:

```
Firewall
  ↓
Collector
  ↓
Buffer
  ↓
SIEM
```

If SIEM processing slows:

```
Buffer
  ↓
Temporary Storage
  ↓
SIEM Recovers
  ↓
Logs Forwarded
```

Buffering strategy depends on the architecture and available tooling.

26. Disaster Recovery

DR = Disaster Recovery

Ask:

What happens if the SIEM infrastructure fails?

Possible scenarios:

==> Server failure

==> Storage failure

==> Database/index failure

==> Network failure

==> Data corruption

==> Site failure

A DR plan should define:

Detection

↓

Recovery

↓

Validation

↓

Service Restoration

27. Backup

Backup requirements depend on what data and configuration need recovery.

Possible backup targets:

==> Configuration

==> Detection rules

==> Dashboards

==> Parsers

==> Important case data

==> Metadata

==> Critical supporting databases

Not every SIEM deployment should back up raw event data in the same way; retention and archival architecture should be designed deliberately.



28. Restore Testing

A backup is not enough.

You need to test restoration.

Backup

↓

Restore

↓

Validate

↓

Application Test

↓

Data Test

Important question:

Can we actually restore the SIEM when we need it?



29. RPO and RTO

RPO

Recovery Point Objective

How much data loss is acceptable after a failure.

Example:

RPO = 15 minutes

The organization may accept losing up to approximately 15 minutes of data, depending on the actual recovery design.

RTO

Recovery Time Objective

How quickly the service should be restored.

Example:

RTO = 1 hour

These should be defined according to business and security requirements.



30. SIEM Upgrade

SIEM platforms require maintenance and upgrades.

Before an upgrade:

```
Backup
↓
Check Compatibility
↓
Read Release Documentation
↓
Test
↓
Maintenance Window
↓
Upgrade
↓
Validate
↓
Monitor
```

Never assume an upgrade is risk-free.



31. Patch Management

Patch:

==> Operating System

==> SIEM components

==> Database/index components

==> Agents

==> Supporting software

Prioritize patches based on:

==> Security risk

==> Exposure

==> Vendor guidance

==> Business impact

==> Compatibility



32. SIEM Health Monitoring

A SIEM Engineer should continuously monitor:

- CPU
- Memory
- Disk
- Network
- EPS
- Queue
- Latency
- Service Status
- Index Health
- Storage
- Agent Health
- Search Performance

Example:

SIEM Health Dashboard

Ingestion	→ Healthy
Storage	→ 68%
CPU	→ 54%
Memory	→ 62%
EPS	→ 2,340
Indexing	→ Healthy
Search	→ Normal
Collectors	→ Healthy



33. Log Pipeline Health

A healthy SIEM requires a healthy data pipeline.

Monitor:

```
graph TD; Source --> Collector; Collector --> Transport; Transport --> Ingestion; Ingestion --> Parsing; Parsing --> Indexing; Indexing --> Storage;
```

Source
↓
Collector
↓
Transport
↓
Ingestion
↓
Parsing
↓
Indexing
↓
Storage

If one stage fails, the entire pipeline can be affected.



34. Detecting Missing Logs

Suppose:

Firewall logs stopped arriving.

Troubleshooting:

```
Firewall
  ↓
Network
  ↓
Port
  ↓
Collector
  ↓
Parser
  ↓
Indexer
  ↓
SIEM Search
```

Check each stage.

Do not immediately assume the SIEM itself is broken.



35. Log Latency

Logs may arrive late.

Example:

```
Event Time = 10:00  
SIEM Time  = 10:08
```

Latency:

8 minutes

High latency can affect real-time detection.

36. Search Performance

Slow searches can affect SOC operations.

Possible causes:

- ==> Very large time range
- ==> High-cardinality fields
- ==> Expensive joins/correlations
- ==> Poor query design
- ==> Insufficient resources
- ==> Too much data
- ==> Poor indexing strategy

Improve performance through:

- ==> Better queries
- ==> Appropriate indexes/data streams
- ==> Time filtering
- ==> Data lifecycle management
- ==> Resource scaling

The exact optimization techniques depend on the SIEM platform.



37. Capacity Planning

Capacity planning means preparing infrastructure for future growth.

Consider:

```
Current EPS
+
Peak EPS
+
Expected Growth
+
Retention
+
Replication
+
Search Load
+
Detection Load
```

Example:

```
Current EPS = 2,000
Expected Growth = 50%

Future EPS ≈ 3,000
```

Plan infrastructure before reaching the limit.



38. Scaling

There are two common approaches.

Vertical Scaling

Increase resources on existing systems.

CPU ↑
RAM ↑
Storage ↑

Horizontal Scaling

Add more systems.

Node 1
Node 2
Node 3

Horizontal scaling can provide greater distribution but usually increases operational complexity.



39. Data Tiering

A mature architecture may use:

```
Hot Storage
  ↓
Warm Storage
  ↓
Cold Storage
  ↓
Archive
```

This balances:

==> Search performance

==> Storage cost

==> Retention

==> Investigation requirements



40. Encryption

Security data should be protected.

Consider encryption:

```
Log Source
  ↓
Encrypted Transport
  ↓
SIEM
  ↓
Encrypted Storage
```

For network transport, secure protocols such as TLS may be appropriate where supported.



41. Certificate Management

If TLS is used, certificates must be managed.

Monitor:

==> Expiration

==> Trust chain

==> Hostname validation

==> Key rotation

==> Certificate deployment

A certificate expiration can unexpectedly break log ingestion.



42. Infrastructure Automation

SIEM infrastructure benefits from automation.

Possible tools include:

==> Ansible

==> Terraform

==> Shell scripting

==> Python

==> CI/CD

Example:

```
Configuration
  ↓
Git
  ↓
Automation
  ↓
SIEM Infrastructure
```

Infrastructure-as-Code can make deployments more consistent.



43. Configuration Management

Important configurations should be documented and version controlled where appropriate.

Examples:

==> Parsers

==> Detection rules

==> Dashboards

==> Deployment configuration

==> Automation scripts

==> Infrastructure configuration

This helps with:

Change



Review



Testing



Deployment



Rollback

44. Change Management

Before changing production SIEM infrastructure:

```
Change Request
↓
Risk Assessment
↓
Testing
↓
Approval
↓
Implementation
↓
Validation
↓
Documentation
```

This is particularly important for changes affecting:

==> Log ingestion

==> Detection

==> Storage

==> Authentication

==> Network access

45. SIEM Infrastructure Testing

Test:

==> Ingestion

==> Parsing

==> Indexing

==> Searching

==> Detection

==> Alerting

==> Dashboard

==> API

==> Backup

==> Restore

==> Failover

==> Recovery

Example:

Firewall Event

↓

SIEM

↓

Detection

↓

Alert

↓

Dashboard

Verify the complete path.



46. Common Infrastructure Failures

Failure 1 — Storage Full

Logs stop

Failure 2 — CPU Saturation

Processing slows

Failure 3 — Memory Pressure

Services become unstable

Failure 4 — Network Failure

Logs are delayed/lost

Failure 5 — Certificate Expired

Secure connection fails

Failure 6 — Broken Parser

Events arrive
but important fields are missing

Failure 7 — Index Failure

Data may not be searchable



47. SIEM Troubleshooting Method

Use a structured approach:

```
Problem
↓
Define Scope
↓
Check Recent Changes
↓
Check Service Health
↓
Check Network
↓
Check Logs
↓
Check Resources
↓
Check Data Pipeline
↓
Identify Root Cause
↓
Fix
↓
Validate
↓
Document
```



48. Root Cause Analysis

Do not stop at:

“The logs were missing.”

Ask:

Why were logs missing?

Maybe:

```
Firewall
  ↓
Configuration Changed
  ↓
Syslog Destination Wrong
  ↓
Collector Received Nothing
```

Root cause:

Incorrect Syslog Configuration

This produces a better long-term solution.



49. SIEM Infrastructure Monitoring Dashboard

A useful infrastructure dashboard:

SIEM INFRASTRUCTURE	
EPS	2,340
Storage	68%
CPU	54%
Memory	62%
Search Latency	Normal
Collectors	
Collector-01	Healthy
Collector-02	Healthy
Collector-03	Warning
Index Health	
Queue	Normal

50. Practical SIEM Infrastructure Lab

You can build a learning environment using:

```
Linux VM
+
Wazuh
+
Docker
+
Windows VM
+
Linux VM
+
Network Logs
```

Architecture:

```
Windows
  ↓
Linux
  ↓
Wazuh Agent
  ↓
Wazuh Server
  ↓
Indexer
  ↓
Dashboard
```

Monitor:

==> CPU

==> RAM

==> Disk

==> Network

==> Agent status

==> Alert rate

==> Storage



51. Docker-Based SIEM Lab

A development lab can use containers where the selected SIEM platform supports containerized deployment.

Conceptually:

Docker Host

|

├─ SIEM Component

├─ Index Component

├─ Dashboard

└─ Supporting Services

For production, always follow the platform's supported architecture and deployment requirements.



52. SIEM Infrastructure Security Checklist

- [] Server hardening
- [] Strong authentication
- [] RBAC
- [] MFA where supported
- [] Least privilege
- [] Network segmentation
- [] TLS where appropriate
- [] Secure secret storage
- [] Patch management
- [] Backup
- [] Restore testing
- [] Audit logging
- [] Monitoring
- [] Capacity planning
- [] Storage monitoring
- [] Certificate monitoring
- [] Failover planning
- [] Disaster recovery
- [] Change management



53. SIEM Engineer Interview Questions

Q1. What is SIEM infrastructure?

Answer:

SIEM infrastructure includes the systems, network, storage, collectors, processing components, indexing/search components, security controls, monitoring, and supporting services required to operate a SIEM reliably.

Q2. What is EPS?

Answer:

EPS means Events Per Second and represents the rate at which events are generated or ingested, depending on where it is measured.

Q3. How do you calculate SIEM storage requirements?

Answer:

I would consider average and peak ingestion rate, event size, daily data volume, retention period, indexing overhead, compression, replication, and backup or archive requirements.

Q4. What happens when SIEM storage becomes full?

Answer:

Depending on the platform, ingestion, indexing, or other services may become degraded or stop. I would monitor storage proactively and implement appropriate retention, tiering, alerting, and capacity planning.

Q5. What is High Availability?**Answer:**

High Availability is an architecture designed to reduce service disruption by providing redundancy and failover mechanisms.

Q6. What is the difference between vertical and horizontal scaling?**Answer:**

Vertical scaling increases resources on an existing system, such as CPU or RAM. Horizontal scaling adds additional systems or nodes to distribute workload.

Q7. How would you troubleshoot missing logs?**Answer:**

I would trace the complete pipeline: source configuration, network connectivity, transport port, collector, ingestion, parsing, indexing, and SIEM search. I would also check timestamps, queues, service health, and recent configuration changes.

Q8. Why is log latency important?**Answer:**

High log latency can delay detection and investigation, especially for security events that require near-real-time monitoring.

Q9. What is RPO?

Answer:

RPO, or Recovery Point Objective, defines the acceptable amount of data loss measured in time after a disruption.

Q10. What is RTO?

Answer:

RTO, or Recovery Time Objective, defines the target time within which a service should be restored after a disruption.

Q11. Why is SIEM backup important?

Answer:

Backup helps recover important configurations and other required data after failure, corruption, or operational mistakes. Backup requirements should be based on the organization's recovery design.

Q12. Why should restore testing be performed?

Answer:

Because a backup that has never been restored cannot be assumed to be recoverable. Restore testing validates that the backup and recovery process actually works.

Q13. What SIEM infrastructure metrics would you monitor?

Answer:

I would monitor EPS, CPU, memory, disk utilization, disk I/O, network utilization, queue depth, ingestion latency, indexing health, search performance, service status, and storage capacity.

Q14. How would you plan SIEM capacity?

Answer:

I would analyze current and peak EPS, data volume, expected growth, retention, replication, search workload, detection workload, and available resources, then plan capacity with sufficient headroom.



54. Complete SIEM Infrastructure Checklist

- [] Understand SIEM architecture
- [] Understand log flow
- [] Understand collectors
- [] Understand ingestion
- [] Understand indexing
- [] Understand storage
- [] Calculate data volume
- [] Understand EPS
- [] Monitor CPU
- [] Monitor RAM
- [] Monitor Disk
- [] Monitor Network
- [] Monitor ingestion latency
- [] Monitor queues
- [] Plan retention
- [] Plan storage tiers
- [] Implement access control
- [] Harden infrastructure
- [] Plan HA
- [] Plan DR
- [] Configure backups

☐ Test restoration

☐ Plan RPO/RT0

☐ Monitor certificates

☐ Automate configuration

☐ Document changes

☐ Test upgrades

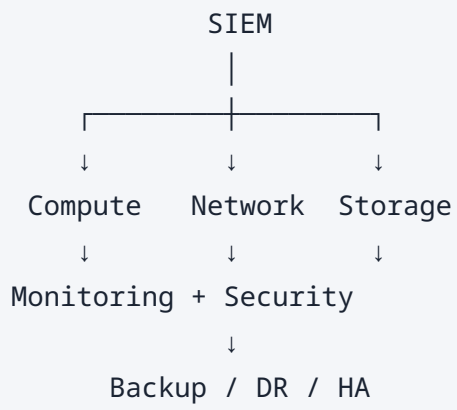
☐ Monitor continuously

55. Complete SIEM Engineer Infrastructure Flow

Remember this complete flow:

```
Log Sources
  ↓
Collectors
  ↓
Network / Transport
  ↓
Ingestion
  ↓
Parsing
  ↓
Normalization
  ↓
Processing
  ↓
Indexing
  ↓
Storage
  ↓
Detection
  ↓
Alert
  ↓
Investigation
  ↓
Dashboard
  ↓
Reporting
```

And infrastructure operates underneath the entire pipeline:



56. Complete SIEM Engineer Skill Map

After completing this series, you should understand:

```
graph TD; 01[01. Log Management] --> 02[02. SIEM Platforms]; 02 --> 03[03. Log Integration]; 03 --> 04[04. Detection Engineering]; 04 --> 05[05. Query & Investigation]; 05 --> 06[06. SOC Operations]; 06 --> 07[07. Threat Intelligence]; 07 --> 08[08. Automation & SOAR]; 08 --> 09[09. Dashboard & Reporting]; 09 --> 10[10. SIEM Infrastructure];
```

01. Log Management
↓
02. SIEM Platforms
↓
03. Log Integration
↓
04. Detection Engineering
↓
05. Query & Investigation
↓
06. SOC Operations
↓
07. Threat Intelligence
↓
08. Automation & SOAR
↓
09. Dashboard & Reporting
↓
10. SIEM Infrastructure

These areas are strongly connected.



57. Final Takeaway

A SIEM Engineer is not simply someone who installs a SIEM.

A strong SIEM Engineer understands the complete security data lifecycle:

Collect → Integrate → Normalize → Detect → Investigate → Automate → Visualize → Maintain

And underneath everything:

Infrastructure → Performance → Storage → Availability → Security

The most important mindset is:

A SIEM should not only collect security data. It should reliably transform that data into useful security visibility.



SIEM Engineer — Complete Learning Series

- #01 — Log Management
 - #02 — SIEM Platforms
 - #03 — Log Integration
 - #04 — Detection Engineering
 - #05 — Query & Investigation
 - #06 — SOC Operations
 - #07 — Threat Intelligence
 - #08 — Automation & SOAR
 - #09 — Dashboard & Reporting
 - #10 — SIEM Infrastructure ← You are here
 - #11 — SIEM Architecture Design → Next
 - #12 — SIEM Capacity Planning
 - #13 — SIEM Performance Tuning
 - #14 — Advanced Detection Engineering
 - #15 — Threat Hunting with SIEM
 - #16 — SIEM Data Quality & Troubleshooting
 - #17 — Hands-On SIEM: Multi-Platform Practical Guide
-

What Next?

After completing these core topics, continue with #11 Architecture Design → #12–#13 sizing & tuning → #14–#16 advanced topics, then finish with the #17 hands-on capstone.

A good learning path is:

```
graph TD; Theory --> Lab; Lab --> Build_SIEM[Build SIEM]; Build_SIEM --> Integrate_Logs[Integrate Logs]; Integrate_Logs --> Create_Detection[Create Detection]; Create_Detection --> Investigate_Alerts[Investigate Alerts]; Investigate_Alerts --> Automate_Response[Automate Response]; Automate_Response --> Build_Dashboard[Build Dashboard]; Build_Dashboard --> Monitor_Infrastructure[Monitor Infrastructure]; Monitor_Infrastructure --> Document_Everything[Document Everything];
```

Try to build your own small SOC lab instead of only reading theory.

Navigation

← **Previous Article: #09 — Dashboard & Reporting**

SIEM Engineer Index

Next Article: #11 — SIEM Architecture Design

SIEM Engineer #11 — SIEM Architecture Design

A SIEM Engineer শুধু SIEM platform install বা alert monitor করে না।

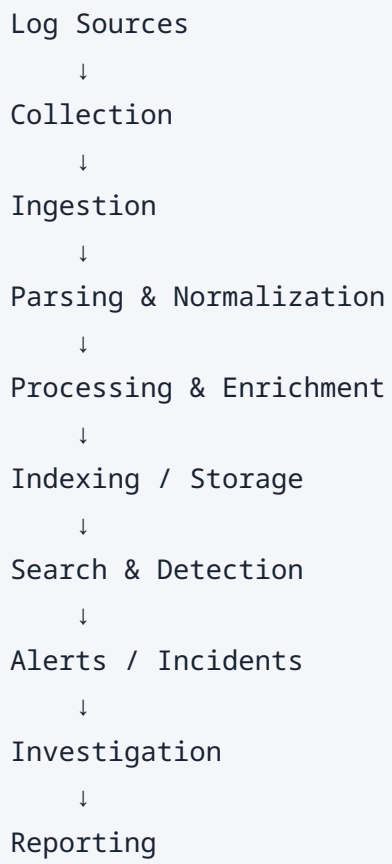
একজন SIEM Engineer-এর গুরুত্বপূর্ণ দায়িত্ব হলো এমন একটি **SIEM Architecture** design করা, যেখানে হাজার হাজার বা millions of security events reliably collect, process, store, search, detect এবং investigate করা যায়।

এই article-এ আমরা SIEM Architecture-এর foundation থেকে **Enterprise Architecture, Single-node vs Distributed Architecture, Collector, Indexer, Search Layer, High Availability (HA), Scalability, Security, Disaster Recovery** এবং **Real-World Design** পর্যন্ত শিখব।

1. What is SIEM Architecture?

SIEM Architecture হলো একটি SIEM environment-এর বিভিন্ন components কীভাবে একে অপরের সাথে connected এবং কাজ করবে তার overall design।

একটি সাধারণ SIEM flow:



উদাহরণ:

```

Firewall
Windows
Linux
Active Directory
Cloud
EDR
Network Devices
Applications
    |
    v
+-----+
| Log Collectors |
+-----+
    |
    v
+-----+
| Ingestion      |
+-----+
    |
    v
+-----+
| Processing     |
| Parsing        |
| Normalization  |
| Enrichment     |
+-----+
    |
    v
+-----+
| Index / Storage|
+-----+
    |
    v
+-----+
| Search / SIEM  |
+-----+
    |
    v
+-----+
| Detection      |
+-----+

```

|

v

+-----+

| SOC Analyst |

+-----+



2. Why does SIEM Architecture matter?

একটি SIEM ছোট environment-এ খুব সহজ হতে পারে।

কিন্তু environment বড় হলে কয়েকটি সমস্যা দেখা দেয়:

==> অনেক বেশি logs

==> High EPS

==> Storage growth

==> Search performance

==> Network latency

==> Single Point of Failure

==> Data loss

==> High availability requirement

==> Multiple SOC analysts

==> Compliance requirements

==> Disaster Recovery requirement

তাই SIEM deploy করার আগে architecture design করা অত্যন্ত গুরুত্বপূর্ণ।



3. SIEM Architecture Design Principles

একটি ভালো architecture design করার সময় কয়েকটি বিষয় মাথায় রাখতে হয়।

মূল principles:

==> Scalability

==> Availability

==> Performance

==> Security

==> Reliability

==> Maintainability

==> Observability

==> Disaster Recovery

==> Cost efficiency

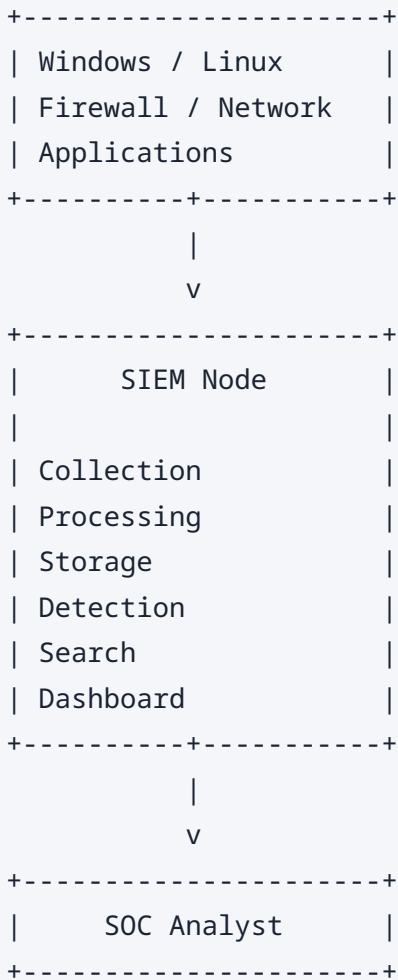
একটি ভালো architecture শুধু আজকের **workload** handle করবে না।

এটি future growth-এর জন্যও প্রস্তুত থাকবে।



4. Small SIEM Architecture

ছোট organization বা lab environment-এর জন্য single-node architecture যাথেষ্ট হতে পারে।



এখানে একটি server অনেকগুলো কাজ করতে পারে।

==> Log collection

==> Processing

==> Storage

==> Detection

==> Search

==> Dashboard

সুবিধা

==> Simple deployment

==> Low cost

==> Easy management

==> Lab-এর জন্য ভালো

সীমাবদ্ধতা

==> Single Point of Failure

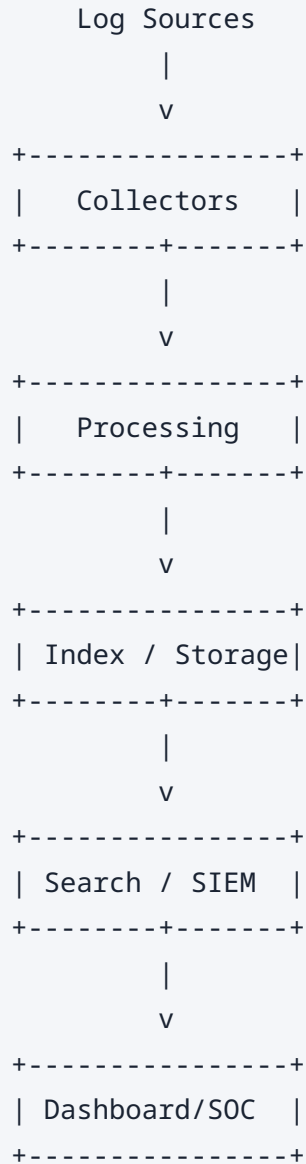
==> Limited scalability

==> Resource contention

==> Large workload-এ performance সমস্যা হতে পারে

5. Medium SIEM Architecture

Medium environment-এ components আলাদা করা শুরু করা যায়।



এতে workload আলাদা করা যায়।

উদাহরণ:

==> Collector শুধু logs collect করবে

==> Processing layer parsing করবে

==> Indexer data store করবে

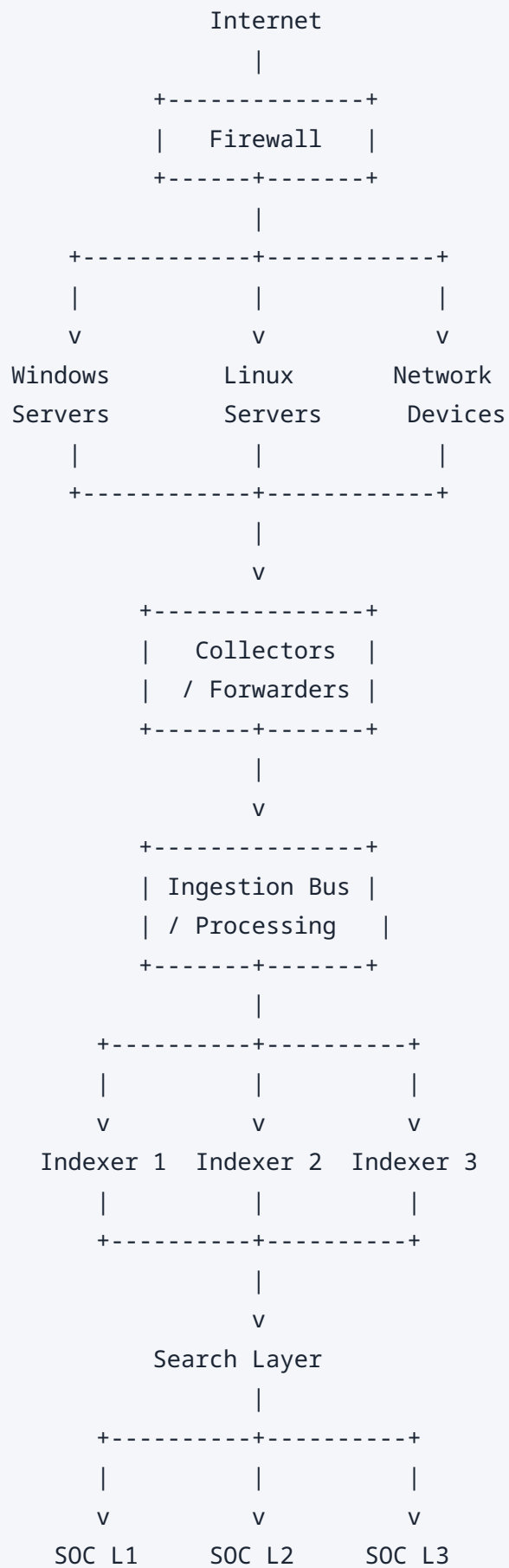
==> Search layer queries handle করবে

==> Dashboard analyst interface provide করবে



6. Enterprise SIEM Architecture

Large enterprise environment-এ distributed architecture বেশি প্রয়োজন হয়।



এখানে system-এর বিভিন্ন অংশ আলাদা করে scale করা যায়।

7. Centralized vs Distributed Architecture

Centralized Architecture

সব logs একটি central SIEM environment-এ পাঠানো হয়।

Sources

```
|
+-----> SIEM
|
+-----> SIEM
|
+-----> SIEM
```

সুবিধা

- ==> Simple management
- ==> Centralized visibility
- ==> Easy investigation
- ==> Easier policy management

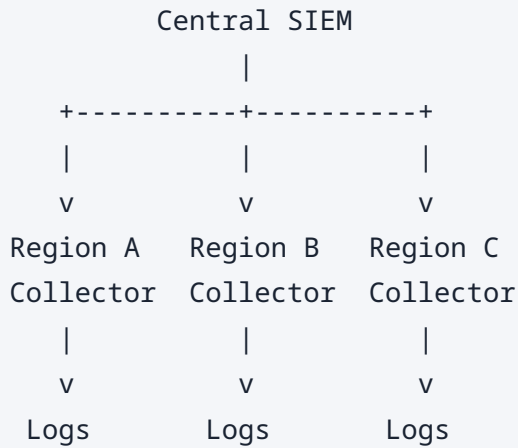
সমস্যা

- ==> Network dependency
- ==> Central bottleneck হতে পারে
- ==> Large scale-এ scalability challenge



8. Distributed Architecture

Large environment-এ বিভিন্ন location বা network segment-এ collectors রাখা যায়।



উদাহরণ:

==> Dhaka Data Center

==> Chattogram Data Center

==> Cloud Environment

==> Remote Office

প্রতিটি location local collection করতে পারে এবং central SIEM-এ normalized data পাঠাতে পারে।



9. Single-node vs Multi-node

Single-node

একটি node বিভিন্ন services চালায়।

```
+-----+
|      SIEM Node      |
|                      |
| Collector            |
| Processing           |
| Storage              |
| Search               |
| Detection             |
| Dashboard            |
+-----+
```

এটি সাধারণত:

==> Lab

==> Small organization

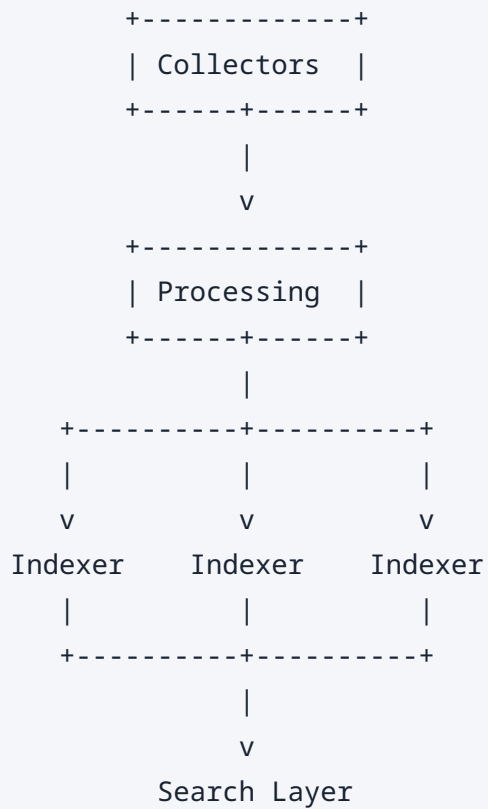
==> POC

==> Training environment

-এর জন্য ব্যবহারযোগ্য।

Multi-node

Different workloads different nodes-এ distribute করা হয়।



সুবিধা

==> Better scalability

==> Better availability

==> Workload separation

==> Easier capacity planning

==> Better performance isolation



10. Collector

Collector হলো এমন component যা বিভিন্ন source থেকে logs/events collect করে।

Sources হতে পারে:

==> Windows

==> Linux

==> Firewall

==> Router

==> Switch

==> Active Directory

==> Database

==> Web Server

==> Application

==> Cloud service

==> EDR

==> IDS/IPS

Collector-এর কাজ:

Source



Collect



Buffer



Forward



11. Forwarder

Forwarder একটি source বা intermediate system থেকে logs SIEM-এর দিকে forward করে।

```
Server
  |
  v
Forwarder
  |
  v
Collector
  |
  v
SIEM
```

Forwarder architecture-এর সুবিধা:

==> Network traffic control

==> Local buffering

==> Reliable forwarding

==> Centralized ingestion

==> Remote site support



12. Ingestion Layer

Ingestion Layer হলো SIEM-এর সেই অংশ যেখানে collected events system-এর ভিতরে প্রবেশ করে।

```
Logs
↓
Network Transport
↓
Ingestion
↓
Queue / Buffer
↓
Processing
```

Ingestion layer-এর গুরুত্বপূর্ণ বিষয়:

==> Throughput

==> Buffering

==> Backpressure

==> Reliability

==> Authentication

==> Encryption

==> Error handling



13. Processing Layer

Raw logs সরাসরি investigation-এর জন্য সবসময় useful নয়।

Processing layer data-কে usable format-এ convert করে।

```
Raw Log
  ↓
Parsing
  ↓
Normalization
  ↓
Field Extraction
  ↓
Enrichment
  ↓
Correlation
  ↓
Processed Event
```

উদাহরণ:

```
Raw:
Failed password for admin from 10.10.10.50

Normalized:

event.category = authentication
event.action   = login_failed
user.name      = admin
source.ip      = 10.10.10.50
```

এতে detection এবং investigation সহজ হয়।

14. Indexer / Storage Layer

Processed events storage/indexing system-এ যায়।

Processed Events

|

v

+-----+

| Indexer |

+-----+

|

v

+-----+

| Storage |

+-----+

Indexer-এর primary responsibilities:

==> Index events

==> Store searchable data

==> Support queries

==> Manage data lifecycle

==> Support retention policies

Large deployment-এ একাধিক indexer ব্যবহার করা যেতে পারে।



15. Search Layer

Search layer analysts এবং detection systems-কে data query করতে সাহায্য করে।

```
SOC Analyst
  |
  v
Search Query
  |
  v
Search Layer
  |
  v
Indexes
  |
  v
Results
```

উদাহরণ:

```
Find all failed logins
from source IP 10.10.10.50
during the last 24 hours.
```

Search layer-এর performance SIEM-এর overall user experience-এ বড় ভূমিকা রাখে।



16. Dashboard Layer

Dashboard হলো analyst এবং management-এর জন্য visualization layer ।

এখানে থাকতে পারে:

==> Security Alerts

==> Authentication Events

==> Failed Logins

==> Malware Alerts

==> Firewall Events

==> Top Source IPs

==> Top Destination IPs

==> Detection Trends

==> Incident Metrics

==> System Health



17. Detection Layer

Detection engine incoming events analyse করে suspicious activity identify করে।

```
graph TD; Event --> DR[Detection Rule]; DR --> CM[Condition Match?]; CM --> Yes; Yes --> Alert; Alert --> SOC[SOC Investigation];
```

Detection logic হতে পারে:

==> Signature-based

==> Threshold-based

==> Correlation-based

==> Behavioral

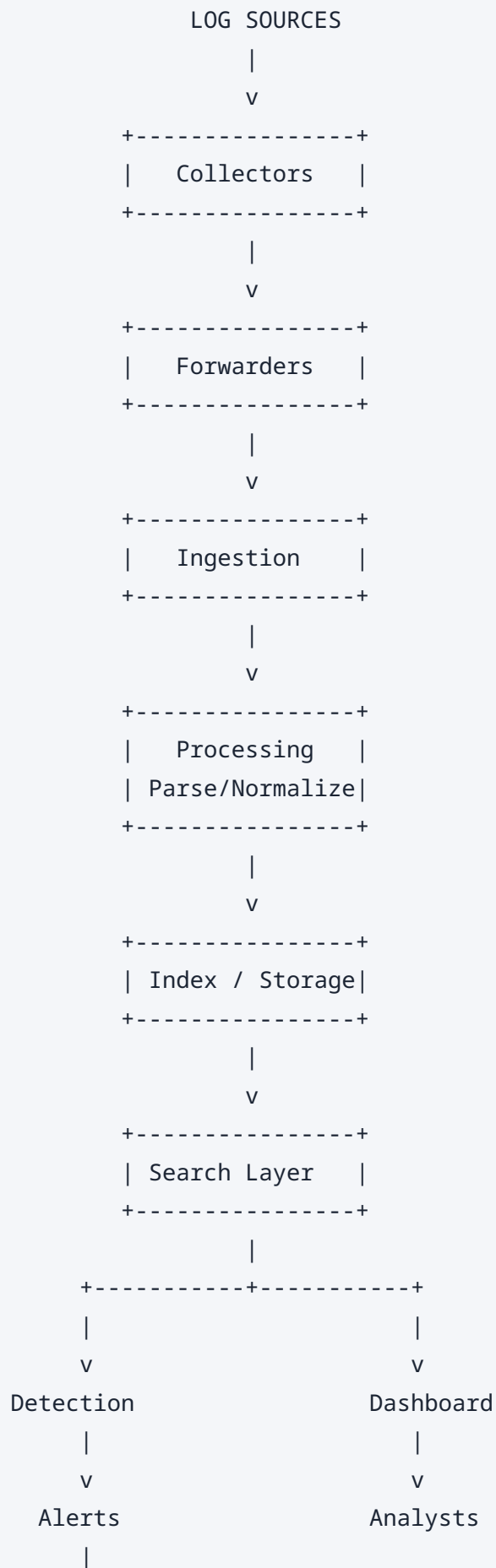
==> Sequence-based

==> Risk-based

Detection Engineering নিয়ে বিস্তারিত আমরা **#14 Advanced Detection Engineering**-এ দেখব।

18. SIEM Components — Complete View

একটি enterprise SIEM-কে এভাবে চিন্তা করতে পারেন:



v
Incident



19. High Availability — HA

High Availability (HA) মানে system-এর একটি component failure হলেও service যতটা সম্ভব available রাখা।

ধরুন:

Collector

|

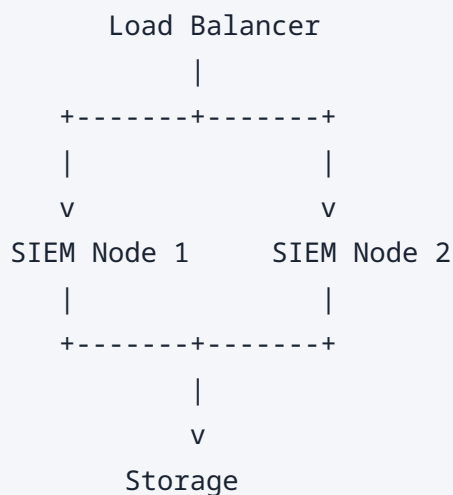
v

SIEM Node

SIEM node down হলে ingestion বন্ধ হয়ে যেতে পারে।

এটি Single Point of Failure।

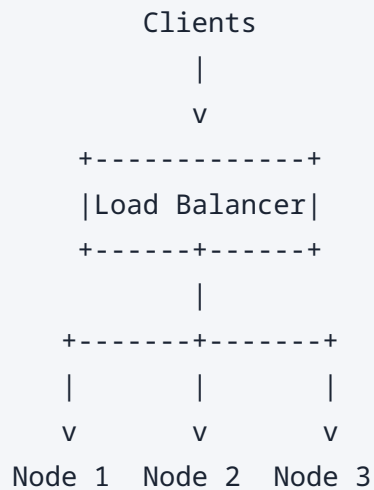
HA architecture:



এতে একটি node failure হলেও অন্য node service continue করতে পারে—যদি platform এবং architecture সেই failover support করে।

20. Load Balancing

একটি node-এর উপর সব workload না দিয়ে traffic distribute করা যায়।



Load balancing ব্যবহার করা যায়:

==> Log ingestion

==> API traffic

==> Search requests

==> Web interface

==> Collector traffic



21. Scalability

Scalability হলো workload বাড়লে system-এর capacity বাড়ানোর ability।

দুই ধরনের scaling গুরুত্বপূর্ণ।

Vertical Scaling

একটি server-এর resources বাড়ানো।

8 CPU
32 GB RAM
↓
16 CPU
64 GB RAM

Horizontal Scaling

আরও nodes যোগ করা।

Node 1
Node 2
Node 3
↓
Node 4
Node 5

Large SIEM environment-এ workload অনুযায়ী horizontal scaling গুরুত্বপূর্ণ হতে পারে।

22. Architecture Design-এর আগে কী জানতে হবে?

SIEM architecture design করার আগে requirements collect করতে হবে।

Questions:

==> কতগুলো log source আছে?

==> কত EPS generate হচ্ছে?

==> Peak EPS কত?

==> প্রতিদিন কত GB logs?

==> Retention কতদিন?

==> কতজন analyst?

==> কতগুলো office/data center?

==> Cloud environment আছে?

==> Compliance requirement কী?

==> HA requirement কী?

==> RPO কী?

==> RTO কী?

==> Future growth কত?

এই প্রশ্নগুলোর উত্তর ছাড়া architecture blindly design করা উচিত নয়।



23. EPS — Events Per Second

SIEM architecture-এর সবচেয়ে গুরুত্বপূর্ণ sizing inputs-এর একটি হলো **EPS**।

EPS মানে:

Events Per Second

উদাহরণ:

Average EPS = 5,000

Peak EPS = 12,000

Architecture design করার সময় শুধু average EPS দেখলে হবে না।

Peak workload consider করতে হবে।

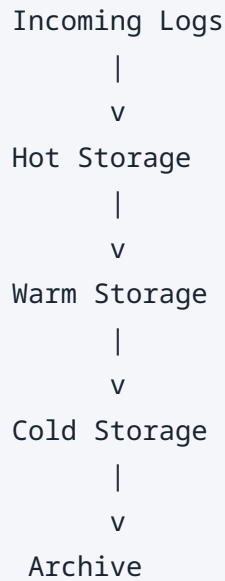
কারণ attack বা incident-এর সময় event volume অনেক বেড়ে যেতে পারে।

Capacity Planning আমরা #12-এ বিস্তারিত করব।

24. Storage Architecture

SIEM environment-এ storage খুব গুরুত্বপূর্ণ।

একটি conceptual storage model:



Hot

Frequently searched data।

Warm

Less frequently accessed data।

Cold

Long-term retention-এর জন্য।

Archive

Compliance বা historical requirement-এর জন্য রাখা data।

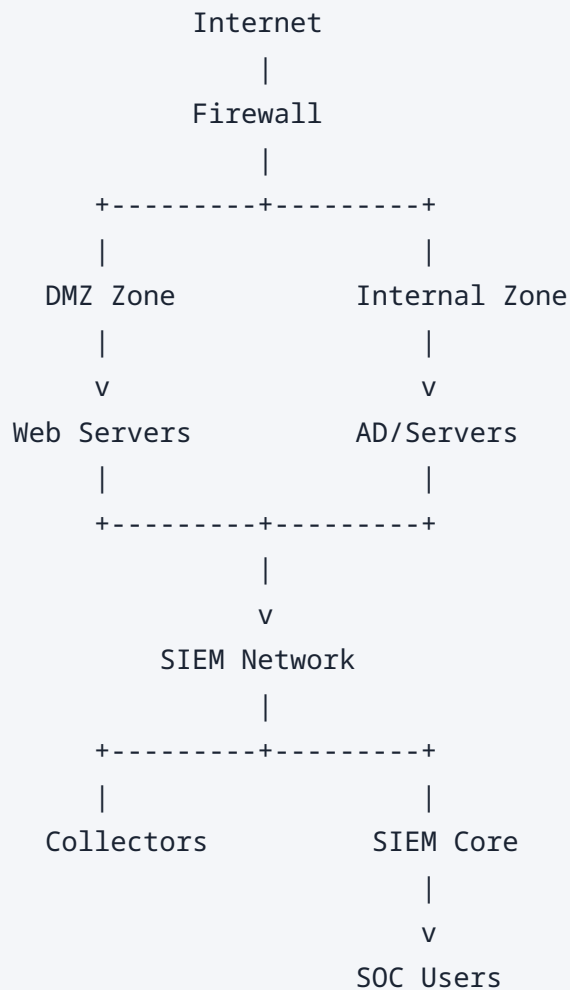
Actual tiers এবং capabilities platform অনুযায়ী আলাদা হতে পারে।



25. SIEM Network Architecture

SIEM environment-কে security zones-এর মধ্যে design করা ভালো।

উদাহরণ:



Principles:

==> Least privilege

==> Network segmentation

==> Restricted management access

==> TLS where supported

==> Firewall rules

==> Administrative access control

==> Monitoring

26. SIEM Security Architecture

SIEM নিজেই একটি highly sensitive system।

কারণ SIEM-এর মধ্যে থাকতে পারে:

==> Authentication logs

==> User information

==> IP addresses

==> Security alerts

==> Investigation data

==> Threat intelligence

==> Incident information

তাই SIEM security গুরুত্বপূর্ণ।

Security controls:

==> RBAC

==> MFA যেখানে supported

==> TLS encryption

==> Secure API authentication

==> Secrets management

==> Admin network restriction

==> Audit logging

==> Certificate management

==> Regular patching

==> Backup protection

==> Strong authentication

27. Role-Based Access Control — RBAC

সব analyst-এর একই permission থাকা উচিত নয়।

Example:

```

SOC L1
  ↓
Read + Triage

SOC L2
  ↓
Investigation + Case Management

SOC L3
  ↓
Advanced Investigation + Detection

SIEM Engineer
  ↓
Configuration + Integration

Administrator
  ↓
Platform Administration
  
```

Exact roles organization এবং platform অনুযায়ী আলাদা হবে।



28. On-Prem SIEM Architecture

On-prem environment:

```
Servers
  |
Network
  |
Collectors
  |
SIEM Infrastructure
  |
Storage
  |
SOC
```

Advantages:

==> Infrastructure control

==> Data residency control

==> Custom networking

==> Internal integration

Challenges:

==> Hardware cost

==> Maintenance

==> Scaling

==> Backup

==> DR

==> Infrastructure management

29. Cloud SIEM Architecture

Cloud SIEM environment-এ cloud services এবং cloud-native data sources integrate করা হয়।

```
Cloud Services
  |
  v
Cloud Logs
  |
  v
SIEM / Analytics
  |
  v
Detection
  |
  v
SOC
```

Considerations:

==> API integration

==> Cloud identity

==> Network connectivity

==> Data transfer

==> Storage cost

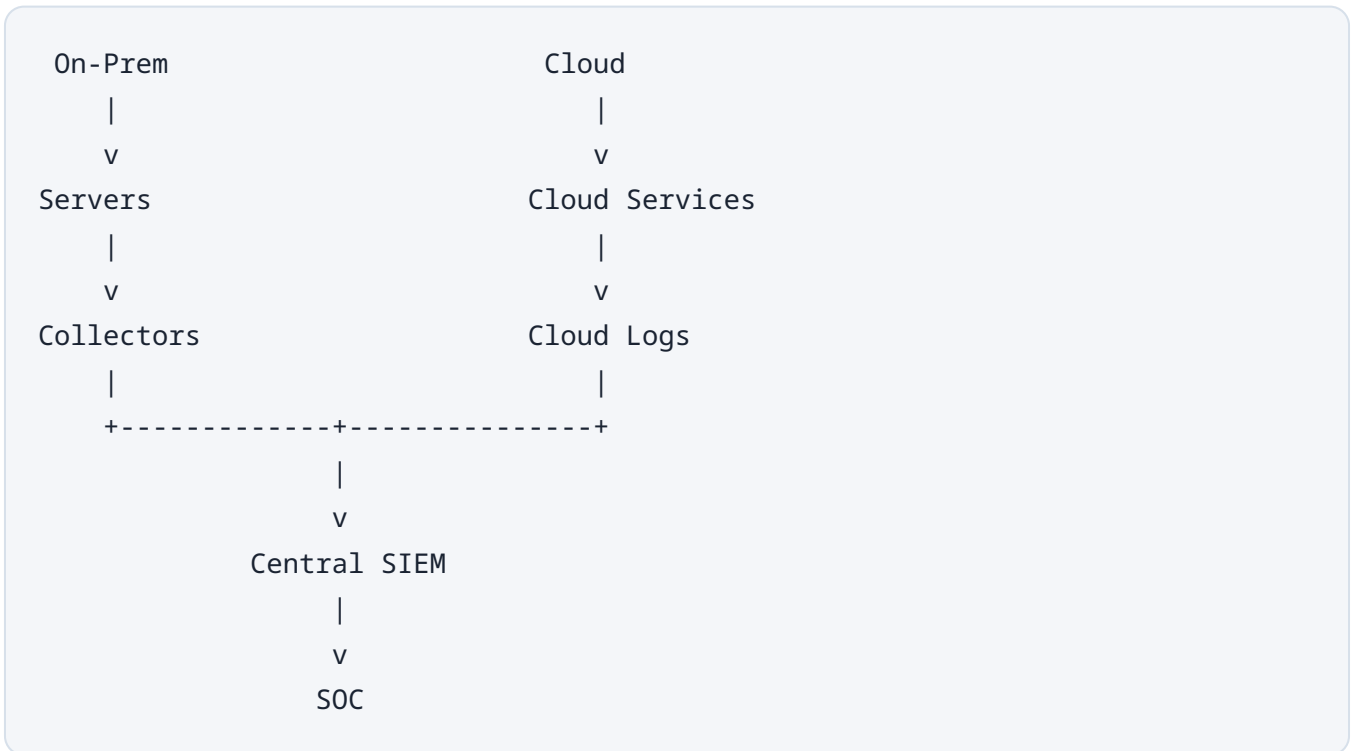
==> Access control

==> Multi-account architecture



30. Hybrid SIEM Architecture

অনেক organization-এর on-prem + cloud দুই environment থাকে।



Hybrid architecture-এ data integration এবং secure connectivity খুব গুরুত্বপূর্ণ।



31. Disaster Recovery — DR

SIEM architecture শুধু primary environment নিয়ে design করা উচিত নয়।

ধরুন:

Primary SIEM

|

X

Failure

|

v

DR SIEM

DR planning-এর গুরুত্বপূর্ণ বিষয়:

==> Backup

==> Replication যেখানে supported

==> Configuration backup

==> Detection rule backup

==> Dashboard backup

==> Integration configuration

==> Certificate/secret recovery process

==> Restore testing

32. RPO and RTO

RPO — Recovery Point Objective

Failure-এর পরে সর্বোচ্চ কত data loss acceptable?

Example:

RPO = 15 minutes

মানে organization এমন recovery design করতে চায় যাতে data-loss window roughly 15 minutes-এর মধ্যে রাখা যায়।

RTO — Recovery Time Objective

Service কত সময়ের মধ্যে restore করতে হবে?

Example:

RTO = 1 hour

অর্থাৎ recovery process-এর target হলো service প্রায় 1 hour-এর মধ্যে restore করা।

Actual targets organization-এর business requirements অনুযায়ী নির্ধারিত হবে।



33. Observability & Health Monitoring

SIEM-এর নিজের health monitor করতে হবে।

Monitor করুন:

==> EPS

==> Ingestion rate

==> Event latency

==> Queue size

==> CPU

==> RAM

==> Disk usage

==> Disk I/O

==> Network

==> Search latency

==> Failed ingestion

==> Collector status

==> Index health

==> Certificate expiry

==> Service availability

একজন SIEM Engineer-এর জন্য **SIEM health monitoring** অত্যন্ত গুরুত্বপূর্ণ।



✖

34. Common SIEM Architecture Mistakes

✖

Mistake 1 — শুধু current workload দেখা

Future growth consider না করলে কয়েক মাস পরে capacity problem হতে পারে।

✖

Mistake 2 — Single Point of Failure

একটি server-এর উপর পুরো SIEM depend করা।

✖

Mistake 3 — Peak EPS ignore করা

Average EPS দিয়ে architecture design করা।

✖

Mistake 4 — Storage underestimate করা

Retention এবং growth ঠিকভাবে calculate না করা।

✖

Mistake 5 — Network architecture ignore করা

সব logs direct central system-এ পাঠানো কিন্তু bandwidth এবং latency analysis না করা।

✖

Mistake 6 — Security ignore করা

SIEM-কে normal application server হিসেবে treat করা।

✖

Mistake 7 — Backup test না করা

Backup আছে কিন্তু restore কখনও test করা হয়নি।

✖

Mistake 8 — Monitoring না রাখা

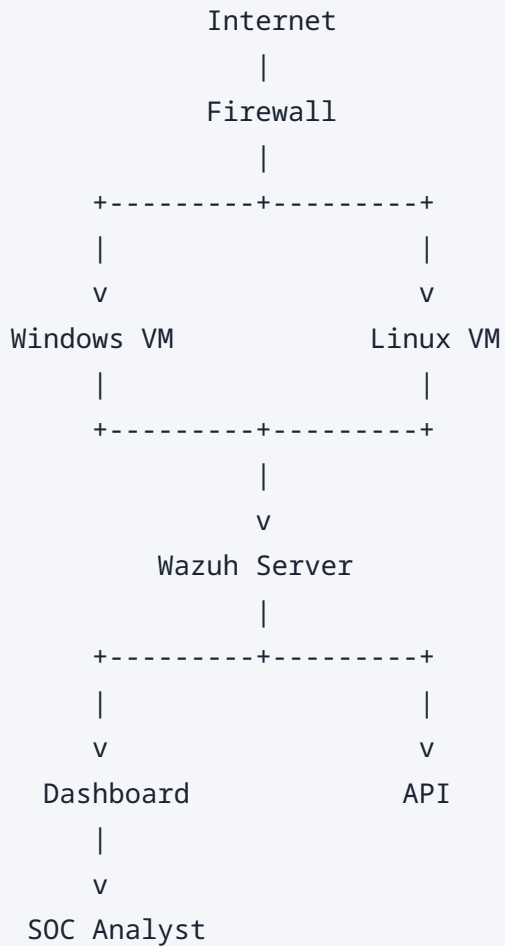
SIEM itself unhealthy হলেও কেউ জানতে পারছে না।

Mistake 9 — Documentation না করা

কোন source কোথায় connected—তা documented না থাকা।

35. Practical SIEM Architecture Lab

একটি learning lab-এর জন্য আপনি এমন architecture তৈরি করতে পারেন:



তারপর ধীরে ধীরে add করুন:

Firewall



Network Device



Threat Intelligence



Detection Rules



Automation



Ticketing

এভাবে একটি ছোট SOC architecture ধীরে ধীরে enterprise-style architecture-এ evolve করা যায়।

36. Real-World Example

ধরুন একটি organization-এর:

==> 2,000 endpoints

==> Windows Servers

==> Linux Servers

==> Active Directory

==> Firewalls

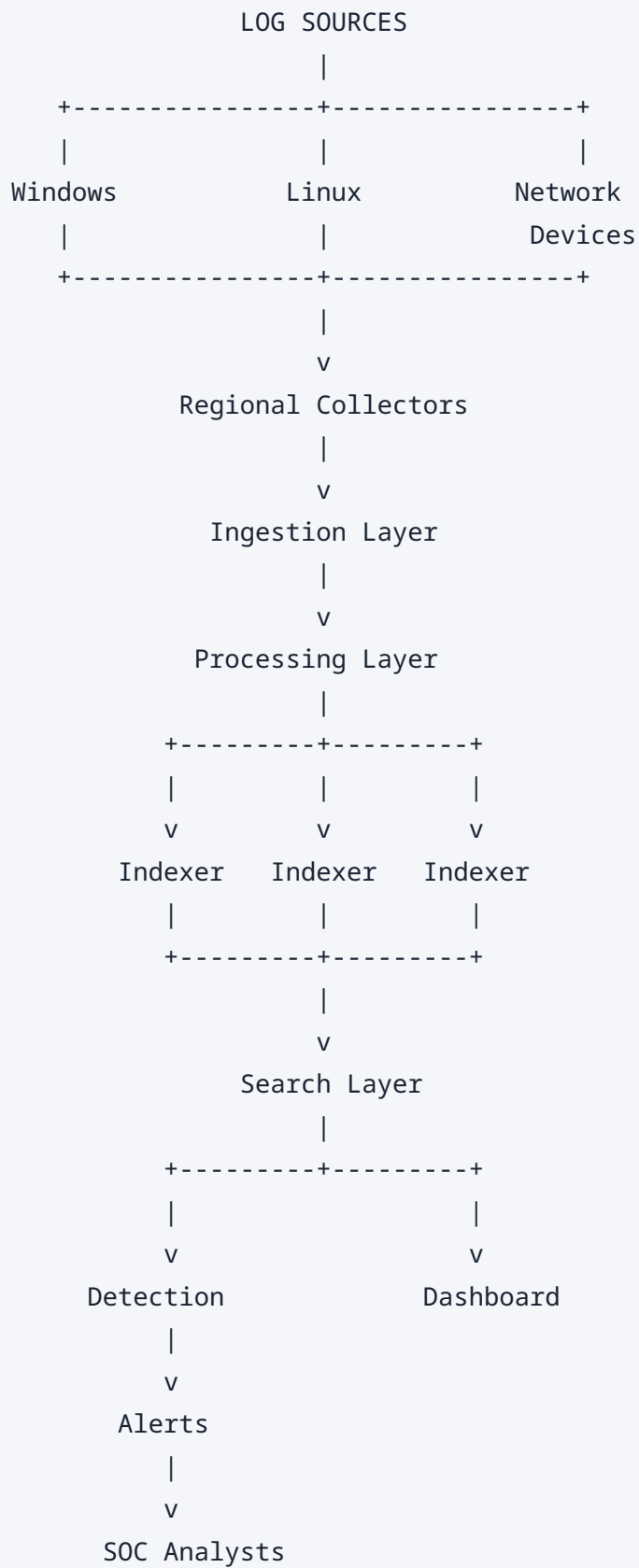
==> Routers/Switches

==> Web Applications

==> Cloud workloads

এবং একটি SOC team আছে।

একটি conceptual architecture হতে পারে:



এর সাথে:

Threat Intelligence

|

v

Enrichment

|

v

Detection

এবং:

SIEM

|

v

SOAR / Automation

|

+-----> Ticketing

|

+-----> Notification

|

+-----> Controlled Response



37. SIEM Architecture Design Process

একজন SIEM Engineer architecture design করার সময় এই process follow করতে পারেন:

```
graph TD;
    A[Requirements] --> B[Asset & Log Source Discovery];
    B --> C[EPS & Data Volume];
    C --> D[Retention];
    D --> E[Capacity Planning];
    E --> F[Component Design];
    F --> G[Network Design];
    G --> H[Security Design];
    H --> I[HA Design];
    I --> J[DR Design];
    J --> K[Implementation];
    K --> L[Testing];
    L --> M[Monitoring];
    M --> N[Documentation];
    N --> O[Continuous Improvement];
```

এটাই একটি practical architecture design lifecycle।



38. Architecture Documentation

Architecture design করার পরে documentation করতে হবে।

Documentation-এ থাকতে পারে:

- ==> Architecture diagram
 - ==> IP/network zones
 - ==> Data sources
 - ==> Collectors
 - ==> Ingestion paths
 - ==> Storage
 - ==> Retention
 - ==> Detection components
 - ==> Integrations
 - ==> HA configuration
 - ==> DR configuration
 - ==> Backup process
 - ==> Monitoring
 - ==> Ports and protocols
 - ==> Authentication
 - ==> Ownership
 - ==> Troubleshooting procedure
-



39. SIEM Platform Architecture — Conceptual View

Different SIEM platforms-এর terminology এবং internal architecture আলাদা হতে পারে।

তবে conceptual layers অনেক ক্ষেত্রে একই ধরনের:

```
graph TD;
    A[Data Sources] --> B[Collection];
    B --> C[Ingestion];
    C --> D[Processing];
    D --> E[Storage / Indexing];
    E --> F[Search / Analytics];
    F --> G[Detection];
    G --> H[Alert / Incident];
    H --> I[SOC];
```

Wazuh

Conceptually:

Agents / Network Devices



Collection



Wazuh Manager



Rules / Analysis



Indexed / Stored Data



Dashboard

Splunk

একটি conceptual Splunk architecture-এ সাধারণত:

Data Sources



Forwarding / Collection



Indexing



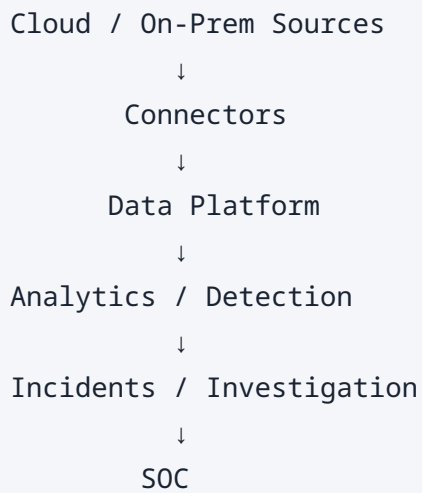
Search / Analytics



Dashboards / Alerts

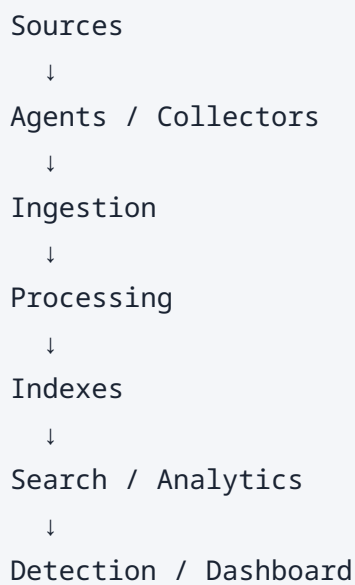
Microsoft Sentinel

Cloud-oriented conceptual architecture:



Elastic / OpenSearch

Conceptually:



Important: এগুলো conceptual architecture । Actual components, deployment models এবং capabilities platform/version অনুযায়ী পরিবর্তিত হতে পারে ।

40. SIEM Architecture Interview Questions & Answers

Q1. What is SIEM Architecture?

Answer:

SIEM Architecture defines how log sources, collectors, ingestion, processing, storage, indexing, search, detection, dashboards and SOC workflows are connected to provide centralized security monitoring.

Q2. Single-node and distributed SIEM-এর মধ্যে পার্থক্য কী?

Answer:

Single-node architecture-এ multiple SIEM functions একই node-এ run করে।

Distributed architecture-এ collection, processing, indexing, search এবং অন্যান্য workloads multiple nodes-এ distribute করা হয়।

Distributed architecture generally provides better scalability and workload separation.

Q3. What is a SIEM Collector?

Answer:

A collector receives security events from sources such as servers, firewalls, network devices and applications and forwards them to the SIEM processing or ingestion layer.

Q4. What is an Indexer?

Answer:

An indexer processes and organizes events into searchable structures and stores the data so that analysts and detection systems can query it efficiently.

Q5. Why is High Availability important in SIEM?

Answer:

SIEM is a critical security monitoring system. If a single component fails, organizations may lose visibility or miss security events. HA reduces dependency on a single component and improves service availability.

Q6. What is the difference between vertical and horizontal scaling?

Answer:

Vertical scaling means increasing resources such as CPU, RAM or storage on an existing node.

Horizontal scaling means adding more nodes to distribute the workload.

Q7. What factors do you consider when designing SIEM architecture?

Answer:

I consider:

==> EPS

==> Peak EPS

==> Data volume

==> Retention

==> Log sources

==> Network architecture

==> Storage

==> Search workload

==> HA

==> Security

==> DR

==> RPO/RTO

==> Future growth

==> Cost

Q8. Why should peak EPS be considered?

Answer:

Average traffic does not represent the workload during incidents or attack periods. Security events can increase significantly during abnormal activity, so architecture should handle expected peak workload with an appropriate safety margin.

Q9. What is RPO?

Answer:

RPO stands for Recovery Point Objective. It defines the acceptable amount of data loss measured in time after a failure.

Q10. What is RTO?

Answer:

RTO stands for Recovery Time Objective. It defines the target time within which a service should be restored after a failure.

Q11. How would you troubleshoot a SIEM architecture where logs are delayed?

Answer:

I would check:

```
graph TD; Source --> Collector; Collector --> Network; Network --> Ingestion; Ingestion --> Queue; Queue --> Processing; Processing --> Indexer; Indexer --> Search;
```

Source
↓
Collector
↓
Network
↓
Ingestion
↓
Queue
↓
Processing
↓
Indexer
↓
Search

Then check:

==> Source timestamp

==> Network latency

==> Collector health

==> Queue/backpressure

==> Processing latency

==> Indexing performance

==> Storage I/O

==> Search latency

This approach helps identify where the delay occurs.



41. SIEM Architecture Checklist

Architecture final করার আগে check করুন:

Requirements

- ==> Log sources identified
- ==> EPS measured
- ==> Peak EPS identified
- ==> Retention defined
- ==> Compliance requirements identified

Infrastructure

- ==> CPU planned
- ==> RAM planned
- ==> Storage planned
- ==> Network planned
- ==> Scaling strategy defined

Architecture

- ==> Collector design complete
- ==> Ingestion design complete
- ==> Processing design complete
- ==> Indexing design complete
- ==> Search design complete

Availability

- ==> HA requirement defined
- ==> Failover tested
- ==> Load balancing considered
- ==> Single Points of Failure identified

Security

- ==> RBAC configured
- ==> TLS considered
- ==> Admin access restricted
- ==> Secrets protected
- ==> Audit logging enabled

DR

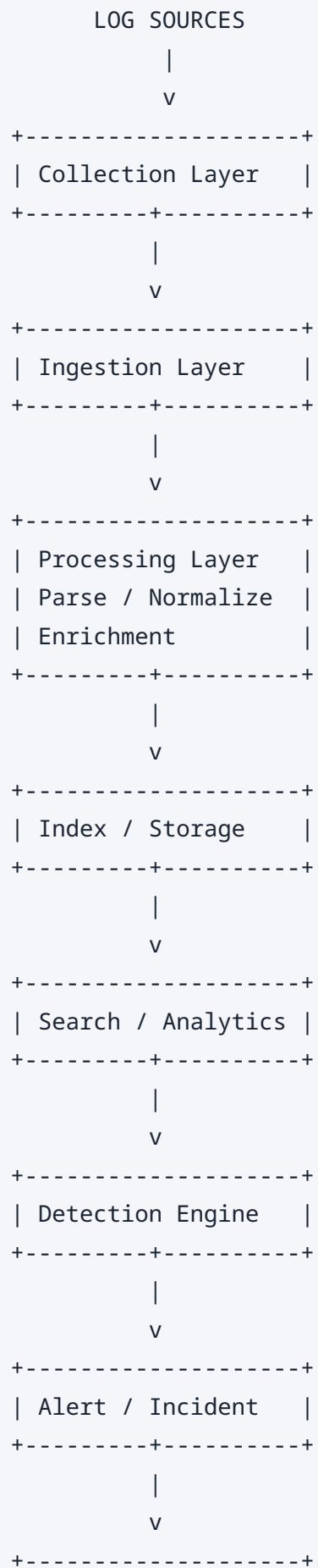
- ==> Backup defined
- ==> RPO defined
- ==> RTO defined
- ==> Restore tested

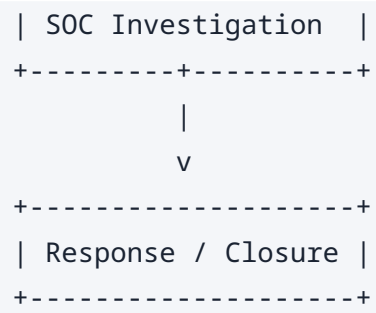
Operations

- ==> Monitoring configured
 - ==> Alerting configured
 - ==> Documentation completed
 - ==> Troubleshooting runbook available
-

42. Complete SIEM Architecture Flow

একজন SIEM Engineer হিসেবে এই flow-টা মনে রাখুন:





এটাই SIEM-এর complete logical architecture flow।



43. What should a SIEM Engineer remember?

একটি SIEM architecture design করার সময় শুধু “কোন **SIEM software** ব্যবহার করব?” এই প্রশ্ন করা যথেষ্ট নয়।

একজন SIEM Engineer-কে ভাবতে হবে:

How much data?

↓

Where is data coming from?

↓

How will we collect it?

↓

How will we transport it?

↓

How will we process it?

↓

How will we store it?

↓

How will analysts search it?

↓

How will we detect threats?

↓

How will we survive failures?

↓

How will we scale?

↓

How will we secure the SIEM?

↓

How will we recover it?

এটাই একজন **SIEM Engineer**-এর **architecture mindset**।



Practical Security Takeaway

SIEM Architecture হলো পুরো SOC/SIEM environment-এর foundation।

একটি ভালো architecture:

==> Reliable log collection নিশ্চিত করে

==> Detection-এর জন্য quality data দেয়

==> Investigation দ্রুত করে

==> Single Point of Failure কমায়

==> Future growth support করে

==> Security এবং compliance requirements support করে

==> Disaster-এর সময় recovery সহজ করে

Remember:

Good SIEM Architecture = Reliable Data + Scalable Infrastructure + Fast Detection + High Availability + Secure Operations



Next in the SIEM Engineer Series

Previous

==> #10 — **SIEM Infrastructure**

Current

==> #11 — **SIEM Architecture Design**

Next

==> #12 — **SIEM Capacity Planning**

Next article-এ আমরা practical calculation শিখব:

==> EPS Calculation

==> Events Per Day

==> GB/day

==> Storage Calculation

==> Retention Planning

==> Peak Load

==> Growth Estimation

==> Capacity Buffer

==> Real-world SIEM sizing example

Navigation

← **Previous Article: #10 — SIEM Infrastructure**

SIEM Engineer Index

Next Article: #12 — SIEM Capacity Planning →

SIEM Engineer #12 — SIEM Capacity Planning

A SIEM Engineer-এর জন্য শুধু SIEM install করা যথেষ্ট নয়।

একটি SIEM environment-এ কতগুলো **events per second (EPS)** আসবে, প্রতিদিন কত **GB/TB data** তৈরি হবে, কতদিন data retain করতে হবে, কত storage লাগবে এবং future-এ workload কতটা বাড়তে পারে—এসব আগে থেকেই plan করতে হয়।

এই process-কে বলা হয় **SIEM Capacity Planning**।

এই article-এ আমরা practicalভাবে শিখব:

==> EPS Calculation ==> Events Per Day ==> GB/day ==> Storage Planning ==> Retention ==> Peak Load ==> Growth Estimation ==> Capacity Buffer ==> Real-world SIEM sizing



1. What is SIEM Capacity Planning?

SIEM Capacity Planning হলো future এবং current workload অনুযায়ী SIEM-এর required compute, storage, network এবং processing capacity নির্ধারণ করার process।

সহজভাবে:

How much data?

↓

How fast?

↓

How long?

↓

How much storage?

↓

How much CPU/RAM?

↓

How much network?

↓

How much future growth?



2. Why does Capacity Planning matter?

Capacity ঠিকভাবে plan না করলে SIEM environment-এ বিভিন্ন সমস্যা হতে পারে।

==> Logs drop হতে পারে

==> Events delayed হতে পারে

==> Search slow হতে পারে

==> Storage full হতে পারে

==> Detection delayed হতে পারে

==> CPU/RAM saturation হতে পারে

==> Network congestion হতে পারে

==> Investigation difficult হতে পারে

==> Future growth handle করা কঠিন হতে পারে

একজন SIEM Engineer-এর লক্ষ্য হলো আজকের **workload** এবং আগামী দিনের **growth**—দুটোই বিবেচনা করা।

3. Key Capacity Metrics

SIEM capacity planning-এর সময় কয়েকটি metric সবচেয়ে গুরুত্বপূর্ণ।

==> EPS

==> Peak EPS

==> Events Per Day

==> Average Event Size

==> GB/day

==> Retention Period

==> Storage Requirement

==> Search Workload

==> CPU

==> RAM

==> Network Bandwidth

==> Growth Rate



4. What is EPS?

EPS = Events Per Second

এটি প্রতি second-এ SIEM কতগুলো events receive/process করছে তা বোঝায়।

উদাহরণ:

Average EPS = 1,000

Peak EPS = 3,000

অর্থাৎ সাধারণ সময়ে প্রায় 1,000 events/sec আসতে পারে, কিন্তু peak period-এ 3,000 events/sec পর্যন্ত যেতে পারে।



5. EPS Calculation

ধরুন:

Total Events = 86,400,000
Time = 24 hours

24 hours:

$24 \times 60 \times 60$
= 86,400 seconds

তাহলে:

EPS = Total Events / Total Seconds

EPS = 86,400,000 / 86,400

EPS = 1,000

অর্থাৎ average EPS = 1,000 EPS।

6. Events Per Day

যদি EPS জানা থাকে:

$$\text{Events/day} = \text{EPS} \times 86,400$$

Example:

$$\text{EPS} = 1,000$$

$$\begin{aligned}\text{Events/day} &= 1,000 \times 86,400 \\ &= 86,400,000 \text{ events/day}\end{aligned}$$

অর্থাৎ প্রতিদিন প্রায় **86.4 million events**।



7. Average Event Size

Storage planning-এর জন্য event size জানা দরকার।

ধরুন average event size:

1 KB

এবং:

EPS = 1,000

তাহলে প্রতি second:

$1,000 \times 1 \text{ KB}$
 $= 1,000 \text{ KB/sec}$

অর্থাৎ প্রায়:

1 MB/sec



8. GB Per Day Calculation

ধরুন:

EPS = 1,000
Average Event Size = 1 KB

Events/day:

86,400,000

Raw data:

$86,400,000 \times 1 \text{ KB}$
 $= 86,400,000 \text{ KB}$

Decimal storage হিসেবে এটি প্রায়:

86.4 GB/day

অর্থাৎ:

1,000 EPS × 1 KB event ≈ 86.4 GB/day raw data

এটি একটি simplified estimation।

Actual SIEM storage raw data-এর চেয়ে বেশি বা কম হতে পারে, কারণ indexing, metadata, replication, compression এবং platform-specific overhead থাকতে পারে।



9. General Storage Formula

একটি basic estimation:

$$\begin{aligned} \text{Daily Storage} \\ = \\ \text{EPS} \times \text{Average Event Size} \times 86,400 \end{aligned}$$

তারপর retention:

$$\begin{aligned} \text{Retention Storage} \\ = \\ \text{Daily Storage} \times \text{Retention Days} \end{aligned}$$

Example:

$$\begin{aligned} \text{Daily Data} &= 100 \text{ GB} \\ \text{Retention} &= 30 \text{ days} \\ \\ 100 \times 30 \\ &= 3,000 \text{ GB} \\ \\ &\approx 3 \text{ TB} \end{aligned}$$

এটি raw-data estimate।

Actual provisioned storage-এর জন্য additional overhead এবং safety margin consider করতে হবে।



10. Storage Planning

SIEM storage planning-এর সময় শুধু raw data calculate করলেই হবে না।

Consider করুন:

==> Raw event data

==> Index overhead

==> Metadata

==> Replication

==> Compression

==> Hot/warm/cold storage

==> Backup

==> Temporary files

==> Free-space requirement

==> Growth buffer



11. Hot Storage

Hot storage-এ frequently searched data রাখা হয়।

উদাহরণ:

Last 7 days

এখানে SOC analyst frequently investigation করতে পারে।

Advantages:

==> Fast search

==> Fast investigation

==> Recent incidents সহজে analyse করা যায়



12. Warm Storage

Warm storage তুলনামূলকভাবে কম frequently accessed data-এর জন্য ব্যবহার করা যায়।

উদাহরণ:

8-30 days

এটি organization-এর retention এবং platform architecture অনুযায়ী পরিবর্তিত হবে।



13. Cold Storage

Long-term retention-এর জন্য cold storage ব্যবহার করা যেতে পারে।

উদাহরণ:

30-180 days

এখানে data সাধারণত hot data-এর মতো frequently queried হয় না।

14. Archive

কিছু organization compliance বা investigation requirement-এর জন্য আরও দীর্ঘ সময় data রাখতে পারে।

Example:

Primary SIEM

↓

Hot

↓

Warm

↓

Cold

↓

Archive

Retention policy business এবং compliance requirements অনুযায়ী নির্ধারণ করতে হবে।



15. Retention Planning

Retention মানে কতদিন security logs রাখা হবে।

উদাহরণ:

Hot = 7 days
Warm = 23 days
Archive = 1 year

এটি শুধু example।

প্রতিটি organization-এর requirements আলাদা।

Retention নির্ধারণের সময় consider করুন:

==> Compliance

==> Legal requirements

==> Incident investigation

==> Threat hunting

==> Business requirements

==> Storage cost

==> Privacy requirements



16. Growth Estimation

আজ আপনার SIEM:

1,000 EPS

কিন্তু আগামী বছর:

1,500 EPS

হতে পারে।

কারণ:

==> New servers

==> New applications

==> More endpoints

==> Cloud adoption

==> New branches

==> More security tools

==> Increased logging

তাই capacity planning-এ growth estimate রাখতে হবে।

17. Simple Growth Example

বর্তমান:

1,000 EPS

ধরি annual growth:

20%

তাহলে next year:

$1,000 \times 1.20$
 $= 1,200 \text{ EPS}$

দুই বছর পরে:

$1,200 \times 1.20$
 $= 1,440 \text{ EPS}$

অর্থাৎ capacity planning-এ শুধু current EPS ব্যবহার করা উচিত নয়।



18. Average EPS vs Peak EPS

এটি খুব গুরুত্বপূর্ণ।

ধরুন:

Average EPS = 2,000

Peak EPS = 8,000

যদি architecture শুধু 2,000 EPS-এর জন্য design করা হয়, attack বা incident-এর সময় সমস্যা হতে পারে।

Peak হতে পারে:

==> Authentication storm

==> Malware outbreak

==> DDoS-related logging

==> Firewall event spike

==> Large-scale scanning

==> Incident response activity

তাই:

Capacity Planning = Average Workload + Peak Workload + Growth



19. Capacity Buffer

Production SIEM-এ পুরো capacity 100% ব্যবহার করে design করা ভালো practice নয়।

ধরুন:

Expected Peak = 8,000 EPS

তারপর architecture-এ additional capacity রাখা হলো।

Conceptually:

Expected Peak
+
Safety Margin
=
Planned Capacity

Safety margin organization-এর risk tolerance, workload variability এবং platform behavior অনুযায়ী নির্ধারণ করা উচিত।



20. Network Capacity Planning

শুধু storage নয়, network bandwidth-ও plan করতে হবে।

ধরুন:

```
SIEM traffic  
= 20 MB/sec
```

তাহলে network architecture-এ এই traffic-এর জন্য adequate capacity রাখতে হবে।

Consider করুন:

==> Log traffic

==> Management traffic

==> Search traffic

==> Replication traffic

==> Backup traffic

==> API traffic

একই network link-এ সব traffic থাকলে congestion হতে পারে।



21. CPU Planning

CPU requirement depend করে:

==> EPS

==> Parsing

==> Normalization

==> Correlation

==> Detection

==> Enrichment

==> Compression

==> Indexing

==> Search workload

একটি SIEM-এর CPU requirement শুধু log volume দিয়ে determine করা যায় না।



22. RAM Planning

RAM গুরুত্বপূর্ণ হতে পারে:

==> Search

==> Caching

==> Indexing

==> Query processing

==> Detection

==> Application services

==> Buffers

==> JVM/runtime workloads যেখানে প্রয়োজ্য

তাই RAM planning workload-based হওয়া উচিত।



23. Disk I/O Planning

High-volume SIEM-এর জন্য শুধু disk capacity নয়, **disk performance**-ও গুরুত্বপূর্ণ।

Consider করুন:

==> Write IOPS

==> Read IOPS

==> Throughput

==> Latency

==> Concurrent searches

একটি storage-এর capacity অনেক TB হতে পারে, কিন্তু I/O performance কম হলে SIEM search এবং ingestion slow হতে পারে।

এটি পরের #13 **SIEM Performance Tuning**-এ আরও বিস্তারিত দেখব।



24. Replication Factor

Distributed SIEM environment-এ data replication থাকতে পারে।

Conceptually:

Original Data

|

+-----> Copy 1

|

+-----> Copy 2

যদি replication factor বাড়ে:

Storage Requirement ↑

Availability ↑

অর্থাৎ redundancy বাড়লে storage requirement-ও বাড়তে পারে।

Exact behavior platform অনুযায়ী আলাদা।



25. Complete Storage Estimation Example

ধরি:

Average EPS	= 2,000
Average Event	= 1 KB
Retention	= 30 days

Events/day:

$$2,000 \times 86,400$$

$$= 172,800,000 \text{ events/day}$$

Raw data/day:

$$172,800,000 \times 1 \text{ KB}$$

$$\approx 172.8 \text{ GB/day}$$

30 days:

$$172.8 \times 30$$

$$\approx 5,184 \text{ GB}$$

$$\approx 5.18 \text{ TB}$$

এটি **raw-data estimate**।

Actual storage planning-এ additional consideration:

Raw Data

+

Index Overhead

+

Replication

+

Metadata

+

System Overhead

+

Growth Buffer

=

Provisioned Storage

26. Real-World Example

ধরুন একটি company-এর:

==> 1,500 Windows endpoints

==> 100 Linux servers

==> 20 network devices

==> 5 firewalls

==> Active Directory

==> Web applications

==> Cloud services

প্রথমে source অনুযায়ী event generation estimate করতে হবে।

Windows	→	1,500	EPS
Linux	→	300	EPS
Network	→	200	EPS
Firewall	→	500	EPS
Applications	→	700	EPS
Cloud	→	300	EPS

Total	→	3,500	EPS

এটি example estimate।

তারপর:

Average EPS = 3,500
Peak EPS = measured/projected peak

এরপর calculate করতে হবে:

==> Events/day

==> GB/day

==> Retention

==> Storage

==> Network

==> CPU

==> RAM

==> Indexing

==> Search workload



27. Capacity Planning Worksheet

একজন SIEM Engineer এই ধরনের table maintain করতে পারেন:

Metric	Value
Average EPS	3,500
Peak EPS	8,000
Average Event Size	1 KB
Events/Day	302.4M
Raw Data/Day	~302.4 GB
Retention	30 days
Raw Retention	~9.07 TB
Growth	20%
HA/Replication	Required
Backup	Required

Note: এটি example planning data; production sizing-এর আগে real measurements এবং platform-specific sizing guidance ব্যবহার করা উচিত.



Capacity Planning Checklist

- [] Average EPS and Peak EPS measured from real sources
 - [] Events/day calculated: $\text{EPS} \times 86400$
 - [] GB/day calculated: $\text{events/day} \times \text{avg event size}$
 - [] Retention storage calculated: $\text{GB/day} \times \text{retention days}$
 - [] Index + replication + buffer overhead added (see #10 storage tiers)
 - [] Hot / Warm / Cold / Archive split defined
 - [] CPU / RAM / Disk IOPS and network sized for peak + 20-30% buffer
 - [] Capacity alerts set (e.g. 80% warn, 90% critical)
 - [] Worksheet reviewed before procurement / expansion
-



28. Capacity Monitoring

Capacity planning একবার করে শেষ হয়ে যায় না।

Continuously monitor করুন:

- ==> EPS trend
 - ==> Daily GB
 - ==> Storage utilization
 - ==> CPU utilization
 - ==> RAM utilization
 - ==> Disk I/O
 - ==> Search latency
 - ==> Ingestion latency
 - ==> Queue size
 - ==> Event drops
 - ==> Network utilization
-



29. Capacity Alerting

Threshold-based monitoring করা যেতে পারে।

Example:

Storage > 80%

↓

Warning

Storage > 90%

↓

Critical

আরও monitor করা যায়:

EPS > Planned Capacity

↓

Alert

এতে capacity issue হওয়ার আগেই SIEM Engineer action নিতে পারে।

30. Capacity Planning Lifecycle

একটি practical lifecycle:



```
graph TD; Measure --> Analyze; Analyze --> Calculate; Calculate --> Design; Design --> Deploy; Deploy --> Monitor; Monitor --> Forecast; Forecast --> Scale; Scale --> Review;
```

Measure
↓
Analyze
↓
Calculate
↓
Design
↓
Deploy
↓
Monitor
↓
Forecast
↓
Scale
↓
Review

এটি continuous process।



31. Practical Lab

একটি ছোট lab তৈরি করুন।

Step 1

5–10টি log source তৈরি করুন।

Windows
Linux
Firewall
Web Server
Application

Step 2

প্রতিটি source থেকে event rate measure করুন।

Step 3

Average EPS বের করুন।

Step 4

Peak EPS record করুন।

Step 5

Average event size estimate করুন।

Step 6

GB/day calculate করুন।

Step 7

30/90-day retention-এর storage estimate করুন।

Step 8

Growth factor যোগ করুন।

Step 9

Storage এবং infrastructure utilization monitor করুন।

Step 10

Capacity threshold configure করুন।

32. Interview Questions & Answers

Q1. What is SIEM Capacity Planning?

Answer:

SIEM Capacity Planning is the process of estimating the compute, storage, network and processing resources required to handle current and future SIEM workloads.

Q2. What is EPS?

Answer:

EPS means Events Per Second. It represents how many events a SIEM receives or processes per second.

Q3. How do you calculate events per day?

Answer:

$$\text{Events/day} = \text{EPS} \times 86,400$$

because one day contains 86,400 seconds.

Q4. What factors affect SIEM storage?

Answer:

==> EPS

==> Event size

==> Retention

==> Index overhead

==> Replication

==> Compression

==> Metadata

==> Backup

==> Growth

Q5. Why is peak EPS important?

Answer:

Average EPS does not represent the maximum workload. During security incidents, authentication storms or large event spikes, EPS can increase significantly. Therefore, SIEM architecture should account for expected peak workload.

Q6. What is retention?

Answer:

Retention defines how long SIEM data is stored and available according to business, security, legal and compliance requirements.

Q7. Is raw GB/day enough for SIEM storage planning?

Answer:

No. Raw data is only the starting point. I also consider indexing overhead, metadata, replication, compression behavior, backups, system overhead and future growth.

Q8. How would you handle future SIEM growth?

Answer:

I would monitor historical EPS and data-volume trends, estimate growth, maintain a capacity buffer, and design the architecture so that additional resources or nodes can be added when required.

Q9. What happens if SIEM storage becomes full?

Answer:

Depending on the platform and configuration, ingestion may fail, indexing may stop, services may become unhealthy, or data retention behavior may be affected. I would immediately check storage utilization, ingestion health, retention policies and platform-specific safeguards.

Q10. Which metrics do you monitor for capacity?

Answer:

I monitor:

==> EPS

==> Peak EPS

==> GB/day

==> Storage utilization

==> CPU

==> RAM

==> Disk I/O

==> Network

==> Search latency

==> Ingestion latency

==> Queue size

==> Event drops

33. Common Capacity Planning Mistakes

Mistake 1

শুধু average EPS calculate করা।

Mistake 2

Event size consider না করা।

Mistake 3

Retention ভুল estimate করা।

Mistake 4

Index overhead ignore করা।

Mistake 5

Replication/storage overhead ignore করা।

Mistake 6

Future growth consider না করা।

Mistake 7

Peak workload ignore করা।

Mistake 8

Disk I/O ignore করা।

Mistake 9

Network capacity ignore করা।

Mistake 10

Capacity monitor না করা।

34. SIEM Engineer Mindset

একজন SIEM Engineer যখন শুনবে:

“আমাদের 5,000 EPS SIEM লাগবে।”

তখন শুধু server কিনবে না।

বরং প্রশ্ন করবে:

5,000 EPS average or peak?

↓

Average event size কত?

↓

GB/day কত?

↓

Retention কতদিন?

↓

Search workload কত?

↓

Replication লাগবে?

↓

HA লাগবে?

↓

Backup কোথায়?

↓

Growth কত?

↓

Peak-এর জন্য capacity কত?

এই mindset-টাই একজন **SIEM Engineer**-কে আলাদা করে।

35. Complete Capacity Planning Flow

```
Log Sources
  ↓
Measure EPS
  ↓
Find Peak EPS
  ↓
Measure Event Size
  ↓
Calculate Events/Day
  ↓
Calculate GB/Day
  ↓
Define Retention
  ↓
Calculate Raw Storage
  ↓
Add Index/Replication Overhead
  ↓
Add Growth Buffer
  ↓
Plan CPU/RAM
  ↓
Plan Network
  ↓
Plan Disk I/O
  ↓
Deploy
  ↓
Monitor
  ↓
Forecast
  ↓
Scale
```



Final Takeaway

SIEM Capacity Planning-এর মূল বিষয় হলো:

Data Volume + EPS + Peak Load + Retention + Storage + Growth

একজন ভালো SIEM Engineer আগে থেকে বুঝতে পারে:

==> কত data আসবে

==> কত দ্রুত আসবে

==> কতদিন রাখতে হবে

==> কত storage লাগবে

==> কত processing power লাগবে

==> কখন capacity বাড়তে হবে

Remember:

Don't size a SIEM only for today. Design it for today's workload, tomorrow's growth, and incident-time peaks.



SIEM Engineer Series Navigation

Previous

==> #11 — SIEM Architecture Design

Current

==> #12 — SIEM Capacity Planning

Next

==> #13 — SIEM Performance Tuning

পরের article-এ আমরা দেখব:

==> Slow Queries

==> CPU/RAM Bottlenecks

==> Disk I/O

==> Index Optimization

==> Search Optimization

==> Query Performance

==> Ingestion Performance

==> Troubleshooting

Navigation

← Previous Article: #11 — SIEM Architecture Design

SIEM Engineer Index

Next Article: #13 — SIEM Performance Tuning →

SIEM Engineer #13 — SIEM Performance Tuning

A SIEM environment may have enough CPU, RAM and storage, but it can still become slow.

SOC Analyst যখন search করে:

“Show me all failed logins from the last 24 hours.”

যদি result আসতে অনেক সময় লাগে, তাহলে investigation slow হয়ে যায়।

একজন **SIEM Engineer**-এর কাজ শুধু SIEM চালু রাখা নয়। তাকে বুঝতে হয়:

==> Why is the SIEM slow?

==> Where is the bottleneck?

==> Is ingestion delayed?

==> Is storage slow?

==> Is the query inefficient?

==> Is CPU overloaded?

==> Is memory under pressure?

==> Is indexing the problem?

এই process-কে বলা হয় **SIEM Performance Tuning**।



1. Why does SIEM Performance matter?

SIEM performance directly affects SOC operations।

Slow SIEM হলে:

==> Alert processing delay হতে পারে

==> Investigation slow হতে পারে

==> Threat hunting difficult হতে পারে

==> Detection latency বাড়তে পারে

==> Search timeout হতে পারে

==> Dashboard slow হতে পারে

==> Analyst productivity কমতে পারে

==> Incident response delay হতে পারে

তাই performance tuning একটি continuous SIEM engineering activity।

2. SIEM Performance-এর Main Areas

SIEM performance broadly কয়েকটি layer-এ analyse করা যায়:

```
graph TD; A[Log Sources] --> B[Collection]; B --> C[Network]; C --> D[Ingestion]; D --> E[Processing]; E --> F[Indexing]; F --> G[Storage]; G --> H[Search]; H --> I[Dashboard];
```

Log Sources
↓
Collection
↓
Network
↓
Ingestion
↓
Processing
↓
Indexing
↓
Storage
↓
Search
↓
Dashboard

যে কোনো layer bottleneck হতে পারে।



3. Performance Troubleshooting Mindset

একজন SIEM Engineer কখনো সরাসরি server restart দিয়ে troubleshooting শুরু করবে না।

প্রথমে প্রশ্ন করবে:

What is slow?

↓

Where is it slow?

↓

When did it start?

↓

What changed?

↓

Which resource is saturated?

↓

What is the root cause?

↓

What is the safest fix?

4. Important Performance Metrics

Monitor করুন:

==> EPS

==> Ingestion latency

==> Processing latency

==> Queue size

==> CPU utilization

==> RAM utilization

==> Disk utilization

==> Disk IOPS

==> Disk latency

==> Network throughput

==> Search latency

==> Query execution time

==> Indexing rate

==> Event drops

==> Error rate

==> Node health



5. Slow Query — First Problem

SOC environment-এ সবচেয়ে common performance problem হলো **slow search/query**।

Example:

Search:
All authentication events
from all systems
for the last 2 years

এটি huge amount of data scan করতে পারে।

Better approach:

Time Range
↓
Relevant Index/Data
↓
Relevant Fields
↓
Specific Conditions
↓
Result



6. Time Range Optimization

Query performance improve করার সবচেয়ে সহজ উপায়গুলোর একটি হলো appropriate time range ব্যবহার করা।

Bad:

Search last 2 years

যদি incident last 24 hours-এর মধ্যে ঘটে থাকে।

Better:

Search last 24 hours

তারপর প্রয়োজন হলে:

24h

↓

7d

↓

30d

অর্থাৎ broad-to-narrow investigation approach ব্যবহার করুন।

7. Filter Early

Query-তে যত দ্রুত relevant data filter করা যায়, তত ভালো।

ধরুন আপনার কাছে millions of events আছে।

Bad approach:

```
Search everything
  ↓
Process everything
  ↓
Filter later
```

Better:

```
Time filter
  ↓
Event type
  ↓
Host
  ↓
User
  ↓
IP
  ↓
Result
```



8. Search the Right Data

সব query-এর জন্য সব data scan করার প্রয়োজন নেই।

যদি authentication investigation হয়:

Authentication Data

যদি firewall investigation হয়:

Firewall Data

যদি endpoint investigation হয়:

Endpoint Data

Relevant data source/index ব্যবহার করলে unnecessary scanning কমে।



9. Query Optimization

একটি efficient query সাধারণত:

==> Specific time range ব্যবহার করে

==> Relevant dataset ব্যবহার করে

==> প্রয়োজনীয় fields ব্যবহার করে

==> Early filtering করে

==> Unnecessary wildcard avoid করে

==> Unnecessary expensive operations avoid করে

==> Result size সীমিত রাখে

==> Aggregation carefully ব্যবহার করে



10. Wildcard Problem

অনেক SIEM query engine-এ broad wildcard expensive হতে পারে।

Example:

```
*admin*
```

এটি অনেক field বা অনেক records scan করতে পারে।

Better:

```
user = "admin"
```

যদি exact field এবং exact value জানা থাকে।

Exact query সাধারণত investigation-এর জন্য বেশি efficient হতে পারে।



11. Too Many Fields

Query-তে অপ্রয়োজনীয় সব field return করা উচিত নয়।

ধরুন analyst-এর শুধু দরকার:

```
timestamp  
user  
source_ip  
destination_ip  
action
```

তাহলে পুরো event-এর hundreds of fields return করার প্রয়োজন নাও থাকতে পারে।

কম data return করলে:

==> Query response ছোট হয়

==> Network usage কমে

==> UI rendering দ্রুত হতে পারে



12. Aggregation Performance

Aggregation useful কিন্তু expensive হতে পারে।

Example:

```
Count failed login  
by source IP  
for last 24 hours
```

এটি useful detection query।

কিন্তু huge dataset-এর উপর multiple high-cardinality aggregation চালালে performance impact হতে পারে।

তাই:

==> Time range সীমিত করুন

==> Relevant fields ব্যবহার করুন

==> Result size control করুন



13. CPU Performance

CPU high হলে first identify করতে হবে কোন component CPU consume করছে।

Possible causes:

==> High EPS

==> Complex parsing

==> Heavy detection rules

==> Expensive correlation

==> Large searches

==> Many concurrent queries

==> Compression

==> Indexing workload

==> Background tasks



14. High CPU Troubleshooting Flow

CPU High

↓

Check Which Process

↓

Check Ingestion Rate

↓

Check Search Load

↓

Check Detection Load

↓

Check Parsing

↓

Check Recent Changes

↓

Identify Bottleneck

↓

Tune / Scale

শুধু CPU usage দেখে solution নেওয়া উচিত নয়।



15. RAM / Memory Performance

Memory pressure হলে:

==> Search slow হতে পারে

==> Cache efficiency কমতে পারে

==> Processes restart করতে পারে

==> Garbage collection pressure হতে পারে

==> System swap ব্যবহার করতে পারে

==> Overall latency বাড়তে পারে

Monitor করুন:

==> Used memory

==> Available memory

==> Cache

==> Swap

==> Process memory

==> Runtime/JVM memory যেখানে applicable



16. Swap Problem

যদি SIEM workload-এর কারণে system বারবার swap ব্যবহার করে:

RAM Pressure



Swap Usage



Disk I/O



Higher Latency



Slow SIEM

অতএব memory pressure এবং disk performance একে অপরের সাথে connected হতে পারে।



17. Disk I/O

SIEM-এর জন্য disk performance অত্যন্ত গুরুত্বপূর্ণ।

কারণ:

Incoming Event



Processing



Indexing



Disk Write

আর search-এর সময়:

Query



Index



Disk Read



Result

অর্থাৎ একই storage system read এবং write workload handle করতে পারে।

18. Important Disk Metrics

Monitor করুন:

==> Disk utilization

==> IOPS

==> Read throughput

==> Write throughput

==> Latency

==> Queue depth

==> Free space

==> Filesystem health



19. High Disk I/O Symptoms

যদি disk bottleneck হয়:

==> Ingestion slow

==> Search slow

==> Indexing delay

==> Queue বৃদ্ধি

==> Event latency বৃদ্ধি

==> Dashboard delay

হতে পারে।

20. Storage Optimization

Storage optimization-এর মধ্যে থাকতে পারে:

==> Appropriate storage type

==> Proper indexing strategy

==> Retention policy

==> Data lifecycle management

==> Compression where appropriate

==> Old data tiering

==> Unnecessary data reduction

==> Storage monitoring

21. Index Optimization

Index হলো SIEM search performance-এর গুরুত্বপূর্ণ অংশ।

Conceptually:

```

Raw Events
  ↓
Processed Events
  ↓
Index
  ↓
Search
    
```

ভালো indexing-এর লক্ষ্য:

==> Fast search

==> Efficient storage

==> Manageable index size

==> Predictable performance

Index strategy platform-specific হতে পারে।



22. Index Size

Index খুব বড় হলে:

==> Search scope বাড়ে

==> Storage usage বাড়ে

==> Maintenance difficult হতে পারে

অন্যদিকে excessive small indexes/shards/partitions তৈরি করলেও overhead বাড়তে পারে।

তাই balance দরকার।



23. Sharding / Partitioning

Distributed search systems data-কে multiple logical partitions-এ ভাগ করতে পারে।

Conceptually:

Large Dataset

```
|  
+---- Shard 1  
|  
+---- Shard 2  
|  
+---- Shard 3  
|  
+---- Shard 4
```

Query multiple shards-এ execute হতে পারে।

Too many বা poorly sized partitions performance-এর উপর negative effect ফেলতে পারে।

Exact implementation platform অনুযায়ী আলাদা।

24. Replication vs Performance

Replication availability এবং resilience বাড়াতে সাহায্য করতে পারে।

কিন্তু replication-এর কারণে:

==> Additional storage

==> Additional network traffic

==> Additional write workload

প্রয়োজন হতে পারে।

তাই architecture এবং performance একসাথে design করতে হবে।



25. Ingestion Performance

Suppose:

Incoming EPS = 10,000
Processing Capacity = 6,000 EPS

তাহলে:

Incoming
↓
10,000 EPS
↓
Processing
↓
6,000 EPS

Backlog তৈরি হতে পারে।

Incoming Rate > Processing Rate
↓
Queue ↑
↓
Event Latency ↑

এটি একটি important performance issue।



26. Queue Monitoring

Queue একটি useful performance indicator।

যদি:

Queue = Normal

তাহলে system healthy হতে পারে।

কিন্তু:

Queue
100
500
1,000
10,000
50,000

এভাবে continuously increase করলে ingestion bottleneck থাকতে পারে।

27. Event Latency

Event Latency হলো event generate হওয়ার পর SIEM-এ searchable/processed হতে কত সময় লাগে।

Conceptually:

Event Generated

↓

Network

↓

Collector

↓

Ingestion

↓

Processing

↓

Indexing

↓

Searchable

এই পুরো সময় monitor করা গুরুত্বপূর্ণ।

28. Performance Baseline

Performance troubleshooting-এর আগে baseline তৈরি করা উচিত।

Example:

Normal EPS	= 5,000
Normal CPU	= 55%
Normal RAM	= 65%
Normal Disk I/O	= 45%
Normal Search	= 2 sec
Normal Latency	= 5 sec

তারপর incident-এর সময় compare করুন।

```
Baseline
  ↓
Current Metrics
  ↓
Difference
  ↓
Potential Cause
```



29. Before & After Testing

কোনো optimization করার আগে baseline record করুন।

Example:

Before:
Query = 30 sec

Optimization-এর পরে:

After:
Query = 5 sec

তাহলে change-এর impact measurable হয়।

30. Performance Tuning Process

একটি practical process:

```
graph TD; A[Identify Problem] --> B[Collect Metrics]; B --> C[Establish Baseline]; C --> D[Find Bottleneck]; D --> E[Form Hypothesis]; E --> F[Make Controlled Change]; F --> G[Test]; G --> H[Measure]; H --> I[Document]; I --> J[Monitor];
```

Identify Problem
↓
Collect Metrics
↓
Establish Baseline
↓
Find Bottleneck
↓
Form Hypothesis
↓
Make Controlled Change
↓
Test
↓
Measure
↓
Document
↓
Monitor



31. Real-World Problem — Slow Search

Problem:

SOC analyst reports that authentication searches are taking 30–60 seconds.

Investigation:

```
graph TD; A[Check Time Range] --> B[Check Query]; B --> C[Check Dataset]; C --> D[Check CPU]; D --> E[Check RAM]; E --> F[Check Disk I/O]; F --> G[Check Index Health]; G --> H[Check Concurrent Searches]; H --> I[Identify Bottleneck];
```

Possible causes:

==> Large time range

==> Inefficient query

==> Heavy aggregation

==> Storage latency

==> High concurrent searches

==> Indexing pressure

==> Resource saturation



32. Real-World Problem — Logs Delayed

Problem:

Firewall logs are arriving 10 minutes late.

Troubleshooting:

```
graph TD; Firewall --> Collector; Collector --> Network; Network --> Ingestion; Ingestion --> Queue; Queue --> Processing; Processing --> Indexer;
```

প্রতিটি layer check করুন।

যদি queue continuously grows:

```
Incoming Rate
>
Processing Rate
```

তাহলে processing বা indexing bottleneck থাকতে পারে।



33. Real-World Problem — High CPU

Problem:

CPU = 95%

Check করুন:

==> Which process?

==> EPS কি বেড়েছে?

==> New log source added?

==> New detection rule deployed?

==> Large query চলছে?

==> Parsing expensive?

==> Search concurrency বেড়েছে?

==> Recent configuration change?

তারপর root cause identify করুন।

34. Performance Optimization Areas

একটি SIEM-এর performance improve করার জন্য কয়েকটি জায়গা বিবেচনা করা যায়:

Query

==> Time range optimization

==> Field filtering

==> Efficient search logic

Detection

==> Unnecessary rules remove/tune

==> Expensive correlation review

==> Duplicate detections reduce

Storage

==> Appropriate storage

==> Retention optimization

==> Index strategy

Infrastructure

==> CPU

==> RAM

==> Disk I/O

==> Network

Architecture

==> Horizontal scaling

==> Workload separation

==> Load balancing



35. Detection Rule Performance

একটি detection rule logically correct হলেও resource-heavy হতে পারে।

Example:

Search millions of events

↓

Every few seconds

↓

Complex correlation

এটি expensive হতে পারে।

তাই detection engineering-এর সময় consider করতে হবে:

==> Search frequency

==> Dataset size

==> Time window

==> Query complexity

==> Field availability

==> Result volume



36. Dashboard Performance

Dashboard-ও SIEM performance impact করতে পারে।

ধরুন dashboard-এ:

```
20 panels
+
20 queries
+
Large time range
```

সব query একই সময়ে execute হলে search workload বাড়তে পারে।

Better approach:

==> প্রয়োজনীয় panels

==> Appropriate time range

==> Efficient queries

==> Cached/summary data যেখানে appropriate

==> Drill-down dashboards



37. Search Concurrency

ধরুন:

1 Analyst → 1 Query
10 Analysts → 10 Queries
50 Analysts → 50 Queries

একই সময়ে অনেক analyst complex searches করলে search layer-এর উপর load বাড়তে পারে।

তাই enterprise architecture-এ concurrent search workload consider করতে হবে।



38. Performance vs Capacity

এই দুইটি একই জিনিস নয়।

Capacity

System কত workload handle করতে পারে?

Performance

System সেই workload কত efficiently handle করছে?

Example:

SIEM Capacity = 10,000 EPS
Current Load = 5,000 EPS

তারপরও query যদি 60 seconds নেয়, তাহলে performance issue থাকতে পারে।



39. Practical Lab

আপনার lab-এ এই experiment করতে পারেন:

Step 1

Normal performance measure করুন।

CPU
RAM
Disk
EPS
Search latency

Step 2

একটি broad query চালান।

Large time range

Step 3

Query execution time record করুন।

Step 4

Time range reduce করুন।

Step 5

Relevant fields filter করুন।

Step 6

আবার execution time measure করুন।

Step 7

Before/After compare করুন।

Before Optimization

↓

30 sec

After Optimization

↓

5 sec

Step 8

Change documentation করুন।



40. Performance Troubleshooting Checklist

Query

- ==> Time range checked
- ==> Query complexity checked
- ==> Wildcards reviewed
- ==> Fields reviewed
- ==> Aggregations reviewed
- ==> Result size reviewed

Infrastructure

- ==> CPU checked
- ==> RAM checked
- ==> Disk checked
- ==> IOPS checked
- ==> Network checked

SIEM

- ==> EPS checked
- ==> Queue checked
- ==> Ingestion latency checked
- ==> Index health checked
- ==> Search concurrency checked

==> Detection workload checked

Operations

==> Baseline available

==> Recent changes reviewed

==> Optimization tested

==> Before/after measured

==> Documentation updated

41. Interview Questions & Answers

Q1. What is SIEM Performance Tuning?

Answer:

SIEM Performance Tuning is the process of identifying and reducing bottlenecks in log ingestion, processing, indexing, storage and search so the SIEM can operate efficiently and provide timely results.

Q2. What would you check if SIEM searches are slow?

Answer:

I would check:

==> Query time range

==> Query complexity

==> Dataset/index selection

==> CPU

==> RAM

==> Disk I/O

==> Index health

==> Search concurrency

==> Ingestion/indexing workload

==> Recent configuration changes

Q3. How do you troubleshoot high CPU?

Answer:

First I identify which process or workload is consuming CPU. Then I correlate it with EPS, searches, detection rules, parsing and recent changes. After identifying the bottleneck, I make a controlled optimization or scale the relevant component.

Q4. Why is disk I/O important for SIEM?

Answer:

SIEM systems continuously write incoming events and read indexed data during searches. High disk latency or insufficient I/O performance can therefore affect both ingestion and search performance.

Q5. What is event latency?

Answer:

Event latency is the time between an event being generated and becoming available for processing or searching in the SIEM.

Q6. What causes SIEM ingestion delay?

Answer:

Possible causes include:

==> High EPS

==> Network congestion

==> Collector overload

==> Processing bottleneck

==> Queue buildup

==> Indexing bottleneck

==> Storage latency

Q7. How can you optimize a SIEM query?

Answer:

I would start with an appropriate time range, target the relevant data source, filter early, select only necessary fields, avoid unnecessary expensive operations, and limit the result set.

Q8. What is the difference between capacity and performance?

Answer:

Capacity describes how much workload the system can handle, while performance describes how efficiently and quickly the system handles that workload.

42. Common Performance Tuning Mistakes

- ==> Blindly restarting services
 - ==> Increasing CPU without finding the bottleneck
 - ==> Adding RAM without measuring memory pressure
 - ==> Ignoring disk I/O
 - ==> Ignoring query design
 - ==> Searching huge time ranges unnecessarily
 - ==> Running expensive queries continuously
 - ==> Creating too many detection rules
 - ==> Ignoring dashboard workload
 - ==> Making changes without baseline
 - ==> Not measuring before/after performance
-

43. SIEM Engineer Performance Mindset

যখন কেউ বলবে:

“SIEM slow.”

তখন একজন SIEM Engineer-এর response হওয়া উচিত:

```
What is slow?  
  ↓  
Search?  
Ingestion?  
Dashboard?  
Detection?  
API?  
  ↓  
Where is the bottleneck?  
  ↓  
CPU?  
RAM?  
Disk?  
Network?  
Query?  
Index?  
  ↓  
What changed?  
  ↓  
What is the safest optimization?  
  ↓  
Test  
  ↓  
Measure
```

“SIEM slow” কোনো root cause নয়।

এটি শুধু একটি symptom ।

44. Complete SIEM Performance Flow

```
graph TD; A[Performance Issue] --> B[Identify Layer]; B --> C[Collect Metrics]; C --> D[Compare Baseline]; D --> E[Find Bottleneck]; E --> F[Analyze Root Cause]; F --> G[Optimize Query / Rule / Resource]; G --> H[Test]; H --> I[Measure]; I --> J[Document]; J --> K[Monitor];
```

Performance Issue

↓

Identify Layer

↓

Collect Metrics

↓

Compare Baseline

↓

Find Bottleneck

↓

Analyze Root Cause

↓

Optimize Query / Rule / Resource

↓

Test

↓

Measure

↓

Document

↓

Monitor



Final Security Takeaway

SIEM performance tuning-এর মূল লক্ষ্য শুধু “**server fast করা**” নয়।

লক্ষ্য হলো:

==> Reliable ingestion

==> Low event latency

==> Efficient indexing

==> Fast investigation

==> Efficient detection

==> Stable dashboards

==> Predictable performance

==> Scalable operations

Remember:

Measure → Identify → Optimize → Test → Measure Again

কোনো performance change করার আগে **baseline** নিন, bottleneck identify করুন এবং change-এর পরে measurable improvement verify করুন।



SIEM Engineer Series Navigation

Previous

==> #12 — **SIEM Capacity Planning**

Current

==> #13 — **SIEM Performance Tuning**

Next

==> #14 — **Advanced Detection Engineering**

Next article-এ আমরা আরও advanced level-এ যাব:

==> Behavioral Detection

==> Correlation

==> Sequence Detection

==> Risk-Based Detection

==> Detection Tuning

==> False Positive Reduction

==> Detection Testing

==> MITRE ATT&CK Mapping

Navigation

← **Previous Article: #12 — SIEM Capacity Planning**

SIEM Engineer Index

Next Article: #14 — Advanced Detection Engineering →

SIEM Engineer #14 — Advanced Detection Engineering

Introduction

A SIEM can collect millions of events, but collecting logs alone does not make a SOC effective.

The real value comes from identifying **suspicious behavior** from those events.

This is where **Detection Engineering** becomes important.

Basic detection may look for one event:

```
Failed login detected
```

Advanced detection asks:

```
Why did this happen?  
Who performed it?  
From where?  
How many times?  
What happened before and after?  
Does this behavior match an attack pattern?
```

Advanced Detection Engineering focuses on building detections that are:

==> Accurate ==> Context-aware ==> Scalable ==> Testable ==> Maintainable ==> Low-noise ==>

Useful for SOC Analysts

What is Advanced Detection Engineering?

Advanced Detection Engineering is the process of designing SIEM detection logic that identifies suspicious activities by analyzing:

==> Multiple events ==> User behavior ==> Host behavior ==> Network activity ==> Authentication patterns ==> Process activity ==> Threat Intelligence ==> Attack techniques ==> Historical behavior

Instead of depending on a single event, advanced detection can combine multiple signals.

Basic Detection

Failed Login

Advanced Detection

```
10 Failed Logins
    ↓
Successful Login
    ↓
Same Source IP
    ↓
Privileged Account
    ↓
Suspicious Activity
    ↓
High-Risk Alert
```

This provides much more context to the SOC Analyst.

Why does this matter?

A large organization can generate thousands or millions of events every day.

If every suspicious-looking event becomes an alert, the SOC may experience:

==> Alert Fatigue ==> High False Positives ==> Analyst Overload ==> Missed Threats ==> Slow Investigation

Good detection engineering tries to answer:

Which events actually deserve analyst attention?

The goal is not simply to create more alerts.

The goal is to create **better alerts**.



Detection Engineering Lifecycle

A mature detection normally follows this lifecycle:

```
Idea
↓
Security Hypothesis
↓
Identify Data Sources
↓
Understand Fields
↓
Create Detection Logic
↓
Build Rule
↓
Test
↓
Validate
↓
Deploy
↓
Monitor
↓
Tune
↓
Review
```

A SIEM Engineer should understand every stage.



Security Hypothesis

Before creating a rule, define what behavior you want to detect.

For example:

An attacker may perform password spraying
against multiple user accounts from a single source.

Now identify the required evidence.

==> Authentication logs ==> Username ==> Source IP ==> Destination system ==> Authentication
result ==> Timestamp ==> Account type

The hypothesis becomes the foundation of the detection.

Data Source Identification

A detection is only as good as the data available to it.

For example:

Windows Detection

Possible sources:

==> Windows Security Logs ==> Sysmon ==> PowerShell logs ==> Endpoint Security ==> EDR

Linux Detection

==> SSH logs ==> Authentication logs ==> Audit logs ==> Process logs ==> Shell history where appropriate

Network Detection

==> Firewall ==> IDS/IPS ==> Proxy ==> DNS ==> VPN ==> NetFlow

Identity Detection

==> Active Directory ==> Azure/Cloud Identity ==> VPN ==> SSO ==> MFA

Understand Your Fields

Before writing detection logic, understand the available fields.

For example:

```
src_ip
dst_ip
username
hostname
event_type
process_name
command_line
timestamp
action
status
severity
```

Bad field understanding can create bad detections.

Always verify:

==> Field name ==> Data type ==> Example values ==> Normal values ==> Missing values ==>
Normalization ==> Timestamp format

Single-Event Detection

The simplest detection uses one event.

Example:

```
event_type = "privilege_change"  
AND  
target_group = "Domain Admins"
```

This may generate an alert.

Single-event detections are useful, but they often lack context.

For example:

```
User added to privileged group
```

does not tell us:

==> Was this authorized? ==> Who performed it? ==> From which host? ==> Was there suspicious activity before it? ==> Is this normal for the user?

This is why correlation becomes important.

Correlation Detection

Correlation means combining multiple events to identify a meaningful security pattern.

Example:

```
Failed Login
+
Failed Login
+
Failed Login
+
Successful Login
```

Together, these events may indicate a possible brute-force attack.

Another example:

```
PowerShell Execution
+
Encoded Command
+
External Network Connection
```

Together, these signals may deserve more attention than any single event.

Threshold Detection

Threshold detection looks for repeated activity.

Example:

```
More than 20 failed logins  
from the same IP  
within 5 minutes
```

Conceptually:

```
count(failed_login)  
GROUP BY src_ip  
WHERE count > 20  
TIME WINDOW = 5 minutes
```

Thresholds should be based on:

==> Environment baseline ==> User behavior ==> Authentication system ==> Business requirements ==> Historical activity

Avoid choosing arbitrary thresholds.

Sequence Detection

Sequence detection focuses on the **order of events**.

Example:

```
Login Failure
  ↓
Successful Login
  ↓
Privilege Escalation
  ↓
Suspicious Process
  ↓
Outbound Connection
```

Each event alone may not be enough.

The sequence creates stronger context.

Example

1. Multiple failed SSH logins
2. Successful SSH login
3. sudo execution
4. New process starts
5. External connection

This could represent a potential attack chain.

The SIEM Engineer should investigate the relationship between these events.

Behavioral Detection

Behavioral detection focuses on **what is unusual**, rather than only looking for known indicators.

For example:

A user normally logs in:

```
09:00-18:00
Dhaka
Corporate Network
```

Suddenly:

```
03:15
Unknown Location
New Device
VPN
```

Instead of simply asking:

```
Is this IP malicious?
```

behavioral detection asks:

```
Is this behavior unusual for this user?
```

Baseline Analysis

A baseline represents normal activity.

For example:

Normal Login Count:
20-50/day

Suddenly:

Login Count:
500/day

That deviation may deserve investigation.

Baselines can be created for:

==> Users ==> Hosts ==> Applications ==> IP addresses ==> Network traffic ==> Processes ==>
Authentication

But remember:

Unusual does not automatically mean malicious.

It should become a signal for investigation.

Entity-Based Detection

Advanced SIEM detections often analyze entities.

Common entities include:

```
User
Host
IP
Application
Account
Device
```

Example:

```
User: admin
Host: SERVER-01
Source IP: 10.10.10.50
```

The SIEM can build context around these entities.

For example:

```
admin
  ↓
Multiple Failed Logins
  ↓
Successful Login
  ↓
Privilege Change
  ↓
Suspicious Process
```

Now the SOC Analyst has a much stronger investigation context.

Risk-Based Detection

Instead of treating every event equally, we can assign risk values.

Conceptually:

Failed Login	+5
Suspicious PowerShell	+20
Privilege Escalation	+30
Known Malicious IP	+40

Total:

$$5 + 20 + 30 + 40 = 95$$

The SIEM may then treat the entity as high risk.

A generic model could be:

```
Risk =  
Event Risk  
+  
Behavior Risk  
+  
Threat Intelligence Risk  
+  
Entity Context
```

The exact implementation depends on the SIEM platform.

Risk scoring should be carefully tested because poor scoring can create excessive alerts.

Severity vs Risk vs Confidence

These concepts should not be confused.

Severity

How serious the detected activity could be.

Low
Medium
High
Critical

Confidence

How confident we are that the detection represents suspicious activity.

Low Confidence
Medium Confidence
High Confidence

Risk

Potential overall security impact based on multiple signals.

Example:

Severity: High
Confidence: Medium
Risk: 75

These values provide different types of context.

False Positive Reduction

A detection can be technically correct but operationally noisy.

Example:

```
PowerShell detected
```

PowerShell itself is not malicious.

Administrators and applications may legitimately use it.

Therefore, detection logic may consider:

==> User ==> Parent process ==> Command line ==> Destination ==> Host role ==> Time ==>

Frequency ==> Known administrative activity

Instead of:

```
PowerShell = Alert
```

Use contextual logic.

Exceptions and Allowlists

Sometimes legitimate activity should be excluded.

Examples:

- Approved vulnerability scanner
- Approved administrator
- Approved backup server
- Approved monitoring system
- Approved automation account

But exceptions must be controlled carefully.

Bad approach:

- Ignore entire IP

Better approach:

- Ignore specific activity
under specific conditions
for a documented reason.

Every exception should have:

==> Owner ==> Reason ==> Scope ==> Expiration/review date ==> Approval

Suppression

If the same event repeatedly creates alerts, suppression can reduce noise.

Example:

```
Same User  
Same Host  
Same Detection  
Repeated 100 times
```

Instead of creating 100 separate alerts, the SIEM may group or suppress them according to defined logic.

However, suppression should never hide meaningful changes in behavior.

Detection for Brute Force

A simple brute-force detection could be:

```
Multiple authentication failures
+
Same source
+
Short time window
```

Advanced version:

```
Multiple failed logins
      ↓
Multiple accounts targeted
      ↓
Same source IP
      ↓
Successful authentication
```

The second detection provides more context.

PowerShell Detection

PowerShell is widely used for legitimate administration.

Therefore, detection should focus on suspicious characteristics.

Potential signals:

==> Encoded commands ==> Suspicious parent process ==> Unusual user ==> Unexpected server
==> Download activity ==> External connection ==> Abnormal execution time ==> Suspicious
command-line patterns

A stronger detection may correlate:

Office Application



PowerShell



Encoded Command



Network Connection

Privilege Escalation Detection

Possible signals include:

```
User privilege change
+
New privileged group membership
+
Administrative command execution
```

For Linux:

```
Normal User
  ↓
sudo
  ↓
Root Command
```

For Windows:

```
Standard Account
  ↓
Privilege Change
  ↓
Administrative Activity
```

Context is important because legitimate administrators also perform these actions.

Lateral Movement Detection

Lateral movement often involves one system accessing another.

Example:

```
Host A
  ↓
Host B
  ↓
Host C
```

Possible signals:

==> New remote login ==> RDP ==> SMB ==> WinRM ==> SSH ==> Remote administration tools

A useful detection can correlate:

```
New Source Host
+
Privileged Account
+
Remote Authentication
+
Multiple Destination Hosts
```

Firewall Scanning Detection

A single connection is usually not enough.

Instead:

```
One source IP
    ↓
Many destination ports
    ↓
Many destination hosts
    ↓
Short time period
```

This can indicate scanning behavior.

Example logic:

```
COUNT(unique_destination_ports) > threshold
```

and/or:

```
COUNT(unique_destination_hosts) > threshold
```

The threshold should be adapted to the environment.

MITRE ATT&CK Mapping

Detection engineers should understand **MITRE ATT&CK**.

A detection can be mapped to:

Tactic

↓

Technique

↓

Sub-technique

For example, a detection involving suspicious PowerShell activity can be mapped to the appropriate ATT&CK technique.

This helps the SOC understand:

==> What behavior are we detecting? ==> Which attack technique does it represent? ==> Which techniques are missing? ==> Where are our detection gaps?

Detection Coverage

A mature SOC should not only count the number of rules.

It should understand coverage.

Example:

ATT&CK Technique



Data Source



Detection



Alert



Investigation



Response

A technique without the required data source cannot be reliably detected.

Therefore:

Detection coverage = Data + Logic + Validation + Operational Response

Sigma Rules

Sigma is a generic rule format for describing SIEM detections.

Conceptually:

```
Log Source
Detection Conditions
Fields
Severity
Tags
References
```

The advantage is portability.

A detection idea can be represented independently of one specific SIEM query language and then adapted to platforms such as:

==> Splunk ==> Elastic ==> Microsoft Sentinel ==> Other SIEM platforms

Always validate the converted query against your actual data.

Minimal Sigma example structure (see #17 for runnable YAML + `sigma convert`):

```
title: Multiple Failed SSH Logins
logsource:
  product: linux
detection:
  selection:
    EventID: 4625
  condition: selection | count() > 5
tags:
  - attack.credential_access
  - attack.t1110
```

Detection-as-Code

Detection-as-Code treats detection rules like software.

That means using:

```
Git
Version Control
Code Review
Testing
Documentation
CI/CD
Change History
```

Example workflow:

```
Developer
  ↓
Create Detection
  ↓
Git Commit
  ↓
Review
  ↓
Test
  ↓
Deploy
  ↓
Monitor
```

This makes detection management more controlled and repeatable.

Detection Testing

Never deploy an important detection without testing it.

Testing should include:

Positive Test

Generate the behavior that should trigger the rule.

Expected:
Alert = YES

Negative Test

Generate legitimate activity.

Expected:
Alert = NO

Edge Case

Test unusual but legitimate scenarios.

Expected:
Correct handling

This helps identify:

==> False positives ==> False negatives ==> Missing fields ==> Logic errors ==> Performance problems

Detection Performance

A detection may be logically correct but computationally expensive.

For example:

```
Search entire dataset
+
Long time range
+
Complex wildcard
+
Large aggregation
```

This can consume significant SIEM resources.

Detection engineers should consider:

==> Search scope ==> Time window ==> Indexed fields ==> Query complexity ==> Event volume
==> Scheduling frequency ==> Correlation strategy

A good detection should be both **accurate and efficient**.

Detection Monitoring

After deployment, the work is not finished.

Monitor:

- Alert Volume
- False Positive Rate
- True Positive Rate
- Execution Time
- Rule Errors
- Data Availability
- Detection Coverage
- Analyst Feedback

For example:

Before tuning:
1,000 alerts/day

After tuning:
100 alerts/day

But reducing alerts is not automatically success.

You must verify that important detections are still firing.

Detection Tuning Process

A practical tuning process:

```
graph TD; A[Review Alert] --> B[Understand Why It Triggered]; B --> C[Determine TP / FP]; C --> D[Identify Noise Pattern]; D --> E[Modify Detection]; E --> F[Test]; F --> G[Deploy]; G --> H[Monitor];
```

Never tune only to reduce alert numbers.

The objective is to improve **signal quality**.

Real-World Example — Suspicious Login

Suppose the SIEM detects:

```
03:15 – Login from IP A
03:17 – Login from IP B
03:18 – MFA failure
03:20 – Successful login
03:21 – Privileged action
```

A basic detection might only detect:

```
Successful Login
```

An advanced detection considers the complete context:

```
Unusual Time
+
New Source
+
MFA Failure
+
Successful Login
+
Privileged Activity
```

This creates a stronger investigation signal.

Practical Detection Engineering Lab

You can build a small lab using:

```
Wazuh
Linux VM
Windows VM
Firewall
Syslog
```

Lab 01 — SSH Brute Force

Generate:

```
Multiple Failed SSH Logins
```

Detect:

```
Same Source IP
+
Multiple Failures
+
Short Time Window
```

Lab 02 — Successful Login After Failures

Create:

```
Failed Login
Failed Login
Failed Login
Successful Login
```

Build a correlation rule.

Lab 03 — Privilege Escalation

Generate controlled activity:

```
User
↓
sudo
↓
Privileged Command
```

Create a detection.

Lab 04 — Suspicious Process

Monitor:

```
Process Name
Parent Process
User
Command Line
Network Connection
```

Create a contextual detection.

Lab 05 — Network Scanning

Generate authorized lab traffic:

One Source

↓

Many Ports

↓

Many Hosts

Create a threshold-based detection.

Detection Engineering Documentation

Every important detection should have documentation.

A useful template:

```
Detection Name:
Description:
Objective:
Data Sources:
Required Fields:
Detection Logic:
MITRE ATT&CK:
Severity:
Confidence:
Risk:
False Positive Scenarios:
Exceptions:
Testing Method:
Owner:
Version:
Last Review:
```

This is extremely useful for SOC operations.

Detection Engineering + SOC

Detection engineering and SOC operations should work together.

Detection Engineer



Creates Detection



SOC Analyst



Investigates Alert



Provides Feedback



Detection Engineer



Tunes Detection

Analyst feedback is extremely valuable.

The SOC Analyst sees real-world alert behavior that may not be visible during initial rule development.

Common Mistakes

Avoid these mistakes:

==> Creating alerts for every suspicious event ==> Using arbitrary thresholds ==> Ignoring normal user behavior ==> Not testing negative cases ==> No documentation ==> No MITRE mapping ==> No exception management ==> Excessive allowlisting ==> Ignoring query performance ==> Never reviewing old detections ==> Measuring success only by alert reduction ==> Deploying rules without validating data

SIEM Engineer Interview Questions

Q1. What is Detection Engineering?

Answer:

Detection Engineering is the process of designing, testing, deploying, monitoring, and tuning security detections that identify suspicious activity from security data.

Q2. What is correlation detection?

Answer:

Correlation detection combines multiple related events to identify a meaningful security pattern instead of relying on a single event.

Q3. What is behavioral detection?

Answer:

Behavioral detection identifies unusual or abnormal activity based on patterns, baselines, or entity behavior.

Q4. What is the difference between severity and confidence?

Answer:

Severity describes the potential seriousness of the activity, while confidence describes how strongly the available evidence supports the detection.

Q5. How do you reduce false positives?

Answer:

I first analyze why the detection triggered, identify legitimate patterns, and then improve the logic using context such as user, host, process, destination, frequency, baseline, and approved exceptions. I then test the updated rule before deployment.

Q6. What is sequence detection?

Answer:

Sequence detection identifies a meaningful order of events, such as multiple login failures followed by a successful login and then privilege escalation.

Q7. Why is baseline important?

Answer:

A baseline helps establish normal behavior so that unusual deviations can be identified and investigated.

Q8. What is Detection-as-Code?

Answer:

Detection-as-Code applies software engineering practices such as Git, version control, testing, code review, and controlled deployment to security detections.

Q9. What is Sigma?

Answer:

Sigma is a generic format for describing SIEM detection rules in a platform-independent way.

Q10. How do you validate a detection?

Answer:

I perform positive, negative, and edge-case testing, verify the required data fields, check false positives, validate the generated alert, and monitor the rule after deployment.

Detection Engineering Checklist

Before deploying a detection:

- ☒ Security hypothesis defined
- ☒ Data sources identified
- ☒ Required fields verified
- ☒ Detection logic created
- ☒ Threshold reviewed
- ☒ Correlation considered
- ☒ Baseline considered
- ☒ False positives identified
- ☒ Positive test completed
- ☒ Negative test completed
- ☒ MITRE mapping completed
- ☒ Severity defined
- ☒ Confidence considered
- ☒ Documentation completed
- ☒ Performance checked
- ☒ Deployment approved
- ☒ Monitoring enabled
- ☒ Review date defined

Complete Advanced Detection Flow

The complete SIEM detection engineering process can be remembered as:

```
Security Problem
  ↓
Hypothesis
  ↓
Data Sources
  ↓
Field Validation
  ↓
Detection Logic
  ↓
Correlation / Behavior / Sequence
  ↓
Risk & Context
  ↓
Rule Development
  ↓
Testing
  ↓
Validation
  ↓
MITRE Mapping
  ↓
Deployment
  ↓
Monitoring
  ↓
SOC Feedback
  ↓
Tuning
  ↓
Review
```



Final Takeaway

Advanced Detection Engineering is not about writing complicated queries.

It is about creating **reliable security signals from raw data**.

A strong SIEM Engineer should understand:

==> Detection Logic ==> Correlation ==> Behavioral Detection ==> Sequence Detection ==> Risk-Based Detection ==> Baselines ==> False Positive Reduction ==> MITRE ATT&CK ==> Sigma ==> Detection-as-Code ==> Testing ==> Performance ==> Continuous Tuning

Remember:

Good detection creates useful signals. Great detection creates useful context.

Practical Security Takeaway

Don't ask only:

"Did an event happen?"

Ask:

"Is this behavior unusual?"

"How is it connected to other events?"

"Who is involved?"

"What happened before and after?"

"Can I confidently explain why this deserves investigation?"

That mindset is the foundation of advanced Detection Engineering.

Follow the SIEM Engineer learning series — Boni Yeamin



SIEM Engineer Learning Series

- #01 — Log Management
 - #02 — SIEM Platforms
 - #03 — Log Integration
 - #04 — Detection Engineering
 - #05 — Query & Investigation
 - #06 — SOC Operations
 - #07 — Threat Intelligence
 - #08 — Automation & SOAR
 - #09 — Dashboard & Reporting
 - #10 — SIEM Infrastructure
 - #11 — SIEM Architecture Design
 - #12 — SIEM Capacity Planning
 - #13 — SIEM Performance Tuning
 - #14 — Advanced Detection Engineering ← You are here
 - #15 — Threat Hunting with SIEM → Next
 - #16 — SIEM Data Quality & Troubleshooting
 - #17 — Hands-On SIEM: Multi-Platform Practical Guide
-

Navigation

← **Previous Article: #13 — SIEM Performance Tuning**

SIEM Engineer Index

Next Article: #15 — Threat Hunting with SIEM →

SIEM Engineer #15 — Threat Hunting with SIEM

Introduction

A traditional SOC often works like this:

```
Alert
  ↓
Investigation
  ↓
Response
```

But attackers do not always generate an obvious alert.

Sometimes suspicious activity exists inside normal-looking logs.

This is where **Threat Hunting** becomes important.

Threat Hunting means proactively searching security data to discover:

==> Unknown threats ==> Hidden attacker activity ==> Suspicious behavior ==> Compromised accounts ==> Persistence ==> Lateral movement ==> Data exfiltration ==> Detection gaps

A SIEM Engineer should understand how SIEM data can support continuous threat hunting.

What is Threat Hunting?

Threat Hunting is a proactive security activity where analysts search available data for evidence of malicious or suspicious activity.

Traditional detection:

```
graph TD;
  A[Attacker] --> B[Detection Rule];
  B --> C[Alert];
  C --> D[SOC Analyst];
```

Threat hunting:

```
graph TD;
  A[Threat Hypothesis] --> B[Search SIEM Data];
  B --> C[Find Suspicious Pattern];
  C --> D[Investigate];
  D --> E[Validate];
  E --> F[Improve Detection];
```

The key difference is that hunting does not always wait for an alert.

Why does this matter?

Attackers may:

==> Use legitimate tools ==> Compromise valid accounts ==> Modify existing processes ==> Blend into normal traffic ==> Avoid known malware signatures ==> Operate slowly ==> Exploit detection gaps

Therefore:

No alert does not always mean no threat.

Threat hunting helps search for activity that existing detections may have missed.



Threat Hunting vs Alert Investigation

These two activities are related but different.

Alert Investigation

You already have a signal:

```
SIEM Alert
↓
Validate
↓
Investigate
↓
Verdict
```

Threat Hunting

You start with a question or hypothesis:

```
Hypothesis
↓
Search
↓
Investigate
↓
Evidence
↓
Verdict
```

Example:

Alert Investigation

"Multiple failed SSH logins detected."

Threat Hunting

"Are there any unusual SSH login patterns that our current detections are not identifying?"



Threat Hunting Lifecycle

A practical hunting lifecycle is:

```
graph TD; A[Define Objective] --> B[Create Hypothesis]; B --> C[Identify Data Sources]; C --> D[Build Search]; D --> E[Analyze Results]; E --> F[Pivot]; F --> G[Correlate]; G --> H[Enrich]; H --> I[Validate]; I --> J[Scope]; J --> K[Document]; K --> L[Improve Detection];
```

Define Objective
↓
Create Hypothesis
↓
Identify Data Sources
↓
Build Search
↓
Analyze Results
↓
Pivot
↓
Correlate
↓
Enrich
↓
Validate
↓
Scope
↓
Document
↓
Improve Detection

The final step is extremely important.

A successful hunt can produce a **new detection rule**.

Start with a Hypothesis

Don't begin with:

"Let me search everything."

That creates enormous amounts of data.

Start with a focused hypothesis.

Example:

I suspect that a compromised account may be performing unusual authentication activity.

Then ask:

==> Which users? ==> Which hosts? ==> Which source IPs? ==> What time period? ==> What authentication methods? ==> What behavior is unusual?

Types of Threat Hunting

Common hunting approaches include:

IOC Hunting

Search for known indicators.

- IP
- Domain
- URL
- Hash
- Email

TTP Hunting

Search for attacker behavior associated with techniques.

- PowerShell
- Credential Access
- Persistence
- Lateral Movement
- Command Execution

User Hunting

Investigate unusual user behavior.

- Login time
- Source location
- Device
- Privilege
- Authentication failures

Host Hunting

Look for suspicious host behavior.

- New process
- New service
- Unexpected connection
- Privilege change
- Unusual login

Network Hunting

Analyze:

- DNS
- Firewall
- Proxy
- VPN
- NetFlow
- IDS/IPS

IOC Hunting

IOC means **Indicator of Compromise**.

Examples:

==> Malicious IP ==> Malicious domain ==> URL ==> File hash ==> Email address

Suppose you have:

```
Suspicious IP:  
203.0.113.50
```

A hunter may search:

```
src_ip = "203.0.113.50"  
OR  
dst_ip = "203.0.113.50"
```

Then investigate:

==> Which systems communicated with it? ==> When? ==> How frequently? ==> Which user was involved? ==> Which process generated the traffic? ==> Was there related DNS activity?

Historical IOC Hunting

One powerful feature of a SIEM is historical search.

Suppose an IP is identified as malicious today.

Don't only search:

Today

Search historical data according to your retention period.

Example:

Last 30 days

Then determine:

First Seen
Last Seen
Total Connections
Affected Hosts
Affected Users

This helps determine potential scope.

TTP Hunting

IOC hunting depends on known indicators.

TTP hunting focuses on attacker behavior.

For example:

Suspicious PowerShell

Instead of searching only for one known hash or IP, search for behaviors such as:

==> Encoded commands ==> Unusual PowerShell execution ==> Suspicious parent-child processes
==> Unexpected network connections ==> Unusual users executing PowerShell

This can help identify previously unknown infrastructure.

MITRE ATT&CK and Threat Hunting

MITRE ATT&CK provides a useful framework for describing adversary behavior.

A hunter can start with:

```
Tactic
↓
Technique
↓
Expected Behavior
↓
Required Data
↓
SIEM Search
```

For example:

```
Execution
↓
PowerShell
↓
PowerShell execution
↓
Windows process logs
↓
SIEM query
```

The exact data required depends on the environment.

Data Sources for Threat Hunting

A SIEM hunter may use:

Endpoint

==> Windows Security Logs ==> Sysmon ==> Linux audit logs ==> EDR

Network

==> Firewall ==> IDS/IPS ==> DNS ==> Proxy ==> NetFlow

Identity

==> Active Directory ==> VPN ==> SSO ==> MFA ==> Cloud Identity

Application

==> Web Server ==> Database ==> API Gateway ==> Application logs

The more useful telemetry available, the more hunting possibilities exist.

Search Strategy

A good hunt usually moves:

```
Broad
↓
Narrow
↓
Pivot
↓
Correlate
↓
Deep Investigation
```

Example:

```
All authentication events
↓
Unusual login hours
↓
Specific user
↓
Specific source IP
↓
Related host activity
↓
Process activity
↓
Network activity
```

Don't immediately search millions of events with complicated logic.

Start small and expand based on evidence.

Time-Based Hunting

Time is one of the most important dimensions in SIEM investigations.

Example:

```
12:00  
Login  
  
12:03  
PowerShell  
  
12:05  
Privilege Change  
  
12:08  
Network Connection
```

A timeline can reveal relationships that individual events do not show.

Always ask:

```
What happened before?  
What happened during?  
What happened after?
```

Baseline Hunting

Threat hunters should understand normal behavior.

Example:

```
User A
Normally:
08:30-18:00
Corporate Network
Laptop-01
```

Suddenly:

```
02:15
New Device
External IP
Administrative Login
```

This is an anomaly worth investigating.

But remember:

Anomaly $\neq$ confirmed attack.

The analyst must validate the context.

User Behavior Hunting

Consider:

```
Username  
Source IP  
Host  
Login Time  
Authentication Method  
Privilege  
Application
```

Example:

```
User: admin01  
  
Normal:  
10 logins/day  
  
Observed:  
150 login attempts/day
```

Questions:

==> Is this legitimate? ==> Was the password recently changed? ==> Is an application generating the activity? ==> Is the account under attack? ==> Was there a successful login afterward?

Host-Based Hunting

A hunter can investigate unusual hosts.

Example:

SERVER-01

Search:

Process Activity
Authentication
Network Connections
File Changes
Service Changes
Privilege Changes

Then build a timeline.

Login
↓
Process
↓
Privilege Change
↓
Network Connection

This provides stronger context.

Process Hunting

Process telemetry can provide valuable information.

Useful fields:

```
process_name
parent_process
command_line
user
hostname
timestamp
```

Example:

```
Parent:
winword.exe

Child:
powershell.exe
```

This relationship may deserve investigation depending on the environment and context.

The key is not simply:

```
PowerShell = malicious
```

Instead:

Who?

What?

When?

Where?

Parent process?

Command line?

Network activity?

Network Threat Hunting

Network hunting can investigate:

==> Rare destinations ==> High-volume connections ==> Unusual ports ==> Multiple destination hosts ==> DNS anomalies ==> Repeated failed connections ==> Unexpected outbound traffic

Example:

```
Host-01
  ↓
100 different external IPs
  ↓
within 10 minutes
```

Investigate:

==> What process created the connections? ==> Is the destination legitimate? ==> Is this normal for the host? ==> What DNS requests occurred?

DNS Hunting

DNS is extremely useful for threat hunting.

Search for:

==> Rare domains ==> Newly observed domains ==> High-frequency queries ==> Suspicious query patterns ==> Unexpected external domains

Example:

```
Host-01
  ↓
Rare Domain
  ↓
DNS Query
  ↓
Connection
```

Correlating DNS with firewall and endpoint data can provide additional context.

Authentication Hunting

Authentication hunting is especially useful for detecting account compromise.

Look for:

- Failed Login
- Successful Login
- New Source
- New Device
- Unusual Time
- Privilege Change

Example:

```
20 Failed Logins
    ↓
Successful Login
    ↓
New Device
    ↓
Administrative Activity
```

This creates a useful investigation hypothesis.

Impossible Travel / Abnormal Location

A user's authentication activity may show:

```
Login A  
Location A  
10:00
```

```
Login B  
Location B  
10:20
```

If the locations or network paths are inconsistent with realistic travel, this may require investigation.

However, VPNs, proxies, cloud services, mobile networks, and corporate gateways can create misleading locations.

Therefore:

Impossible travel should be treated as an investigation signal, not automatic proof of compromise.

Threat Hunting with Multiple Data Sources

A strong hunt often combines multiple sources.

Example:

Authentication

+

Endpoint

+

DNS

+

Firewall

Suppose:

Authentication:

Unusual login

Endpoint:

Suspicious process

DNS:

Rare domain

Firewall:

Outbound connection

Together, these events provide stronger context than any single source.

Pivoting

Pivoting means moving from one finding to another related piece of evidence.

Example:

```
Suspicious IP
  ↓
Affected Host
  ↓
Affected User
  ↓
Process
  ↓
DNS
  ↓
Other Hosts
```

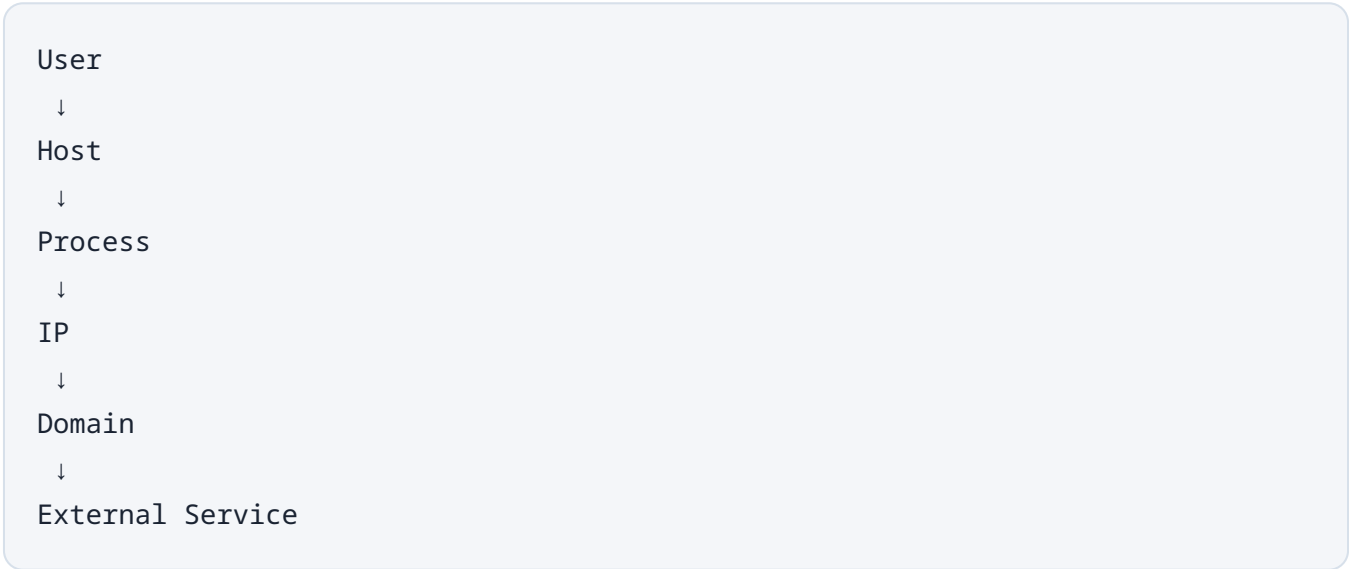
Another example:

```
Suspicious User
  ↓
Login Events
  ↓
Hosts
  ↓
Processes
  ↓
Network Connections
```

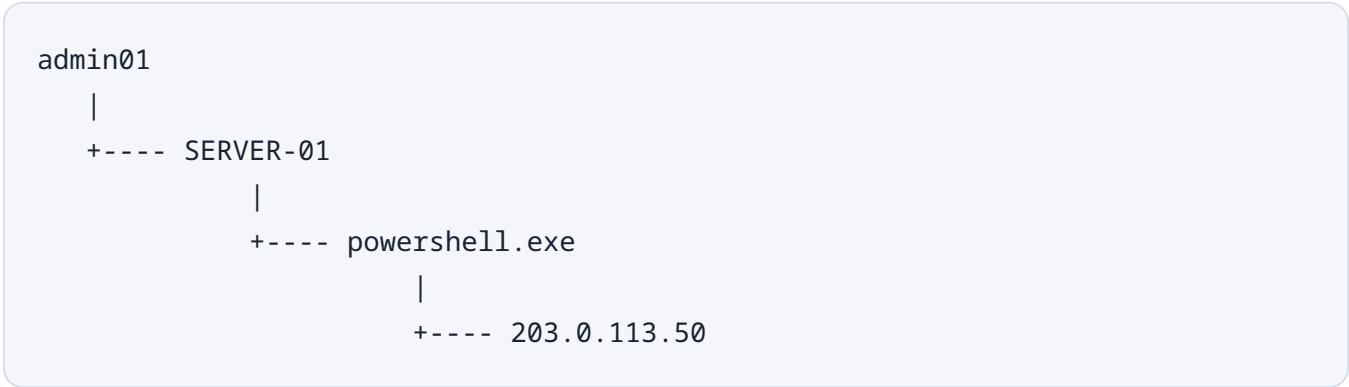
Pivoting is one of the most important SIEM investigation skills.

Entity Relationship Hunting

Think about relationships:



This can be represented as:



This relationship-based thinking makes complex investigations easier.

Threat Intelligence Enrichment

Threat Intelligence can add context to hunting.

Example:

```
Observed IP
  ↓
Threat Intelligence Lookup
  ↓
Reputation / Context
  ↓
Historical Activity
  ↓
Investigation
```

But threat intelligence should not be treated as absolute truth.

Consider:

==> Source reputation ==> Confidence ==> Age ==> Expiration ==> Context ==> Multiple sources

Hunt Results

A hunt can produce several outcomes.

Outcome 1 — No Suspicious Activity

No relevant evidence found.

This is still useful because it documents the hypothesis and search scope.

Outcome 2 — Suspicious Activity

Suspicious activity found.

Continue investigation.

Outcome 3 — Confirmed Incident

Evidence supports malicious activity.

Follow incident response procedures.

Outcome 4 — Detection Gap

You discover behavior that existing rules did not detect.

This should lead to:

New Detection

↓

Testing

↓

Deployment

Threat Hunting → Detection Engineering

One of the strongest outcomes of hunting is creating new detections.

Example:

```
Threat Hunt
  ↓
Suspicious PowerShell Pattern
  ↓
Repeated Observation
  ↓
Detection Logic
  ↓
Rule
  ↓
Testing
  ↓
Production Detection
```

This creates a continuous improvement cycle:

```
Hunt
  ↓
Discover
  ↓
Detect
  ↓
Monitor
  ↓
Hunt Again
```

Practical Hunt — SSH Activity

Suppose you want to investigate SSH activity.

Start with:

Authentication logs

Search:

SSH login

Then identify:

Source IP
Username
Destination Host
Success/Failure
Timestamp

Next:

Top source IPs

Then:

Failed login count

Then:

Successful login after failures

Then:

Post-login activity

Finally:

```
sudo  
process  
network  
file
```

This is a real hunting workflow.

Practical Hunt — Suspicious PowerShell

Start with:

PowerShell events

Then analyze:

User
Host
Parent Process
Command Line
Timestamp
Destination

Pivot into:

Network Connections

Then:

DNS

Then:

Authentication

Finally determine:

Normal
Suspicious
Malicious
Unknown

Practical Hunt — Lateral Movement

Start with remote authentication:

```
RDP
SSH
SMB
WinRM
VPN
```

Then ask:

```
Which user?
Which source host?
Which destination?
How many destinations?
Was the behavior normal?
What happened after login?
```

Example:

```
User A
  ↓
Host 01
  ↓
Host 02
  ↓
Host 03
  ↓
Host 04
```

Multiple remote connections in a short period may warrant investigation depending on the user's role and normal behavior.

Threat Hunting Query Design

A good query should be:

==> Focused ==> Understandable ==> Reusable ==> Efficient ==> Documented

Start with a simple query.

Then add:

```
Time
↓
User
↓
Host
↓
IP
↓
Process
↓
Network
```

Avoid writing one huge query before understanding the data.

Hunting Notebook

Maintain a hunting record.

Example:

Hunt ID:

TH-2026-001

Hypothesis:

Possible compromised privileged account

Data Sources:

Authentication

Endpoint

Firewall

Time Range:

30 Days

Queries:

...

Findings:

...

Affected Entities:

...

Evidence:

...

Verdict:

...

Detection Gap:

...

New Detection:

...

Analyst:

...

Date:

...

This improves repeatability and knowledge sharing.

Threat Hunting Metrics

Threat hunting should not be measured only by the number of hunts.

Useful metrics include:

==> Hunts completed ==> Suspicious findings ==> Confirmed incidents ==> Detection gaps discovered ==> New detections created ==> Existing detections improved ==> Time spent per hunt ==> Data sources used ==> Coverage improvements

A hunt that finds a detection gap can be valuable even when no incident is discovered.

Common Threat Hunting Mistakes

Avoid:

==> Searching without a hypothesis ==> Searching too much data at once ==> Ignoring normal behavior ==> Trusting one IOC blindly ==> Ignoring historical data ==> Not correlating data sources ==> Not documenting findings ==> Treating anomalies as confirmed attacks ==> Never converting findings into detections ==> Ignoring query performance

Threat Hunting Lab

You can create a practical lab with:

Wazuh
Linux VM
Windows VM
Firewall
Syslog
Network Traffic

Hunt 01

Investigate:

SSH Failed Login

Hunt 02

Investigate:

Successful Login After Multiple Failures

Hunt 03

Investigate:

Unusual PowerShell Activity

Hunt 04

Investigate:

New Privileged Account Activity

Hunt 05

Investigate:

One Host Connecting to Many Destinations

Hunt 06

Investigate:

Rare DNS Domains

For each hunt:

Hypothesis

↓

Query

↓

Evidence

↓

Pivot

↓

Correlation

↓

Verdict

↓

Detection Improvement

SIEM Engineer Interview Questions

Q1. What is Threat Hunting?

Answer:

Threat Hunting is a proactive process of searching security data for suspicious or malicious activity that may not have generated an existing SIEM alert.

Q2. What is the difference between Threat Hunting and Alert Investigation?

Answer:

Alert investigation starts with an existing security alert, while threat hunting starts with a hypothesis or security question and searches available data proactively.

Q3. What is IOC hunting?

Answer:

IOC hunting searches SIEM and security telemetry for known indicators such as malicious IPs, domains, URLs, hashes, or other indicators.

Q4. What is TTP hunting?

Answer:

TTP hunting focuses on attacker behaviors and techniques rather than depending only on known indicators.

Q5. How do you start a threat hunt?

Answer:

I start by defining a clear objective and hypothesis, identify the required data sources, validate the available fields, create focused queries, analyze results, pivot into related evidence, correlate events, and document the findings.

Q6. Why is historical data important?

Answer:

Historical data can reveal previous activity associated with a newly discovered IOC, account, host, or behavior and helps determine the potential scope and timeline.

Q7. What do you do when a hunt discovers suspicious activity?

Answer:

I preserve the relevant evidence, investigate the scope and context, determine whether the activity is benign, suspicious, or malicious, and follow the organization's incident response process when appropriate.

Q8. How can Threat Hunting improve SIEM?

Answer:

Threat hunting can identify detection gaps, improve existing rules, create new detections, identify missing telemetry, and improve overall detection coverage.

Q9. Why is baseline important in threat hunting?

Answer:

A baseline helps distinguish unusual behavior from normal operational activity. It provides context for determining whether an observed deviation deserves further investigation.

Q10. What is pivoting?

Answer:

Pivoting means moving from one investigation artifact to related evidence, such as moving from an IP to a host, then to a user, process, DNS activity, and network connections.

Threat Hunting Checklist

Before completing a hunt:

- ☒ Objective defined
 - ☒ Hypothesis created
 - ☒ Data sources identified
 - ☒ Fields validated
 - ☒ Time range defined
 - ☒ Initial query created
 - ☒ Baseline considered
 - ☒ Historical data checked
 - ☒ Pivot points identified
 - ☒ Multiple data sources correlated
 - ☒ Threat Intelligence considered
 - ☒ Evidence documented
 - ☒ Scope evaluated
 - ☒ Verdict recorded
 - ☒ Detection gap considered
 - ☒ New detection identified
 - ☒ Final report completed
-

Complete Threat Hunting Flow

Remember this flow:

```
graph TD; A[Threat Hypothesis] --> B[Data Sources]; B --> C[SIEM Search]; C --> D[Initial Finding]; D --> E[Pivot]; E --> F[Correlation]; F --> G[Enrichment]; G --> H[Timeline]; H --> I[Scope]; I --> J[Evidence]; J --> K[Verdict]; K --> L[Detection Improvement]; L --> M[Documentation];
```

Threat Hypothesis
↓
Data Sources
↓
SIEM Search
↓
Initial Finding
↓
Pivot
↓
Correlation
↓
Enrichment
↓
Timeline
↓
Scope
↓
Evidence
↓
Verdict
↓
Detection Improvement
↓
Documentation

Final Takeaway

A SIEM Engineer should not think only in terms of:

"Which alert fired?"

A mature security mindset also asks:

"What could be happening that we are not detecting?"

Threat Hunting helps answer that question.

Remember:

==> **IOC Hunting** finds known indicators. ==> **TTP Hunting** finds attacker behavior. ==> **Behavioral Hunting** finds abnormal activity. ==> **Historical Hunting** finds previous exposure. ==> **Correlation** connects separate events. ==> **Pivoting** expands the investigation. ==> **Detection Engineering** turns hunting knowledge into repeatable detection.

The strongest SIEM workflow is not:

Alert → Investigation → Close

It is:

Alert

↓

Investigation

↓

Threat Hunt

↓

Detection Gap

↓

New Detection

↓

Continuous Improvement



Practical Security Takeaway

Don't wait for every threat to generate an alert.

Build the habit of asking:

What behavior should I expect?

What behavior is unusual?

What evidence do I have?

What else is connected?

What happened before and after?

What detection gap did I discover?

That is the mindset of a **Threat Hunter + SIEM Engineer**.

Follow the SIEM Engineer learning series — Boni Yeamin

SIEM Engineer Learning Series

- #01 — Log Management
- #02 — SIEM Platforms
- #03 — Log Integration
- #04 — Detection Engineering
- #05 — Query & Investigation
- #06 — SOC Operations
- #07 — Threat Intelligence
- #08 — Automation & SOAR
- #09 — Dashboard & Reporting
- #10 — SIEM Infrastructure
- #11 — SIEM Architecture Design
- #12 — SIEM Capacity Planning
- #13 — SIEM Performance Tuning
- #14 — Advanced Detection Engineering
- #15 — Threat Hunting with SIEM ← You are here
- #16 — SIEM Data Quality & Troubleshooting → Next
- #17 — Hands-On SIEM: Multi-Platform Practical Guide

Navigation

← Previous Article: #14 — Advanced Detection Engineering

SIEM Engineer Index

Next Article: #16 — SIEM Data Quality & Troubleshooting →

SIEM Engineer #16 — SIEM Data Quality & Troubleshooting

Introduction

A SIEM is only as useful as the data it receives.

Imagine your SIEM has thousands of detection rules, dashboards, and correlation logic—but:

```
Firewall logs are missing
Windows logs are delayed
Timestamps are incorrect
Events are duplicated
Fields are not parsed
```

Then even a well-designed SIEM may produce incorrect results.

This is why **SIEM Data Quality** is a critical responsibility for a SIEM Engineer.

A SIEM Engineer should be able to answer:

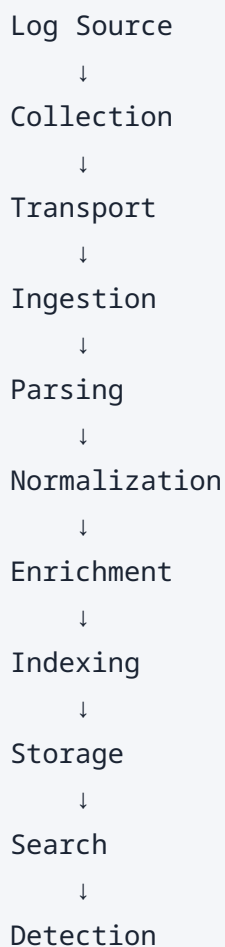
==> Are the logs arriving? ==> Are they complete? ==> Are they parsed correctly? ==> Are timestamps correct? ==> Are events duplicated? ==> Are important fields available? ==> Is there ingestion delay? ==> Are events being dropped? ==> Is the data trustworthy enough for detection?

What is SIEM Data Quality?

SIEM Data Quality means ensuring that security data entering and stored inside the SIEM is:

==> Available ==> Complete ==> Accurate ==> Timely ==> Correctly parsed ==> Properly normalized ==> Consistent ==> Searchable ==> Reliable

Think of the SIEM pipeline:



```
graph TD; A[Log Source] --> B[Collection]; B --> C[Transport]; C --> D[Ingestion]; D --> E[Parsing]; E --> F[Normalization]; F --> G[Enrichment]; G --> H[Indexing]; H --> I[Storage]; I --> J[Search]; J --> K[Detection];
```

Log Source
↓
Collection
↓
Transport
↓
Ingestion
↓
Parsing
↓
Normalization
↓
Enrichment
↓
Indexing
↓
Storage
↓
Search
↓
Detection

A problem at any stage can affect security operations.

Why does this matter?

Suppose a firewall stops sending logs.

The SIEM may show:

No Firewall Alerts

A SOC Analyst might think:

"No attack detected."

But the real situation could be:

```

Firewall
  X
  ↓
No Logs
  ↓
No Detection
  ↓
No Alert
    
```

This is a **visibility problem**, not necessarily a security improvement.

Therefore:

No security event can sometimes mean no telemetry.



Data Quality Dimensions

Important dimensions include:

1. Availability

Are the expected logs arriving?

2. Completeness

Are important fields and events present?

3. Accuracy

Does the data represent the real event correctly?

4. Timeliness

Are logs arriving without excessive delay?

5. Consistency

Are the same fields formatted consistently?

6. Uniqueness

Are events being duplicated?

7. Validity

Do values follow the expected format?



Common SIEM Data Problems

Common problems include:

- Missing Logs
- Duplicate Logs
- Parsing Failure
- Timestamp Problems
- Log Delay
- Field Mapping Problems
- Wrong Source Identification
- Malformed Events
- Encoding Problems
- Dropped Events
- Network Connectivity Problems
- Certificate Problems
- Authentication Problems
- Storage Problems

A SIEM Engineer needs a structured troubleshooting approach.



The Data Troubleshooting Flow

Use this process:

```
Problem Detected
  ↓
Identify Source
  ↓
Check Source Health
  ↓
Check Network
  ↓
Check Collector
  ↓
Check Ingestion
  ↓
Check Parsing
  ↓
Check Normalization
  ↓
Check Indexing
  ↓
Check Search
  ↓
Validate Detection
  ↓
Document Root Cause
```

Do not immediately change configuration.

First identify **where the data pipeline is failing**.

Missing Logs

One of the most common SIEM problems is:

"Why are logs not coming?"

Possible causes:

==> Source service stopped ==> Agent stopped ==> Network problem ==> Firewall blocking traffic
==> Wrong destination ==> Authentication failure ==> Certificate problem ==> Collector failure ==>
Parser problem ==> Storage problem ==> Queue overflow ==> Configuration error

Missing Log Troubleshooting

Suppose:

Firewall logs stopped arriving.

Check:

Step 1 — Source

Is the firewall generating logs?

Firewall
↓
Logging Enabled?

Step 2 — Destination

Is the firewall configured with the correct SIEM/collector destination?

Step 3 — Network

Can the source communicate with the collector?

Step 4 — Collector

Is the collector listening?

Step 5 — Ingestion

Are events entering the SIEM?

Step 6 — Parsing

Are the events being recognized?

Step 7 — Search

Can you find the events?

This prevents random troubleshooting.

Source Health Monitoring

Every important log source should have health monitoring.

Example:

Firewall-01

Expected:

Continuous logs

Observed:

No events for 30 minutes

This should create a health signal.

A SIEM should ideally monitor:

==> Last event time ==> Event count ==> EPS ==> Connection status ==> Agent status ==> Queue status ==> Error rate

Log Silence Detection

A useful operational detection is:

Expected Log Source

↓

No Events

↓

Defined Time Period

↓

Health Alert

For example:

Firewall-01

No events for 15 minutes

The exact threshold depends on the source.

This is sometimes called a **log source health** or **log silence** detection.

Duplicate Logs

Duplicate logs happen when the same event reaches the SIEM more than once.

Example:

Original Event



Collector A



SIEM

Original Event



Collector B



SIEM

Now the same event may appear twice.

Possible causes:

==> Multiple forwarding paths ==> Misconfigured agents ==> Retry behavior ==> Collector duplication ==> Load-balancing issues ==> Replayed events

Why Duplicate Logs Matter

Duplicates can affect:

==> Alert volume ==> EPS calculations ==> Storage ==> Dashboards ==> Correlation rules ==>
Threshold detections ==> Incident investigation

Example:

Actual:

5 failed logins

SIEM sees:

10 events

A threshold rule might incorrectly trigger.

Therefore, duplicate detection is important.

Detecting Duplicates

Useful fields may include:

```
event_id
timestamp
source
host
message
hash
sequence_number
```

Conceptually:

```
Same Event
+
Same Source
+
Same Timestamp
+
Same Identifier
```

may indicate duplication.

However, not every identical-looking event is necessarily a duplicate. Validate the source behavior first.

Parsing Failure

Suppose the raw log is:

```
<134>Sep 16 15:20:10 firewall01  
src=10.10.10.20 dst=10.10.20.10 action=deny
```

But the SIEM cannot extract:

```
src_ip  
dst_ip  
action
```

Then detection rules may fail.

This is a **parsing problem**.

Why Parsing Matters

Raw:

```
src=10.10.10.20  
dst=10.10.20.10  
action=deny
```

Normalized:

```
source.ip = 10.10.10.20  
destination.ip = 10.10.20.10  
event.action = deny
```

Detection logic becomes easier when fields are normalized.

For example:

```
source.ip  
destination.ip  
user.name  
process.name  
event.action
```

Parsing Troubleshooting

When parsing fails:

```
Raw Event
  ↓
Inspect Format
  ↓
Identify Pattern
  ↓
Check Parser
  ↓
Check Field Extraction
  ↓
Test Sample Logs
  ↓
Validate Fields
  ↓
Test Detection
```

Do not modify a parser without first examining real raw events.

Field Mapping Problems

Different vendors may use different names.

Example:

Vendor A:

`src_ip`

Vendor B:

`sourceAddress`

Vendor C:

`source.ip`

If your detection expects:

`source.ip`

but the field is never mapped, the detection may not work.

Therefore, normalization is important.

Timestamp Problems

Time is one of the most important parts of SIEM data.

A log may contain:

```
Source Time:  
15:00
```

But the SIEM may display:

```
09:00
```

Possible causes:

==> Time zone mismatch ==> NTP problem ==> Incorrect parser ==> Timestamp format issue ==>
Collector clock problem ==> Daylight-saving handling where applicable

Why Timestamp Accuracy Matters

Consider:

```
10:00 Login  
10:02 PowerShell  
10:05 Privilege Change
```

If timestamps are wrong:

```
10:05 Login  
09:50 PowerShell  
10:20 Privilege Change
```

The investigation timeline becomes misleading.

This can affect:

==> Correlation ==> Sequence detection ==> Incident investigation ==> Threat hunting ==> Root-cause analysis

Time Synchronization

Important systems should use reliable time synchronization.

Conceptually:

```
NTP
↓
Firewall
Server
Windows
Linux
Collector
SIEM
```

All systems should maintain consistent time.

A SIEM Engineer should monitor:

==> Clock drift ==> Time zone configuration ==> NTP synchronization ==> Timestamp parsing

Log Latency

Log latency means the delay between when an event occurs and when it becomes available for analysis.

Example:

Event occurs:

10:00:00

SIEM receives:

10:00:03

Latency:

3 seconds

But imagine:

Event occurs:

10:00

SIEM receives:

10:20

A 20-minute delay can seriously affect SOC response.

Sources of Log Latency

Latency may occur in:

```
graph TD; Source --> Network; Network --> Collector; Collector --> Queue; Queue --> Parser; Parser --> Indexer; Indexer --> Storage;
```

Source
↓
Network
↓
Collector
↓
Queue
↓
Parser
↓
Indexer
↓
Storage

Possible causes:

==> Network congestion ==> Collector overload ==> Queue backlog ==> CPU saturation ==> Disk I/O ==> Indexing delay ==> Rate limiting ==> Backpressure

Queue and Backpressure

Suppose:

Incoming:
10,000 EPS

Processing capacity:
7,000 EPS

Then:

10,000
↓
Queue
↓
7,000 processed

The remaining events may accumulate.

If the queue becomes full:

Queue Full
↓
Event Drop

This creates a data-loss risk.

Monitor:

==> Queue depth ==> Queue growth ==> Processing rate ==> Event drops ==> Retry rate

Event Drops

Event drops are particularly important.

Possible causes:

==> Queue overflow ==> Network failure ==> Collector overload ==> Parser errors ==> Storage failure ==> Rate limiting ==> Resource exhaustion

If events are dropped, detection coverage may be affected.

The SIEM Engineer should identify:

How many?
When?
Which source?
Which component?
Why?

Log Completeness

Completeness means checking whether the expected data is present.

Suppose a firewall should send:

```
Allow
Deny
VPN
Authentication
Admin
System
```

But only:

```
Deny
```

events appear.

The source may technically be connected, but the dataset is incomplete.

Therefore:

Connected does not always mean complete.

Data Quality Validation

A practical validation framework:

```
graph TD; A[Availability] --> B[Completeness]; B --> C[Parsing]; C --> D[Normalization]; D --> E[Timestamp]; E --> F[Timeliness]; F --> G[Uniqueness]; G --> H[Searchability]; H --> I[Detection Validation];
```

Availability
↓
Completeness
↓
Parsing
↓
Normalization
↓
Timestamp
↓
Timeliness
↓
Uniqueness
↓
Searchability
↓
Detection Validation

Log Source Onboarding Validation

When onboarding a new source:

Step 1

Confirm source configuration.

Step 2

Generate test events.

Step 3

Verify transport.

Step 4

Confirm ingestion.

Step 5

Inspect raw event.

Step 6

Validate parsing.

Step 7

Validate normalized fields.

Step 8

Check timestamp.

Step 9

Test detection.

Step 10

Document the source.

Raw vs Parsed Data

Always understand both.

Raw Data

Original event exactly as received.

Parsed Data

Structured fields extracted from the event.

Example:

Raw:
user=admin src=10.0.0.10 action=login

Parsed:

```
user.name = admin
source.ip = 10.0.0.10
event.action = login
```

When troubleshooting detection failures, checking raw data is often essential.

Detection Failure Because of Data Quality

Suppose you have this detection:

```
source.ip = suspicious_ip
```

But the firewall logs contain:

```
clientAddress
```

and no mapping exists.

The detection may return:

```
0 results
```

The problem may not be the detection rule.

It may be the **data model**.

This is why SIEM Engineers should always verify the data before blaming the rule.

Data Quality and Threat Hunting

Threat hunting depends heavily on reliable telemetry.

Suppose you hunt:

PowerShell activity

But PowerShell logs are missing.

Your hunt may incorrectly conclude:

No suspicious PowerShell activity found.

The correct conclusion may be:

PowerShell telemetry is incomplete.
Hunt confidence is limited.

This distinction is extremely important.

Data Quality Monitoring Dashboard

A useful SIEM health dashboard can include:

- Log Sources
- EPS
- Last Event Time
- Event Drops
- Parsing Errors
- Queue Size
- Latency
- Storage
- Collector Health
- Agent Health

Example:

Source	EPS	Last Event
Firewall-01	450	10 sec ago
Windows-DC01	800	5 sec ago
Linux-Server-01	120	8 sec ago
VPN-01	0	25 min ago ⚠

This immediately highlights a potential problem.

Data Quality Alerts

Automate health monitoring.

Examples:

No Logs

No events for defined period
→ Health Alert

EPS Drop

Normal: 500 EPS
Current: 20 EPS
→ Investigate

Parsing Error

Parsing failure rate > threshold
→ Alert

Queue Growth

Queue continuously increasing
→ Investigate

Latency

Normal: <10 sec

Observed: 5 min

→ Investigate

Troubleshooting Example — Firewall Logs Missing

Problem:

Firewall logs disappeared.

Investigation

```
Firewall
↓
Is logging enabled?
↓
Is destination correct?
↓
Network connectivity?
↓
Collector listening?
↓
Events arriving?
↓
Parser working?
↓
Indexing working?
↓
Search working?
```

Suppose you discover:

```
Firewall
  ↓
Network
  ↓
Collector
  X
```

Root cause:

Collector service stopped.

After restarting:

```
Firewall
  ↓
Collector
  ↓
SIEM
  ↓
Search
  ↓
Detection
```

Validate before closing the incident.

Troubleshooting Example — Duplicate Events

Problem:

Alert volume suddenly doubled.

Investigation:

```
Alert Volume
  ↓
Event Count
  ↓
Source
  ↓
Raw Events
  ↓
Compare Event IDs
  ↓
Check Forwarders
```

Suppose:

Collector A → SIEM
Collector B → SIEM

Both are forwarding the same events.

Root cause:

Duplicate forwarding path

Fix:

==> Correct forwarding configuration ==> Validate event count ==> Confirm alert volume ==>
Check storage impact

Troubleshooting Example — Wrong Timestamp

Problem:

Events appear several hours in the past.

Check:

```
Source timezone
↓
Server timezone
↓
NTP
↓
Collector
↓
Parser
↓
SIEM timestamp field
```

Identify where the time changed.

Never simply modify the SIEM display without understanding the source of the problem.

Troubleshooting Example — Parsing Failure

Problem:

Firewall events arrive,
but detection cannot find source IP.

Check:

Raw Event
↓
Parser
↓
Field Extraction
↓
Field Mapping

If raw data contains:

src=10.10.10.10

but the parser does not extract it into:

source.ip

the detection may fail.

Fix the parsing or normalization layer, then retest.

Data Quality SLA

Organizations can define operational expectations.

For example:

Log Availability:

≥ 99.5% per critical source (example - adapt to org)

Log Latency:

≤ 5 min for critical sources, ≤ 15 min standard (example)

Parsing Success:

≥ 99% parsed, < 1% unparsed (example)

Critical Source Silence:

Alert if no events for > 15 min (example)

The exact targets should be based on business and security requirements.

Not every source requires identical thresholds.

Critical vs Non-Critical Sources

Prioritize monitoring based on importance.

Critical

==> Domain Controllers ==> Firewalls ==> Identity Systems ==> EDR ==> VPN ==> Critical Servers

Important

==> Application Servers ==> Web Servers ==> Databases

Lower Priority

==> Development Systems ==> Test Systems

The classification depends on the organization's architecture.

Data Quality and Compliance

Reliable logging can also support:

==> Auditability ==> Investigation ==> Incident response ==> Security monitoring ==> Compliance requirements

But compliance requirements vary by organization, industry, and jurisdiction.

Therefore, SIEM retention and logging requirements should be mapped to the organization's actual policies and applicable regulations.

Practical Lab

Build a lab using:

Wazuh
Linux VM
Windows VM
Firewall
Syslog

Lab 01 — Missing Logs

Stop a log source or agent in your isolated lab.

Observe:

Last Event Time
EPS
Health Status

Create a health alert.

Lab 02 — Parsing

Send a sample Syslog event.

Verify:

Raw Event
Source IP
Destination IP
Action
Timestamp

Lab 03 — Timestamp

Generate logs from systems with different timezone configurations in a controlled lab.

Observe the effect on:

Timeline
Search
Correlation

Then correct the configuration.

Lab 04 — Duplicate Logs

Configure two forwarding paths in the lab.

Observe:

Event Count
EPS
Alert Count

Then remove the duplicate path.

Lab 05 — Log Latency

Generate controlled events and measure:

Event Time

↓

Collection Time

↓

SIEM Search Time

Calculate approximate ingestion delay.

SIEM Data Quality Checklist

- ☒ Log source identified
 - ☒ Source is generating logs
 - ☒ Network path verified
 - ☒ Collector healthy
 - ☒ Authentication verified
 - ☒ Events arriving
 - ☒ Raw logs inspected
 - ☒ Parser validated
 - ☒ Fields mapped
 - ☒ Timestamp verified
 - ☒ Time synchronization checked
 - ☒ Duplicate events checked
 - ☒ Event latency measured
 - ☒ Event drops checked
 - ☒ Queue health checked
 - ☒ Storage health checked
 - ☒ Search validated
 - ☒ Detection tested
 - ☒ Root cause documented
-

SIEM Engineer Interview Questions

Q1. What is SIEM Data Quality?

Answer:

SIEM Data Quality is the process of ensuring that security data is available, complete, accurate, timely, correctly parsed, normalized, and reliable for detection and investigation.

Q2. What would you check if logs suddenly stopped?

Answer:

I would check the source, logging configuration, network connectivity, collector status, authentication or certificates, ingestion pipeline, queues, parsing, indexing, and finally search availability.

Q3. What is log latency?

Answer:

Log latency is the time difference between when an event occurs and when it becomes available in the SIEM for analysis.

Q4. What causes duplicate logs?

Answer:

Common causes include duplicate forwarding paths, multiple agents, retry mechanisms, collector configuration problems, replayed events, or misconfigured load-balancing paths.

Q5. Why is parsing important?

Answer:

Parsing converts raw events into structured fields. Detection rules, correlation, dashboards, and investigations often depend on those structured fields.

Q6. How do you troubleshoot a parsing problem?

Answer:

I first inspect the raw event, understand its format, verify the parser, check field extraction and mapping, test representative events, and then validate the resulting fields in the SIEM.

Q7. Why is timestamp accuracy important?

Answer:

Accurate timestamps are essential for event correlation, attack timelines, threat hunting, incident investigation, and determining the sequence of activities.

Q8. What is backpressure?

Answer:

Backpressure occurs when incoming data arrives faster than downstream components can process it, causing queues to grow and potentially resulting in increased latency or event loss.

Q9. How can you detect a silent log source?**Answer:**

Monitor the source's expected event rate or last-event timestamp and generate a health alert when activity falls below an appropriate threshold or stops for a defined period.

Q10. What is the relationship between data quality and detection engineering?**Answer:**

Detection logic depends on reliable telemetry. Missing, incorrectly parsed, delayed, or incorrectly mapped data can cause false negatives or misleading detections.

Complete SIEM Data Quality Flow

Remember this:

```
graph TD; A[Log Source] --> B[Generate Event]; B --> C[Transport]; C --> D[Collector]; D --> E[Ingestion]; E --> F[Queue]; F --> G[Parsing]; G --> H[Normalization]; H --> I[Timestamp]; I --> J[Enrichment]; J --> K[Indexing]; K --> L[Storage]; L --> M[Search]; M --> N[Detection]; N --> O[Investigation];
```

Log Source
↓
Generate Event
↓
Transport
↓
Collector
↓
Ingestion
↓
Queue
↓
Parsing
↓
Normalization
↓
Timestamp
↓
Enrichment
↓
Indexing
↓
Storage
↓
Search
↓
Detection
↓
Investigation

At every stage ask:

Is the data:
Available?
Complete?
Accurate?
Timely?
Unique?
Valid?
Searchable?



Final Takeaway

A SIEM Engineer should never think:

"Logs are visible, so everything is fine."

Instead, think:

Are all expected logs arriving?
Are important fields present?
Are timestamps correct?
Are events delayed?
Are events duplicated?
Are events being dropped?
Can detection rules trust this data?

Because:

Bad data can create bad detection.

A strong SIEM Engineer therefore works on both sides:

Data Quality
+
Detection Engineering
+
Performance
+
Troubleshooting

Together, these create a reliable SIEM environment.

Practical Security Takeaway

Before creating a complex detection rule, first ask:

"Can I trust the data?"

If the answer is no, fix the **data pipeline** first.

Good SIEM engineering starts with good data.

Follow the SIEM Engineer learning series — Boni Yeamin



SIEM Engineer Learning Series

- #01 — Log Management
- #02 — SIEM Platforms
- #03 — Log Integration
- #04 — Detection Engineering
- #05 — Query & Investigation
- #06 — SOC Operations
- #07 — Threat Intelligence
- #08 — Automation & SOAR
- #09 — Dashboard & Reporting
- #10 — SIEM Infrastructure
- #11 — SIEM Architecture Design
- #12 — SIEM Capacity Planning

#13 — SIEM Performance Tuning

#14 — Advanced Detection Engineering

#15 — Threat Hunting with SIEM

#16 — SIEM Data Quality & Troubleshooting ← **You are here**

#17 — Hands-On SIEM: Multi-Platform Practical Guide → **Next**

Navigation

← **Previous Article: #15 — Threat Hunting with SIEM**

SIEM Engineer Index

Next Article: #17 — Hands-On SIEM: Multi-Platform Practical Guide →

SIEM Engineer #17 — Hands-On SIEM: Multi-Platform Practical Guide

Introduction

এখন পর্যন্ত আমরা যা শিখেছি তার বেশিরভাগই **theory**।

এই article-এর মূল কথা একটাই:

শুধু Wazuh নয় — একটি **unified lab** দিয়ে **overall SIEM practical** শেখা, যেখানে একই **exercise** একাধিক **platform**-এ করা যায়।

একজন SIEM Engineer-এর জন্য দরকার:

==> একটি platform ভালোভাবে জানা (primary skill)

==> অন্য platform-গুলোতে একই কাজ করতে পারা (adaptability)

==> কোন platform-এ কোন কাজ কীভাবে হয় তার ধারণা (concept portability)

এই guide-এ আমরা একটি **lab**, একটি **dataset**, এবং একই **workflow** ব্যবহার করে শিখব:

Wazuh → Splunk → Elastic Security → OpenSearch → Microsoft Sentinel

সবগুলো একসাথে — একটাই structured plan-এ।

1. Platform-Independent Core Skills

প্রথমে বুঝুন — platform বদলালেও **skill** বদলায় না।

প্রতিটি SIEM-এ আপনাকে একই ৮টি কাজ করতে হয়:

- | | |
|--------------------|----------------------------|
| 1. Data Onboarding | → logs ঢুকানো |
| 2. Parsing | → fields বের করা |
| 3. Normalization | → common schema-তে map করা |
| 4. Searching | → data খোঁজা |
| 5. Detection | → rule লেখা |
| 6. Investigation | → alert তদন্ত করা |
| 7. Dashboard | → visualize করা |
| 8. Tuning | → false positive কমানো |

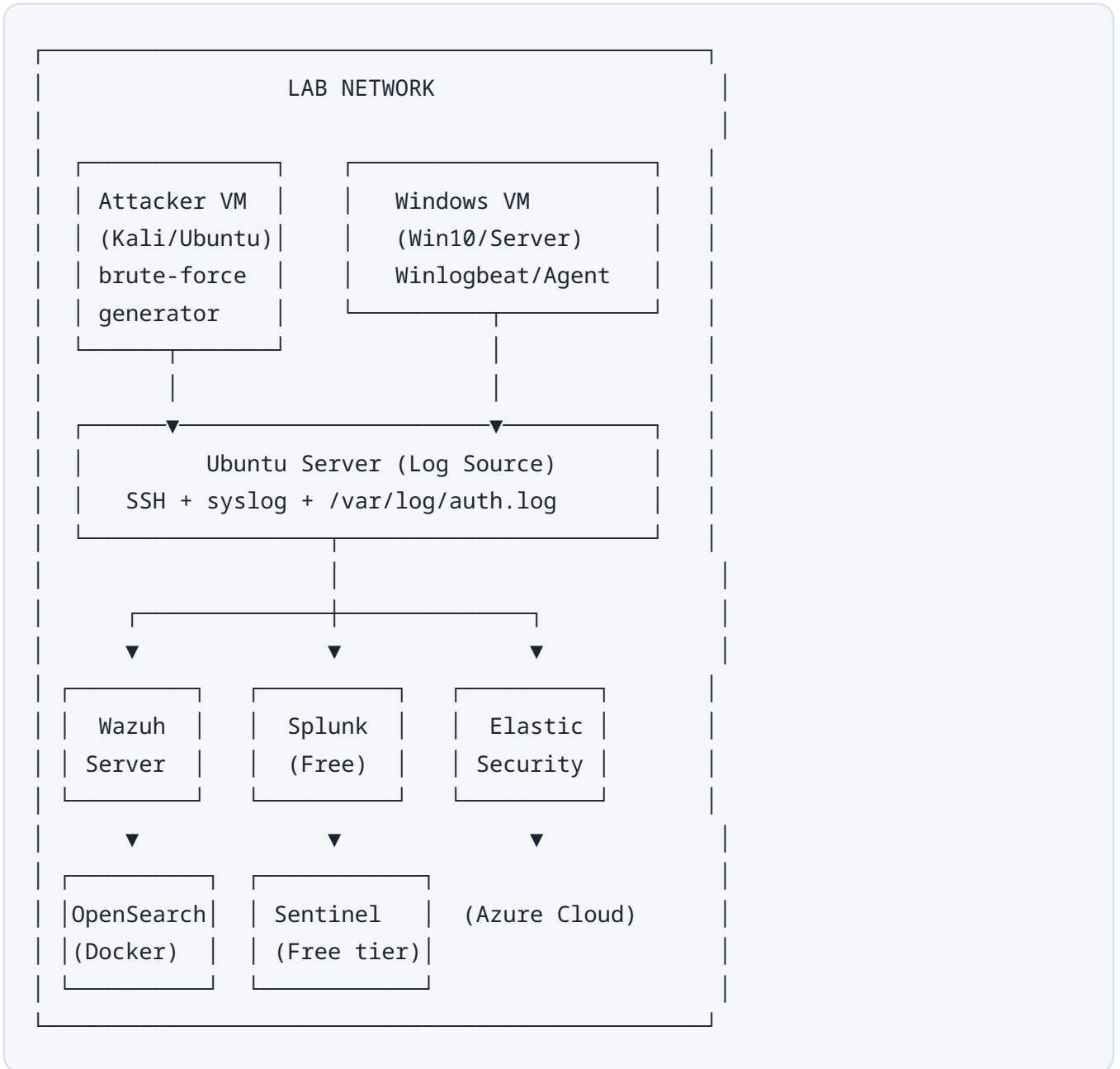
তাই আমাদের lab plan হবে:

একই ৮টি কাজ → ৫টি platform-এ practice।

Platform শেখা সহজ হয়ে যাবে, কারণ আপনি আগেই জানবেন **কী করতে হবে** — শুধু **কীভাবে** খুঁজে নিতে হবে।

2. The Unified Lab Architecture

একটি lab environment দিয়েই সব platform test করা যাবে:



Minimum hardware requirement

Option A – সব একসাথে (challenging):

32 GB RAM host

Option B – Recommended (প্র্যাকটিক্যাল):

16 GB RAM host

একসময়ে এক/দুইটি SIEM চালান, বাকিগুলো stop করুন

Exercise শেষ হলে পরের platform-এ যান

Option C – Low resource:

Wazuh (VM) + Sentinel (cloud, নিজের মেশিনে resource লাগে না)

VM specification (recommended)

Ubuntu Server (log source + Wazuh) → 4 GB RAM, 2 vCPU

Windows VM → 4 GB RAM, 2 vCPU

Attacker VM → 2 GB RAM, 1 vCPU

SIEM platform (per platform) → 4-8 GB RAM

Cloud alternative: AWS/Azure free tier-এ Ubuntu VM নিয়েও lab করা যায়, কিন্তু brute-force simulation নিজের isolated lab-এই করা ভালো।



3. Platform Setup — কোনটা কীভাবে

3.1 Wazuh (Free, Open-Source) — Primary

সবচেয়ে সহজ start:

```
# Ubuntu Server-এ all-in-one install
curl -sO https://packages.wazuh.com/4.14/wazuh-install.sh
sudo bash ./wazuh-install.sh -a
```

Components:

```
Wazuh Manager → analysis + rules
Wazuh Indexer → storage (OpenSearch-based)
Dashboard      → web UI (port 443)
Agent          → Windows/Linux endpoint-এ
```

3.2 Splunk (Free/Trial Lab Option — 500 MB/day)

```
# Docker (easiest)
docker run -d -p 8000:8000 -p 8088:8088 -p 9997:9997 \
  -e SPLUNK_START_ARGS="--accept-license" \
  -e SPLUNK_PASSWORD="YourStrongPass123" \
  --name splunk splunk/splunk:latest
```

অথবা direct install: splunk.com → Splunk Enterprise free trial/free license option। Splunk Enterprise Trial/Free সাধারণত standalone lab use-এর জন্য 500 MB/day indexing limit দেয়; install করার আগে official Splunk page verify করুন।

Web UI → <http://localhost:8000>
 HEC port → 8088 (HTTP Event Collector)
 UF port → 9997 (Universal Forwarder)

3.3 Elastic Security (Basic License Lab Option)

```
# Docker Compose (official quickstart - see elastic.co docs for current URL)
curl -sO https://raw.githubusercontent.com/elastic/elasticsearch/main/docs/reference/setup/install/docker/docker-compose.yml

# অথবা simplified:
docker network create elastic
docker run -d --name es01 --net elastic -p 9200:9200 \
  -e "discovery.type=single-node" \
  -e "xpack.security.enabled=false" \
  docker.elastic.co/elasticsearch/elasticsearch:9.5.3
docker run -d --name kibana --net elastic -p 5601:5601 \
  docker.elastic.co/kibana/kibana:9.5.3
```

তরপর Kibana → **Security** → **Install Elastic Agent** → endpoint-এ agent install।

3.4 OpenSearch (Open-Source Lab Option)

```
# Docker Compose
git clone https://github.com/opensearch-project/opensearch-devops
# অথবা simple:
docker network create opensearch-net
docker run -d --name opensearch --net opensearch-net -p 9200:9200 -p 9600:9600 \
  -e "discovery.type=single-node" \
  -e "DISABLE_SECURITY_PLUGIN=true" \
  opensearchproject/opensearch:latest
docker run -d --name opensearch-dashboards --net opensearch-net -p 5601:5601 \
  -e "OPENSEARCH_HOSTS=[\"http://opensearch:9200\"]" \
  -e "DISABLE_SECURITY_DASHBOARDS_PLUGIN=true" \
  opensearchproject/opensearch-dashboards:latest
```

Security plugin disable করা শুধু isolated lab/testing-এর জন্য। Production setup official secured deployment guide follow করবে।

3.5 Microsoft Sentinel (Cloud)

1. Azure Free Account (\$200 credit + free services)
2. Create → Log Analytics Workspace
3. Microsoft Sentinel → ওই workspace-এ attach করুন
4. Data Connectors → প্রয়োজনীয় connector enable করুন
 - Windows Security Events (via AMA agent)
 - Syslog (via CEF/AMA)
 - SigninLogs / AuditLogs (Entra ID)

Sentinel-এ data ingestion cost হয়। Lab-এ ছোট volume রাখুন, কাজ শেষে connector disable করুন।

4. The One-Dataset Approach — একই Data, সব Platform

এটাই এই guide-এর সবচেয়ে দরকারী trick:

একই attack simulation চালান, একই logs ওটি platform-এ পাঠান, একই চটি exercise করুন।

4.1 Standard Dataset — যা generate করবেন

```

| DATASET 1 – SSH Brute Force |
| → Ubuntu-তে attacker থেকে 30টি failed SSH login |
|
| DATASET 2 – Successful Login After Failures |
| → শেষে একটি successful login (attack chain) |
|
| DATASET 3 – Windows Failed Logins (Event 4625) |
| → Windows-এ ভুল password দিয়ে 10 বার login try |
|
| DATASET 4 – Account Creation (Event 4720) |
| → Windows-এ নতুন user তৈরি |
|
| DATASET 5 – Firewall Traffic (optional) |
| → pfSense/FortiGate demo syslog |

```

4.2 Attack Simulation Script (SSH Brute Force)

Attacker VM থেকে:

```
#!/bin/bash
# brute.sh – controlled lab simulation ONLY
TARGET="192.168.56.10"    # আপনার lab Ubuntu-র IP
USER="admin"

for i in $(seq 1 30); do
  sshpass -p "WrongPass$i" ssh -o StrictHostKeyChecking=no \
    $USER@$TARGET exit 2>/dev/null
  sleep 1
done
```

শুধুমাত্র নিজের **isolated lab**-এ চালান। অন্য কারো system-এ চালানো আইনত অপরাধ।

Ubuntu-র /var/log/auth.log-এ দেখবেন:

```
Failed password for admin from 192.168.56.20 port 44812 ssh2
Failed password for admin from 192.168.56.20 port 44814 ssh2
...
```

এই একই **dataset** এখন প্রতিটি SIEM-এ যাবে।

4.3 Log Delivery — কোন platform-এ কীভাবে পৌঁছাবে

Ubuntu SSH logs

├─ Wazuh Agent (Ubuntu-তে install)	→ Wazuh Manager
├─ Universal Forwarder	→ Splunk Indexer
├─ Elastic Agent / Filebeat	→ Elasticsearch
├─ Fluent Bit / Logstash	→ OpenSearch
└─ Syslog (rsyslog → collector/AMA)	→ Sentinel Log Analytics

Windows events-এর জন্য:

Windows Event Log

—	Wazuh Agent	→ Wazuh
—	Splunk UF	→ Splunk
—	Winlogbeat / Elastic Agent	→ Elastic
—	AMA agent	→ Sentinel

One source, many destinations: rsyslog দিয়ে একই log একাধিক জায়গায় পাঠানো যায় (@@remote-host multiple action lines)। তাই একই একটি failed login ঘটনা ৫টি SIEM-এই দেখা যাবে।

5. Same Task, Different Platform — Mapping Table

এই table-টা ধরে রাখুন। এটাই **overall SIEM শেখার shortcut**:

Task	Wazuh	Splunk	Elastic	OpenSearch
Onboard agent	Wazuh Agent	Universal Forwarder	Elastic Agent	Fluentbeat/Logstash
Syslog input	remote connection	TCP/UDP input	Logstash input	Ingest pipeline
Search language	Dashboard filter / DQL	SPL	KQL (Lucene-style) / DSL	DQL
Failed login field	data.srcip, rule.id:5710	EventCode=4625	event.action:ssh_login_failed	event.action:failed_login
Detection	Custom rule (XML)	Correlation Search	Detection Rule (EQL/Threat)	Security Analytics rule
Dashboard	Modules / Visualize	Dashboard Studio	Kibana Lens	OpenSearch Dashboards
Case management	Basic	ES + SOAR	Elastic Cases	—
Automation	Active Response	SOAR app	Connector/Watcher	Notification
MITRE mapping	Rule mitre field	ES ATT&CK content pack	Rule references	—



6. Exercise 1 — Search: Same Query, 4 Languages

প্রশ্ন: “কোন *source IP* সবচেয়ে বেশি *failed SSH login* করেছে?”

Wazuh

Dashboard → Security events → filter:

```
rule.groups: authentication_failed
```

অথবা API:

```
curl -k -H "Authorization: Bearer $TOKEN" \
  "https://localhost:55000/events?rule.groups=authentication_failed"
```

Splunk (SPL)

```
index=linux sourcetype=linux_secure "Failed password"
| rex "Failed password for \w+ (?<user>\S+) from (?<src_ip>\S+)"
| stats count by src_ip
| sort - count
```

Elastic (Kibana KQL)

```
event.module:system AND event.dataset:system.auth AND message:"Failed password"
```

Aggregation (Lens/Timeline):

Group by: source.ip → Count → Sort desc

Sentinel (Kusto KQL)

Syslog

```
| where ProcessName == "sshd" and SyslogMessage contains "Failed password"  
| extend SrcIp = extract(@"from (\d+\.\d+\.\d+\.\d+)", 1, SyslogMessage)  
| summarize FailedCount = count() by SrcIp  
| order by FailedCount desc
```

লক্ষ্য করুন — **logic identical**: filter → extract field → group → count → sort। শুধু syntax আলাদা। এটাই “query language শেখা” আসলে সহজ — একবার mindset বুঝলেই হয়।



7. Exercise 2 — Detection: Brute Force Rule Everywhere

Logic:

```
IF failed_ssh >= 10
FROM same src_ip
WITHIN 5 minutes
THEN alert (severity: high)
```

7.1 Wazuh Custom Rule

/var/ossec/etc/rules/local_rules.xml:

```
<group name="linux,sshd,brute_force,">
  <!-- Frequency-based brute force using if_matched_sid -->
  <rule id="100100" level="10" frequency="10" timeframe="300">
    <if_matched_sid>5710</if_matched_sid>  <!-- sshd auth failed -->
    <description>SSH brute force: multiple failures from same IP.</
      description>
    <mitre><id>T1110</id></mitre>
  </rule>
</group>
```

```
sudo systemctl restart wazuh-manager
```

Test: brute.sh চালান → Dashboard-এ rule.id:100100 alert দেখুন।

7.2 Splunk Correlation Search

```
index=linux sourcetype=linux_secure "Failed password"
| rex "from (?<src_ip>\S+)"
| stats count by src_ip, index, host
| where count >= 10
```

Save as **Alert** → Trigger when number of results > 0 → Cron/real-time window 5 min

7.3 Elastic Detection Rule

Kibana → Security → Rules → **Create new rule** → **Threshold:**

```
Query:      event.module:system AND message:"Failed password"
Group by:   source.ip
Threshold:  count >= 10
Window:     5 minutes
Severity:   High
```

7.4 OpenSearch Security Analytics

Dashboards → Security Analytics → Detector:

```
Input:      syslog index
Workflow:   ssh_brute_force
Trigger:    count >= 10 in 5m → Notification channel (email/Slack)
```

7.5 Sentinel Analytics Rule (KQL)

Syslog

```
| where SyslogMessage contains "Failed password"  
| extend SrcIp = extract(@"from (\d+\.\d+\.\d+\.\d+)", 1, SyslogMessage)  
| where isnotempty(SrcIp)  
| summarize FailCount = count() by SrcIp, bin(TimeGenerated, 5m)  
| where FailCount >= 10
```

Query scheduling: every 5 minutes, lookback 5 minutes

Incident: create, severity High

MITRE: Credential Access → T1110 Brute Force

একই rule ৫ ভাষায় লেখা শেষ — একবার করলে detection engineering আর ভুলবেন না।



8. Exercise 3 — Investigation: Attack Chain খোঁজা

Scenario: brute force-এর পরে একটি **successful login** হয়েছে (Dataset 2)।

প্রতিটি platform-এ এই pivot chain চালান:

```
Step 1: Alert-এর src_ip নিন
Step 2: ওই IP-র সব activity দেখুন
Step 3: Successful login খুঁজুন (Accepted password / 4624)
Step 4: Login-এর পরে কী হয়েছে (command, process, sudo)
Step 5: Timeline বানান
```

Splunk version:

```
index=linux src_ip="192.168.56.20"
| sort _time
| table _time, action, user, src_ip, dest, message
```

Sentinel version:

```
union Syslog, SecurityEvent
| where TimeGenerated > ago(1h)
| where SourceIP == "192.168.56.20" or IPAddress == "192.168.56.20"
| project TimeGenerated, EventType, Account, SourceIP, Activity
| order by TimeGenerated asc
```

Timeline (খাতায় লিখুন):

```
14:20:01 Failed password (attempt 1)
14:20:35 Failed password (attempt 30)
14:21:10 Accepted password for admin ← breach point
14:21:15 sudo: admin : TTY=pts/0 ; COMMAND=/bin/whoami
14:22:40 new user created (if Dataset 4 exists)
```

এই exercise-টা প্রতিটি platform-এ করুন। **Investigation** মানেই এটা — #05 article-এর practical রূপ।



9. Exercise 4 — Dashboard: একই Panel, সব Platform

প্রতিটি SIEM-এ এই ৫টি panel বানান:

1. Failed logins over time (time-series line)
2. Top 10 source IPs (bar/table)
3. Top targeted usernames (bar)
4. Successful vs Failed ratio (pie/metric)
5. Rule-wise alert count (table)

Platform	কোথায় বানাবেন
Wazuh	Dashboard → Visualize / module override
Splunk	Dashboard Studio (SPL-backed panels)
Elastic	Kibana Lens (drag-drop fields)
OpenSearch	Dashboards → Visualize
Sentinel	Workbooks (KQL-backed)

Dashboard বানানোর আগে query validate করুন — #09 article-এর validation flow follow করুন।



10. Exercise 5 — Automation: SOAR Mini-Lab

10.1 Webhook Alert → Python Script

Wazuh Integration (একটি উদাহরণ):

/var/ossec/etc/ossec.conf:

```
<integration>
  <name>custom-webhook</name>
  <hook_url>http://192.168.56.30:5000/alert</hook_url>
  <level>10</level>
  <alert_format>json</alert_format>
</integration>
```

Python listener:


```

from flask import Flask, request
import requests

app = Flask(__name__)

TI_API = "https://api.example-ti.com/v1/ip/" # যেকোনো TI source

@app.route("/alert", methods=["POST"])
def alert():
    data = request.json
    src_ip = (data.get("data") or {}).get("srcip")
    if src_ip:
        rep = requests.get(TI_API + src_ip, timeout=5)
        print(f"[ALERT] {src_ip} → TI: {rep.status_code}")
    return "ok", 200

app.run(host="0.0.0.0", port=5000)

```

10.2 Sentinel Playbook (Logic App)

```

Incident created
  ↓
Get incident entity (IP)
  ↓
HTTP → TI lookup
  ↓
Add comment to incident

```

10.3 Shuffle (Free SOAR)

Wazuh → webhook → Shuffle workflow:
 Extract IP → TI lookup → Create case (TheHive/Google Sheet) → Notify

মনে রাখুন — automated containment (firewall block, account disable) শুধু **approval**-এর পরে।
Human-in-the-loop!



11. Exercise 6 — Threat Intel Integration

MISP Lab (Free TIP)

```
# MISP docker
git clone https://github.com/MISP/misp-docker
cd misp-docker && docker compose up -d
```

1. MISP-এ event create করুন → আপনার lab attacker IP add করুন
2. API key নিন
3. SIEM-এ connect করুন:
 - Wazuh → CDB list / integration script
 - Splunk → Threat Intel Management
 - Elastic → Threat Intel connector (IOC index)
 - Sentinel → TI connector (STIX/TAXII)
4. Attacker IP দিয়ে traffic জেনারেট করুন → TI match alert দেখুন

12. 6-Week Combined Learning Plan — সব একসাথে

WEEK 1 – Foundation Lab

- └─ Lab setup: Ubuntu + Windows + Attacker VM
- └─ Wazuh install + agents (Ubuntu, Windows)
- └─ Dataset 1-2 generate (SSH brute force + success)
- └─ Exercise: search + verify logs in Wazuh

WEEK 2 – Detection on Wazuh

- └─ Custom brute-force rule (XML)
- └─ Windows rule test (4720 account creation)
- └─ Investigation: attack chain timeline
- └─ Dashboard: 5 panels

WEEK 3 – Splunk

- └─ Install (Docker), UF on Ubuntu + Windows
- └─ Same dataset ingest → SPL search
- └─ Brute-force alert (SPL)
- └─ Dashboard + same investigation

WEEK 4 – Elastic + OpenSearch

- └─ Docker setup, agents
- └─ KQL/DSL search, threshold rule
- └─ Same investigation, same dashboard
- └─ (Optional) OpenSearch detector

WEEK 5 – Sentinel (Cloud)

- └─ Azure free account + workspace
- └─ Connectors: Syslog(AMA) + Windows Events
- └─ KQL: same queries, analytics rules
- └─ Incident + Playbook (Logic App)
- └─ কাজ শেষে connector off / data cleanup

WEEK 6 – Advanced + Portfolio

- └─ MISP TI integration → IOC match alert
- └─ Shuffle/Python automation
- └─ Sigma rules লিখুন → convert → deploy
- └─ Portfolio: GitHub repo + screenshots + write-up

13. Sigma — একবার লিখুন, সব জায়গায় চালান

আগের exercise-এর rule-টাই Sigma-তে:

```

title: SSH Brute Force Detection
id: 100100-bf-ssh
status: stable
description: Multiple failed SSH logins from same source in short window
author: Your Name
date: 2026/09/16
logsource:
  product: linux
  service: auth
detection:
  keywords:
    - 'Failed password'
  condition: keywords | count() by source.ip >= 10
falsepositives:
  - Approved security scanner
level: high
tags:
  - attack.credential_access
  - attack.t1110

```

```

pip install sigma-cli pySigma
sigma convert -t splunk -p splunk rule.yml # → SPL
sigma convert -t lucene -p ecs-elastic rule.yml # → Elastic query

```

এটাই **Detection-as-Code** — rule একবার লিখুন, Git-এ রাখুন, যেকোনো SIEM-এ convert করুন।



14. Windows Event ID Cheat Sheet (Lab-এ যা collect করবেন)

```
4624 Successful logon
4625 Failed logon          → brute force detection
4634 Logoff
4648 Explicit credential logon
4663 File/object access attempt
4672 Special privileges assigned
4688 Process creation      → command-line monitoring
4720 User account created
4722 User account enabled
4724 Password reset
4726 User account deleted
4728 Member added to security group
4732 Member added to local group
4740 Account locked out
1102 Audit log cleared     → tampering indicator
```

Wazuh rule test:

```
# Windows VM-এ test user বানান → 4720 → SIEM-এ verify
net user testuser Pass123! /add
```



✖ 15. Common Multi-Platform Lab Mistakes

✖ Mistake 1 — সব platform একসাথে চালানো

RAM শেষ, সব slow। একসময়ে এক/দুইটা।

✖ Mistake 2 — শুধু install করে থেমে যাওয়া

Install ≠/learning। Exercise complete করুন।

✖ Mistake 3 — একই dataset না ব্যবহার করা

আলাদা data মানে তুলনা করতে পারবেন না।

✖ Mistake 4 — Alert tune না করে পরের platform-এ যাওয়া

Tuning-ই আসল skill। Scanner IP exclude করার practice করুন।

✖ Mistake 5 — Documentation না রাখা

প্রতিটি step screenshot + note। এটাই portfolio।

Mistake 6 — Cloud cost ignore করা

Sentinel-এর ingestion budget মনিটর করুন। Lab শেষে বন্ধ করুন।



16. Multi-Platform SIEM Checklist

- [] Lab VMs ready (Ubuntu, Windows, Attacker)
- [] Dataset scripts ready (brute force, Windows events)
- [] Wazuh installed + 2 agents
- [] Splunk installed + UF
- [] Elastic installed + agent
- [] (Optional) OpenSearch detector tested
- [] Sentinel workspace + connectors
- [] Same dataset ingested in each platform
- [] Brute-force rule written in each platform
- [] Attack-chain investigation done in each platform
- [] 5-panel dashboard built in each platform
- [] Webhook/SOAR automation tested
- [] MISP TI integration tested
- [] Sigma rules written + converted
- [] GitHub portfolio repo created
- [] README + screenshots + detection rules committed



17. Interview Questions

Q1. আপনি কোন কোন SIEM platform-এ কাজ করেছেন?

Answer: আমি প্রাথমিকভাবে Wazuh-এ hands-on কাজ করেছি — agent deployment, custom rules, decoders, dashboards। এছাড়াও একই use cases Splunk (SPL), Elastic Security এবং Microsoft Sentinel (KQL)-এ implement করেছি — যেমন SSH brute-force detection, Windows event monitoring, dashboards এবং basic automation। আমার approach হলো concept-first: platform আলাদা হলেও onboarding, detection, investigation workflow একই।

Q2. Splunk আর Sentinel-এ একই detection কীভাবে লিখবেন?

Answer: Logic একই থাকে — failed login events filter করা, source IP দিয়ে group করা, threshold apply করা। Splunk-এ SPL (`stats count by src_ip | where count >= 10`) আর Sentinel-এ KQL (`summarize count() by IpAddress`) ব্যবহার করি। Splunk-এ Alert/Scheduled Search, Sentinel-এ Analytics Rule হিসেবে deploy করি।

Q3. Wazuh-এর সীমাবদ্ধতা কী, আর enterprise SIEM কোথায় এগিয়ে?

Answer: Wazuh দুর্দান্ত open-source endpoint monitoring ও compliance platform, কিন্তু বড় scale-এ advanced correlation, SOAR integration, long-term retention ও compliance reporting-এ enterprise platform যেমন Splunk/Sentinel বেশি mature। Lab ও small environment-এ Wazuh খুব শক্তিশালী।

Q4. একটি নতুন SIEM platform শিখতে কতদিন লাগবে?

Answer: Core concept জানা থাকলে — onboarding, search, detection, dashboard — এই ৪টি কাজ প্রথম সপ্তাহেই করা যায়। Platform-specific deep features (SOAR, data models, cost management) আরও ৩-৪ সপ্তাহ। আমি প্রতিবার একই dataset দিয়ে same exercise করি, তাই শেখা দ্রুত হয়।

Q5. Sigma rule কেন ব্যবহার করবেন?

Answer: Sigma দিয়ে detection logic platform-independent ভাবে লেখা যায়। একই rule Splunk, Elastic, Sentinel-এ convert করা যায়, Git-এ version control করা যায়, team-এ share করা যায়। এটা Detection-as-Code-এর ভিত্তি।



Final Takeaway

একটি Lab → একটি Dataset
একই Exercise → ৫টি Platform
একটি Skill → Overall SIEM

মনে রাখুন:

SIEM শেখা মানে কোনো product-এর menu মুখস্থ করা নয়। SIEM শেখা মানে — data onboard করা, threat detect করা, incident investigate করা — যেকোনো platform-এ।

Platform বদলাবে, skill থাকবে।

Wazuh দিয়ে শুরু করুন, বাকিগুলোতে repeat করুন — ৬ সপ্তাহ পরে আপনি “overall SIEM” practical জানবেন।



SIEM Engineer Learning Series

#01 — Log Management #02 — SIEM Platforms #03 — Log Integration #04 — Detection Engineering #05 — Query & Investigation #06 — SOC Operations #07 — Threat Intelligence #08 — Automation & SOAR #09 — Dashboard & Reporting #10 — SIEM Infrastructure #11 — SIEM Architecture Design #12 — SIEM Capacity Planning #13 — SIEM Performance Tuning #14 — Advanced Detection Engineering #15 — Threat Hunting with SIEM #16 — SIEM Data Quality & Troubleshooting #17 — Hands-On SIEM: Multi-Platform Practical Guide ← You are here

Navigation

← **Previous Article: #16 — SIEM Data Quality & Troubleshooting**

SIEM Engineer Index

You have completed the full #01–#17 series
